Appunti di Algorithm Engineering

Appunti del Corso

Anno Accademico 2026/2027

Indice

1 Progettazione di Algoritmi, Modelli Computazionali e Gerarchia di Memoria **1**
- 1.1 Definizione Formale di un Algoritmo 1
- 1.2 Misurazione dell'Efficienza e Analisi Asintotica 2
- 1.3 Modelli Computazionali: RAM e Hardware Reale 4
- 1.4 Località e Modello a Due Livelli 7
- 1.5 Esempio Pratico: Scansione di un Array Lineare 9

2 Accesso a Passi, B-Tree e Impatto della Paginazione Virtuale **11**
- 2.1 Accesso Non Sequenziale: L'Algoritmo $A_{s,b}$ 11
- 2.2 Limiti Algoritmici e Accelerazione Hardware 13
- 2.3 Ricerca in Insiemi Ordinati: Ricerca Binaria contro B-Tree . . 14
- 2.4 Algoritmi I/O-Aware e Paginazione Virtuale 16
- 2.5 Punti Chiave 17

3 Permutazione, Ordinamento e Binary Merge Sort in Memoria Esterna **19**
- 3.1 Il Problema della Permutazione vs. Ordinamento 19
- 3.2 Complessità di Permutazione in Memoria Esterna 24
- 3.3 Binary Merge Sort in Memoria Esterna 26
- 3.4 Equivalenza Asintotica del Costo di Ordinamento Esterno . . 29

4 Multiway Merge Sort, Snow Plow e Disk Striping **31**
- 4.1 Multiway Merge Sort (k-Way Merge Sort) 31
- 4.2 Ottimizzazione dei Run Iniziali: Lo Spazzaneve (*Snow Plow*) . 34
- 4.3 Parallelismo su Più Dischi: *Disk Striping* 37

5 Limiti Inferiori per l'Ordinamento, Quicksort Dual-Pivot e Bounded Quicksort **41**
- 5.1 Modello RAM: Limiti Inferiori e Alberi di Decisione 41
- 5.2 Modello a Due Livelli: Limite Inferiore di Aggarwal-Vitter . . 45

5.3 Quicksort: Gestione dei Duplicati e Dual-Pivot Quicksort . . . 49
5.4 Bounded Quicksort (BQS) ed Eliminazione della Tail Recursion 56

6 Statistiche d'Ordine, Trade-off Multi-Pivot e Analisi Formale del Caso Medio di Quicksort 63
6.1 Il Problema della Selezione (*Selection of the k-th Item*) 63
6.2 Il Dilemma del Multi-Pivot (*Why Not Many Pivots?*) 70
6.3 Quicksort: Partizionamento a 3 Vie e Implicazioni Ricorsive . 73
6.4 Costo Medio di Quicksort ($\Theta(n \log_2 n)$) 76
6.5 Sintesi Comparativa dei Risultati 81

7 Ordinamento Esterno (Distribution Sort), Over-Sampling e Fondamenti di Random Sampling 83
7.1 Il Problema dell'Ordinamento Esterno e il Multi-Way Partitioning 83
7.2 La Tecnica dell'Over-Sampling 85
7.3 Analisi Probabilistica del Fallimento e Limite di Chernoff . . . 88
7.4 Introduzione al Random Sampling e Limiti dell'Approccio Non-Streaming 93

8 Algoritmi di Random Sampling su Data Stream 97
8.1 Fondamenti del Modello di Streaming e del Random Sampling 97
8.2 Campionamento con Lunghezza Nota 99
8.3 Campionamento con Lunghezza Incognita 106
8.4 Dimostrazione Formale per Induzione del Reservoir Sampling . 111
8.5 Sintesi Comparativa 113

9 Strutture Dati Randomizzate: Il Problema del Dizionario e Skip List 115
9.1 Introduzione e Motivazione: Il Problema del Dizionario 115
9.2 Dalla Ricerca Binaria su Lista alla Skip List 118
9.3 Skip List: Struttura e Analisi Probabilistica Formale 122
9.4 Conclusioni e Prospettive: Verso gli Alberi di Ricerca Randomizzati 128

10 Alberi di Ricerca Randomizzati: Treap, Operazioni Dinamiche e Applicazioni Geometriche 129
10.1 Introduzione e Definizione Formale del Treap 129
10.2 Operazioni Elementari: Rotazioni nel Treap 132
10.3 Algoritmi Fondamentali e Operazioni Dinamiche 133
10.4 Analisi Probabilistica Formale (Seidel e Aragon) 141
10.5 Isomorfismo Geometrico e Applicazioni Multidimensionali . . . 144

11 Algoritmi di Intersezione per Posting Lists nell'Information Retrieval 154
11.1 Architettura di un Motore di Ricerca e Contesto del Problema 154
11.2 Algoritmi Fondamentali e Analisi Comparativa 157
11.3 Mutual Partition (Divide et Impera) 160
11.4 Doubling Search (Exponential Search / Galloping Search) . . . 163
11.5 Block Partitioning / Skip Pointers (Approccio Pratico a Blocchi) 168

12 Ricerca per Interpolazione su Bucket e Ordinamento di Stringhe 171
12.1 Ricerca per Interpolazione Basata su Bucket 171
12.2 Ordinamento di Stringhe e Analisi del Limite Inferiore 178
12.3 Multi-Key Quicksort (3-Way Radix Sort) 182

13 Strutture Dati e Algoritmi per Stringhe: TST e Radix Sort 189
13.1 Introduzione e Modello Computazionale per Stringhe 189
13.2 Ternary Search Trees: Modello Teorico e Struttura del Nodo . 190
13.3 Costruzione del TST e Partizionamento a 3 Vie 193
13.4 Operazioni Fondamentali e Analisi di Complessità nei TST . . 197
13.5 Radix Sort: Tassonomia, Funzionamento e Complessità 199
13.6 Ottimizzazione di LSD Radix Sort per Stringhe Binarie 203
13.7 Analisi Comparativa Critica e Scelta Architetturale 207

14 Alberi Digitali per Stringhe: Trie, MSD Radix Sort e Trie Compatti 211
14.1 Relazione Fondamentale tra MSD Radix Sort e Trie 211
14.2 Implementazione Standard tramite Vettori (Array-Based Trie) 214
14.3 Ottimizzazione della Memoria: Trie Basato su Tabelle Hash . 216
14.4 Trie Compatti (Compacted Trie o Patricia Trie) 218
14.5 Estensioni per Prefix Search con Frequenze (Autocompletamento) 224

15 Algoritmi e Strutture Dati per la Ricerca di Stringhe in Memoria Gerarchica 228
15.1 Introduzione e Modelli di Calcolo per la Ricerca di Stringhe . 228
15.2 Organizzazione e Compressione su Disco: Front-Coding 232
15.3 L'Architettura a Due Livelli e il Compacted Trie come Indice Primario 235
15.4 La Patricia Trie (Blind Trie) e la Compressione Lossy 237
15.5 L'Algoritmo di Ricerca a Tre Fasi (Three-Phase Blind Search) 239
15.6 Esempi Applicativi Completi e Simulazione Passo-Passo 243
15.7 Quadro Comparativo e Considerazioni Conclusive 249

16 Indicizzazione e Ricerca di Sottostringhe: Suffix Array, Suffix Tree e Array LCP **252**
16.1 Dal Problema del Dizionario alla Ricerca di Sottostringhe . . . 252
16.2 La Struttura Dati Suffix Array (SA) 257
16.3 La Struttura Dati Suffix Tree 262
16.4 Algoritmi di Costruzione e l'Array LCP 266
16.5 Quadro Comparativo e Sintesi Conclusiva 271

Bibliografia **274**

Capitolo 1

Progettazione di Algoritmi, Modelli Computazionali e Gerarchia di Memoria

1.1 Definizione Formale di un Algoritmo

Un *algoritmo* è una sequenza finita e ordinata di istruzioni elementari non ambigue che, a partire da un insieme specificato di dati di input, produce un corrispondente insieme di dati di output in un lasso di tempo finito.

Nella formulazione classica presentata in *The Art of Computer Programming* [1], un algoritmo deve soddisfare cinque proprietà fondamentali:

Finitudine. L'algoritmo deve sempre terminare dopo un numero finito di passi di calcolo per ogni configurazione di input ammissibile.

Definitezza. Ogni istruzione deve essere formulata in modo chiaro, preciso e senza ambiguità; lo stato di esecuzione successivo deve essere univocamente determinato.

Input. L'algoritmo riceve zero o più quantità specificate dall'esterno prima che l'esecuzione abbia inizio.

Output. L'algoritmo produce una o più quantità di output che hanno una relazione specificata con i dati di input. L'output deve essere significativo per il problema che si sta risolvendo.

Efficacia. Ogni operazione prescritta deve essere elementare e fisicamente eseguibile in tempo finito utilizzando una quantità limitata di risorse, come carta e penna o una macchina sequenziale.

1.1.1 Algoritmi Esatti, Intelligenza Artificiale ed Euristiche

Un classico algoritmo esatto si distingue da un metodo di intelligenza artificiale o euristico per le garanzie che fornisce e per il modo in cui esplora lo spazio delle decisioni.

Tabella 1.1: Confronto tra algoritmi esatti, intelligenza artificiale e metodi euristici.

Dimensione	Algoritmo esatto classico	Metodo di IA o euristico
Garanzia di correttezza	Produce una soluzione corretta per ogni istanza ammissibile; l'ottimalità è garantita quando la specifica del problema richiede un ottimo.	Può privilegiare la qualità empirica o l'efficienza pratica e può mancare di una garanzia universale per ogni istanza.
Terminazione	Può essere dimostrata in anticipo utilizzando funzioni potenziale o varianti di ciclo.	Spesso legata a criteri di arresto euristici, come un numero massimo di epoche o la convergenza del gradiente.
Spazio delle decisioni	Determinato da una procedura specificata in modo preciso, che può essere deterministica o randomizzata.	Esplorato tramite ricerca euristica, inferenza statistica o ottimizzazione di una funzione di perdita.

La *programmazione euristica* non cerca una dimostrazione formale di ottimalità assoluta su ogni istanza ammissibile. Piuttosto, cerca un compromesso efficiente per problemi computazionalmente intrattabili, come i problemi NP-completi o NP-ardui.

1.2 Misurazione dell'Efficienza e Analisi Asintotica

La valutazione quantitativa delle prestazioni di un algoritmo richiede un impianto metodologico formale capace di separare la bontà intrinseca del progetto dai dettagli contingenti dell'hardware e della toolchain di compilazione.

In questa sezione esaminiamo le motivazioni teoriche che guidano l'adozione della notazione asintotica e la scelta prudenziale dell'analisi al caso peggiore.

1.2.1 Perché Non Contare il Numero Esatto di Istruzioni?

Definire il costo esatto di esecuzione $T_A(n)$ come il numero effettivo di cicli di clock o di istruzioni elementari presenta diverse difficoltà:

1. **Dipendenza dall'architettura:** il risultato dipenderebbe dalla specifica CPU, dall'architettura del set di istruzioni (ISA) e dalla frequenza di clock.
2. **Dipendenza dalla toolchain:** varierebbe a seconda del compilatore, del livello di ottimizzazione e del sistema operativo.
3. **Mancanza di generalità:** oscurerebbe la qualità intrinseca dell'idea algoritmica.

Per questo motivo, l'analisi degli algoritmi utilizza la notazione asintotica. Essa astrae dai dettagli implementativi e dalle costanti moltiplicative, e studia il tasso di crescita della funzione di costo al tendere della dimensione dell'input all'infinito: $n \to \infty$.

O-grande (limite superiore asintotico):

$$f(n) = \mathcal{O}(g(n)) \iff \exists c > 0,\ n_0 > 0 \text{ tali che } 0 \leq f(n) \leq c\, g(n), \quad \forall n \geq n_0.$$

Omega-grande (limite inferiore asintotico):

$$f(n) = \Omega(g(n)) \iff \exists c > 0,\ n_0 > 0 \text{ tali che } 0 \leq c\, g(n) \leq f(n), \quad \forall n \geq n_0.$$

Theta-grande (limite asintotico stretto):

$$f(n) = \Theta(g(n)) \iff f(n) = \mathcal{O}(g(n)) \wedge f(n) = \Omega(g(n)).$$

1.2.2 Perché Preferire l'Analisi al Caso Pessimo?

- **Garanzia assoluta:** un limite superiore garantisce che l'algoritmo non impiegherà mai più tempo del limite teorico. Questo è essenziale per i sistemi real-time e mission-critical.
- **Semplice derivazione formale:** l'analisi al caso pessimo non richiede assunzioni probabilistiche sulla distribuzione dell'input, le quali sono spesso necessarie per l'analisi al caso medio.

1.3 Modelli Computazionali: RAM e Hardware Reale

Un algoritmo non viene eseguito in un vuoto astratto: viene eseguito su una macchina con processori, memorie e meccanismi di trasferimento dati. Un modello computazionale specifica quali dettagli della macchina sono rilevanti per l'analisi e quali vengono deliberatamente ignorati. Il modello RAM è il punto di partenza standard; l'hardware reale spiega poi quando tale astrazione diviene insufficiente.

1.3.1 Il Modello Random Access Machine

Il modello Random Access Machine (RAM) è un'astrazione sequenziale, basata su parole (word), ispirata all'architettura di von Neumann. Una RAM è costituita da un processore, un programma finito e una memoria indirizzabile di parole. Le sue assunzioni abituali sono:

- **Esecuzione sequenziale:** un processore esegue un'istruzione elementare alla volta.
- **Dati della dimensione di una parola:** l'aritmetica, i confronti, le assegnazioni e il calcolo degli indirizzi su un numero costante di parole richiedono tempo costante.
- **Accesso uniforme alla memoria:** la lettura o la scrittura di qualsiasi indirizzo costa $\mathcal{O}(1)$, indipendentemente dall'indirizzo e dalla storia degli accessi.
- **Memoria sufficiente:** l'input e le strutture dati risiedono interamente nello spazio di indirizzamento modellato; il costo di allocazione dello spazio viene analizzato separatamente quando risulta rilevante.

Il modello RAM è utile perché rende confrontabili le idee algoritmiche. Ad esempio, un ciclo che esamina n elementi di un array esegue $\Theta(n)$ operazioni RAM, indipendentemente dal particolare processore utilizzato per eseguirlo. Il modello non afferma che una macchina reale possieda memoria infinita o una latenza veramente costante; esso dichiara quali costi vengono astratti.

1.3.2 Hardware Reale: Gerarchia di Memoria e Memory Wall

Su una macchina moderna, il processore è molto più veloce delle memorie che contengono grandi insiemi di dati. Questo crescente divario tra calcolo e

movimento dei dati è noto come *memory wall*. Inoltre, la memoria locale non è una singola risorsa uniforme: è organizzata in livelli con capacità, latenze e granularità di trasferimento differenti.

Una gerarchia semplificata, ordinata dal livello più veloce e più piccolo a quello più lento e più grande, è:

$$\text{Registri della CPU} \longrightarrow \text{Cache L1} \longrightarrow \text{Cache L2} \longrightarrow \text{Cache L3} \longrightarrow \text{DRAM} \longrightarrow \text{NVMe/SSD/disco} \longrightarrow \text{Storage remoto / cloud.}$$

I registri e le cache fanno parte del sistema di memoria veloce del processore; la DRAM è l'usuale memoria principale; gli SSD e i dischi forniscono archiviazione persistente. Lo storage remoto e i servizi cloud sono inclusi qui solo come una più ampia gerarchia di accesso ai dati, in quanto sono raggiunti attraverso una rete piuttosto che attraverso il bus di memoria locale.

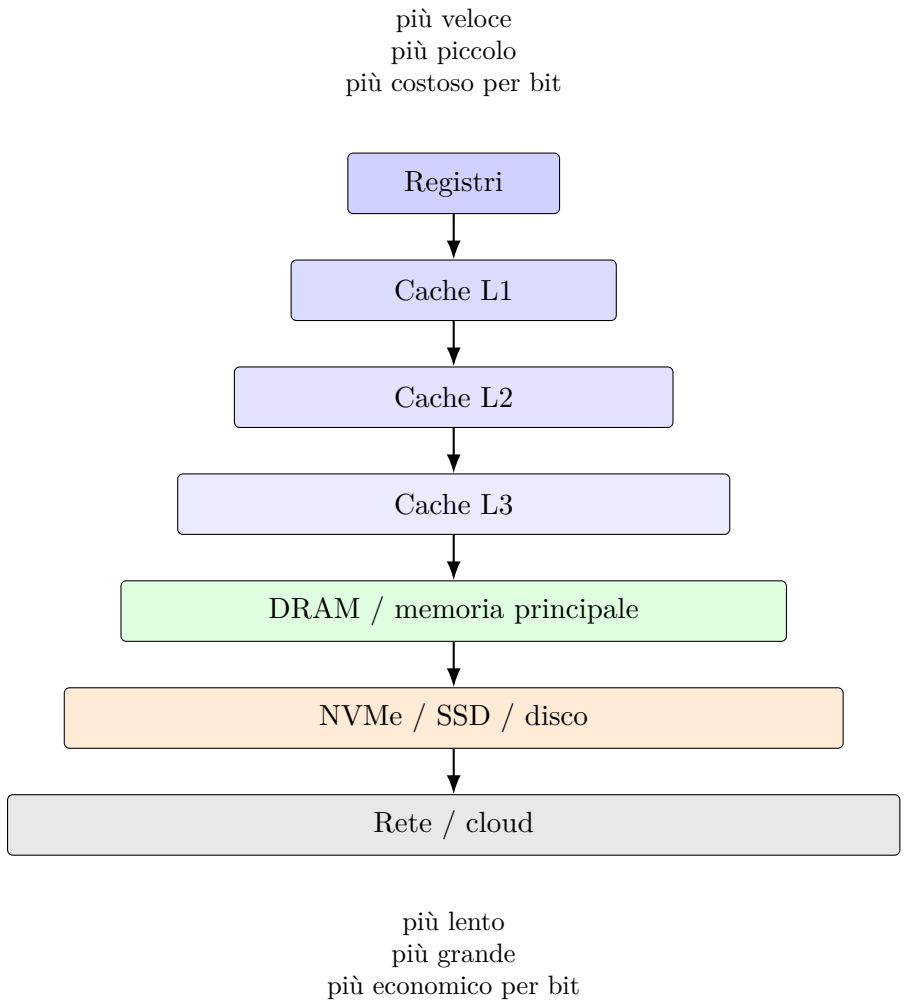


Figura 1.1: Una vista gerarchica della memoria e dell'archiviazione.

Figura 1.1 descrive i principali livelli di memoria locale: ogni strato inferiore scambia una minore velocità di accesso per una maggiore capacità.

Spostandosi verso il basso nella Figura 1.1, la capacità generalmente cresce mentre crescono anche la latenza e il costo di trasferimento. I rapporti esatti dipendono dall'hardware e dal carico di lavoro, per cui valori come da 100 a 200 volte per la DRAM rispetto alla L1, o da 10^4 a 10^6 volte per il disco rispetto alla L1, dovrebbero essere letti come esempi di ordine di grandezza piuttosto che come costanti universali.

1.3.3 Quando il Modello RAM Non È Sufficiente

Il modello RAM è una buona prima approssimazione quando il calcolo è prevalentemente sequenziale, il working set risiede interamente nella memoria principale e il costo del movimento dei dati non è il fattore dominante. Un modello diverso è preferibile quando uno dei seguenti effetti controlla le prestazioni:

Calcolo su GPU. L'esecuzione SIMT (Single Instruction, Multiple Threads) utilizza molti core di elaborazione e distingue tra spazi di memoria condivisa, locale e globale.

Memoria esterna. Nel calcolo out-of-core, il dataset non risiede interamente nella memoria veloce, per cui i trasferimenti a blocchi e le operazioni di I/O dominano il costo.

Calcolo parallelo e distribuito. Diversi processori o macchine eseguono simultaneamente e la sincronizzazione e la comunicazione diventano costi espliciti.

Informatica quantistica. Qubit, sovrapposizione, entanglement e porte unitarie richiedono un modello basato su operazioni quantistiche piuttosto che su parole RAM.

L'insegnamento ingegneristico fondamentale consiste quindi non nello scartare il modello RAM, bensì nell'utilizzare l'astrazione più parsimoniosa capace di catturare il collo di bottiglia del sistema target.

1.4 Località e Modello a Due Livelli

I sistemi moderni sfruttano due proprietà statistiche dei programmi reali:

1. **Località temporale:** se una locazione di memoria viene referenziata al tempo t, è probabile che venga referenziata nuovamente a breve, al tempo $t + \Delta t$.

2. **Località spaziale:** se una locazione di memoria x viene referenziata, locazioni vicine come $x + 1$, $x + 2$ e così via, verranno probabilmente referenziate a breve.

1.4.1 Il Modello di Memoria a Due Livelli

Quando si elaborano gigabyte o terabyte di dati, la gerarchia multilivello può essere astratta come un modello a due livelli, associato al modello a memoria esterna [2], noto anche come modello Disk Access Machine.

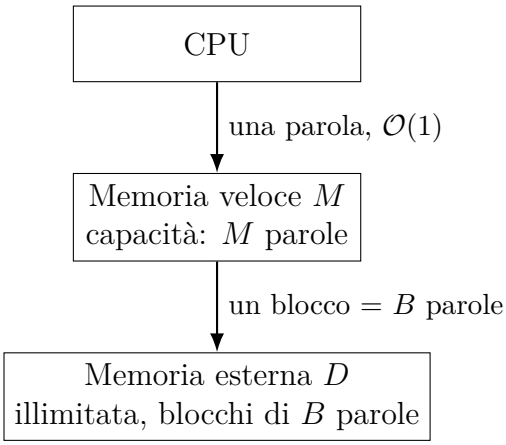


Figura 1.2: Modello di memoria a due livelli con memoria veloce M e memoria esterna D.

Figura 1.2 descrive l'astrazione a due livelli: la CPU opera su parole nella memoria veloce M, mentre i trasferimenti tra M e la memoria esterna D avvengono in blocchi di B parole.

Parametri del Modello

N: numero totale di elementi del problema da elaborare.

M: dimensione della memoria veloce interna, misurata come il numero massimo di elementi che possono essere contenuti simultaneamente.

B: dimensione del blocco trasferito in una singola operazione di I/O, con $M \geq 2B$.

Metriche di Costo

1. **Complessità di I/O (metrica primaria):** numero di trasferimenti di blocchi di dimensione B tra la memoria lenta e quella veloce.
2. **Tempo di CPU (metrica secondaria):** numero di operazioni logiche e aritmetiche eseguite dopo che i dati sono stati caricati in M.

3. **Spazio:** numero di blocchi allocati sulla memoria secondaria.

Questa astrazione è frattale: può modellare sia i trasferimenti RAM–disco sia i trasferimenti cache L3–RAM, ridefinendo il significato di M e B.

1.5 Esempio Pratico: Scansione di un Array Lineare

Consideriamo un array contiguo di N interi scansionato in sequenza, ad esempio per calcolare la somma $\sum_{i=1}^{N} A[i]$.

Listing 1.1: Scansione sequenziale di un array

```
long sum = 0;
for (int i = 0; i < N; i++) {
    sum += A[i];
}
```

1.5.1 Analisi nel Classico Modello RAM

- Accessi in memoria: N.
- Istruzioni di addizione: N.
- Tempo di calcolo asintotico: $\Theta(N)$.

1.5.2 Analisi nel Modello a Due Livelli

Assumiamo che la memoria secondaria memorizzi l'array A in modo consecutivo in blocchi di dimensione B e che $A[0]$ sia allineato con il confine di un blocco. Senza questa assunzione di allineamento, la scansione potrebbe richiedere al più un trasferimento di blocco aggiuntivo, senza alterare il limite asintotico:

1. Al primo accesso ad $A[0]$, si verifica un miss. Il sistema trasferisce il blocco contenente $A[0], \ldots, A[B-1]$ nella memoria veloce con un singolo I/O.
2. I successivi $B-1$ accessi trovano i loro dati già nella memoria veloce, non richiedendo ulteriori I/O. Questo sfrutta la località spaziale.
3. Un nuovo trasferimento di I/O è necessario solo quando l'indice attraversa il confine di un blocco, ovvero, una volta ogni B elementi.

Pertanto, il costo totale di I/O di una scansione lineare è

$$\text{Scan}(N) = \left\lceil \frac{N}{B} \right\rceil = \Theta\left(\frac{N}{B}\right) \text{ operazioni di I/O.}$$

1.5.3 Accesso Lineare contro Accesso Sparso

- **Accesso sequenziale (alta località spaziale):** la lettura di N elementi richiede N/B operazioni di I/O. Poiché B varia tipicamente da 64 byte per una linea di cache a centinaia di kilobyte o megabyte per pagine e blocchi del disco, lo speedup rispetto al modello ingenuo può essere di un fattore approssimativamente pari a B.
- **Accesso casuale o passo $k > B$ (nessuna località spaziale):** ogni elemento richiesto potrebbe risiedere in un blocco diverso, causando N operazioni di I/O indipendenti. Il costo degrada a $\Theta(N)$ I/O, il che è asintoticamente più lento di un fattore B.

Capitolo 2

Accesso a Passi, B-Tree e Impatto della Paginazione Virtuale

2.1 Accesso Non Sequenziale: L'Algoritmo $A_{s,b}$

Sia S un array contenente n elementi. Il sistema di archiviazione trasferisce i dati in blocchi fisici di dimensione B (un blocco del disco o una linea di cache). Definiamo inoltre una dimensione del blocco logico b, dove $b \leq B$ e b divide B. I corrispondenti numeri di blocchi logici e fisici sono

$$N' = \frac{n}{b}, \qquad N_{\text{phys}} = \frac{n}{B}.$$

L'algoritmo $A_{s,b}$ calcola la somma degli elementi visitando i blocchi logici con un pattern a passi (stride). Il blocco logico visitato al passo j è

$$\text{block}(j) = (js) \bmod N', \qquad j = 0, 1, \ldots, N' - 1.$$

2.1.1 Condizione di Coprimalità

La sequenza visita ogni blocco logico esattamente una volta se e solo se

$$\gcd\left(s, \frac{n}{b}\right) = 1.$$

In tal caso, la moltiplicazione per s è una permutazione dei residui modulo N'. Se il massimo comun divisore è $d > 1$, la sequenza si ripete dopo $N'/d = n/(bd)$ blocchi logici distinti e di conseguenza copre solo una frazione dell'array.

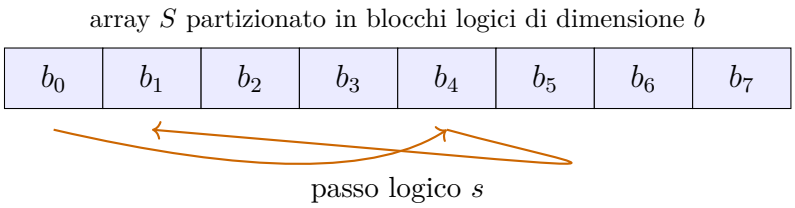


Figura 2.1: Un passo modulare visita i blocchi logici in un ordine non sequenziale.

La Figura 2.1 illustra l'ordine logico degli accessi. Il costo fisico dipende da come questo ordine si mappa su blocchi di dimensione B, piuttosto che unicamente dal numero di operazioni aritmetiche.

2.1.2 Costo di I/O

Una prima utile approssimazione confronta la distanza fisica tra accessi logici successivi, sb, con la dimensione del blocco fisico B.

Tabella 2.1: Comportamento approssimato di I/O del pattern di accesso a passi.

Condizione	Località e comportamento	Costo di I/O
$sb < B$	Diversi accessi logici possono condividere un singolo blocco fisico; una parziale località spaziale è preservata.	$\Theta(n/B)$
$sb \geq B$	Gli accessi successivi tipicamente ricadono su blocchi fisici differenti. Se $M < n$, il working set non può rimanere interamente nella memoria veloce.	$\Theta(n/b)$
$b = 1$ e $s \geq B$	Ogni accesso logico è un singolo elemento e il passo distrugge il beneficio dei trasferimenti a blocchi.	$\Theta(n)$

I limiti nella Tabella 2.1 descrivono il tipico regime del caso pessimo o dei capacity miss. Le costanti esatte dipendono dall'allineamento, dalla politica

di rimpiazzo e dall'interazione tra la sequenza modulare e i blocchi fisici. Nell'ultimo caso, il costo è peggiore di un fattore pari a circa B rispetto ad una scansione lineare, che può sfruttare ogni elemento trasferito.

2.2 Limiti Algoritmici e Accelerazione Hardware

Fissiamo un tempo di esecuzione massimo ammissibile T_0 e assumiamo che l'hardware o il parallelismo accelerino il calcolo di un fattore K. La macchina accelerata può pertanto permettersi un tempo di esecuzione originale pari a

$$\frac{T(n)}{K} = T_0 \qquad \Longleftrightarrow \qquad T(n) = KT_0.$$

L'effetto sulla dimensione massima dell'input risolvibile dipende dal tasso di crescita di T.

Tabella 2.2: Effetto di un'accelerazione hardware di un fattore K sulla dimensione dell'input risolvibile.

Algoritmo	Complessità	Hardware di base	Hardware accelerato	Guadagno effettivo
A_1	cn	$n_1 = T_0/c$	$n_1' = Kn_1$	Moltiplicativo, K
A_2	cn^2	$n_2 = \sqrt{T_0/c}$	$n_2' = \sqrt{K}\, n_2$	Sublineare, $\sqrt{K}$
A_3	$c2^n$	$n_3 = \log_2(T_0/c)$	$n_3' = n_3 + \log_2 K$	Additivo, $+\log_2 K$

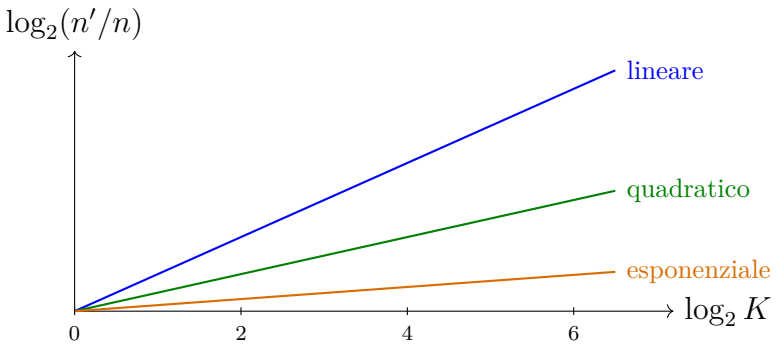


Figura 2.2: Beneficio relativo dell'accelerazione hardware per tassi di crescita rappresentativi.

La Figura 2.2 mostra perché i miglioramenti hardware non rimuovono un algoritmo asintoticamente inefficiente. Per un algoritmo esponenziale, persino uno speedup di 1000 volte aumenta la dimensione dell'input risolvibile di soli $\log_2(1000) \approx 10$ elementi.

2.3 Ricerca in Insiemi Ordinati: Ricerca Binaria contro B-Tree

La ricerca puntuale di chiavi all'interno di insiemi ordinati di grandi dimensioni evidenzia la profonda divergenza tra complessità computazionale in memoria centrale e traffico di I/O verso la memoria secondaria. In questa sezione confrontiamo l'efficienza della ricerca binaria tradizionale con la struttura a blocchi e ad alto fan-out dei B-Tree.

2.3.1 Ricerca Binaria in Memoria Secondaria

Per una sequenza ordinata di n chiavi memorizzate in blocchi di dimensione B, la ricerca binaria esegue $\Theta(\log_2 n)$ confronti sulla CPU. Il suo costo di I/O è maggiore di quanto il numero di confronti suggerisca: durante le prime iterazioni, le posizioni esaminate sono separate da $n/2$, $n/4$ e così via. Finché l'intervallo di ricerca non si restringe ad al più B elementi, ogni sondaggio può indirizzare un blocco fisico differente. Pertanto,

$$\mathrm{I/O}_{\text{ricerca binaria}} \approx \log_2 n - \log_2 B = \log_2\left(\frac{n}{B}\right).$$

L'approssimazione ignora l'allineamento e la possibilità che un blocco rimanga nella cache, ma cattura il comportamento dominante in memoria esterna.

2.3.2 B-Tree e B$^+$-Tree

Un B-Tree sceglie la dimensione del nodo in modo che corrisponda ad un blocco fisico. Ogni nodo memorizza $\Theta(B)$ chiavi e puntatori ai figli, pertanto il suo fan-out è $\Theta(B)$. Un B$^+$-Tree usa lo stesso principio e normalmente memorizza i record o i puntatori ai record nelle sue foglie, rendendolo particolarmente adatto per le query ad intervallo.

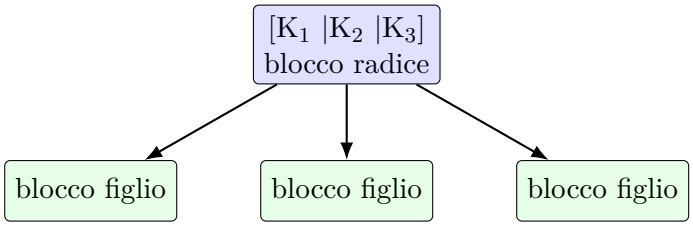


Figura 2.3: Un B-Tree orientato ai blocchi con alto fan-out.

La Figura 2.3 mostra la decisione fondamentale di layout: un I/O carica un intero nodo e fornisce molte chiavi di instradamento e puntatori. L'altezza, e di conseguenza il numero di letture di blocchi per una point query, è approssimativamente

$$h = \log_B\left(\frac{n}{B}\right) = \frac{\log_2(n/B)}{\log_2 B}.$$

Più precisamente, i dettagli implementativi introducono arrotondamenti per eccesso e fattori di occupazione. Lo spazio dei nodi interni è piccolo in confronto a quello delle foglie poiché

$$\text{nodi interni} \le \frac{n}{B^2} + \frac{n}{B^3} + \cdots = \frac{n}{B(B-1)} \ll \frac{n}{B}.$$

2.3.3 Confronto Quantitativo

Ignorando gli arrotondamenti per eccesso, lo speedup di I/O di un B-Tree rispetto alla ricerca binaria è

$$\frac{\mathrm{I/O}_{\text{ricerca binaria}}}{\mathrm{I/O}_{\text{B-Tree}}} = \frac{\log_2(n/B)}{\log_2(n/B)/\log_2 B} = \log_2 B.$$

Tabella 2.3: Confronto illustrativo per un grande dataset ordinato.

Parametro	Valore
Dimensione del dataset	$n = 2^{40}$ elementi, approssimativamente 10^{12} record
Dimensione del blocco	$B = 2^{14} = 16\,384$ record
I/O della ricerca binaria	$40 - 14 = 26$ accessi ai blocchi
I/O del B-Tree	$(40 - 14)/14 \approx 1.85$, quindi circa 2 accessi ai blocchi

Per una latenza di archiviazione su scala dei millisecondi, la differenza tra approssimativamente 26 e 2 accessi ai blocchi cambia una point query da centinaia di millisecondi a soli pochi millisecondi, subordinatamente alla cache e ai dettagli hardware.

2.4 Algoritmi I/O-Aware e Paginazione Virtuale

Ci sono due modi complementari per gestire i trasferimenti tra memoria veloce e lenta:

- **Progettazione I/O-aware (esplicita):** L'algoritmo modella B e la capacità della memoria veloce M in modo diretto, e gestisce buffer, layout dei dati e trasferimenti in modo programmatico.
- **Memoria virtuale e paginazione (implicita):** Il sistema operativo e l'hardware muovono le pagine in modo trasparente utilizzando tabelle delle pagine e politiche di rimpiazzo come le approssimazioni LRU.

2.4.1 Paginazione con un Working Set Sovradimensionato

Assumiamo che la memoria fisica abbia capacità M e sia divisa in M/B frame. Se l'applicazione richiede

$$n = (1 + \varepsilon)M = M + \varepsilon M, \qquad \varepsilon > 0,$$

allora almeno εM unità del working set devono risiedere nella memoria secondaria. Sia α la frazione di istruzioni che eseguono operazioni di memoria. Un tipico profilo di istruzione è:

- frazione $1 - \alpha$: operazioni aritmetiche o logiche, con costo di circa un ciclo di CPU;
- frazione α: operazioni `LOAD` o `STORE`;
- probabilità $1 - P(\varepsilon)$: i dati referenziati sono residenti in RAM, con tempo di accesso t_m;
- probabilità $P(\varepsilon)$: l'accesso causa un page fault, con tempo di servizio t_{disco}.

Il tempo medio risultante per istruzione è

$$T_{\text{medio}} = (1 - \alpha) \cdot 1 + \alpha \left[(1 - P(\varepsilon)) t_m + P(\varepsilon) t_{\text{disco}} \right].$$

Poiché $t_m \ll t_{\text{disco}}$, il termine del page fault può dominare:

$$T_{\text{medio}} \approx \alpha P(\varepsilon) t_{\text{disco}}.$$

Questa approssimazione è un avvertimento sulla sensibilità, non un'identità: quando $P(\varepsilon)$ è zero, i termini di base della CPU e della RAM hanno ancora rilevanza.

2.4.2 Esempio Numerico di Thrashing

Prendiamo $\alpha = 0.3$, una penalità per page fault di $t_{\text{disco}} = 10^5$ cicli di clock, e una probabilità di fault di soli $P(\varepsilon) = 10^{-3}$. Allora la penalità attesa è

$$T_{\text{penalità}} \approx 0.3 \cdot 10^{-3} \cdot 10^5 = 30 \text{ cicli per istruzione.}$$

Un page fault ogni mille accessi in memoria può pertanto aggiungere in media circa 30 cicli ad ogni istruzione. Rispetto ad una baseline di un ciclo, questo può produrre un rallentamento di un ordine di grandezza, potenzialmente nel range di 10×–30× a seconda della latenza RAM ordinaria e dell'effettivo comportamento del working set. Quando il tasso di fault aumenta ulteriormente, il processo entra in *thrashing*: il sistema spende la maggior parte del suo tempo a servire page fault invece di progredire con computazione utile.

2.5 Punti Chiave

1. Un passo modulare visita tutti i blocchi logici solo quando il passo e il numero di blocchi logici sono coprimi.

2. Layout e pattern di accesso block-aware possono cambiare la complessità di I/O di un fattore B, anche quando il lavoro della CPU rimane inalterato.

3. I B-Tree riducono la profondità di ricerca in memoria esterna facendo corrispondere il loro fan-out alla dimensione del blocco fisico.

4. L'accelerazione hardware migliora gli algoritmi lineari e polinomiali in modi differenti, ma fornisce solo un miglioramento additivo agli algoritmi esponenziali.

5. La memoria virtuale è conveniente, ma un working set sovradimensionato può trasformare una piccola probabilità di page fault in un costo prestazionale dominante.

Capitolo 3

Permutazione, Ordinamento e Binary Merge Sort in Memoria Esterna

3.1 Il Problema della Permutazione vs. Ordinamento

Il problema della permutazione arbitraria di un insieme di elementi costituisce uno dei banchi di prova più significativi per comprendere l'impatto della gerarchia di memoria. In questa sezione formalizziamo la nozione matematica di permutazione e confrontiamo le strategie di riarrangiamento diretto rispetto ai vincoli posti dalla memoria secondaria.

3.1.1 Formulazione del Problema

Siano dati due vettori di dimensione n:

- Un array di dati $S[1 \dots n]$.
- Un vettore di permutazione $\pi[1 \dots n]$, contenente una permutazione biiettiva degli indici $\{1, 2, \dots, n\}$.

L'obiettivo consiste nel riordinare gli elementi di S in un nuovo array S' in modo tale che ogni elemento originario $S[i]$ sia ricollocato nella posizione target indicata da $\pi[i]$, ovvero:

$$S'[\pi[i]] = S[i], \qquad \forall i \in \{1, \dots, n\}. \tag{3.1}$$

Esempio 3.1.1 (Permutazione di indici)**.** Consideriamo un array di 4 elementi e il relativo vettore di permutazione:

$$\begin{aligned} \text{Indici:} \quad & 1 \quad 2 \quad 3 \quad 4 \\ S = \quad & [\,A, \quad B, \quad C, \quad D\,] \\ \pi = \quad & [\,3, \quad 1, \quad 2, \quad 4\,] \end{aligned}$$

Applicando la definizione elemento per elemento:

$$\begin{aligned} S'[\pi[1]] &= S'[3] = A, \\ S'[\pi[2]] &= S'[1] = B, \\ S'[\pi[3]] &= S'[2] = C, \\ S'[\pi[4]] &= S'[4] = D. \end{aligned}$$

L'array permutato finale risulta pertanto:

$$S' = [\,B, \quad C, \quad A, \quad D\,].$$

In Figura 3.1 viene illustrata visivamente la mappatura degli elementi dall'array originale al vettore permutato.

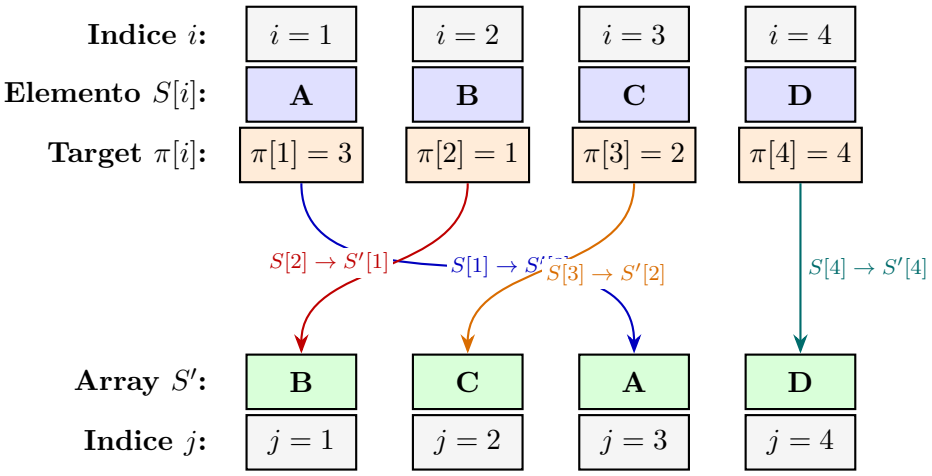


Figura 3.1: Mappatura della permutazione: ogni elemento $S[i]$ viene ricollocato nella posizione target specificata da $\pi[i]$ nell'array S'.

3.1.2 Implementazione Ingenua su Modello RAM

Nel modello RAM classico, in cui ogni accesso in lettura o scrittura a qualsiasi locazione di memoria ha costo unitario $\mathcal{O}(1)$, l'algoritmo si implementa tramite una singola scansione lineare:

Algorithm 1: Permutazione Diretta su RAM

Dati: Array $S[1 \dots n]$, vettore di permutazione $\pi[1 \dots n]$
Risultato: Array S permutato in loco

```
1 for i ← 1 to n do
2     S'[π[i]] ← S[i]
3 S ← copy(S')
```

La complessità temporale nel modello RAM è strettamente lineare:

$$T_{\text{RAM}}(n) = \Theta(n).$$

In memoria centrale, pertanto, permutare è asintoticamente più efficiente rispetto all'ordinamento basato su confronti, il quale richiede notoriamente un lower bound di $\Omega(n \log n)$ operazioni elementari.

3.1.3 Principio di Simulazione (RAM contro Due Livelli)

Teorema 3.1.1 (Teorema di Simulazione)**.** *Qualsiasi algoritmo sequenziale formulato per il modello RAM con complessità temporale $T(n)$ richiede al più $\mathcal{O}(T(n))$ trasferimenti di blocchi (I/O) nel modello di memoria a due livelli con parametri (M, B).*

Dimostrazione (Idea). Nel caso peggiore, ogni istruzione elementare RAM che effettua un accesso in memoria (lettura o scrittura) indirizza una locazione appartenente a un blocco non presente in memoria veloce M. Ciascun accesso scatena pertanto un trasferimento di blocco dal disco, producendo esattamente 1 operazione di I/O. Moltiplicando il numero complessivo di passi $T(n)$ per il costo massimo di 1 I/O per passo, si ottiene il bound superiore immediato di $\mathcal{O}(T(n))$ I/O. □

Sebbene il principio di simulazione garantisca che un algoritmo RAM possa sempre essere eseguito in memoria esterna, il bound $\mathcal{O}(T(n))$ è spesso inefficiente e lontano dall'ottimalità, poiché ignora completamente il fattore di aggregazione B offerto dai trasferimenti a blocchi.

3.1.4 La Relazione tra Ordinamento e Permutazione

Nel modello a due livelli la gerarchia di complessità si ribalta, come sintetizzato nel prospetto comparativo in Tabella 3.1:

Problema	Modello RAM	Memoria a Due Livelli (I/O)
Permutazione $\mathrm{Perm}(n)$	$\Theta(n)$	$\Theta\Big(\min\big\{n, \mathrm{Sort}(n, M, B)\big\}\Big)$
Ordinamento $\mathrm{Sort}(n)$	$\Theta(n \log_2 n)$	$\Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$

Tabella 3.1: Confronto di complessità asintotica tra RAM e Memoria Esterna.

Per comprendere questa equivalenza nei sistemi a blocchi, consideriamo le due riduzioni:

1. **Ordinamento tramite Permutazione:** Dato un array S, determinare la sequenza ordinata richiede dapprima di individuare la permutazione ordinante π_S (calcolabile tramite ordinamento delle chiavi) e successivamente di applicare π_S all'array S.

2. **Permutazione tramite Ordinamento (Algoritmo a Riduzione di Coppie):** La permutazione di un array S può essere trasformata in un problema di ordinamento di record secondo il seguente procedimento a tre stadi:

   - **Passo 1 (Scansione e Creazione Coppie):** Si effettuano scansioni parallele e sequenziali di S e π per costruire una collezione di n record composti dalla coppia $\langle S[i], \pi[i] \rangle$. Poiché si tratta di una scansione sequenziale lineare, il costo è:

   $$\mathrm{I/O}_{\text{Passo 1}} = \mathcal{O}\left(\frac{n}{B}\right).$$

   - **Passo 2 (Ordinamento per Destinazione):** Si ordinano le n coppie generate assumendo come chiave primaria la seconda componente $\pi[i]$. Il costo coincide esattamente con l'ordinamento in memoria esterna:

   $$\mathrm{I/O}_{\text{Passo 2}} = \mathrm{Sort}(n, M, B).$$

   - **Passo 3 (Scansione ed Estrazione):** Avendo ordinato le coppie per indice di destinazione crescente, è sufficiente una scansione lineare per estrarre la prima componente $S[i]$, riversandola nel vettore finale S'. Costo:

   $$\mathrm{I/O}_{\text{Passo 3}} = \mathcal{O}\left(\frac{n}{B}\right).$$

Il costo I/O aggregato di questa strategia è:

$$\mathrm{I/O}_{\text{Sort-based}} = \mathcal{O}\left(\frac{n}{B}\right) + \mathrm{Sort}(n, M, B) + \mathcal{O}\left(\frac{n}{B}\right) = \Theta(\mathrm{Sort}(n, M, B)). \quad (3.2)$$

Esempio 3.1.2 (Tracciamento della Riduzione a Coppie con Array Reali)**.** Per visualizzare concretamente la riduzione, consideriamo i dati dell'Esempio 3.1.1:

$$S = [A, B, C, D], \qquad \pi = [3, 1, 2, 4].$$

L'algoritmo opera attraverso tre stati successivi dell'array di record:

- **Passo 1 (Accoppiamento):** Si accoppia sequenzialmente ciascun elemento con la propria coordinata target:

$$P = \Big[\, \langle A, 3\rangle,\ \ \langle B, 1\rangle,\ \ \langle C, 2\rangle,\ \ \langle D, 4\rangle \,\Big].$$

- **Passo 2 (Sort esterno su $\pi[i]$):** L'array viene ordinato assumendo come chiave di comparazione il secondo valore della tupla (la destinazione finale):

$$P_{\text{ordinato}} = \Big[\, \langle B, \mathbf{1}\rangle,\ \ \langle C, \mathbf{2}\rangle,\ \ \langle A, \mathbf{3}\rangle,\ \ \langle D, \mathbf{4}\rangle \,\Big].$$

- **Passo 3 (Estrazione sequenziale):** Proiettando la prima coordinata di ciascun elemento ordinato, si ricompone il vettore finale:

$$S' = [B, C, A, D],$$

  che realizza esattamente la permutazione originaria poiché $S'[1] = B$, $S'[2] = C$, $S'[3] = A$, $S'[4] = D$.

La pipeline di riduzione delle coppie è illustrata graficamente in Figura 3.2.

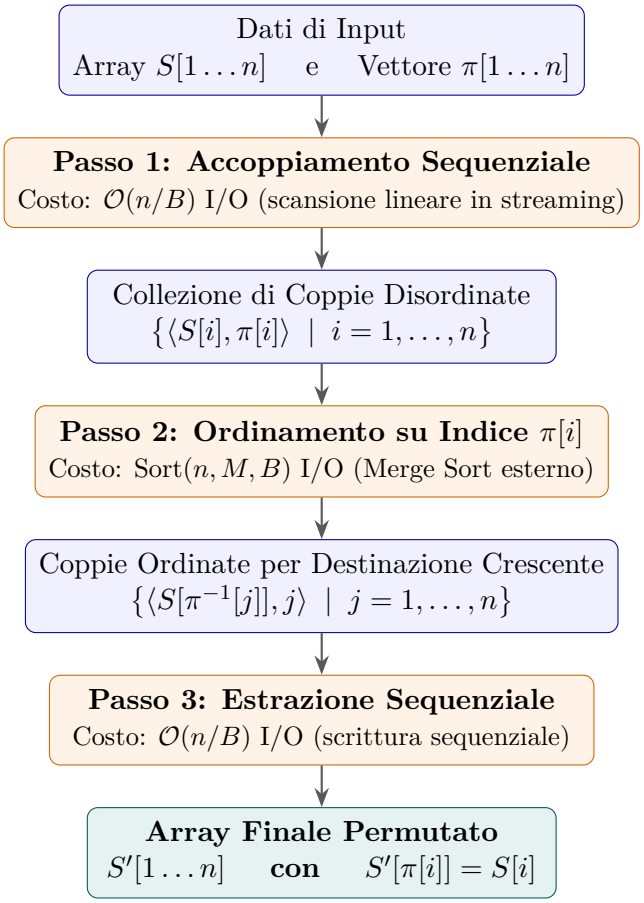


Figura 3.2: Flusso operativo dell'algoritmo di permutazione basato sulla riduzione a coppie.

3.2 Complessità di Permutazione in Memoria Esterna

Il lower bound fondamentale per il problema della permutazione in memoria a due livelli è stato stabilito da Aggarwal e Vitter [2]:

$$\mathrm{Perm}(n, M, B) = \Theta\left(\min\left\{n,\ \frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right\}\right). \tag{3.3}$$

Definendo $L = \log_{M/B}(n/B)$ come il numero di passate richieste dalla fusione multi-via nell'ordinamento:

$$\mathrm{Sort}(n, M, B) = \Theta\left(\frac{n}{B} \cdot L\right). \tag{3.4}$$

3.2.1 Analisi dei Casi Limite

Il confronto tra l'algoritmo ingenuo diretto e la strategia basata sull'ordinamento delle coppie dipende dal valore relativo tra la dimensione del blocco B e il numero di livelli L:

- **Caso 1 ($n \geq \frac{n}{B} \cdot L \iff B \geq L$):** La dimensione del blocco B è maggiore o uguale alla profondità dell'albero di fusione. In questa situazione, ordinare le coppie richiede meno I/O rispetto all'accesso casuale elemento per elemento:

$$\text{Perm}(n, M, B) = \Theta(\text{Sort}(n, M, B)).$$

- **Caso 2 ($n < \frac{n}{B} \cdot L \iff B < L$):** Se la dimensione del blocco fosse patologicamente piccola rispetto al volume complessivo dei dati ($B < L$), eseguire n accessi diretti (al più n I/O) risulterebbe asintoticamente inferiore al costo dell'ordinamento:

$$\text{Perm}(n, M, B) = \Theta(n).$$

3.2.2 Verifica Numerica su Parametri Architetturali Reali

Per comprendere quale dei due regimi asintotici si manifesti nei sistemi reali, consideriamo parametri architetturali realistici ed estremi:

- Memoria interna veloce: $M = 16$ GB $= 2^4 \cdot 2^{30}$ elementi $= 2^{34}$ elementi.
- Dimensione del blocco disco: $B = 32$ KB $= 2^5 \cdot 2^{10}$ elementi $= 2^{15}$ elementi.
- Dimensione del dataset di input: $n = 2^{60}$ elementi (scala Exabyte).

Calcoliamo il numero di passate L:

$$L = \log_{M/B}\left(\frac{n}{M}\right) = \frac{\log_2(n/M)}{\log_2(M/B)}.$$

Sostituendo i valori esponenziali:

$$\frac{n}{M} = \frac{2^{60}}{2^{34}} = 2^{26} \implies \log_2\left(\frac{n}{M}\right) = 26,$$
$$\frac{M}{B} = \frac{2^{34}}{2^{15}} = 2^{19} \implies \log_2\left(\frac{M}{B}\right) = 19.$$

Otteniamo:

$$L = \frac{26}{19} \approx 1{,}36 < 2.$$

Poiché $B = 2^{15} = 32\,768$ e $L \leq 2$, la condizione $B \geq L$ è verificata con un margine di oltre quattro ordini di grandezza ($32\,768 \gg 2$). Di conseguenza, in qualsiasi sistema reale la permutazione è strettamente governata dal costo di ordinamento:

$$\mathrm{Perm}(n) = \Theta(\mathrm{Sort}(n)).$$

3.3 Binary Merge Sort in Memoria Esterna

L'algoritmo ricorsivo classico di Merge Sort si estende alla memoria esterna suddividendo l'array in due metà, ordinando ricorsivamente ciascuna metà e fondendo i due sottoarray ordinati (*runs*):

Algorithm 2: Binary Merge Sort

Dati: Array S, estremi i, j
Risultato: Sottoarray $S[i \dots j]$ ordinato

```
1 if i < j then
2     m ← ⌊(i + j)/2⌋
3     MS(S, i, m)
4     MS(S, m + 1, j)
5     MERGE(S, i, m, j)
```

3.3.1 Architettura dei Buffer nella Procedura di Fusione

Dati due segmenti ordinati (*runs*) consecutivi di lunghezza l, la procedura di fusione genera un unico run ordinato di lunghezza $2l$ su un array temporaneo S'.

- **Nel Modello RAM:** Si effettuano al più $2l$ confronti e $2l$ trasferimenti in memoria, determinando un costo computazionale di $\Theta(l)$.
- **Nel Modello a Due Livelli:** Per eseguire la fusione minimizzando gli I/O, la memoria interna M viene partizionata in 3 buffer dedicati, ciascuno di dimensione esattamente pari a un blocco B:
    1. **Input Buffer 1:** ospita un blocco (dimensione B) del primo run ordinato.

2. **Input Buffer 2:** ospita un blocco (dimensione B) del secondo run ordinato.
3. **Output Buffer:** raccoglie gli elementi fusi in ordine crescente fino al riempimento di un blocco B.

In Figura 3.3 è rappresentata la gestione della memoria interna e dei flussi da/verso il disco durante la fusione binaria.

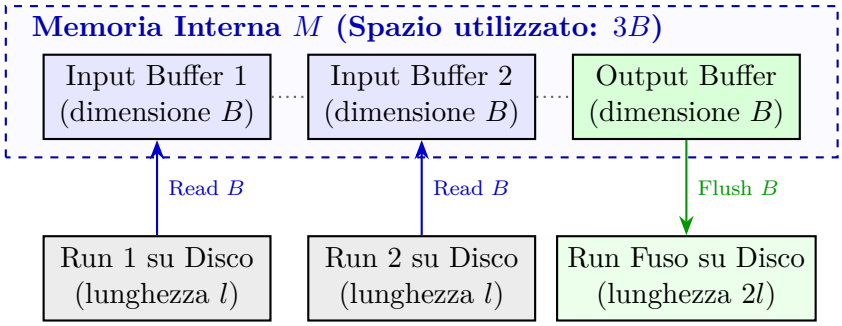


Figura 3.3: Architettura dei buffer di memoria interna durante la procedura di fusione binaria.

Il funzionamento del protocollo di trasferimento è il seguente:

1. Gli elementi in testa ai due buffer di input vengono confrontati in CPU; l'elemento minore viene spostato nell'output buffer.
2. Quando uno dei buffer di input si svuota, viene emessa una lettura da disco per caricare il blocco successivo del rispettivo run ($\frac{l}{B}$ I/O per ciascun run di input).
3. Non appena l'output buffer si riempie con B elementi, viene scaricato (*flushed*) su disco con una singola scrittura ($\frac{2l}{B}$ I/O complessivi di scrittura).

Il costo aggregato della fusione di due run di lunghezza l è:

$$\mathrm{I/O}_{\mathrm{Merge}}(l) = \frac{l}{B} + \frac{l}{B} + \frac{2l}{B} = \frac{4l}{B} = \Theta\left(\frac{l}{B}\right). \tag{3.5}$$

3.3.2 Complessità dell'Albero di Ricorsione

La ricorsione si arresta non appena la dimensione del sottoarray scende al di sotto della capacità della memoria interna M. A questo livello base, ciascun

sottoarray di dimensione M viene letto interamente in memoria, ordinato localmente tramite un algoritmo interno efficiente (es. QuickSort o HeapSort) con zero I/O aggiuntivi, e riscritto su disco con $\mathcal{O}(M/B)$ I/O.

Si generano così inizialmente:

$$N_{\text{run}} = \frac{n}{M} \text{ run ordinati di dimensione } M.$$

L'albero di fusione binaria presenta un'altezza complessiva pari a:

$$h = \log_2\left(\frac{n}{M}\right) \text{ livelli.} \tag{3.6}$$

In Figura 3.4 è illustrata la struttura dell'albero di fusione per il Binary Merge Sort esterno.

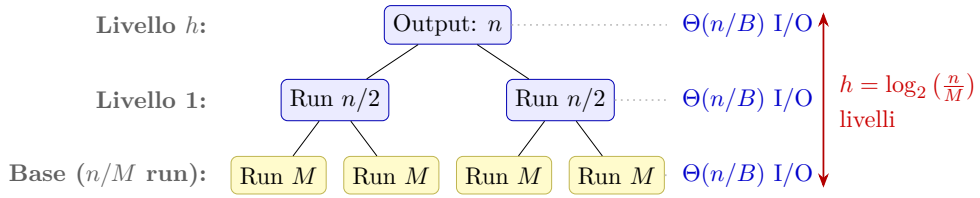


Figura 3.4: Albero di ricorsione del Binary Merge Sort esterno con indicazione dei costi I/O per livello.

Ad ogni livello dell'albero di fusione, tutti gli n elementi dell'array vengono scanditi esattamente una volta, richiedendo $\Theta(n/B)$ operazioni di I/O per livello. Moltiplicando per l'altezza dell'albero si ottiene la complessità totale:

$$\text{I/O}_{\text{BinaryMergeSort}} = \mathcal{O}\left(\frac{n}{B}\log_2\left(\frac{n}{M}\right)\right). \tag{3.7}$$

3.3.3 Criticità e Limiti di Efficienza

Nonostante l'algoritmo migliori drasticamente rispetto a un approccio ingenuo, esso presenta un grave collo di bottiglia strutturale:

- **Sottoutilizzo drastico della memoria interna:** Durante la fusione binaria vengono occupati contemporaneamente soltanto 3 blocchi ($3B$ elementi) all'interno dello spazio disponibile M. Tutta la restante porzione di memoria interna ($M - 3B$) rimane completamente inutilizzata.

- **Base del logaritmo sub-ottimale:** La base del logaritmo è rigidamente ancorata a 2, mentre la memoria M permetterebbe di mantenere aperti simultaneamente fino a:
$$k \approx \frac{M}{B} - 1 \text{ buffer di input.}$$
- **Perdita del fattore di convergenza** $\log_2(M/B)$**:** La mancata estensione a una fusione a k vie (*Multiway Merge Sort*) comporta un numero di passate significativamente superiore, impedendo di raggiungere il lower bound ottimale:
$$\mathcal{O}\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right).$$

In Figura 3.5 viene evidenziata graficamente la disparità tra l'allocazione binaria e la piena saturazione della memoria veloce.

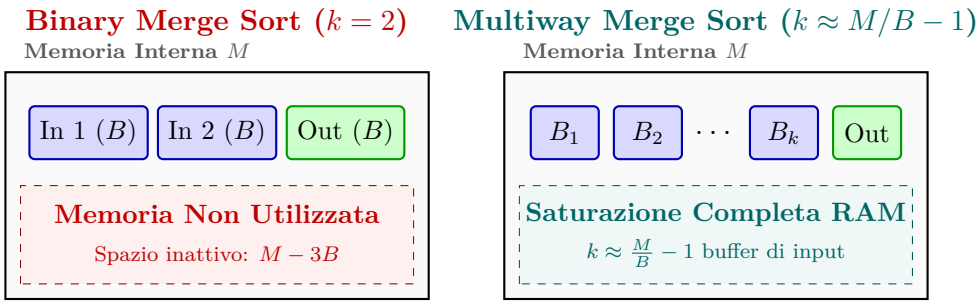


Figura 3.5: Confronto dell'efficienza nell'allocazione della memoria interna: grave sottoutilizzo nel Binary Merge Sort rispetto alla saturazione ottimale nel Multiway Merge Sort.

Il superamento di questa limitazione strutturale conduce in modo naturale all'architettura dell'ordinamento multi-via, analizzata in dettaglio nelle sezioni seguenti.

3.4 Equivalenza Asintotica del Costo di Ordinamento Esterno

Nella trattazione della complessità computazionale in memoria secondaria, il costo I/O dell'ordinamento esterno viene espresso nella letteratura specializzata [2] attraverso due notazioni apparentemente distinte:
$$\Theta\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{M}\right)\right) \qquad \text{oppure} \qquad \mathcal{O}\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right). \tag{3.8}$$

Queste due formulazioni sono asintoticamente perfettamente equivalenti per qualsiasi istanza $n \geq M$.

Per dimostrare formalmente l'equivalenza, applichiamo le proprietà fondamentali dei logaritmi espandendo l'argomento del termine $\log_{M/B}(n/B)$:

$$\begin{aligned} \log_{M/B}\left(\frac{n}{B}\right) &= \log_{M/B}\left(\frac{n}{M} \cdot \frac{M}{B}\right) \\ &= \log_{M/B}\left(\frac{n}{M}\right) + \log_{M/B}\left(\frac{M}{B}\right) \\ &= \log_{M/B}\left(\frac{n}{M}\right) + 1. \end{aligned} \tag{3.9}$$

Moltiplicando l'identità per il fattore comune $\frac{n}{B}$, otteniamo:

$$\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right) = \frac{n}{B}\log_{M/B}\left(\frac{n}{M}\right) + \frac{n}{B}. \tag{3.10}$$

Dal punto di vista algoritmico, il termine additivo lineare $\frac{n}{B}$ corrisponde esattamente alla passata iniziale (*Passo 0*) necessaria alla formazione dei *run* ordinati di partenza tramite caricamento e ordinamento in RAM.

Allorché $n \gg M$, si ha $\log_{M/B}(n/M) \geq 1$, per cui il termine additivo $+1$ viene pienamente assorbito dalla costante nascosta nella notazione asintotica Θ:

$$\Theta\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right) = \Theta\left(\frac{n}{B}\left(\log_{M/B}\left(\frac{n}{M}\right) + 1\right)\right) = \Theta\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{M}\right)\right). \tag{3.11}$$

Ne consegue che è possibile utilizzare indifferentemente $\frac{n}{M}$ oppure $\frac{n}{B}$ all'interno dell'argomento logaritmico senza alterare la rigorosità dell'analisi.

Capitolo 4

Multiway Merge Sort, Snow Plow e Disk Striping

4.1 Multiway Merge Sort (k-Way Merge Sort)

Nel *Binary Merge Sort*, descritto nella sezione 3.3, si allocano simultaneamente soltanto 3 blocchi di memoria interna, lasciando inutilizzato lo spazio residuo $M - 3B$. Il **Multiway Merge Sort** (o ordinamento per fusione a k vie) supera radicalmente questa inefficienza sfruttando l'intera capacità M per fondere simultaneamente k sequenze ordinate in un'unica passata (*fan-in* pari a k).

4.1.1 Configurazione e Architettura della Memoria Interna

La memoria primaria M viene partizionata in unità di trasferimento di dimensione B:

- $k = \frac{M}{B} - 1$ **buffer di input**, ciascuno di capacità esattamente pari a un blocco B, associati stabilmente a ciascuno dei k run attivi su memoria secondaria.
- 1 **buffer di output**, anch'esso di capacità B, deputato ad accumulare progressivamente gli elementi risultanti in ordine crescente prima del trasferimento fisico su disco.
- Una struttura dati ausiliaria residente interamente in memoria interna: un **Min-Heap** (o coda di priorità) contenente k nodi. Ciascun nodo memorizza la chiave corrente minima di uno dei k blocchi attivi insieme all'identificatore del buffer di provenienza.

In Figura 4.1 viene illustrato lo schema architettonico dei buffer e il flusso dei dati coordinato dal Min-Heap.

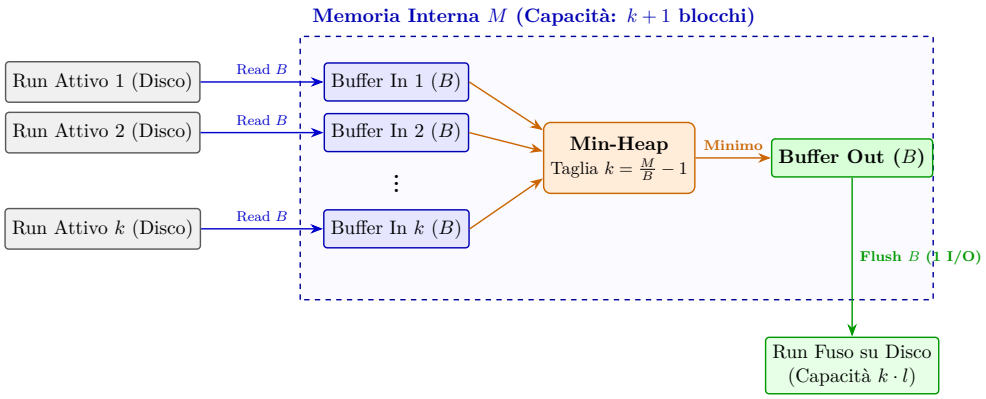


Figura 4.1: Architettura della memoria interna durante la fusione multi-via (k-Way Merge Sort) con Min-Heap di coordinamento.

Il Min-Heap consente di estrarre l'elemento minimo globale tra i k flussi con costo computazionale in memoria interna pari a $\mathcal{O}(\log_2 k)$ confronti. Quando il buffer di input da cui proviene il minimo estratto si esaurisce, viene emessa una singola operazione di lettura I/O per caricare il successivo blocco da B elementi del corrispondente run. Analogamente, ogni qualvolta il buffer di output raccoglie B elementi ordinati, viene eseguito un *flush* fisico su disco tramite una singola scrittura I/O.

4.1.2 Dinamica dell'Algoritmo ed Evoluzione dei Livelli

L'algoritmo si articola in due macro-fasi strutturate:

1. **Fase 0 (Formazione dei Run Iniziali):** L'array non ordinato di n elementi residenti su disco viene scandito a porzioni contigue di dimensione massima gestibile in memoria veloce, pari a M. Ciascun blocco di M elementi viene caricato in RAM, ordinato localmente con un algoritmo in-memory efficiente (es. QuickSort o HeapSort) e riscritto ordinato su disco:

   - Numero di run iniziali generati:

$$N_0 = \frac{n}{M}. \tag{4.1}$$

- Costo I/O della Fase 0: lettura e scrittura sequenziale di tutti gli n elementi, pari a:

$$\mathrm{I/O}_{\mathrm{RunFormation}} = 2 \cdot \frac{n}{B} = \mathcal{O}\left(\frac{n}{B}\right). \tag{4.2}$$

2. **Fasi Successive di Fusione (k-Way Merge):** A ogni livello dell'albero di fusione, i run disponibili vengono raggruppati a gruppi di k. Ciascun gruppo di k run, ciascuno di lunghezza corrente l, viene fuso simultaneamente producendo un nuovo run ordinato di estensione $k \cdot l$:

   - Costo di I/O per fondere un singolo gruppo: $\mathcal{O}\left(\frac{k \cdot l}{B}\right)$.
   - Costo I/O complessivo per livello: poiché ogni elemento dell'array appartiene a esattamente un gruppo a ciascun livello, tutti gli n elementi vengono letti e riscritti esattamente una volta per ogni passata, con un consumo di:

$$\mathrm{I/O}_{\mathrm{Livello}} = 2 \cdot \frac{n}{B} = \mathcal{O}\left(\frac{n}{B}\right). \tag{4.3}$$

All'i-esimo livello dell'albero di fusione, ciascun run possiede una dimensione pari a $k^i \cdot M$. Il procedimento ricorsivo termina non appena rimane un unico run complessivo di dimensione n:

$$k^i \cdot M = n \implies k^i = \frac{n}{M} \implies i = \log_k\left(\frac{n}{M}\right) \text{ livelli.} \tag{4.4}$$

Sostituendo il massimo grado di fusione ammissibile $k \approx \frac{M}{B} - 1 = \Theta\left(\frac{M}{B}\right)$, otteniamo:

$$\text{Numero di Fasi di Merge} = \mathcal{O}\left(\log_{M/B}\left(\frac{n}{M}\right)\right). \tag{4.5}$$

In Figura 4.2 viene illustrata la gerarchia dei livelli di fusione a k vie, evidenziando il fan-in espanso e i costi di I/O sostenuti a ciascun livello dell'albero.

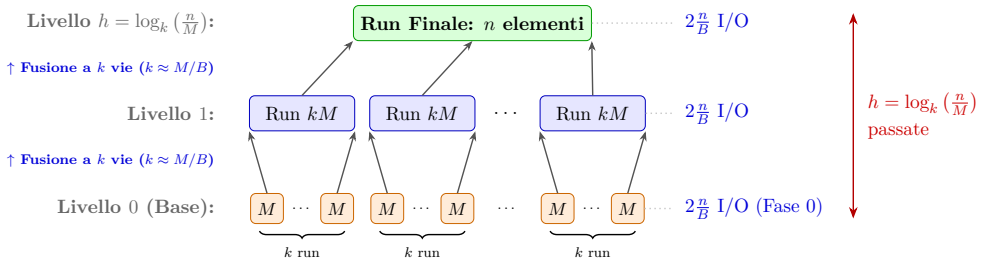


Figura 4.2: Evoluzione dei livelli e albero di fusione a k vie nel Multiway Merge Sort.

La complessità I/O complessiva dell'algoritmo risulta pertanto:

$$\begin{aligned} \mathrm{I/O}_{\text{Totale}} = & \underbrace{\mathcal{O}\left(\frac{n}{B}\right)}_{\text{Run Formation}} + \underbrace{\mathcal{O}\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{M}\right)\right)}_{\text{Fasi di Fusione}} \\ & = \mathcal{O}\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right) \text{ I/O.} \end{aligned} \tag{4.6}$$

Questo risultato eguaglia asintoticamente il lower bound teorico per l'ordinamento in memoria gerarchica derivato da Aggarwal e Vitter [2] (la cui derivazione formale mediante albero di decisione I/O viene sviluppata in dettaglio nel Capitolo 5, sezione 5.2), a conferma dell'ottimalità asintotica del Multiway Merge Sort.

4.2 Ottimizzazione dei Run Iniziali: Lo Spazzaneve (*Snow Plow*)

Sebbene il Multiway Merge Sort standard generi run iniziali di dimensione limitata alla capacità della memoria interna M, l'altezza dell'albero di fusione $\lceil \log_{M/B}(n/M) \rceil$ dipende in misura cruciale dal numero iniziale di sequenze prodotte. Produrre run di lunghezza maggiore di M consente di ridurre la profondità della ricorsione, eliminando intere passate di I/O su scala multi-terabyte.

Una tecnica classica ed elegante concepita a questo scopo è la **Replacement Selection**, nota storicamente con la suggestiva metafora dello **Spazzaneve (*Snow Plow*)** [1].

4.2.1 Meccanismo Operativo della Replacement Selection

Invece di riempire staticamente la memoria M, ordinarla e scaricarla per intero, l'algoritmo mantiene costantemente satura la memoria primaria M partizionandola dinamicamente in due insiemi logici:

- **Parte Attiva (Min-Heap corrente):** elementi che risultano maggiori o uguali all'ultimo elemento emesso verso il run in fase di scrittura. Tali elementi sono candidati legittimi ad arricchire il run corrente.
- **Parte Congelata o Disordinata (U):** elementi letti dal flusso di ingresso che risultano strettamente minori dell'ultimo elemento emesso.

Essi non possono essere accodati al run corrente senza violarne l'ordinamento e vengono dunque "congelati", rimanendo parcheggiati in RAM in attesa della formazione del run successivo.

In Figura 4.3 viene rappresentata la partizione dinamica dello spazio in memoria primaria durante l'avanzamento dello spazzaneve.

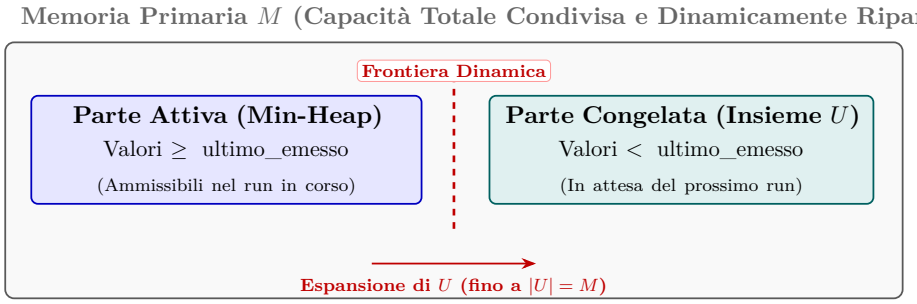


Figura 4.3: Ripartizione dinamica della memoria M durante la Replacement Selection (Spazzaneve).

Il ciclo operativo procede elemento per elemento:

1. Si estrae l'elemento minimo dalla parte attiva tramite la radice del Min-Heap, lo si invia al buffer di output e si registra il suo valore come ultimo_emesso.

2. Si legge un nuovo elemento x dal flusso di input in ingresso:

   - Se $x \geq$ ultimo_emesso, x viene inserito nella parte attiva dell'heap.
   - Se $x <$ ultimo_emesso, x viene inserito nella partizione dei congelati U.

3. Il run corrente termina quando la parte attiva si svuota completamente, ossia quando tutti gli M slot di memoria risultano occupati da elementi congelati ($|U| = M$). A questo punto, l'insieme U viene riconvertito istantaneamente nel nuovo Min-Heap attivo, si apre un nuovo run su disco e il processo riparte.

4.2.2 Esempio Passo-Passo dell'Algoritmo

Consideriamo una memoria rapida di capacità $M = 3$ elementi e una sequenza di input non ordinata:

$$S = [1,\ 7,\ 5,\ 3,\ 2,\ 9,\ 0,\ 4,\ \ldots].$$

In Tabella 4.1 viene tracciata dettagliatamente l'evoluzione dello stato interno della memoria e dei run emessi.

Tabella 4.1: Traccia dell'algoritmo Replacement Selection con capacità $M = 3$.

Passo	**Attivi**	**Congelati (U)**	**Azione Eseguita**	**Emesso**	**Run**
0	$\{1, 5, 7\}$	$\emptyset$	Caricamento di $M = 3$ elementi	—	[ ]
1	$\{5, 7\}$	$\emptyset$	Emette 1; legge 3. Poiché $3 \geq 1$, resta attivo	**1**	[1]
2	$\{5, 7\}$	$\emptyset$	Emette 3; legge 2. Poiché $2 < 3$, 2 **va in** U	**3**	$[1, 3]$
3	$\{7, 9\}$	$\{2\}$	Emette 5; legge 9. Poiché $9 \geq 5$, resta attivo	**5**	$[1, 3, 5]$
4	$\{9\}$	$\{2, 0\}$	Emette 7; legge 0. Poiché $0 < 7$, 0 **va in** U	**7**	$[1, 3, 5, 7]$
5	$\emptyset$	$\{2, 0, 4\}$	Emette 9; legge 4. Poiché $4 < 9$, 4 **va in** U	**9**	$[1, 3, 5, 7, 9]$

Al passo 5, la memoria attiva si azzera ed è interamente saturata dai congelati: $|U| = M = 3$. Il run corrente si chiude formalmente con una lunghezza pari a **5 elementi**, sensibilmente superiore alla capacità nominale della RAM ($M = 3$). L'insieme $\{0, 2, 4\}$ forma immediatamente l'heap di avvio del secondo run ordinato.

Nota sui Sistemi Reali. Nei sistemi operativi e DBMS reali, il caricamento e l'emissione non avvengono elemento per elemento ma a blocchi fisici di dimensione B, integrando opportuni buffer di prefetching in lettura e di accumulo in scrittura (*double buffering*), preservando intatta l'efficienza asintotica di I/O.

4.2.3 Teorema della Lunghezza Media dei Run

Teorema 4.2.1 (Lunghezza Media del Run nello Snow Plow)**.** *Assumendo che le chiavi in ingresso siano variabili aleatorie indipendenti e identicamente distribuite secondo una distribuzione continua uniforme, la lunghezza media dei run generati dall'algoritmo Snow Plow è asintoticamente pari a* $2M$*.*

Dimostrazione Euristica. Analizziamo il comportamento stocastico del sistema a regime:

1. All'avvio della formazione di un run, la memoria contiene M elementi ordinati.

2. Sia τ il numero di elementi addizionali letti dallo stream prima che il run corrente si concluda. Il numero complessivo di elementi appartenenti al run risulterà pari a $M + \tau$.

3. Il run si interrompe esattamente nel momento in cui la partizione dei congelati raggiunge la saturazione: $|U| = M$.

4. Per simmetria distribuzionale e indipendenza statistica, ogni nuovo elemento estratto ha probabilità pari a $P = \frac{1}{2}$ di posizionarsi a monte dell'ultimo valore emesso (venendo assegnato ai congelati U) e probabilità $P = \frac{1}{2}$ di risultare maggiore o uguale (entrando nella parte attiva del run corrente).

5. Il valore atteso del numero di elementi inviati a U dopo aver processato τ ingressi è dunque:

$$\mathbb{E}[|U|] = \tau \cdot P(\text{assegnato a } U) = \tau \cdot \frac{1}{2}. \tag{4.7}$$

6. Imponendo il vincolo di chiusura del run $\mathbb{E}[|U|] = M$, si ottiene:

$$\frac{\tau}{2} = M \implies \tau = 2M. \tag{4.8}$$

Pertanto, il numero medio atteso di elementi generati per ciascun run ammonta a circa $2M$. □

Raddoppiare la lunghezza media dei run ($\approx 2M$) dimezza il numero iniziale di run prodotti $\left(\frac{n}{2M}\right)$, riducendo concretamente il numero di passate globali sull'intero dataset.

4.3 Parallelismo su Più Dischi: *Disk Striping*

Nei sistemi di calcolo ad elevate prestazioni e nei moderni centri dati, lo storage secondario non è affidato a un unico supporto magnetico o a stato solido, ma a un'architettura parallela composta da D dischi fisici indipendenti.

4.3.1 Lower Bound di Ordinamento Parallelo

Quando un sistema dispone di D unità disco operanti in parallelo, ciascuna in grado di trasferire un blocco B per ciclo di I/O, il limite inferiore asintotico di Aggarwal e Vitter [2] per l'ordinamento scala con un fattore ideale D a denominatore:

$$\mathrm{Sort}_D(n, M, B) = \Omega\left(\frac{n}{D \cdot B} \log_{M/B}\left(\frac{n}{M}\right)\right) \text{ I/O paralleli.} \tag{4.9}$$

Tuttavia, progettare un algoritmo che raggiunga tale limite inferiore gestendo D dischi indipendenti in modo asincrono risulta estremamente complesso: è necessario evitare il fenomeno del *bottlenecking* (o congestione dei dischi), in cui le letture si concentrano sbilanciatamente su una singola unità lasciando le restanti $D - 1$ inattive.

4.3.2 L'Architettura del Disk Striping

Per ottenere i vantaggi prestazionali del parallelismo evitando la complessità di una schedulazione asincrona disaccoppiata, si ricorre alla tecnica del **Disk Striping** (concettualmente coincidente con lo schema di archiviazione RAID 0):

- I D dischi fisici indipendenti $d_1, d_2, \ldots, d_D$ vengono raggruppati logicamente in una singola macro-unità.
- Viene definito un **super-blocco logico** di dimensione aggregata:

  $$B' = D \cdot B, \tag{4.10}$$

  costituito dall'unione sincronizzata di un blocco fisico di taglia B allocato al medesimo indirizzo (o cilindro) su ciascuno dei D dischi.
- Qualsiasi operazione di I/O trasferisce sempre un intero super-blocco di ampiezza B' distribuendo il carico uniformemente e simultaneamente su tutti i D canali hardware in un singolo ciclo di clock I/O parallelo.

In Figura 4.4 è rappresentata la mappatura del super-blocco B' sui dischi fisici sottostanti.

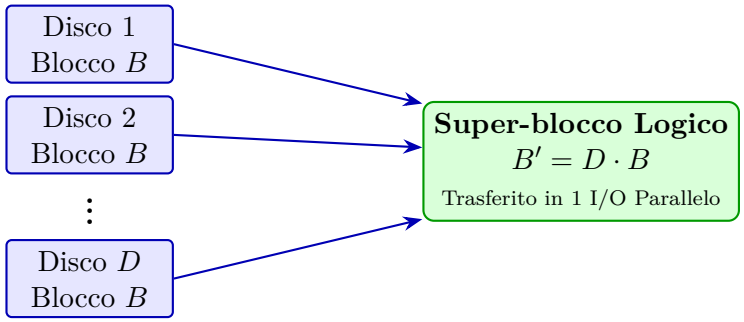


Figura 4.4: Architettura a Disk Striping: raggruppamento di D blocchi fisici in un super-blocco B'.

Applicando il Multiway Merge Sort assumendo la taglia del blocco pari a B', il numero di super-blocchi da scansionare è $\frac{n}{B'} = \frac{n}{D \cdot B}$, mentre la capacità di fan-in di memoria diventa $\frac{M}{B'} = \frac{M}{D \cdot B}$. Il costo I/O parallelo complessivo assume la forma:

$$\mathrm{I/O}_{\mathrm{Striping}} = \mathcal{O}\left(\frac{n}{B'} \log_{M/B'}\left(\frac{n}{M}\right)\right) = \mathcal{O}\left(\frac{n}{D \cdot B} \log_{\frac{M}{D \cdot B}}\left(\frac{n}{M}\right)\right). \tag{4.11}$$

4.3.3 Analisi del Rallentamento (*Slowdown*)

Sebbene il Disk Striping riduca il volume di trasferimenti fisici di un fattore D, esso restringe drasticamente il fan-in a disposizione dell'algoritmo da $\frac{M}{B}$ a $\frac{M}{D \cdot B}$, poiché la memoria deve allocare buffer di ampiezza multipla $B' = D \cdot B$.

Per quantificare la perdita di efficienza rispetto all'algoritmo ideale a dischi indipendenti, valutiamo il rapporto di rallentamento (*Slowdown*):

$$\mathrm{Slowdown} = \frac{\mathrm{I/O}_{\mathrm{Striping}}}{\mathrm{I/O}_{\mathrm{Ideale}}} = \frac{\frac{n}{D \cdot B} \log_{\frac{M}{D \cdot B}}\left(\frac{n}{M}\right)}{\frac{n}{D \cdot B} \log_{\frac{M}{B}}\left(\frac{n}{M}\right)} = \frac{\log_{\frac{M}{D \cdot B}}\left(\frac{n}{M}\right)}{\log_{\frac{M}{B}}\left(\frac{n}{M}\right)}. \tag{4.12}$$

Esprimendo entrambi i logaritmi in base 2 mediante la formula di cambio base $\log_a x = \frac{\log_2 x}{\log_2 a}$:

$$\mathrm{Slowdown} = \frac{\frac{\log_2(n/M)}{\log_2(M/(D \cdot B))}}{\frac{\log_2(n/M)}{\log_2(M/B)}} = \frac{\log_2(M/B)}{\log_2(M/(D \cdot B))} = \frac{\log_2(M/B)}{\log_2(M/B) - \log_2 D}. \tag{4.13}$$

Dividendo numeratore e denominatore per la quantità comune $\log_2(M/B)$:

$$\text{Slowdown} = \frac{1}{1 - \dfrac{\log_2 D}{\log_2(M/B)}}. \tag{4.14}$$

4.3.4 Discussione dei Regimi Asintotici e Limiti Fisici

L'efficienza del Disk Striping è interamente regolata dal rapporto adimensionale:

$$\rho = \frac{\log_2 D}{\log_2(M/B)}. \tag{4.15}$$

Si delineano due comportamenti estremi distinti:

1. **Regime di Memoria Abbondante ($M \to \infty$ oppure $D \ll \frac{M}{B}$):** Quando la memoria interna è molto capiente rispetto al numero di dischi impiegati, la capacità di fan-in della RAM domina marcatamente:

$$\rho = \frac{\log_2 D}{\log_2(M/B)} \to 0 \implies \text{Slowdown} \to \frac{1}{1-0} = 1. \tag{4.16}$$

   In questa circostanza il fattore di rallentamento è trascurabile: il Disk Striping consegue la medesima efficienza asintotica del sistema teorico a canali disaccoppiati, risultando una soluzione ottimale dal punto di vista ingegneristico per la sua estrema semplicità di implementazione.

2. **Saturazione della Memoria Primaria ($\log_2 D \to \log_2(M/B)$):** Allorché il numero di dischi fisici D scala fino a raggiungere la cardinalità massima di blocchi allocabili in memoria ($D \approx \frac{M}{B}$), il rapporto ρ tende a 1. Di conseguenza, il denominatore di (4.14) tende a zero e la funzione diverge positivamente:

$$\lim_{\rho \to 1^-} \frac{1}{1-\rho} = +\infty. \tag{4.17}$$

   Interpretazione fisica: Se $D \geq \frac{M}{B}$, la memoria interna M non possiede spazio sufficiente nemmeno per ospitare un singolo super-blocco $B' = D \cdot B$ per ciascun flusso, determinando l'impossibilità di operare e il collasso strutturale dell'algoritmo di striping.

Capitolo 5

Limiti Inferiori per l'Ordinamento, Quicksort Dual-Pivot e Bounded Quicksort

5.1 Modello RAM: Limiti Inferiori e Alberi di Decisione

Nel modello di calcolo RAM (*Random Access Machine*) standard, un algoritmo basato su confronti (*comparison-based algorithm*) non può accedere alla rappresentazione binaria interna delle chiavi per estrarne cifre o bit (come invece avviene negli algoritmi di tipo *Radix Sort*), ma può acquisire informazioni sull'ordine reciproco degli elementi esclusivamente attraverso confronti binari del tipo:

$$A < B, \quad A = B, \quad \text{oppure} \quad A > B. \tag{5.1}$$

Assumendo, senza perdita di generalità per l'analisi del limite inferiore, che tutte le chiavi in ingresso siano reciprocamente distinte, ogni confronto elementare ammette un esito strettamente dicotomico ($<$ oppure $>$).

5.1.1 Il Modello dell'Albero di Decisione

La classe degli algoritmi deterministici basati su confronti viene formalmente modellata mediante la struttura astratta dell'**Albero di Decisione** (*Decision Tree*):

- **Nodi interni:** Ciascun nodo interno rappresenta un singolo confronto elementare tra una coppia di elementi o posizioni dell'input, denotato convenzionalmente con $[A : B]$. Poiché le chiavi sono distinte, dal nodo si dipartono esattamente due archi uscenti, corrispondenti agli esiti possibili: l'arco sinistro associato all'evento $A < B$ e l'arco destro associato all'evento $A > B$. L'albero risulta pertanto un albero binario proprio.

- **Cammino radice-foglia:** Per ogni specifica istanza di input di dimensione n, l'algoritmo deterministico segue un unico cammino orientato discendente dalla radice verso una foglia. Ciascun nodo visitato lungo il cammino corrisponde a un confronto eseguito; la lunghezza del cammino (numero di archi attraversati) riflette il numero di confronti effettuati per quella specifica configurazione. La lunghezza del cammino più lungo dalla radice a una foglia coincide con l'**altezza** t dell'albero di decisione e quantifica il tempo di esecuzione nel caso peggiore $T(n)$.

- **Foglie:** Ciascuna foglia corrisponde alla terminazione del processo di calcolo e racchiude la risposta o configurazione risolutiva finale del problema (ad esempio, la corretta permutazione degli indici che ordina l'array, o l'indice in cui risiede la chiave cercata).

In Figura 5.1 viene illustrata la topologia generale di un albero di decisione binario per un generico problema basato su confronti.

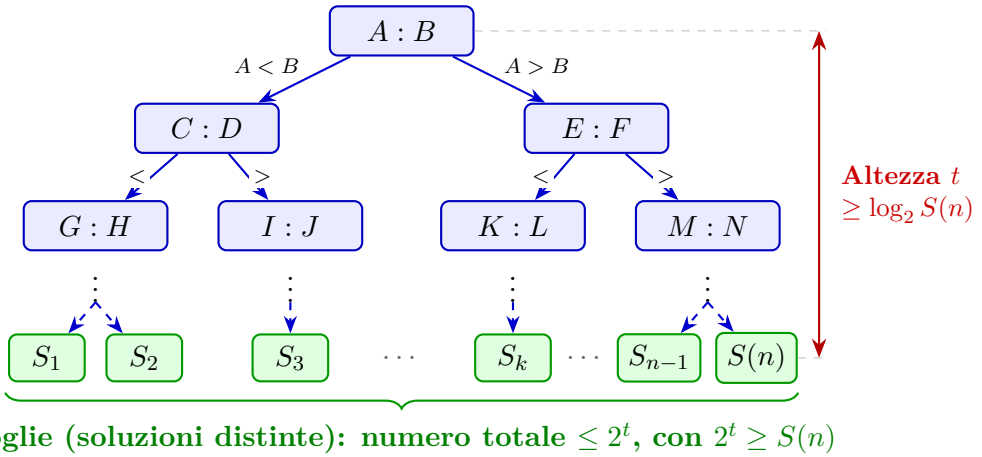


Figura 5.1: Modello dell'albero di decisione binario per algoritmi basati su confronti. I nodi interni rappresentano i confronti elementari; le foglie rappresentano le soluzioni distinte ammissibili $(S_1, S_2, \ldots, S(n))$. Il cammino più lungo determina il tempo nel caso peggiore.

5.1.2 Il Teorema Generale dell'Informazione

Il legame tra il numero di soluzioni distinte di un problema e il numero minimo di confronti necessari a risolverlo è sancito dal seguente teorema fondamentale della teoria dell'informazione applicata agli algoritmi.

Teorema 5.1.1 (Teorema Generale dell'Informazione per Alberi di Decisione)**.** *Sia P un problema computazionale risolvibile tramite un algoritmo deterministico basato su confronti. Se per un input di dimensione n il problema ammette $S(n)$ soluzioni distinte (ciascuna delle quali deve poter essere restituita dall'algoritmo per opportune istanze dell'input), allora qualsiasi algoritmo deterministico basato su confronti richiede nel caso peggiore un tempo:*

$$T(n) = \Omega(\log_2 S(n)). \tag{5.2}$$

Dimostrazione. Consideriamo l'albero di decisione binario associato a un qualsiasi algoritmo deterministico corretto per il problema P. Sia t l'altezza dell'albero, ovvero la lunghezza massima di un cammino dalla radice a una foglia.

Poiché a ogni passo il confronto è binario (avendo assunto chiavi distinte), ogni nodo interno possiede al più 2 figli. Per induzione sull'altezza, un albero binario di altezza t possiede al massimo un numero di foglie L pari a:

$$L \leq 2^t. \tag{5.3}$$

Affinché l'algoritmo sia corretto su qualsiasi istanza ammissibile, l'albero di decisione deve essere in grado di produrre ciascuna delle $S(n)$ possibili risposte distinte. Ne consegue che ogni soluzione distinta deve essere associata ad almeno una foglia distinta dell'albero (se due soluzioni distinte convergessero sulla medesima foglia, l'algoritmo fallirebbe nel distinguerle, restituendo un risultato errato per almeno una delle istanze corrispondenti). Pertanto, deve necessariamente valere la disuguaglianza:

$$2^t \geq L \geq S(n). \tag{5.4}$$

Applicando la funzione logaritmica in base 2 (strettamente monotona crescente) ad ambo i membri, si ottiene:

$$t \geq \log_2 S(n). \tag{5.5}$$

Poiché il tempo di esecuzione nel caso peggiore $T(n)$ coincide per definizione con l'altezza t dell'albero, si conclude che:

$$T(n) \geq \log_2 S(n) \implies T(n) = \Omega(\log_2 S(n)). \tag{5.6}$$

□

5.1.3 Applicazioni Fondamentali: Ricerca e Ordinamento

Il Teorema 5.1.1 consente di ricavare immediatamente i limiti inferiori canonici per due problemi cardine della computazione in memoria interna:

1. **Ricerca di una chiave (*Searching*):** Dato un array ordinato di n elementi e una chiave cercata K, l'algoritmo deve stabilire se K è presente nell'array e, in caso affermativo, restituire l'indice $i \in \{1, 2, \ldots, n\}$ corrispondente; se la chiave è assente, restituisce un simbolo speciale di fallimento $\perp$. L'insieme delle possibili risposte distinte è dunque:

$$S(n) = \{1, 2, \ldots, n\} \cup \{\perp\} \implies |S(n)| = n + 1. \tag{5.7}$$

   Applicando il teorema:

$$T_{\text{Search}}(n) \geq \log_2(n + 1) = \Omega(\log_2 n). \tag{5.8}$$

   Questo limite inferiore attesta l'ottimalità asintotica della *Ricerca Binaria* (*Binary Search*), la quale opera esattamente in $\Theta(\log_2 n)$ confronti.

2. **Ordinamento (*Sorting*):** Dato un vettore di n elementi distinti, ordinare l'array equivale a identificare quale tra tutte le possibili permutazioni degli indici sia quella che dispone le chiavi in ordine non decrescente. Poiché l'insieme $\{1, 2, \ldots, n\}$ ammette esattamente $n!$ permutazioni distinte, il numero di soluzioni possibili corrisponde a:

$$S(n) = n!. \tag{5.9}$$

   Applicando il teorema dei limiti inferiori:

$$T_{\text{Sort}}(n) \geq \log_2(n!). \tag{5.10}$$

   Per esprimere questo valore in forma asintotica esplicita, ricorriamo all'**approssimazione di Stirling** per il fattoriale:

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \implies \ln(n!) = n \ln n - n + \mathcal{O}(\ln n). \tag{5.11}$$

   Convertendo il logaritmo in base 2:

$$\log_2(n!) = n \log_2 n - n \log_2 e + \Theta(\log_2 n) = \Theta(n \log_2 n). \tag{5.12}$$

Alternativamente, mediante una limitazione inferiore elementare per metà dei termini:

$$n! \geq \prod_{i=\lceil n/2 \rceil}^{n} i \geq \left(\frac{n}{2}\right)^{n/2}, \tag{5.13}$$

$$\log_2(n!) \geq \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \frac{n}{2}(\log_2 n - 1) = \Omega(n \log_2 n). \tag{5.14}$$

Si conclude che:

$$T_{\text{Sort}}(n) = \Omega(n \log_2 n). \tag{5.15}$$

Qualsiasi algoritmo deterministico o randomizzato basato su confronti (come *Merge Sort*, *Heap Sort* o il tempo atteso di *Quicksort*) non può scendere al di sotto della soglia asintotica $\Omega(n \log_2 n)$ nel modello RAM.

5.2 Modello a Due Livelli: Limite Inferiore di Aggarwal-Vitter

Nel modello di memoria a due livelli di Aggarwal e Vitter [2], il costo di esecuzione non è misurato dai confronti elementari della CPU, bensì dal numero t di operazioni di I/O, ciascuna delle quali trasferisce un intero blocco di dimensione B tra il disco non volatile e la memoria interna (RAM) di capacità pari a M elementi.

5.2.1 La Natura dell'I/O e i Confronti Gratuiti in Memoria Interna

L'analisi tradizionale dell'albero di decisione binario non è direttamente applicabile al calcolo degli I/O:

- Una singola operazione di lettura dal disco carica contemporaneamente B elementi in memoria rapida.
- Una volta che questi B elementi risiedono nella RAM accanto agli altri $M - B$ elementi già presenti, la CPU può eseguire **un numero arbitrariamente elevato di confronti interni a costo zero di I/O**.
- Tali confronti interni "gratuiti" consentono di riordinare istantaneamente tutti gli M elementi residenti in RAM, restringendo drasticamente il ventaglio delle permutazioni globali ancora compatibili con lo stato dei dati.

Pertanto, per determinare il lower bound su memoria esterna, è necessario costruire un **Albero di Decisione di I/O**, in cui ogni nodo rappresenta un'operazione di I/O (lettura di un blocco di taglia B) e gli archi uscenti corrispondono a tutte le possibili sistemazioni informative derivanti dall'interazione tra i B elementi letti e gli $M - B$ elementi già presenti in memoria interna.

5.2.2 Calcolo del Grado di Diramazione (Fan-out) dell'Albero di I/O

Quando un blocco di B elementi viene trasferito dal disco alla memoria interna (che accoglie M elementi totali, di cui $M - B$ preesistenti e B appena caricati), distinguiamo due scenari operativi:

1. **Lettura di un "Nuovo Blocco" (*New Block*):** Il blocco viene letto dal disco per la primissima volta: gli elementi in esso contenuti non sono mai stati osservati in precedenza da alcun'altra operazione dell'algoritmo.

   - L'ordine reciproco dei B elementi è totalmente ignoto: essi possono presentarsi internamente in una qualsiasi delle $B!$ permutazioni relative possibili.
   - Una volta caricati in RAM, i B nuovi elementi devono essere posizionati reciprocamente rispetto agli $M - B$ elementi già residenti, determinando quale sia la loro collocazione ordinata comune nell'insieme totale di M elementi. Il numero di modi in cui B elementi indistinguibili nella loro posizione possono intercalarsi tra gli M slot totali è dato dal coefficiente binomiale:
   $$\binom{M}{B} = \frac{M!}{B!\,(M-B)!}. \tag{5.16}$$

   Moltiplicando le $B!$ configurazioni d'ordine interno per i $\binom{M}{B}$ modi di intercalamento, il fattore di diramazione (*fan-out*) X generato dalla lettura di un nuovo blocco risulta:
   $$X = (B!) \cdot \binom{M}{B}. \tag{5.17}$$

2. **Rilettura di un "Vecchio Blocco" (*Old Block*):** Il blocco era già stato caricato, ordinato internamente e successivamente scritto su disco in una fase precedente della computazione (si pensi a un run generato durante un passo di merge).

- L'ordine relativo interno dei suoi B elementi è già integralmente noto alla computazione; non si ottiene pertanto alcuna nuova informazione sul loro ordine intrinseco (fattore 1 al posto di $B!$).
- L'unico incremento informativo risiede nell'intercalamento dei B elementi con gli altri $M - B$ elementi attualmente residenti in memoria interna.

Il fattore di diramazione (*fan-out*) Y per la rilettura di un vecchio blocco è quindi:

$$Y = \binom{M}{B}. \tag{5.18}$$

In Figura 5.2 viene schematizzata la biforcazione logica e la cardinalità delle diramazioni per le due tipologie di trasferimento I/O.

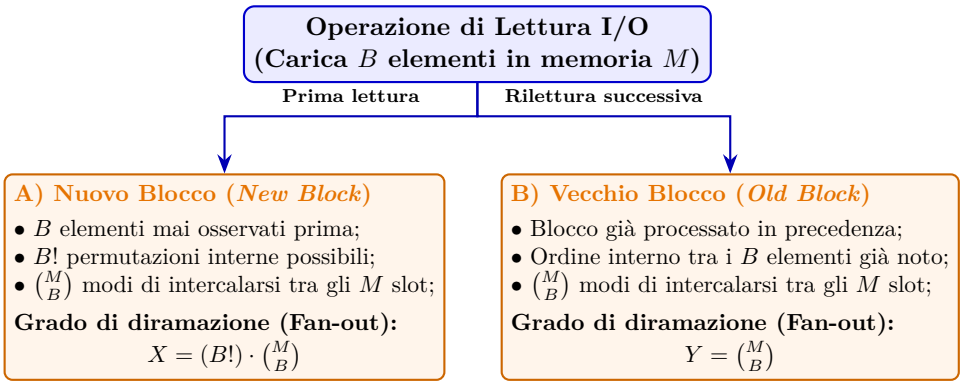


Figura 5.2: Fattori di diramazione (*fan-out*) nell'albero di decisione di I/O nel modello a due livelli: la lettura di un nuovo blocco apporta informazione d'ordine interno ($B!$) e di intercalamento ($\binom{M}{B}$); la rilettura di un vecchio blocco apporta esclusivamente informazione di intercalamento.

5.2.3 Derivazione del Lower Bound di Aggarwal-Vitter

Sia dato un file di ingresso composto da n elementi distinti suddivisi in $N = \frac{n}{B}$ blocchi su disco. Per poter ordinare il file, qualsiasi algoritmo corretto deve necessariamente ispezionare ciascun elemento almeno una volta: ne discende che l'algoritmo deve compiere almeno $\frac{n}{B}$ operazioni di lettura di tipo "Nuovo Blocco".

Se l'algoritmo esegue complessivamente t operazioni di I/O nel caso peggiore, allora:

- Esattamente $\frac{n}{B}$ operazioni saranno di tipo "Nuovo Blocco", ciascuna con fattore di diramazione al più X.
- Le rimanenti $t - \frac{n}{B}$ operazioni saranno (al più) di tipo "Vecchio Blocco", ciascuna con fattore di diramazione al più Y.

Il numero totale di foglie dell'albero di decisione di I/O rappresenta il numero massimo di permutazioni globali distinguibili dalla sequenza di trasferimenti. Affinché l'algoritmo sia in grado di ordinare correttamente qualsiasi istanza, tale numero di foglie deve risultare almeno pari al numero totale di permutazioni ammissibili $n!$:

$$X^{\frac{n}{B}} \cdot Y^{\left(t-\frac{n}{B}\right)} \geq n!. \tag{5.19}$$

Sostituendo le espressioni di X ed Y definite nelle equazioni (5.17) e (5.18):

$$\left[(B!) \cdot \binom{M}{B}\right]^{\frac{n}{B}} \cdot \left[\binom{M}{B}\right]^{t-\frac{n}{B}} \geq n!. \tag{5.20}$$

Raccogliendo le potenze relative alla base comune $\binom{M}{B}$:

$$(B!)^{\frac{n}{B}} \cdot \binom{M}{B}^{\frac{n}{B}} \cdot \binom{M}{B}^{t-\frac{n}{B}} \geq n! \implies (B!)^{\frac{n}{B}} \cdot \binom{M}{B}^{t} \geq n!. \tag{5.21}$$

Applichiamo il logaritmo in base 2 a entrambi i membri della disuguaglianza (5.21):

$$\log_2\left((B!)^{\frac{n}{B}}\right) + \log_2\left(\binom{M}{B}^{t}\right) \geq \log_2(n!), \tag{5.22}$$

da cui:

$$\frac{n}{B}\log_2(B!) + t\log_2\binom{M}{B} \geq \log_2(n!). \tag{5.23}$$

Per sviluppare i termini logaritmici utilizziamo le consuete approssimazioni asintotiche per il fattoriale e per il coefficiente binomiale:

- Per l'approssimazione di Stirling: $\log_2(a!) \approx a\log_2 a - a\log_2 e \approx a\log_2 a$.
- Per il coefficiente binomiale, ricordando che $\binom{a}{b} \leq \left(\frac{e\cdot a}{b}\right)^b$:

$$\log_2\binom{M}{B} \approx B\log_2\left(\frac{M}{B}\right) + \mathcal{O}(B). \tag{5.24}$$

Sostituendo queste espressioni nell'equazione (5.23):

$$\frac{n}{B} \cdot (B \log_2 B) + t \cdot \left(B \log_2 \frac{M}{B} \right) \geq n \log_2 n. \tag{5.25}$$

Semplificando il fattore B nel primo termine:

$$n \log_2 B + t \cdot B \log_2 \left(\frac{M}{B} \right) \geq n \log_2 n. \tag{5.26}$$

Isoliamo il termine contenente l'incognita t:

$$t \cdot B \log_2 \left(\frac{M}{B} \right) \geq n \log_2 n - n \log_2 B = n \left(\log_2 n - \log_2 B \right) = n \log_2 \left(\frac{n}{B} \right). \tag{5.27}$$

Dividendo ambo i membri per la quantità strettamente positiva $B \log_2 \left(\frac{M}{B} \right)$:

$$t \geq \frac{n \log_2 \left(\frac{n}{B} \right)}{B \log_2 \left(\frac{M}{B} \right)} = \frac{n}{B} \cdot \frac{\log_2(n/B)}{\log_2(M/B)}. \tag{5.28}$$

Ricorrendo alla proprietà del cambio di base dei logaritmi ($\frac{\log_2 a}{\log_2 b} = \log_b a$), otteniamo formalmente il celebre teorema di Aggarwal e Vitter.

Teorema 5.2.1 (Limite Inferiore di Aggarwal-Vitter per l'Ordinamento)**.** *Nel modello di memoria a due livelli, qualsiasi algoritmo deterministico o randomizzato basato su confronti richiede nel caso peggiore un numero di trasferimenti I/O pari a:*

$$I/O_{Sort}(n) = \Omega \left(\frac{n}{B} \log_{M/B} \left(\frac{n}{B} \right) \right) = \Omega \left(\frac{n}{B} \frac{\log(n/B)}{\log(M/B)} \right). \tag{5.29}$$

Proprietà 5.2.1 (Ottimalità Asintotica del Multiway Merge Sort)**.** *Il limite inferiore di Aggarwal e Vitter dimostra che il* ***Multiway Merge Sort*** *(k-Way Merge Sort con fattore di merge $k = \frac{M}{B} - 1$) analizzato nel Capitolo 4 (sezione 4.1) è* ***asintoticamente ottimo****: il numero di I/O spesi per ordinare l'array coincide esattamente con la barriera teorica insuperabile imposta dalla teoria dell'informazione su memoria esterna.*

5.3 Quicksort: Gestione dei Duplicati e Dual-Pivot Quicksort

Quicksort è un algoritmo basato sul paradigma *Divide et Impera* di tipo distributivo (*distribution-based*): anziché dividere rigidamente l'array per

posizione (come in *Merge Sort*) per poi unire i risultati ordinati, Quicksort partiziona i dati in base al valore di una o più chiavi privilegiate denominate **pivot**, delegando l'intero lavoro di ordinamento alla fase di distribuzione (*partition*).

5.3.1 Il Problema dei Valori Uguali (Equal Values)

Nelle implementazioni didattiche standard di Quicksort basate sul partizionamento a due vie (*2-way partitioning*, schema di Lomuto o di Hoare):

- L'array viene ripartito in due sole categorie: elementi $\leq p$ da una parte ed elementi $> p$ dall'altra.
- Qualora l'array contenga una frequenza elevata di elementi identici al pivot p (nel caso limite, un array composto esclusivamente da elementi tutti uguali, $S[i] = c, \forall i$), tale schema inserisce tutti gli elementi duplicati all'interno della medesima partizione.
- La dimensione della sotto-istanza si riduce di una sola unità a ogni invocazione (1 contro $n-1$), degradando l'albero di ricorsione a un albero degenere di altezza $\Theta(n)$ e il tempo complessivo di calcolo alla complessità pessima:

$$T(n) = T(n-1) + \Theta(n) = \Theta(n^2). \tag{5.30}$$

Definizione 5.3.1 (Partizionamento a Tre Vie (Dutch National Flag))**.** La soluzione classica al problema dei duplicati, formulata da Edsger Dijkstra [3] con il problema della bandiera olandese, consiste nel partizionare l'array in **tre zone contigue**:

$$[\,< p \quad | \quad = p \quad | \quad > p\,]. \tag{5.31}$$

Gli elementi strettamente uguali al pivot ($= p$) si trovano già nella loro collocazione definitiva e vengono **esclusi dalle successive chiamate ricorsive**, le quali vengono lanciate esclusivamente sulle fasce $[< p]$ e $[> p]$. Su array con elementi tutti uguali, l'algoritmo converge in tempo strettamente lineare $\mathcal{O}(n)$.

5.3.2 Algoritmo di Partizionamento a 3 Vie ed Esempio Esecutivo

Per implementare concretamente il partizionamento a tre vie (*Dutch National Flag*) in Quicksort, si adotta comunemente lo schema lineare ideato da Dijkstra e formalizzato da Sedgewick [4].

Dato un sotto-array $S[\text{lo} \dots \text{hi}]$ e scelto come pivot l'elemento iniziale $p = S[\text{lo}]$, l'algoritmo mantiene tre indici operativi durante la scansione:

- **Puntatore** lt **(*less than*):** delimita a sinistra gli elementi strettamente inferiori al pivot. L'invariante garantisce che $S[k] < p$ per ogni $\text{lo} \leq k < lt$. Inizialmente $lt = \text{lo}$.
- **Puntatore** i **(cursore di scansione):** scorre gli elementi dell'array. L'invariante assicura che $S[k] = p$ per ogni $lt \leq k < i$. Inizialmente $i = \text{lo} + 1$.
- **Puntatore** gt **(*greater than*):** delimita a destra gli elementi strettamente superiori al pivot. L'invariante garantisce che $S[k] > p$ per ogni $gt < k \leq \text{hi}$. Inizialmente $gt = \text{hi}$.
- L'intervallo $S[i \dots gt]$ contiene gli elementi ancora non esaminati.

A ogni iterazione, finché $i \leq gt$, si confronta l'elemento corrente $S[i]$ con il pivot p:

1. **Se** $S[i] < p$**:** si scambia $S[lt]$ con $S[i]$, quindi si incrementano entrambi gli indici ($lt \leftarrow lt + 1$, $i \leftarrow i + 1$). Poiché per invariante la cella $S[lt]$ conteneva un elemento $= p$, lo scambio sposta il valore minore a sinistra e posiziona un duplicato del pivot in $S[i]$, consentendo a i di avanzare.
2. **Se** $S[i] > p$**:** si scambia $S[i]$ con $S[gt]$, quindi si decrementa solo gt ($gt \leftarrow gt - 1$). L'indice i **non avanza**, poiché l'elemento appena scambiato da $S[gt]$ non è ancora stato esaminato.
3. **Se** $S[i] == p$**:** l'elemento si trova già nella collocazione centrale corretta; si incrementa unicamente il cursore di scansione ($i \leftarrow i + 1$).

Al termine della procedura ($i > gt$), l'array risulta ripartito nelle tre zone: $S[\text{lo} \dots lt - 1] < p$, $S[lt \dots gt] = p$ e $S[gt + 1 \dots \text{hi}] > p$. Le successive chiamate ricorsive vengono invocate **esclusivamente** su $S[\text{lo} \dots lt - 1]$ e su $S[gt + 1 \dots \text{hi}]$, escludendo l'intero blocco di duplicati. Le proprietà formali di conservazione dimensionale di tali partizioni e l'integrazione con la selezione rapida (*Quickselect*) e la ricorrenza del tempo atteso vengono approfondite nel Capitolo 6 (sezione 6.3).

In Tabella 5.1 viene riportata l'evoluzione dettagliata passo per passo dei tre puntatori e del contenuto dell'array durante l'esecuzione dell'algoritmo con pivot $p = 4$.

Passo	i	lt	gt	Confronto	Azione	Stato Array
Init	1	0	7	Pivot $p = 4$	Inizializzazione	$[4, 7, 2, 4, 1, 4, 8, 3]$
1	1	0	7	$S[1] = 7 > 4$	SWAP$(S[1], S[7])$, $gt \leftarrow 6$	$[4, 3, 2, 4, 1, 4, 8, 7]$
2	1	0	6	$S[1] = 3 < 4$	SWAP$(S[0], S[1])$, $lt \leftarrow 1$, $i \leftarrow 2$	$[3, 4, 2, 4, 1, 4, 8, 7]$
3	2	1	6	$S[2] = 2 < 4$	SWAP$(S[1], S[2])$, $lt \leftarrow 2$, $i \leftarrow 3$	$[3, 2, 4, 4, 1, 4, 8, 7]$
4	3	2	6	$S[3] == 4$	Avanzamento $i \leftarrow 4$	$[3, 2, 4, 4, 1, 4, 8, 7]$
5	4	2	6	$S[4] = 1 < 4$	SWAP$(S[2], S[4])$, $lt \leftarrow 3$, $i \leftarrow 5$	$[3, 2, 1, 4, 4, 4, 8, 7]$
6	5	3	6	$S[5] == 4$	Avanzamento $i \leftarrow 6$	$[3, 2, 1, 4, 4, 4, 8, 7]$
7	6	3	6	$S[6] = 8 > 4$	SWAP$(S[6], S[6])$, $gt \leftarrow 5$	$[3, 2, 1, 4, 4, 4, 8, 7]$
Fine	**6**	**3**	**5**	$i > gt$	**Terminazione**	$[\mathbf{3}, \mathbf{2}, \mathbf{1} \mid \mathbf{4}, \mathbf{4}, \mathbf{4} \mid \mathbf{8}, \mathbf{7}]$

Tabella 5.1: Traccia puntuale passo-passo del partizionamento a 3 vie con pivot $p = 4$. Al termine, i tre elementi identici 4 occupano gli indici $lt \dots gt$ $(3 \dots 5)$ e vengono esclusi dalle chiamate ricorsive.

5.3.3 Architettura del Dual-Pivot Quicksort (Yaroslavskiy)

Nel 2009, l'ingegnere Vladimir Yaroslavskiy propose un algoritmo innovativo denominato **Dual-Pivot Quicksort** [5]. Dopo intensi benchmark empirici condotti in collaborazione con Joshua Bloch e Robert Sedgewick, l'algoritmo ha sostituito il Quicksort classico a pivot singolo nella libreria standard di Java ('java.util.Arrays.sort') per tutti i tipi primitivi.

A differenza del Quicksort classico, il Dual-Pivot Quicksort seleziona **due pivot** P e Q. Inizialmente si verifica che $P \leq Q$ (scambiandoli se necessario). L'intervallo dell'array $S[\text{inizio} \dots \text{fine}]$ viene suddiviso dinamicamente in quattro regioni gestite tramite tre indici operativi (l, k, g):

- l (*less*): limite superiore degli elementi strettamente minori di P ($S[i] < P$ per ogni inizio $\leq i < l$).
- k: indice di scansione corrente ($l \leq k \leq g$). Gli elementi tra l e $k - 1$ soddisfano l'invariante centrale: $P \leq S[i] \leq Q$.
- g (*great*): limite inferiore degli elementi strettamente maggiori di Q ($S[i] > Q$ per ogni $g < i \leq$ fine).
- La regione compresa tra k e g rappresenta l'insieme degli elementi ancora non esaminati.

In Figura 5.3 viene illustrato lo stato dell'array durante l'esecuzione del partizionamento a due pivot.

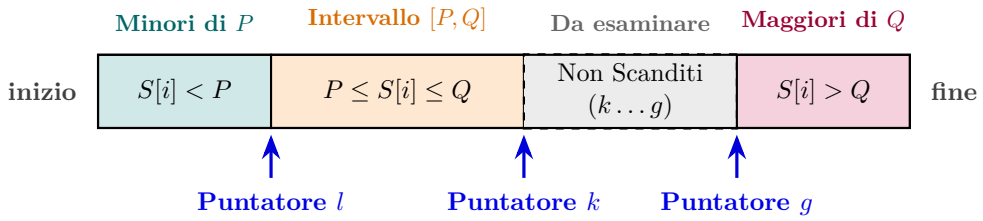


Figura 5.3: Schema del partizionamento nel Dual-Pivot Quicksort di Yaroslavskiy. L'array è partizionato nelle quattro regioni dinamiche: elementi $< P$, elementi compresi tra P e Q, elementi non ancora esaminati tra k e g, ed elementi $> Q$.

5.3.4 Invariante e Logica di Avanzamento

La procedura di partizionamento itera mantenendo l'invariante fino a quando l'indice di scansione k non supera l'indice g ($k \leq g$). A ogni iterazione viene esaminato il valore corrente $S[k]$:

1. **Caso 1:** $S[k] < P$**:** L'elemento appartiene alla prima partizione. Si effettua uno scambio tra $S[k]$ e $S[l]$, dopodiché si incrementano entrambi i puntatori:

$$\text{SWAP}(S[k], S[l]), \quad l \leftarrow l + 1, \quad k \leftarrow k + 1. \tag{5.32}$$

   Poiché per invariante l'elemento preesistente in $S[l]$ apparteneva alla fascia centrale $[P, Q]$, dopo lo scambio la posizione k conterrà un valore valido per la seconda partizione, consentendo a k di avanzare.

2. **Caso 2:** $S[k] > Q$**:** L'elemento appartiene alla partizione superiore. Prima dello scambio, si fa retrocedere il puntatore g finché non si individua una posizione tale che $S[g] \leq Q$ (saltando elementi già corretti per la parte destra). Trovato tale g, si scambia:

$$\text{SWAP}(S[k], S[g]), \quad g \leftarrow g - 1. \tag{5.33}$$

   L'elemento collocato in $S[k]$ (proveniente da g) non è ancora stato classificato rispetto a P: esso viene pertanto **riesaminato al passo successivo senza avanzare** k.

3. **Caso 3:** $P \leq S[k] \leq Q$**:** L'elemento risiede già nella collocazione corretta della fascia centrale. Non si rende necessario alcuno scambio fisico di memoria e l'indice di scansione avanza semplicemente:

$$k \leftarrow k + 1. \tag{5.34}$$

Al termine della scansione ($k > g$), i due pivot P e Q vengono ricollocati scambiandoli rispettivamente nelle loro posizioni definitive $l-1$ e $g+1$. Le tre chiamate ricorsive vengono quindi eseguite in modo indipendente sui tre intervalli:

$$[\text{inizio} \dots l-2], \quad [l \dots g], \quad [g+2 \dots \text{fine}]. \tag{5.35}$$

Algorithm 3: Dual-Pivot Partitioning (Yaroslavskiy)

Dati: Array S, indice iniziale left, indice finale right con left $<$ right
Risultato: Indici di collocazione finale dei pivot (p_1, p_2)

```
1  if S[left] > S[right] then
2      SWAP(S[left], S[right])
3  P ← S[left]
4  Q ← S[right]
5  l ← left + 1
6  g ← right − 1
7  k ← l
8  while k ≤ g do
9      if S[k] < P then
10         SWAP(S[k], S[l])
11         l ← l + 1
12         k ← k + 1
13     else if S[k] > Q then
14         while S[g] > Q and k < g do
15             g ← g − 1
16         SWAP(S[k], S[g])
17         g ← g − 1
18         if S[k] < P then
19             SWAP(S[k], S[l])
20             l ← l + 1
21         k ← k + 1
22     else
23         k ← k + 1
24 l ← l − 1
25 g ← g + 1
26 SWAP(S[left], S[l])
27 SWAP(S[right], S[g])
28 return (l, g)
```

5.3.5 Esempio e Traccia del Dual-Pivot Quicksort

Per illustrare il funzionamento del partizionamento a due pivot descritto nell'algorithm 3, consideriamo un array di 8 elementi:

$$S = [3,\ 8,\ 2,\ 5,\ 1,\ 4,\ 9,\ 7]. \tag{5.36}$$

I pivot vengono scelti agli estremi dell'array: $P = S[0] = 3$ e $Q = S[7] = 7$. Poiché $P \leq Q$ ($3 \leq 7$), l'ordinamento preliminare dei pivot è già soddisfatto. Gli indici iniziali vengono configurati come: $l = 1$, $k = 1$ e $g = 6$.

In Tabella 5.2 viene riassunta la sequenza puntuale dei passaggi salienti dell'esecuzione, evidenziando il movimento dei cursori k, l, g e la ricollocazione finale dei pivot P e Q.

Passo	k	l	g	**Elemento**	**Azione**	**Stato Array**
Init	1	1	6	Pivot $P = 3, Q = 7$	Inizializzazione cursori	$[3, 8, 2, 5, 1, 4, 9, 7]$
1	1	1	6	$S[1] = 8 > 7$	Trova $g = 5$; SWAP$(S[1], S[5])$, $g \leftarrow 4, k \leftarrow 2$	$[3, 4, 2, 5, 1, 8, 9, 7]$
2	2	1	4	$S[2] = 2 < 3$	SWAP$(S[1], S[2])$, $l \leftarrow 2, k \leftarrow 3$	$[3, 2, 4, 5, 1, 8, 9, 7]$
3	3	2	4	$3 \leq S[3] = 5 \leq 7$	Avanzamento $k \leftarrow 4$	$[3, 2, 4, 5, 1, 8, 9, 7]$
4	4	2	4	$S[4] = 1 < 3$	SWAP$(S[2], S[4])$, $l \leftarrow 3, k \leftarrow 5$	$[3, 2, 1, 5, 4, 8, 9, 7]$
Fine	5	3	4	$k > g$	Arresto scansione	$[3, 2, 1, 5, 4, 8, 9, 7]$
Pivot	–	2	5	Ricollocazione	SWAP$(S[0], S[2])$, SWAP$(S[7], S[5])$	$[\mathbf{1}, \mathbf{2} \mid \mathbf{3} \mid \mathbf{5}, \mathbf{4} \mid \mathbf{7} \mid \mathbf{8}, \mathbf{9}]$

Tabella 5.2: Traccia puntuale passo-passo del partizionamento Dual-Pivot con pivot $P = 3$ e $Q = 7$. Al termine, i due pivot sono collocati nelle posizioni definitive (indici 2 e 5) e le chiamate ricorsive procedono separatamente sulle tre sotto-regioni.

Proprietà 5.3.1 (Vantaggi Architetturali del Dual-Pivot Quicksort)**.** *L'adozione di due pivot comporta significativi benefici a livello di ingegneria algoritmica:*

- ***Minore profondità dell'albero di ricorsione:*** *Suddividendo lo spazio in 3 parti anziché in 2, l'altezza media dell'albero di ricorsione si riduce da* $\log_2 n \approx 1.44 \ln n$ *a* $\log_3 n \approx 0.91 \ln n$*, determinando una riduzione del numero complessivo di scansioni della memoria.*

- ***Efficienza di Cache e CPU pipelining:*** *Sebbene il numero teorico di confronti per elemento aumenti leggermente rispetto al Quicksort classico ottimizzato (*$1.9n \ln n$ *contro* $2n \ln n$*), il numero di scritture e di cache miss si riduce marcatamente, sfruttando al massimo la banda di trasferimento della gerarchia di memoria.*

5.4 Bounded Quicksort (BQS) ed Eliminazione della Tail Recursion

Nonostante l'eccellente tempo atteso $\mathcal{O}(n \log_2 n)$, l'implementazione ricorsiva ordinaria di Quicksort presenta una grave criticità legata al consumo di memoria ausiliaria sullo stack di esecuzione (*call stack*).

5.4.1 La Criticità dello Stack di Ricorsione

A ogni chiamata ricorsiva, il sistema operativo alloca un frame di stack contenente i parametri della funzione, le variabili locali e l'indirizzo di ritorno:

- Nel caso ottimo o bilanciato, lo stack raggiunge una profondità pari a $\Theta(\log_2 n)$, richiedendo uno spazio ausiliario trascurabile.
- Tuttavia, qualora il partizionamento risulti costantemente sbilanciato (caso peggiore: pivot posizionato all'estremo 1 contro $n-1$), la profondità dello stack cresce linearmente con la dimensione dell'input:

$$\text{Profondità Stack} = \Theta(n). \tag{5.37}$$

  Per istanze di grandi dimensioni ($n \geq 10^6$), una profondità di stack lineare genera inevitabilmente l'interruzione anomala del programma per esaurimento dello stack (*Stack Overflow Exception*).

5.4.2 Algoritmo Bounded Quicksort (BQS)

Il **Bounded Quicksort (BQS)** risolve questa vulnerabilità strutturale attraverso due tecniche cardine dell'ingegneria del software:

1. **Eliminazione della Ricorsione in Coda (*Tail Call Elimination*):** l'algoritmo non esegue mai due chiamate ricorsive all'interno della stessa funzione. Esegue la ricorsione **esclusivamente sulla partizione di dimensione inferiore o uguale alla metà dell'intervallo corrente** ($\leq \frac{j-i}{2}$). La partizione rimanente (la più grande) viene elaborata in modo puramente iterativo aggiornando gli indici i o j all'interno di un ciclo 'while'.
2. **Cutoff ad Insertion Sort su blocchi piccoli:** quando l'ampiezza dell'intervallo scende al di sotto di una soglia predefinita m_0 (tipicamente $m_0 \in [16, 32]$), la ricorsione viene interrotta a favore dell'ordinamento per inserzione.

Algorithm 4: Bounded Quicksort (BQS)

Dati: Array S, indice iniziale i, indice finale j, soglia m_0
Risultato: Sottoarray $S[i \dots j]$ ordinato

while $(j - i) \geq m_0$ **do**
  $r \leftarrow \text{RANDOM_POSITION}(i, \dots, j)$
  $\text{SWAP}(S[r], S[i])$ `// Pivot casuale portato all'inizio`
  $p' \leftarrow \text{PARTITION}(S, i, j)$ `// p' è la posizione finale del pivot`
  if $p' \leq \lfloor (i + j)/2 \rfloor$ **then**
    `// La partizione SINISTRA è la più piccola: ricorsione`
    $\text{BQS}(S, i, p' - 1, m_0)$
    `// Eliminazione tail recursion sulla DESTRA: iterazione`
    $i \leftarrow p' + 1$
  else
    `// La partizione DESTRA è la più piccola: ricorsione`
    $\text{BQS}(S, p' + 1, j, m_0)$
    `// Eliminazione tail recursion sulla SINISTRA: iterazione`
    $j \leftarrow p' - 1$
$\text{INSERTION_SORT}(S, i, j)$ `// Risoluzione blocchi piccoli di dimensione` $< m_0$

In Figura 5.4 viene illustrato il diagramma di flusso del controllo operativo di BQS, mettendo in risalto la diramazione tra ricorsione vincolata e continuazione iterativa.

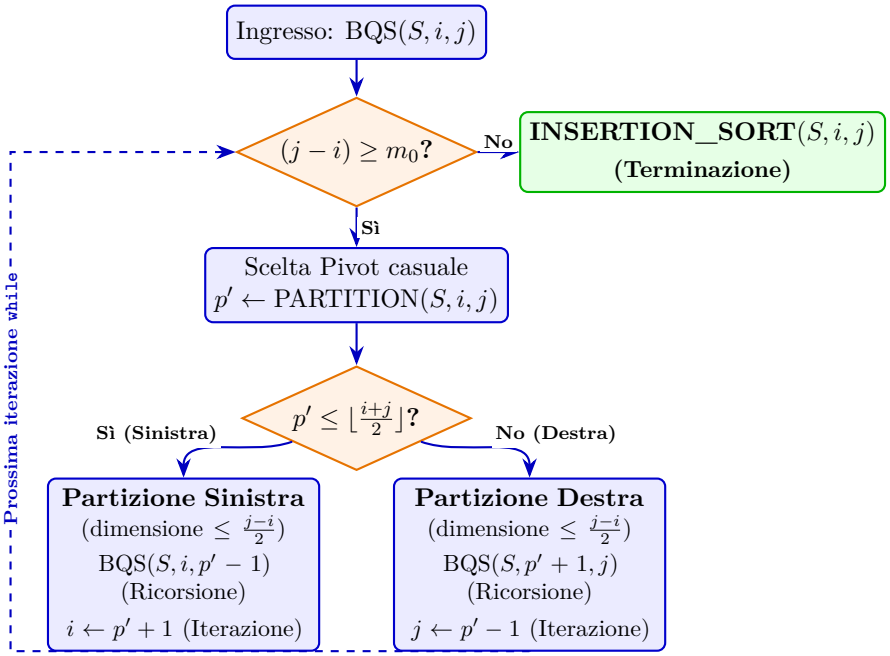


Figura 5.4: Diagramma di controllo del Bounded Quicksort (BQS). La chiamata ricorsiva viene lanciata rigidamente sulla metà di dimensione minore ($\leq (j - i)/2$), mentre la partizione più grande viene elaborata mediante iterazione continua nel ciclo `while`. I blocchi inferiori a m_0 sono ordinati direttamente da Insertion Sort.

5.4.3 Esempio Esecutivo e Gestione dello Stack nel Bounded Quicksort

Per comprendere a fondo il meccanismo di confinamento della profondità dello stack e l'eliminazione della ricorsione in coda (*tail recursion elimination*), consideriamo l'esecuzione dell'algorithm 4 su un array di 8 elementi con soglia di cutoff $m_0 = 3$:

$$S = [7,\ 2,\ 8,\ 4,\ 1,\ 6,\ 3,\ 5]. \tag{5.38}$$

In uno scenario sfavorevole in cui il partizionamento risulta costantemente sbilanciato, un'implementazione ordinaria di Quicksort allocherebbe una sequenza lineare di frame sullo stack di chiamata, raggiungendo una profondità $\Theta(n)$. Al contrario, Bounded Quicksort garantisce che la partizione più voluminosa venga sempre elaborata all'interno dello stesso frame mediante continuazione ciclica (aggiornando i puntatori i o j), mentre la ricorsione viene invocata unicamente sulla partizione di dimensione non superiore alla metà ($\leq L/2$).

In Figura 5.5 vengono illustrati i passaggi esecutivi e il confronto strutturale della memoria dello stack tra Quicksort classico e Bounded Quicksort.

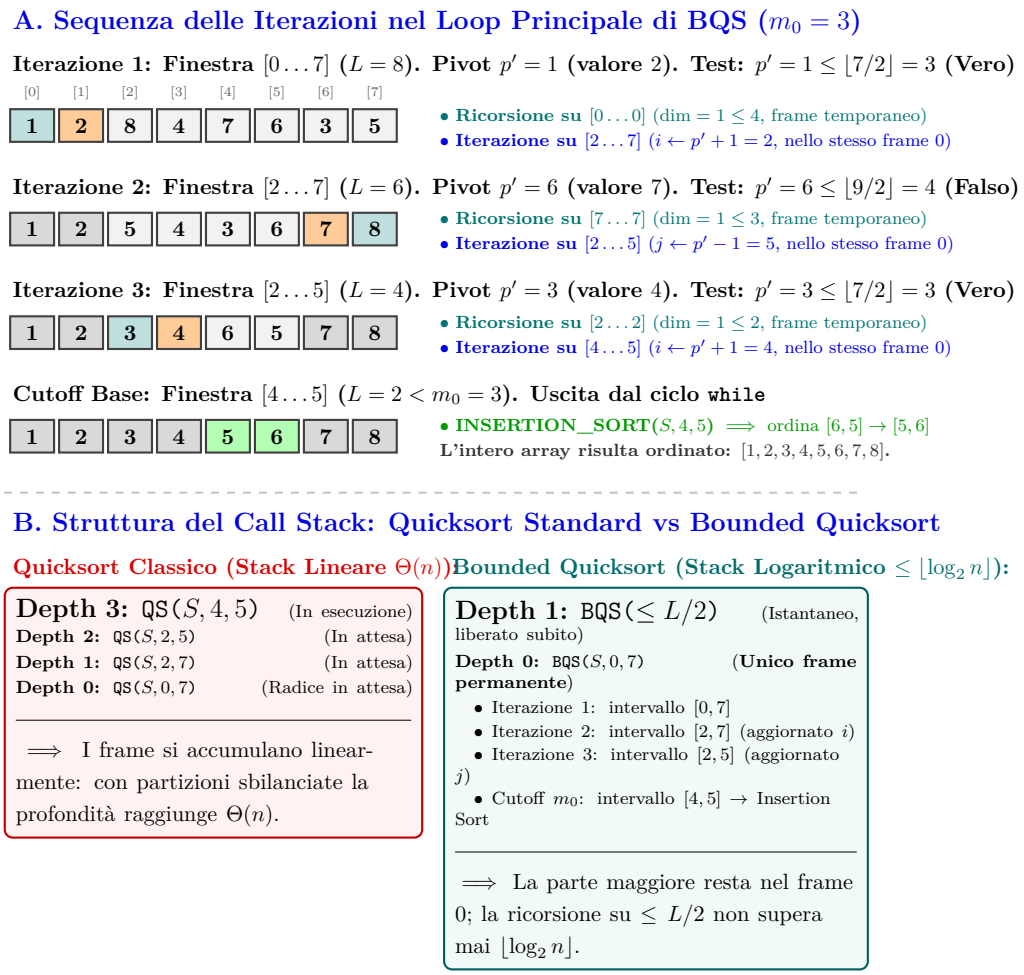


Figura 5.5: Traccia dell'esecuzione di Bounded Quicksort (BQS) con soglia $m_0 = 3$ e confronto della memoria di stack. (**A**) Ad ogni iterazione, la partizione sbilanciata maggiore viene mantenuta nello stesso stack frame aggiornando i o j, mentre solo la partizione dimezzata ($\le L/2$) genera una breve chiamata ricorsiva. (**B**) Il Quicksort ordinario accumula frame lineari; BQS garantisce uno stack a profondità rigorosamente logaritmica.

In Tabella 5.3 viene dettagliata la sequenza puntuale delle iterazioni, evidenziando il test sulla partizione minima, l'azione di tail call elimination e la profondità massima istantanea dello stack di esecuzione.

Passo	$[i, j]$	L	Test Pivot	Azione (Ricorsione & Iterazione)	Stato Array	Stack
Init	$[0, 7]$	8	–	Frame radice BQS$(S, 0, 7)$	$[7, 2, 8, 4, 1, 6, 3, 5]$	0
1	$[0, 7]$	8	$p' = 1 \leq 3$	**Rec:** $[0, 0]$ (dim ≤ 4, istantanea) **Iter:** $i \leftarrow 2$ su $[2, 7]$ (stesso frame)	$[1 \mid \mathbf{2} \mid 8, 4, 7, 6, 3, 5]$	1
2	$[2, 7]$	6	$p' = 6 > 4$	**Rec:** $[7, 7]$ (dim ≤ 3, istantanea) **Iter:** $j \leftarrow 5$ su $[2, 5]$ (stesso frame)	$[1, 2 \mid 5, 4, 3, 6 \mid \mathbf{7} \mid 8]$	1
3	$[2, 5]$	4	$p' = 3 \leq 3$	**Rec:** $[2, 2]$ (dim ≤ 2, istantanea) **Iter:** $i \leftarrow 4$ su $[4, 5]$ (stesso frame)	$[1, 2 \mid 3 \mid \mathbf{4} \mid 6, 5 \mid 7 \mid 8]$	1
Cutoff	$[4, 5]$	2	$L < m_0$	**Exit:** Uscita dal ciclo `while` **Sort:** Insertion Sort su $[4, 5]$	$[1, 2, 3, 4, \mathbf{5}, \mathbf{6}, 7, 8]$	0

Tabella 5.3: Traccia dell'esecuzione passo-passo di Bounded Quicksort (BQS) con soglia di cutoff $m_0 = 3$. In ogni iterazione, la partizione di dimensione minore o uguale a $L/2$ viene risolta con una chiamata ricorsiva limitata, mentre la partizione più grande viene elaborata iterativamente all'interno del frame corrente, mantenendo la profondità dello stack ≤ 1 nell'esempio ($\leq \lfloor \log_2 n \rfloor$ nel caso generale).

5.4.4 Analisi Rigorosa dello Spazio Extra $\Theta(\log_2 n)$

L'aspetto fondamentale di BQS risiede nella garanzia assoluta sul consumo di memoria di sistema:

Teorema 5.4.1 (Garanzia dello Spazio Ausiliario di Bounded Quicksort)**.** *Per qualsiasi input di n elementi, l'algoritmo Bounded Quicksort alloca uno spazio aggiuntivo sullo stack di sistema rigidamente limitato a:*

$$\textit{Spazio Ausiliario} = \Theta(\log_2 n). \tag{5.39}$$

La profondità massima dello stack non eccede $\lfloor \log_2 n \rfloor$ nel caso peggiore assoluto.

Dimostrazione. Sia L la lunghezza della finestra corrente dell'array all'ingresso di una chiamata a BQS, definita da $L = j - i + 1$. Quando il pivot viene posizionato nell'indice p', l'algoritmo effettua una diramazione esaminando la dimensione delle due partizioni:

- La partizione sinistra ha ampiezza $L_{\text{left}} = (p' - 1) - i + 1 = p' - i$.
- La partizione destra ha ampiezza $L_{\text{right}} = j - (p' + 1) + 1 = j - p'$.

La condizione del test $p' \leq \lfloor \frac{i+j}{2} \rfloor$ impone che:

$$p' - i \leq \frac{j - i}{2} < \frac{L}{2}. \tag{5.40}$$

Pertanto, se la condizione è verificata, la chiamata ricorsiva viene invocata sulla partizione sinistra, la cui dimensione è strettamente limitata da:

$$L_{\text{ricorsivo}} \leq \left\lfloor \frac{L}{2} \right\rfloor. \tag{5.41}$$

Nel caso complementare ($p' > \lfloor \frac{i+j}{2} \rfloor$), la partizione destra soddisfa analogamente $j - p' < \frac{L}{2}$, e la chiamata ricorsiva viene invocata sulla partizione destra, verificando la medesima disequazione.

La partizione rimanente possiede dimensione $L_{\text{iterativo}} \geq \frac{L}{2}$, ma essa **non genera alcuna chiamata ricorsiva**: l'algoritmo aggiorna semplicemente il puntatore $i \leftarrow p' + 1$ oppure $j \leftarrow p' - 1$ e itera nel corpo del ciclo 'while' senza allocare alcun nuovo stack frame.

Ne consegue che la dimensione della sotto-istanza associata a ciascuna chiamata ricorsiva nidificata sullo stack si riduce di almeno un fattore 2 rispetto al frame genitore:

$$L^{(k)} \leq \frac{n}{2^k}, \tag{5.42}$$

dove k rappresenta la profondità corrente dello stack. Il processo di ricorsione può proseguire al massimo fintanto che $L^{(k)} \geq m_0 \geq 1$:

$$\frac{n}{2^k} \geq 1 \implies 2^k \leq n \implies k \leq \lfloor \log_2 n \rfloor. \tag{5.43}$$

Poiché ciascun record di attivazione richiede una quantità costante di memoria $\mathcal{O}(1)$, il consumo totale di memoria ausiliaria è rigorosamente limitato a:

$$\text{Extra Space} = \mathcal{O}(1) \cdot \lfloor \log_2 n \rfloor = \Theta(\log_2 n). \tag{5.44}$$

□

5.4.5 Ottimizzazione Ibrida con Cutoff a Insertion Sort

L'introduzione della soglia $m_0 \in [16, 32]$ per il passaggio ad *Insertion Sort* risponde a un criterio fondamentale di *cache locality* ed efficienza esecutiva:

- **Eliminazione dell'overhead ricorsivo:** Quando la dimensione del sottoarray è inferiore a m_0, il costo associato alla chiamata di funzione, al partizionamento e alla gestione dei puntatori supera il costo computazionale dei confronti diretti.
- **Efficienza di Insertion Sort sui piccoli blocchi:** *Insertion Sort* possiede un fattore costante di tempo eccezionalmente ridotto: l'algoritmo non necessita di memoria ausiliaria, esegue istruzioni elementari all'interno di un ciclo compatto e beneficia dell'inlining del codice da parte del compilatore.

- **Sfruttamento ottimale della Cache L1:** Un blocco di $16 \div 32$ elementi occupa circa $128 \div 256$ byte (per elementi da 64 bit), risiedendo integralmente all'interno di una o al massimo due linee di cache L1 (64 byte per linea). Di conseguenza, tutti i confronti e gli scambi avvengono senza generare alcun cache miss, massimizzando il throughput della CPU.
- **Comportamento con quasi-ordinamento:** Le macro-partizioni posizionano gli elementi vicino alla collocazione finale: Insertion Sort sui blocchi ridotti opera su dati quasi ordinati, richiedendo tempo lineare $\mathcal{O}(n)$.

In conclusione, Bounded Quicksort rappresenta una perfetta sintesi di rigore teorico e ingegneria algoritmica, garantendo la complessità temporale attesa ottima $\mathcal{O}(n \log_2 n)$, l'immunità da *Stack Overflow* grazie allo spazio extra $\Theta(\log_2 n)$, e la massima velocità empirica su hardware moderno.

Capitolo 6

Statistiche d'Ordine, Trade-off Multi-Pivot e Analisi Formale del Caso Medio di Quicksort

6.1 Il Problema della Selezione (*Selection of the k-th Item*)

Il problema della selezione costituisce una delle primitive fondamentali dell'informatica teorica e dell'ingegneria degli algoritmi. A differenza del problema dell'ordinamento, in cui l'obiettivo primario è riorganizzare globalmente una sequenza di dati in modo non decrescente, il problema della selezione richiede di identificare un singolo elemento caratterizzato da una determinata proprietà di rango senza dover necessariamente ordinare l'intero insieme.

6.1.1 Formulazione Formale del Problema

Definizione 6.1.1 (k-esima Statistica d'Ordine)**.** Sia $S = [s_1, s_2, \ldots, s_n]$ un vettore o sequenza disordinata di n elementi appartenenti a un dominio totalmente ordinato $(\mathcal{U}, \leq)$, e sia $k \in \{1, 2, \ldots, n\}$ un indice target specificato. La **k-esima statistica d'ordine** di S, comunemente denotata con $S_{(k)}$ o $\text{Select}(S, k)$, è l'elemento che occuperebbe la posizione di indice k qualora la sequenza S venisse disposta in ordine non decrescente:

$$S_{\text{ordinato}} = [s_{(1)}, s_{(2)}, \ldots, s_{(n)}] \quad \text{con } s_{(1)} \leq s_{(2)} \leq \cdots \leq s_{(n)}. \tag{6.1}$$

Pertanto, $\text{Select}(S, k) = S_{\text{ordinato}}[k]$.

Esempio 6.1.1 (Istanza di Esempio Didattica)**.** Si consideri un vettore non ordinato S di cardinalità $n = 6$:

Indice (1-based):	1	2	3	4	5	6
$S =$	2	7	5	3	1	10
$S_{\text{ordinato}} =$	1	2	3	5	7	10

Dall'analisi delle statistiche d'ordine si evince:

- Per $k = 1$: l'obiettivo consiste nell'estrarre il **minimo** assoluto della collezione:
$$S_{(1)} = S_{\text{ordinato}}[1] = 1. \tag{6.2}$$
- Per $k = n = 6$: l'obiettivo consiste nell'individuare il **massimo** assoluto della collezione:
$$S_{(6)} = S_{\text{ordinato}}[6] = 10. \tag{6.3}$$
- Per $k = 3$: la terza statistica d'ordine corrisponde al valore $S_{(3)} = 3$.
- Per $k = \lceil n/2 \rceil = 3$ o $k = \lfloor n/2 \rfloor + 1 = 4$: si ottiene la **mediana** dell'insieme (rispettivamente 3 o 5).

6.1.2 Casi Limite: Ricerca di Minimo e Massimo

Per i casi limite $k = 1$ e $k = n$, la determinazione dell'elemento target non necessita di strutture dati complesse. È sufficiente una scansione lineare iterativa dell'array memorizzando in un registro temporaneo il candidato corrente:

- Si inizializza una variabile temporanea min $\leftarrow S[1]$.
- Si scorrono consecutivamente le restanti $n - 1$ celle da $i = 2$ a n, eseguendo a ogni iterazione il confronto $S[i] <$ min: se la condizione è verificata, si aggiorna min $\leftarrow S[i]$.
- Il numero totale di confronti eseguiti è esattamente pari a $n - 1$.

Proprietà 6.1.1 (Ottimalità Asintotica per $k = 1$ e $k = n$)**.** *Nel modello computazionale basato su confronti, determinare il minimo (o il massimo) di una collezione disordinata di n elementi richiede nel caso peggiore almeno* $n - 1$ *confronti. Pertanto, l'algoritmo di scansione lineare garantisce un costo ottimo:*
$$T(n) = \Theta(n), \quad \textit{con spazio ausiliario } \mathcal{O}(1). \tag{6.4}$$

Dimostrazione per Argomento dell'Avversario. Ogni confronto tra due elementi distinti a e b può eliminare al più un solo elemento dal ruolo di potenziale minimo (l'elemento che risulta maggiore nell'esito del confronto). Poiché all'inizio vi sono n potenziali candidati e alla conclusione deve rimanerne esattamente uno, devono essere eliminati almeno $n-1$ candidati. Ciascun confronto elimina al massimo un candidato; pertanto sono necessari almeno $n-1$ confronti. □

Proprietà 6.1.2 (Ricerca Simultanea di Minimo e Massimo)**.** *Se il problema richiede di individuare simultaneamente sia il minimo ($k=1$) che il massimo ($k=n$), l'esecuzione disgiunta di due scansioni lineari compirebbe $2(n-1)$ confronti. È tuttavia possibile processare gli elementi a coppie:*

1. *Si confrontano gli elementi adiacenti $S[2i]$ e $S[2i+1]$ tra loro (1 confronto).*
2. *Il minore della coppia viene confrontato con il minimo corrente (1 confronto).*
3. *Il maggiore della coppia viene confrontato con il massimo corrente (1 confronto).*

Questo approccio riduce il costo a $\lceil 3n/2 \rceil - 2$ confronti, conseguendo un risparmio del 25% rispetto all'approccio ingenuo.

6.1.3 Strategie Risolutive per k Generico

Quando l'indice target $k \in \{1, \ldots, n\}$ assume un valore generico, la scansione lineare a singolo registro non è più sufficiente, poiché occorre tenere traccia di molteplici elementi concorrenti. In letteratura e nelle applicazioni industriali si distinguono tre strategie fondamentali:

Tabella 6.1: Confronto analitico tra le strategie risolutive classiche per il problema della selezione.

Approccio	Algoritmo / Struttura	Complessità Tempo	Descrizione Architetturale e Meccanismo
1. Ordinamento Globale	Merge Sort / Heap Sort	$\mathcal{O}(n \log_2 n)$	Ordina interamente l'array S; restituisce $S_{\text{ordinato}}[k]$ in tempo $\mathcal{O}(1)$. Sovradimensionato se $k \ll n$.
2. Ordinamento Parziale	Finestra con Insertion Sort	$\mathcal{O}(k \cdot n)$ (*Worst-Case*)	Mantiene una finestra ordinata di dimensione k dei minimi. Ogni nuovo elemento richiede lo scorrimento di al più k posizioni.
3. Coda con Priorità Limitata	Max-Heap di taglia k	$\mathcal{O}(n \log_2 k)$	Mantiene un Max-Heap con i k elementi minimi visti finora. Il massimo dei minimi risiede nella radice e viene filtrato in $\mathcal{O}(\log_2 k)$.

Strategia 1: Ordinamento Globale (*Full Sorting*)

L'approccio più immediato consiste nell'ordinare l'intera sequenza S mediante un algoritmo di ordinamento basato su confronti ottimale, come Merge Sort o Heap Sort, impiegando $\mathcal{O}(n \log_2 n)$ operazioni nel caso peggiore. Una volta che l'array è ordinato, la k-esima statistica d'ordine coincide immediatamente con la cella in posizione k, accessibile in $\mathcal{O}(1)$.

- **Vantaggi:** Estremamente semplice da implementare; ottimale qualora l'applicazione debba rispondere successivamente a molteplici interrogazioni di selezione su indici k differenti (*multi-query workload*).
- **Svantaggi:** Esegue una quantità elevata di lavoro superfluo. Se il sistema necessita unicamente di una specifica statistica d'ordine (ad esempio il percentile 95-esimo o la mediana), ordinare le porzioni di array

che non influenzano la posizione k introduce un overhead computazionale evitabile.

Strategia 2: Ordinamento Parziale e Incrementale (*Insertion Window*)

Qualora k sia sensibilmente inferiore alla cardinalità complessiva n ($k \ll n$), non è necessario ordinare l'intero vettore. Si può mantenere un buffer ausiliario ordinato W di capacità pari a k, contenente i k elementi più piccoli incontrati durante la scansione:

1. Si inseriscono i primi k elementi dell'input in W e si ordinano in tempo $\mathcal{O}(k \log_2 k)$ o $\mathcal{O}(k^2)$.
2. Per ciascuno dei successivi $n - k$ elementi $x \in S[k+1 \dots n]$:
   - Si confronta x con il valore massimo del buffer, situato all'estremità destra $W[k]$.
   - Se $x \geq W[k]$, l'elemento x è maggiore o uguale a ciascuno dei k minimi attuali; pertanto viene immediatamente scartato con un singolo confronto $\mathcal{O}(1)$.
   - Se $x < W[k]$, x appartiene ai nuovi k minimi. Si elimina l'elemento massimo $W[k]$ e si inserisce x all'interno di W mediante lo scorrimento all'indietro tipico di *Insertion Sort*, con un costo pari a $\mathcal{O}(k)$ confronti e spostamenti.

Nel caso peggiore (ad esempio una sequenza strettamente decrescente), ogni nuovo elemento deve essere inserito in testa a W, portando la complessità complessiva a:

$$T(n) = \mathcal{O}(k^2) + \sum_{i=k+1}^{n} \mathcal{O}(k) = \mathcal{O}(k \cdot n). \tag{6.5}$$

Se k è una costante limitata (es. $k \in \{2, 3, 5\}$), il tempo di calcolo è rigorosamente lineare $\mathcal{O}(n)$. Tuttavia, quando $k = \Theta(n)$ (come nel calcolo della mediana $k = n/2$), la complessità degenera a $\mathcal{O}(n^2)$, risultando nettamente peggiore dell'ordinamento globale.

Strategia 3: Coda con Priorità Limitata (*Bounded Max-Heap*)

La Strategia 3 risolve le inefficienze dell'ordinamento parziale sostituendo il buffer sequenziale con un **Max-Heap di dimensione fissa** k.

Proprietà 6.1.3 (Invariante del Bounded Max-Heap)**.** *In qualsiasi istante dell'elaborazione, dopo aver esaminato un prefisso di dimensione $m \geq k$ dell'array originale, il Max-Heap memorizza esattamente i k elementi con valore minore estratti dal sottoinsieme $\{S[1], \ldots, S[m]\}$. La radice dell'heap (top()) coincide sempre con il massimo di questi k elementi, ovvero con il candidato corrente per la k-esima statistica d'ordine.*

Il meccanismo operativo viene illustrato nel diagramma di flusso in Figura 6.1.

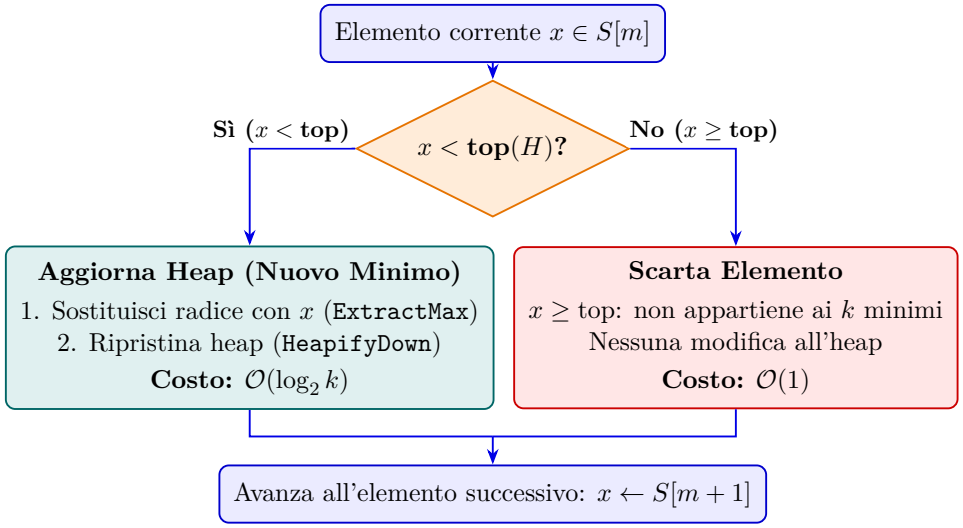


Figura 6.1: Flusso di elaborazione con Max-Heap di dimensione limitata k. Gli elementi che non migliorano i k minimi correnti vengono scartati in tempo costante $\mathcal{O}(1)$.

L'algoritmo formale viene sintetizzato nell'algorithm 5.

Algorithm 5: Selezione con Max-Heap Limitato (*BoundedHeapSelect*)

Dati: Array S di dimensione n, indice target $k \in \{1, \ldots, n\}$
Risultato: Il valore della k-esima statistica d'ordine $S_{(k)}$

```
Inizializza un Max-Heap vuoto H di capacità massima k
for i ← 1 to k do
    H.Insert(S[i])          // Costruzione iniziale in tempo O(k)
for i ← k + 1 to n do
    if S[i] < H.Top() then
        H.ExtractMax()
        H.Insert(S[i])      // Sostituzione e ripristino in tempo
         O(log_2 k)
return H.Top()              // La radice contiene esattamente S_(k)
```

Esempio 6.1.2 (Traccia Esecutiva Passo-Passo del Bounded Max-Heap)**.** Si applichi il algorithm 5 sull'array $S = [2, 7, 5, 3, 1, 10]$ con indice target $k = 3$. L'obiettivo consiste nell'individuare il 3-terzo elemento più piccolo (valore teorico atteso: 3).

Tabella 6.2: Traccia dettagliata dello stato dell'heap per ciascun elemento esaminato ($k = 3$).

Passo	$S[i]$	Confronto con Radice	Azione Intrapresa	Stato Heap H (Top)
1	2	Costruzione iniziale	Inserimento in H	$[2]$
2	7	Costruzione iniziale	Inserimento in H	$[7, 2]$
3	5	Costruzione iniziale	Inserimento in H	$[7, 2, 5]$ (Top $= 7$)
4	3	$3 < 7$ (Sì)	Estrai 7, Inserisci 3	$[5, 2, 3]$ (Top $= 5$)
5	1	$1 < 5$ (Sì)	Estrai 5, Inserisci 1	$[3, 2, 1]$ (Top $= 3$)
6	10	$10 < 3$ (No)	Scarta 10 in $\mathcal{O}(1)$	$[3, 2, 1]$ (Top $= 3$)

Al termine della scansione, la radice del Max-Heap contiene il valore 3, corrispondente in modo esatto a $S_{(3)} = S_{\text{ordinato}}[3]$.

Proprietà 6.1.4 (Analisi della Complessità Temporale e Spaziale)**.** *La costruzione iniziale dell'heap sui primi k elementi richiede tempo lineare $\mathcal{O}(k)$ mediante l'algoritmo di Floyd (`BuildMaxHeap`). Per ciascuno dei successivi $n - k$ elementi, l'algoritmo compie un confronto $\mathcal{O}(1)$; qualora l'elemento sia inferiore alla radice, il costo di ripristino delle proprietà dell'heap è pari a*

$\mathcal{O}(\log_2 k)$. *Nel caso peggiore, il tempo complessivo di esecuzione è pari a:*

$$T(n) = \mathcal{O}(k) + (n-k) \cdot \mathcal{O}(\log_2 k) = \mathcal{O}(n \log_2 k). \tag{6.6}$$

Lo spazio ausiliario occupato è limitato a $\Theta(k)$ locazioni di memoria, rendendo l'algoritmo ideale per scenari di ***data stream*** *in cui $n \gg k$ e l'intero vettore non può risiedere simultaneamente nella cache o nella memoria centrale.*

6.2 Il Dilemma del Multi-Pivot (*Why Not Many Pivots?*)

Nei paradigmi di ripartizione distributiva e negli algoritmi di ordinamento di tipo *Divide et Impera*, la scelta di quanti punti di divisione (*pivot*) impiegare rappresenta una decisione progettuale cruciale. Intuitivamente, suddividere un problema di taglia n in un numero elevato di sotto-problemi permette di contrarre sensibilmente la profondità dell'albero di ricorsione.

6.2.1 L'Illusione Teorica dell'Albero a Molti Rami

Supponiamo di generalizzare lo schema classico di Quicksort selezionando $p \geq 1$ pivot distinti ed ordinati:

$$p_1 < p_2 < \cdots < p_p. \tag{6.7}$$

Tali pivot delimitano nello spazio delle chiavi $p+1$ partizioni disgiunte:

$$(-\infty, p_1), \quad [p_1, p_2), \quad [p_2, p_3), \quad \ldots, \quad [p_p, +\infty). \tag{6.8}$$

Se assumiamo che i pivot siano distribuiti in modo perfettamente bilanciato, ciascun sotto-array conterrà circa $n/(p+1)$ elementi. L'altezza dell'albero di ricorsione si riduce conseguentemente a:

$$h(n) \approx \log_{p+1} n = \frac{\ln n}{\ln(p+1)}. \tag{6.9}$$

Se fosse possibile incrementare p a piacimento (ad esempio ponendo $p = \sqrt{n}$ o $p = \log_2 n$), l'altezza dell'albero collasserebbe a valori estremamente bassi. Tuttavia, negli appunti e nella pratica ingegneristica viene posto il quesito cardine:

"Why not many pivots?" $\implies$ Increase cost of sorting and partitioning.

6.2.2 Costo di Selezione dei Pivot

Per operare una suddivisione bilanciata in $p+1$ intervalli, non è sufficiente estrarre p valori casuali: occorre che tali p pivot siano mutuamente ordinati e rappresentativi della distribuzione statistica dell'input.

- Se i p pivot vengono campionati casualmente, devono essere preliminarmente ordinati tra loro, con un costo locale di $\mathcal{O}(p \log_2 p)$ confronti.
- Se si richiede che i pivot garantiscano frazioni dimensionali bilanciate (ad esempio i percentili esatti), occorre invocare algoritmi deterministici di selezione (quali *Median-of-Medians* di Blum et al. [6]), le cui costanti moltiplicative computazionali superano ampiamente il beneficio derivante dal partizionamento multiplo.

6.2.3 Costo di Classificazione Interna per Elemento

Il fattore maggiormente penalizzante risiede nel numero di confronti necessari per classificare ciascuno degli n elementi all'interno del corretto intervallo tra i $p+1$ disponibili:

1. **Classificazione Lineare Sequenziale:** Se per ogni elemento si esegue una scansione sequenziale confrontandolo progressivamente con $p_1, p_2, \ldots, p_p$, il numero medio di confronti per elemento a ogni livello di ricorsione è proporzionale a p. Il numero totale di confronti effettuati sull'intero albero risulta:

$$C_{\text{tot}}(n) \approx n \cdot p \cdot \frac{\ln n}{\ln(p+1)} = n \ln n \cdot \left(\frac{p}{\ln(p+1)} \right). \tag{6.10}$$

   La funzione $f(p) = \frac{p}{\ln(p+1)}$ è **strettamente crescente** per ogni intero $p \geq 1$:

   - Per $p = 1$: $f(1) = \frac{1}{\ln 2} \approx 1{,}442$.
   - Per $p = 2$: $f(2) = \frac{2}{\ln 3} \approx 1{,}820$ (+26% di confronti rispetto a $p = 1$).
   - Per $p = 7$: $f(7) = \frac{7}{\ln 8} \approx 3{,}366$ (+133% di confronti).

   All'aumentare di p, il numero complessivo di confronti necessari cresce sensibilmente, annullando il vantaggio dell'albero più basso.

2. **Classificazione mediante Ricerca Binaria:** Se per collocare l'elemento nei $p+1$ rami si impiega una ricerca binaria sui p pivot, il

numero di confronti per elemento è pari a $\lceil \log_2(p+1) \rceil$. Moltiplicando per la profondità dell'albero:

$$C_{\text{tot}}(n) \approx n \cdot \log_2(p+1) \cdot \frac{\log_2 n}{\log_2(p+1)} = n \log_2 n. \tag{6.11}$$

Asintoticamente il numero totale di confronti teorici **non diminuisce affatto**, rimanendo invariante rispetto al caso monotematico a singolo pivot.

6.2.4 Colli di Bottiglia Architetturali e Hardware

Nei moderni calcolatori, le prestazioni degli algoritmi non dipendono unicamente dal conteggio algebrico dei confronti, ma dalle interazioni con la microarchitettura della CPU (gerarchia di memoria cache, registri, linee di pipeline e predittori di salto):

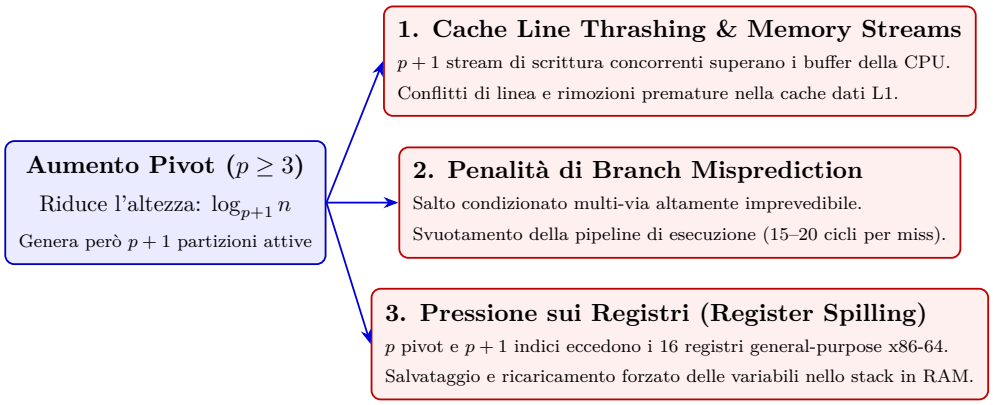


Figura 6.2: Cause microarchitetturali del decadimento prestazionale all'aumentare del numero di pivot p.

- **Stream di Scrittura e Cache Lines:** Con 1 solo pivot si hanno 2 flussi continui di scrittura in memoria (uno dall'estremo sinistro e uno dal destro), che sfruttano appieno i buffer di scrittura sequenziale (*Write-Combining Buffers*). Con p pivot, l'algoritmo deve gestire $p+1$ stream contemporanei. Poiché le cache L1 possiedono un'associatività limitata (solitamente 8 vie), $p+1 \geq 4$ stream concorrenti generano conflitti sistematici di linea di cache (*cache eviction* premature).
- **Predizione dei Salti (*Branch Prediction*):** La classificazione dicotomica ($x < p$) viene predetta con elevata accuratezza dai predittori dinamici moderni. Al contrario, ramificazioni a 3 o più vie creano salti

condizionati difficilmente prevedibili: ogni errore di predizione (*branch misprediction*) comporta una penalità severa di 15–20 cicli di clock sprecati per lo svuotamento della pipeline.

- **Pressione sui Registri (*Register Spilling*):** Su architetture x86-64 vi sono 16 registri ad uso generale. Gestire p valori di pivot, $p+1$ indici di delimitazione, i puntatori di base e le costanti locali eccede il set di registri disponibili quando $p \geq 3$, costringendo il compilatore a riversare temporaneamente le variabili nello stack (*register spilling*), moltiplicando gli accessi lenti in memoria.

Proprietà 6.2.1 (Perché il Dual-Pivot ($p = 2$) Funziona nella Pratica?)**.** *L'algoritmo Dual-Pivot di Yaroslavskiy [5] (adottato nei runtime di Java e C++) rappresenta il punto di compromesso ottimale: sebbene compia mediamente circa il* 5% *in più di confronti rispetto al Quicksort classico a 1 pivot (*$1{,}9n \ln n$ *contro* $2n \ln n$*),* ***riduce del*** 20% ***le operazioni di memorizzazione e swap in memoria****, consentendo una scansione sequenziale efficiente che si adatta perfettamente alla gerarchia di cache moderna. Per* $p \geq 3$*, i costi descritti superano nettamente i benefici.*

6.3 Quicksort: Partizionamento a 3 Vie e Implicazioni Ricorsive

Come introdotto nel Capitolo 5 (sezione 5.3), uno dei problemi più critici del Quicksort tradizionale a due vie risiede nella gestione degli elementi duplicati: se tutti gli elementi sono identici, le partizioni convenzionali a due vie (come lo schema di Lomuto) isolano un solo elemento per passo, producendo un albero di ricorsione lineare di altezza $\Theta(n)$ e degradando il tempo di calcolo a $\Theta(n^2)$.

La soluzione strutturale risiede nel partizionamento a tre vie (*Dutch National Flag*) introdotto da Dijkstra [3]. Mentre nel Capitolo 5 (sottosezione 5.3.2) sono stati esaminati nel dettaglio il meccanismo a tre puntatori (lt, i, gt), il relativo algoritmo in-place e la traccia esecutiva puntuale (si veda la Tabella 5.1 a pagina 52), in questa sezione approfondiamo il **ruolo sistemico del partizionamento a 3 vie nella ricorsione**: gli invarianti dimensionali, l'esclusione della fascia centrale, l'applicazione immediata alla selezione lineare (*Quickselect*) e il raccordo con la ricorrenza del caso medio.

6.3.1 Topologia delle Regioni e Invarianti Dimensionali

Dato un array $S[\text{lo} \dots \text{hi}]$ di cardinalità n e selezionato un pivot p, la procedura a 3 vie riorganizza gli elementi in tre comparti contigui, delimitati dagli indici finali lt e gt, come illustrato nella Figura 6.3.

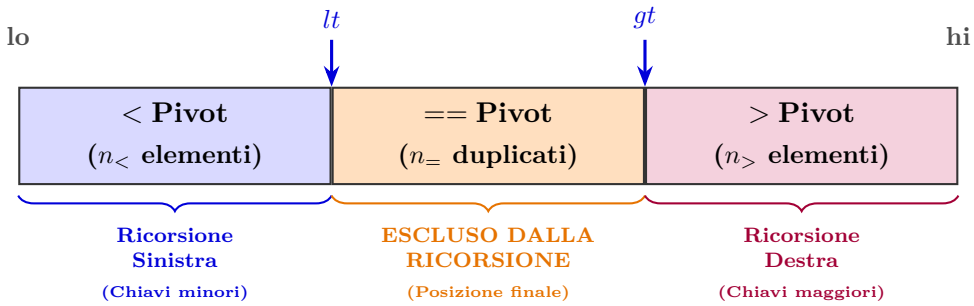


Figura 6.3: Topologia delle tre regioni del partizionamento a 3 vie (*Dutch National Flag*) e puntatori operativi lt e gt.

Proprietà 6.3.1 (Invarianti Dimensionali del Partizionamento a 3 Vie)**.** *Le cardinalità delle tre sezioni soddisfano rigorosamente le seguenti relazioni invarianti:*

1. $n_= \geq 1$*: la partizione centrale contiene sempre almeno il pivot selezionato.*
2. $0 \leq n_< \leq n - 1$*: numero di elementi strettamente minori del pivot.*
3. $0 \leq n_> \leq n - 1$*: numero di elementi strettamente maggiori del pivot.*
4. $n_< + n_= + n_> = n$*: conservazione assoluta della cardinalità totale dell'array.*

Teorema 6.3.1 (Esclusione della Regione Centrale dal Flusso Ricorsivo)**.** *Poiché tutti gli elementi della fascia == Pivot possiedono lo stesso valore e risiedono esattamente nelle celle comprese tra la porzione dei minori e quella dei maggiori, essi si trovano già nella loro* ***collocazione finale definitiva****.*

Nelle successive chiamate ricorsive di Quicksort, la regione centrale viene ***totalmente esclusa****. La ricorsione prosegue unicamente su:*

- *Il sotto-array sinistro di dimensione $n_<$ ($S[lo \dots lt - 1]$).*
- *Il sotto-array destro di dimensione $n_>$ ($S[gt + 1 \dots hi]$).*

La dimensione aggregata dei sotto-problemi residui è pertanto pari a:

$$n_< + n_> = n - n_= \leq n - 1. \tag{6.12}$$

Su un array composto da soli elementi uguali, si ha $n_= = n$ e $n_< = n_> = 0$: l'algoritmo effettua una sola scansione lineare terminando immediatamente in tempo ottimale $\mathcal{O}(n)$.

6.3.2 Applicazione a Quickselect: Selezione del k-esimo Elemento

Il partizionamento a tre vie si integra in modo ideale con il problema della selezione del k-esimo elemento affrontato nella sezione 6.1.

Nell'algoritmo **Quickselect** classico a 2 vie, la presenza di molteplici elementi duplicati può condurre a una sequenza sfortunata di partizioni sbilanciate, portando il tempo di calcolo a $\Theta(n^2)$. Con il partizionamento a 3 vie, la ricerca della k-esima statistica d'ordine gode di una proprietà di **terminazione anticipata**:

- **Se $k < lt$:** l'elemento cercato è strettamente minore del pivot; l'algoritmo scende ricorsivamente **esclusivamente nel ramo sinistro** $S[\text{lo} \dots lt - 1]$ ricercando la posizione k.
- **Se $lt \leq k \leq gt$:** la posizione target k cade esattamente all'interno della fascia centrale dei duplicati del pivot. Poiché tutti gli elementi in tale intervallo sono identici a p, si conclude istantaneamente che:

  $$S_{(k)} = p. \tag{6.13}$$

  L'algoritmo **termina immediatamente in tempo** $\mathcal{O}(1)$ restituendo p, senza alcuna ulteriore chiamata ricorsiva.
- **Se $k > gt$:** l'elemento cercato è strettamente maggiore del pivot; l'algoritmo scende ricorsivamente **esclusivamente nel ramo destro** $S[gt + 1 \dots \text{hi}]$ ricercando la posizione k.

Questa proprietà di *early exit* rende Quickselect a 3 vie eccezionalmente robusto: in insiemi di dati con pochi valori distinti (ad esempio campioni discretizzati o chiavi categoriche), l'algoritmo individua il valore cercato in un numero piccolissimo di iterazioni, con costo medio $\mathcal{O}(n)$ e migliore $\mathcal{O}(n)$ anche nel caso degenere di elementi tutti identici.

6.3.3 Impatto sull'Equazione di Ricorrenza del Caso Medio

La separazione della regione centrale chiarisce la genesi della ricorrenza analizzata nella successiva sezione 6.4.

Nel caso in cui gli elementi siano tutti distinti, la partizione centrale contiene unicamente il pivot stesso ($n_= = 1$), per cui la somma delle dimensioni dei due rami ricorsivi soddisfa:

$$n_< + n_> = n - 1. \tag{6.14}$$

Se il pivot divide l'array in due partizioni di taglia $i-1$ e $n-i$ (con $i \in \{1, \ldots, n\}$ equiprobabile), il costo atteso $T(n)$ soddisfa la ricorrenza classica di Hoare:

$$T(n) = \frac{1}{n} \sum_{i=1}^{n} \Big(T(i-1) + T(n-i) \Big) + \Theta(n). \tag{6.15}$$

Qualora vi siano duplicati ($n_= > 1$), la somma delle taglie dei sotto-problemi ricorsivi $n_< + n_> = n - n_=$ si riduce rigorosamente al di sotto di $n-1$. Ciascun duplicato addizionale sottrae lavoro alla ricorsione successiva senza alcun costo aggiuntivo di I/O o CPU, accelerando la convergenza asintotica dell'algoritmo. L'analisi del caso peggiore per la versione randomizzata coincide pertanto con l'assenza totale di duplicati ($n_= = 1$), la cui trattazione analitica completa viene sviluppata formalmente nella sezione che segue.

6.4 Dimostrazione Matematica del Costo Medio di Quicksort ($\Theta(n \log_2 n)$)

L'analisi del caso medio (*Average-Case Complexity*) di Quicksort è una delle pietre miliari dell'algoritmica classica. Vogliamo determinare formalmente il valore atteso del tempo di esecuzione $T(n)$ assumendo che tutte le $n!$ permutazioni degli elementi in ingresso siano equiprobabili o che, equivalentemente, il pivot sia selezionato a ogni passo in modo casuale ed uniforme.

6.4.1 Derivazione della Relazione di Ricorrenza

Sia S un array di n elementi distinti. La procedura di partizionamento compie un confronto per ciascun elemento tranne il pivot, compiendo un lavoro lineare pari a $c \cdot n$ operazioni elementari (con $c > 0$ costante positiva).

Il pivot selezionato ha probabilità esattamente uniforme pari a $\frac{1}{n}$ di occupare la posizione di rango i (con $i \in \{0, 1, \ldots, n-1\}$) nell'array ordinato. Tale scelta genera due sotto-problemi indipendenti:

- Un sotto-array sinistro di dimensione i.
- Un sotto-array destro di dimensione $n-1-i$.

Il tempo atteso di esecuzione soddisfa pertanto l'equazione integrale:

$$T(n) = c \cdot n + \frac{1}{n} \sum_{i=0}^{n-1} \left(T(i) + T(n-1-i) \right). \tag{6.16}$$

Lemma 6.4.1 (Simmetria delle Sommatorie)**.** *Le due sommatorie parziali nell'Equazione 6.16 contengono esattamente gli stessi termini sommati in ordine inverso. Vale l'identità:*

$$\sum_{i=0}^{n-1} T(n-1-i) = T(n-1) + T(n-2) + \cdots + T(0) = \sum_{i=0}^{n-1} T(i). \tag{6.17}$$

Sostituendo il Lemma nella ricorrenza, l'equazione fondamentale del costo medio di Quicksort diviene:

$$T(n) = c \cdot n + \frac{2}{n} \sum_{i=0}^{n-1} T(i) \quad \text{per } n \geq 2. \tag{6.18}$$

Assumiamo come casi base elementari $T(0) = 0$ e $T(1) = 0$ (o costanti trascurabili).

6.4.2 Risoluzione Algebrica Rigorosa Passo-Passo

La presenza della sommatoria $\sum_{i=0}^{n-1} T(i)$ rende l'equazione non lineare (*full-history recurrence*). Per risolverla, applichiamo il metodo algebrico dell'eliminazione della sommatoria per differenza di termini consecutivi e il metodo del fattore integrante.

Passo 1: Eliminazione della Frazione mediante Moltiplicazione per n

Moltiplichiamo ambo i membri dell'Equazione 6.18 per il fattore n:

$$n\,T(n) = c\,n^2 + 2 \sum_{i=0}^{n-1} T(i). \tag{6.19}$$

Passo 2: Scrittura della Ricorrenza per l'Istanza di Dimensione $n-1$

Scriviamo la medesima relazione sostituendo formalmente n con $n-1$ (valida per $n \geq 3$):

$$(n-1)\,T(n-1) = c\,(n-1)^2 + 2 \sum_{i=0}^{n-2} T(i). \tag{6.20}$$

Passo 3: Sottrazione Membro a Membro (Eq. 6.19 − Eq. 6.20)

Sottraendo l'Equazione 6.20 dall'Equazione 6.19, la sommatoria si cancella quasi completamente, isolando il solo termine $i = n - 1$:

$$2\sum_{i=0}^{n-1} T(i) - 2\sum_{i=0}^{n-2} T(i) = 2\,T(n-1). \tag{6.21}$$

Otteniamo quindi:

$$\begin{aligned} n\,T(n) - (n-1)\,T(n-1) &= c\left[n^2 - (n-1)^2\right] + 2\,T(n-1) && (6.22)\\ &= c\left[n^2 - (n^2 - 2n + 1)\right] + 2\,T(n-1) && (6.23)\\ &= c\,(2n-1) + 2\,T(n-1). && (6.24) \end{aligned}$$

Raggruppiamo i termini in $T(n-1)$ portando $(n-1)T(n-1)$ a secondo membro:

$$\begin{aligned} n\,T(n) &= (n-1)\,T(n-1) + 2\,T(n-1) + c\,(2n-1) && (6.25)\\ n\,T(n) &= (n+1)\,T(n-1) + c\,(2n-1). && (6.26) \end{aligned}$$

Poiché $2n - 1 < 2n$ per ogni $n \geq 1$, possiamo maggiorare per ottenere una comoda limitazione superiore:

$$n\,T(n) \leq (n+1)\,T(n-1) + 2c\,n. \tag{6.27}$$

Passo 4: Metodo del Fattore Integrante (Fattore di Sommazione)

Per ricondurre l'Equazione 6.27 a una forma telescopica, dividiamo entrambi i membri per la quantità quadratica $n(n+1)$:

$$\frac{n\,T(n)}{n(n+1)} \leq \frac{(n+1)\,T(n-1)}{n(n+1)} + \frac{2c\,n}{n(n+1)}. \tag{6.28}$$

Semplificando i fattori comuni a numeratore e denominatore:

$$\frac{T(n)}{n+1} \leq \frac{T(n-1)}{n} + \frac{2c}{n+1}. \tag{6.29}$$

Definiamo la variabile ausiliaria $A_n \coloneqq \frac{T(n)}{n+1}$. L'equazione si trasforma in una ricorrenza additiva elementare di primo grado:

$$A_n \leq A_{n-1} + \frac{2c}{n+1}. \tag{6.30}$$

Passo 5: Sviluppo Telescopico e Numeri Armonici

Srotoliamo la ricorrenza all'indietro per sostituzioni successive fino al caso base A_1:

$$A_n \leq A_{n-1} + \frac{2c}{n+1} \tag{6.31}$$

$$\leq A_{n-2} + \frac{2c}{n} + \frac{2c}{n+1} \tag{6.32}$$

$$\leq A_1 + 2c \sum_{k=2}^{n} \frac{1}{k+1}. \tag{6.33}$$

Effettuando il cambio di variabile di sommatoria $j = k + 1$:

$$\sum_{k=2}^{n} \frac{1}{k+1} = \sum_{j=3}^{n+1} \frac{1}{j} \leq \sum_{j=1}^{n} \frac{1}{j}. \tag{6.34}$$

La sommatoria parziale coincide esattamente con l'n**-esimo Numero Armonico** H_n:

$$H_n := \sum_{j=1}^{n} \frac{1}{j} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}. \tag{6.35}$$

Dall'analisi asintotica tramite approssimazione per integrali di Cauchy (illustrata in Figura 6.4), è noto che:

$$H_n = \ln n + \gamma + \mathcal{O}\left(\frac{1}{n}\right) = \ln n + \mathcal{O}(1), \tag{6.36}$$

dove $\gamma \approx 0{,}577215$ è la costante di Eulero-Mascheroni.

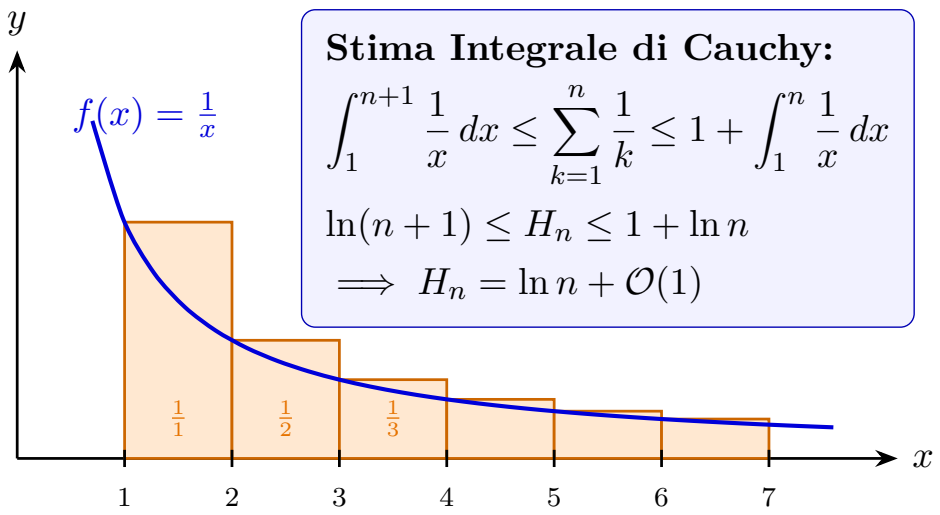


Figura 6.4: Approssimazione integrale della serie armonica $H_n = \sum_{k=1}^{n} \frac{1}{k} \approx \ln n$. L'area dei rettangoli maggiora l'integrale della funzione iperbolica $f(x) = \frac{1}{x}$.

Sostituendo l'espressione asintotica di H_n nella nostra maggiorazione per A_n:

$$A_n = \frac{T(n)}{n+1} \le 2c \ln n + \mathcal{O}(1). \tag{6.37}$$

Moltiplichiamo entrambi i membri per $(n+1)$:

$$T(n) \le 2c\,(n+1) \ln n + \mathcal{O}(n) = 2c\,n \ln n + \mathcal{O}(n). \tag{6.38}$$

6.4.3 Conversione in Base 2 e Significato Informativo

Per esprimere il costo computazionale nell'unità di misura canonica degli algoritmi di confronto (bit di informazione o confronti binari), convertiamo il logaritmo naturale $\ln n$ in logaritmo in base 2 applicando la formula del cambio di base:

$$\ln n = \frac{\log_2 n}{\log_2 e} = (\ln 2) \cdot \log_2 n \approx 0{,}693147 \cdot \log_2 n. \tag{6.39}$$

Sostituendo tale valore nel costo medio atteso:

$$\begin{aligned} T_{\text{medio}}(n) &\le 2c \cdot (\ln 2) \cdot n \log_2 n + \mathcal{O}(n) && (6.40)\\ &\approx (2 \cdot 0{,}693147) \cdot c\,n \log_2 n && (6.41)\\ &\approx 1{,}386294 \cdot c\,n \log_2 n = \Theta(n \log_2 n). && (6.42) \end{aligned}$$

Teorema 6.4.1 (Significato Teorico e Limite di Informazione)**.** *Il limite inferiore teorico imposto dalla teoria dell'informazione (Information-Theoretic Lower Bound) stabilisce che qualsiasi algoritmo di ordinamento basato su confronti necessita, nel caso peggiore e nel caso medio, di almeno:*

$$\log_2(n!) \approx n \log_2 n - n \log_2 e \approx n \log_2 n - 1{,}4427\, n \quad \textit{confronti.} \tag{6.43}$$

Il valore moltiplicativo ottenuto per Quicksort randomizzato mostra che il numero atteso di confronti è pari a:

$$2 \ln 2 \cdot n \log_2 n \approx 1{,}386\, n \log_2 n. \tag{6.44}$$

Proprietà 6.4.1 (Efficienza Ingegneristica di Quicksort)**.** *In media, Quicksort compie solo circa il* 38,6% ***in più di confronti*** *rispetto all'ottimo informativo assoluto ideale. Poiché la procedura di partizione è un ciclo di scansione elementare privo di overhead di allocazione dinamica, che opera in-place ed esibisce un'eccellente località spaziale nella cache L1, il fattore costante c è talmente ridotto da rendere Quicksort sensibilmente più veloce di algoritmi che eseguono meno confronti teorici (come Merge Sort), affermandosi come il motore di ordinamento fondamentale per array in memoria centrale.*

6.5 Sintesi Comparativa dei Risultati

Per consolidare lo studio comparativo delle strutture dati e degli algoritmi analizzati, la Tabella 6.3 riassume le complessità asintotiche per i casi ottimo, medio e peggiore, affiancando i requisiti di memoria ausiliaria e le peculiarità architetturali di ciascun metodo.

Tabella 6.3: Sintesi comparativa degli algoritmi di selezione e ordinamento basati su partizionamento.

Algoritmo	Caso Ottimo	Caso Medio	Caso Peggiore	Spazio	Peculiarità Ingegneristica
Ricerca Min / Max	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\mathcal{O}(1)$	Singolo passaggio lineare sequenziale; $n-1$ confronti ottimi.
Selezione con Max-Heap	$\Theta(n)$	$\Theta(n \log_2 k)$	$\Theta(n \log_2 k)$	$\Theta(k)$	Ideale per streaming e $k \ll n$; scarta gli elementi non idonei in $\mathcal{O}(1)$.
Quickselect (Hoare)	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$\mathcal{O}(\log_2 n)$	Divide et Impera su un solo ramo; tempo atteso lineare per ogni k.
Quicksort Standard	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(n)$	Degenera a $\Theta(n^2)$ con elementi duplicati o partizioni sbilanciate.
Quicksort a 3 Vie	$\Theta(n)$	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(\log_2 n)$	Tempo $\Theta(n)$ se le chiavi sono tutte duplicate; esclude la zona centrale.
Dual-Pivot Quicksort	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(\log_2 n)$	Riduce del 20% le scritture a fronte di +5% confronti; ottimo in cache.
Bounded Quicksort (BQS)	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(\log_2 n)$	Elimina la tail recursion sul ramo maggiore; stack $\mathcal{O}(\log_2 n)$.

6.5.1 Linee Guida per la Scelta dell'Algoritmo

Dall'analisi teorica e ingegneristica sviluppata emergono le seguenti raccomandazioni pratiche:

1. **Selezione Estrema ($k = 1$ o $k = n$):** Utilizzare sempre la scansione lineare a singolo registro; per la ricerca simultanea di minimo e massimo adottare il confronto ad accoppiamento per minimizzare il numero di istruzioni di salto condizionato.

2. **Selezione con Finestra Ridotta ($k \ll n$ in Data Streaming):** Adottare il **Bounded Max-Heap** di capacità k. Consente di processare flussi continui di dati senza allocare un vettore di dimensione n in RAM, filtrando gli elementi superflui in $\mathcal{O}(1)$.

3. **Selezione Generica su Dati Statici ($k = \Theta(n)$):** Ricorrere a **Quickselect** con partizionamento a 3 vie, che garantisce un tempo atteso lineare $\mathcal{O}(n)$ ed evita l'ordinamento superfluo del restante 50% dei dati.

4. **Ordinamento In-Memory ad Alte Prestazioni:** Utilizzare varianti avanzate di Quicksort che combinano il partizionamento a 3 vie (o Dual-Pivot), l'eliminazione della ricorsione in coda (*tail recursion elimination* per garantire spazio di stack $\mathcal{O}(\log_2 n)$) e un cutoff su sotto-array di piccola dimensione ($M \approx 16$–32) delegati a *Insertion Sort* per massimizzare il riuso della cache L1.

Capitolo 7

Ordinamento Esterno (Distribution Sort), Over-Sampling e Fondamenti di Random Sampling

7.1 Il Problema dell'Ordinamento Esterno e il Multi-Way Partitioning

L'ordinamento su memoria secondaria di dataset di dimensioni massicce impone l'abbandono dei paradigmi a partizione dicotomica a favore di strategie multi-via calibrate sulla capacità della RAM e sulla dimensione dei blocchi di I/O. In questa sezione formalizziamo il modello computazionale a due livelli e l'architettura dell'External Distribution Sort basata sulla decomposizione in secchielli.

7.1.1 Il Modello Computazionale I/O (Memoria Secondaria)

Nei sistemi che elaborano grandi moli di dati (*Massive Datasets*), la dimensione dell'insieme da ordinare S supera ampiamente la capacità della memoria centrale ad accesso casuale (RAM). Si adotta il modello standard a due livelli di memoria secondaria introdotto da Aggarwal e Vitter [2]:

- n: numero complessivo di record nella sequenza $S = \langle S[1], \ldots, S[n] \rangle$;
- M: dimensione della memoria interna (RAM), misurata in numero di record ($M \ll n$);

- B: dimensione del blocco di trasferimento del disco, misurata in numero di record trasferiti per singola operazione di Input/Output (I/O).

Il numero di blocchi che possono risiedere simultaneamente nella memoria primaria è pari a $\frac{M}{B}$.

Per ordinare il flusso S con complessità di I/O ottimale, non è sufficiente impiegare un algoritmo ricorsivo a partizione binaria (come il QuickSort classico), poiché esso comporterebbe un albero di ricorsione di altezza $\Theta(\log_2(n/B))$, un valore asintoticamente troppo profondo che saturerebbe il sottosistema disco con eccessivi accessi casuali.

Si ricorre pertanto a un partizionamento a più vie (**Multi-Way Partitioning** o **External Distribution Sort**), dividendo ricorsivamente l'insieme in k partizioni disgiunte (dette *secchielli* o *buckets*), dove il grado di partizione è calibrato sulla memoria disponibile:

$$k = \mathcal{O}\left(\frac{M}{B}\right). \tag{7.1}$$

7.1.2 Definizione Formale dei Secchielli e dei Pivot

Dato il vettore non ordinato $S[1 \dots n]$, l'obiettivo consiste nel selezionare un insieme ordinato di $k-1$ elementi sentinella, denominati **pivot**:

$$s_1 \leq s_2 \leq \cdots \leq s_{k-1}. \tag{7.2}$$

La notazione s evidenzia che tali elementi provengono da una procedura di campionamento (*sampled*).

Per convenzione analitica e per garantire la completezza formale degli estremi della partizione, si definiscono due pivot fittizi sentinella:

$$s_0 \coloneqq -\infty, \qquad s_k \coloneqq +\infty. \tag{7.3}$$

Definizione 7.1.1 (Secchielli o Buckets B_i)**.** I k secchielli $B_1, B_2, \dots, B_k$ sono definiti come le partizioni formate da tutti e soli gli elementi di S che cadono nei rispettivi intervalli semi-aperti determinati dai pivot adiacenti:

$$B_i \coloneqq \Big\{ S[j] \in S \;\mid\; s_{i-1} < S[j] \leq s_i \Big\}, \qquad \forall i \in \{1, 2, \dots, k\}. \tag{7.4}$$

La topologia di suddivisione dello spazio dei dati è illustrata nella Figura 7.1.

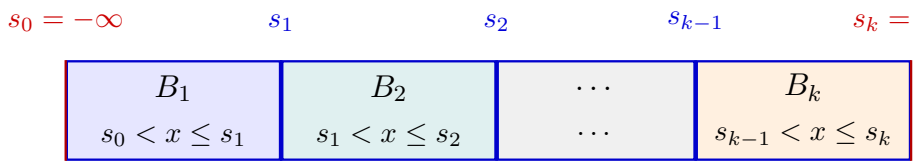


Figura 7.1: Partizionamento dell'insieme S in k secchielli (*buckets*) mediante $k-1$ pivot ordinati.

7.1.3 Condizione di Bilanciamento Ottimale

Affinché l'algoritmo di ordinamento mantenga la complessità ottima nel modello I/O, le dimensioni dei secchielli non devono deviare eccessivamente dalla dimensione media ideale $\frac{n}{k}$.

Proprietà 7.1.1 (Obiettivo Formale di Bilanciamento)**.** *Tutti i secchielli devono possedere una dimensione asintoticamente uniforme:*

$$|B_i| = \Theta\left(\frac{n}{k}\right), \qquad \forall i \in \{1, 2, \ldots, k\}. \tag{7.5}$$

Il significato sistemistico di tale bilanciamento è duplice:

1. **Ottimalità Asintotica dell'Albero di Ricorsione:** Se tutti i secchielli sono bilanciati a ogni livello, la profondità dell'albero di ricorsione si riduce a:
$$\mathcal{O}\left(\log_k \frac{n}{B}\right) = \mathcal{O}\left(\frac{\log(n/B)}{\log(M/B)}\right). \tag{7.6}$$
Poiché distribuire gli elementi tra i k secchielli a ciascun livello richiede una scansione lineare con costo $\mathcal{O}\left(\frac{n}{B}\right)$ I/O (utilizzando un buffer di scrittura per ciascuno dei $k = \mathcal{O}(M/B)$ secchielli), il costo complessivo di I/O raggiunge esattamente il lower bound di Aggarwal e Vitter:
$$\mathcal{O}\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right) \text{ I/O}. \tag{7.7}$$

2. **Prevenzione dell'Overflow in Memoria Centrale:** Nessun secchiello genera saturazione della RAM durante la fase finale in cui i singoli secchielli vengono caricati in memoria interna e ordinati localmente.

7.2 La Tecnica dell'Over-Sampling

La robustezza dell'ordinamento per distribuzione dipende dalla capacità di selezionare pivot che ripartiscano il dataset in secchielli di dimensioni uniformi

ed equilibrate. In questa sezione esaminiamo la tecnica dell'Over-Sampling, che consente di abbattere drasticamente la varianza campionando un insieme sovradimensionato di elementi.

7.2.1 Criticità del Campionamento Banale

Se si estraessero casualmente e direttamente solo $k-1$ pivot da S, la varianza statistica della distanza tra due campioni successivi nell'ordinamento reale risulterebbe estremamente elevata. Di conseguenza, con alta probabilità si verificherebbero forti fluttuazioni: esisterebbero intervalli molto densi e intervalli quasi vuoti, generando secchielli patologici con:

$$|B_i| \gg \frac{n}{k}. \tag{7.8}$$

Tale sbilanciamento farebbe collassare l'efficienza dell'intero ordinamento esterno.

Per ridurre drasticamente tale varianza e garantire matematicamente che nessun secchiello ecceda una prefissata soglia critica, si applica la tecnica del sovracampionamento (**Over-Sampling**): si estrae un numero di elementi significativamente superiore a k, li si ordina in memoria centrale, e si selezionano i pivot finali a intervalli perfettamente regolari.

7.2.2 Formalizzazione dei Parametri di Over-Sampling

Si definisce un parametro di sovracampionamento a:

$$a = \Theta(\log_2 k). \tag{7.9}$$

Nello specifico, per far coincidere con rigore le costanti analitiche della dimostrazione con le disuguaglianze di concentrazione esponenziale, si pone:

$$a + 1 \coloneqq 12 \ln k \qquad (\text{con } k \geq 2). \tag{7.10}$$

La dimensione complessiva del campione casuale estratto $A \subset S$ è pari a:

$$|A| = (a+1)k - 1. \tag{7.11}$$

7.2.3 Algoritmo di Selezione dei Pivot tramite Over-Sampling

La procedura si articola in tre passi sequenziali:

1. **Estrazione Casuale:** Si estraggono uniformemente a caso $(a+1)k-1$ elementi da S, formando l'insieme dei campioni A.

2. **Ordinamento del Campione:** Si ordina l'insieme A in memoria interna (operazione eseguibile a costo zero di I/O aggiuntivi se $|A| \leq M$):

$$A = [A[1], A[2], \ldots, A[|A|]], \qquad \text{con } A[1] \leq A[2] \leq \cdots \leq A[|A|]. \tag{7.12}$$

3. **Selezione dei Pivot Equidistanti:** Per ogni $i \in \{1, \ldots, k-1\}$, il pivot s_i viene scelto come l'elemento in posizione multipla di $(a+1)$ nell'array ordinato A:

$$s_i \coloneqq A\Big[(a+1) \cdot i\Big], \qquad \forall i \in \{1, 2, \ldots, k-1\}. \tag{7.13}$$

Schema Strutturale del Vettore dei Campioni Ordinati A

Il vettore ordinato A viene suddiviso in k blocchi logici composti da a elementi ciascuno, separati dai rispettivi pivot sentinella:

- Tra s_{i-1} e s_i risiedono esattamente a campioni ordinati di A.
- Includendo l'estremo destro s_i, nel semintervallo $(s_{i-1}, s_i]$ cadono esattamente $a+1$ campioni di A.

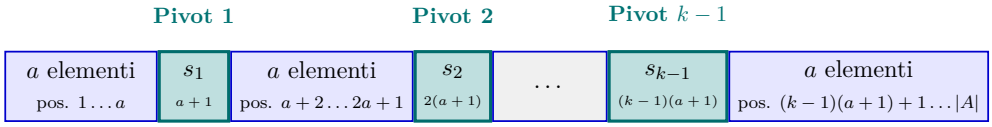


Figura 7.2: Partizione strutturale dell'array ordinato dei campioni A in k blocchi di taglia a intervallati dai pivot.

Algorithm 6: Selezione dei Pivot tramite Over-Sampling

Dati: Dataset non ordinato $S[1 \ldots n]$, numero di vie $k \geq 2$, parametro $a+1 = 12 \ln k$

Risultato: Vettore ordinato dei pivot $s = \langle s_1, s_2, \ldots, s_{k-1} \rangle$

```
m ← (a+1)k − 1
A ← RandomSample(S, m)                    // Estrae m elementi uniformemente a caso
Sort(A)                                   // Ordinamento interno in RAM: A[1] <= ... <= A[m]
for i ← 1 to k − 1 do
    s_i ← A[(a+1) · i]                    // Pivot equidistanti a passo a+1
return s
```

7.3 Analisi Probabilistica del Fallimento e Limite di Chernoff

Per garantire l'ottimalità asintotica dell'External Distribution Sort, è necessario quantificare con precisione la probabilità che uno o più secchielli eccedano la capacità limite della memoria interna. In questa sezione formalizziamo l'evento di fallimento e ne deriviamo una stima rigorosa sfruttando la disuguaglianza esponenziale di Chernoff.

7.3.1 Definizione Formale dell'Evento di Fallimento

Il nostro obiettivo algoritmico consiste nel garantire che nessun secchiello ecceda una dimensione critica. Fissiamo come soglia di fallimento una cardinalità pari o superiore al doppio della dimensione media $\frac{2n}{k}$.

Definizione 7.3.1 (Evento di Fallimento (Failure))**.** L'evento di fallimento si verifica se almeno uno dei k secchielli risulta sovradimensionato:

$$\text{Failure} \iff \exists i \in \{1, 2, \ldots, k\} \text{ tale che } |B_i| \geq \frac{2n}{k}. \tag{7.14}$$

Vogliamo dimostrare formalmente che la probabilità di fallimento è strettamente dominata da una costante:

$$\mathbb{P}(\text{Failure}) \leq \frac{1}{2}. \tag{7.15}$$

Proprietà 7.3.1 (Utilità Pratica della Condizione $\mathbb{P}(\text{Failure}) \leq \frac{1}{2}$)**.** *Una probabilità di insuccesso $\leq \frac{1}{2}$ è considerata ottimale nell'analisi degli algoritmi randomizzati con paradigma **Las Vegas**. Se l'algoritmo verifica a posteriori la condizione di bilanciamento (costo lineare $\frac{n}{B}$ I/O di scansione) e ripete il campionamento in caso di insuccesso, il numero atteso di tentativi prima del successo è una variabile geometrica con valore atteso:*

$$\mathbb{E}[\textit{tentativi}] = \frac{1}{1 - \mathbb{P}(\textit{Failure})} \leq \frac{1}{1 - 1/2} = 2. \tag{7.16}$$

Inoltre, dopo appena r tentativi indipendenti, la probabilità che l'algoritmo non abbia ancora ottenuto una partizione bilanciata decade esponenzialmente a $\leq \left(\frac{1}{2}\right)^r$, rendendo l'approccio robusto ed efficiente in produzione.

7.3.2 Spazio Campionario Ideale S_{sort} e Partizione in Blocchi Fissi

Per analizzare il comportamento stocastico dell'algoritmo, si considera l'ordinamento teorico completo di tutti gli n elementi di S, denotato con S_{sort}:

$$S_{\text{sort}} = [x_1, x_2, \ldots, x_n], \qquad \text{con } x_1 \leq x_2 \leq \cdots \leq x_n. \tag{7.17}$$

Poiché ogni secchiello B_i raccoglie tutti e soli gli elementi compresi nell'intervallo $(s_{i-1}, s_i]$, gli elementi di ciascun B_i occupano uno spazio rigorosamente contiguo all'interno di S_{sort}.

Suddividiamo idealmente l'intero array ordinato S_{sort} in intervalli fissi e disgiunti t_j di dimensione prefissata pari a $\frac{2n}{k}$. Il numero totale di tali blocchi è dato da:

$$\text{Numero di blocchi } t_j = \frac{n}{\frac{2n}{k}} = \frac{k}{2}. \tag{7.18}$$

Indichiamo questi $\frac{k}{2}$ blocchi fissi con $t_1, t_2, \ldots, t_{k/2}$.

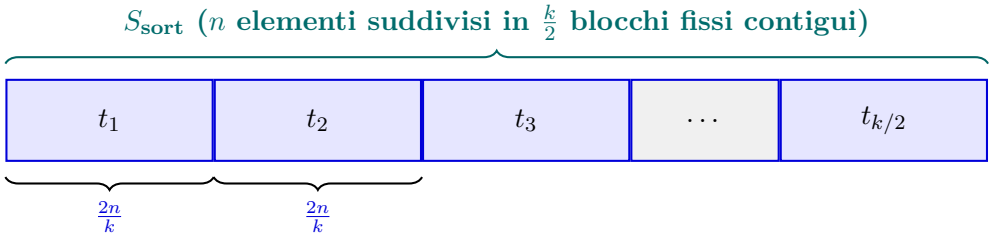


Figura 7.3: Partizione teorica di S_{sort} in $\frac{k}{2}$ blocchi fissi t_j di cardinalità $\frac{2n}{k}$.

7.3.3 Riduzione Logica dell'Evento di Fallimento

Esaminiamo cosa accade quando si verifica l'evento di fallimento per un generico secchiello B_i, ossia quando $|B_i| \geq \frac{2n}{k}$:

1. Poiché B_i è un intervallo contiguo in S_{sort} di lunghezza almeno pari a $\frac{2n}{k}$, esso deve necessariamente contenere al proprio interno (o sovrapporsi interamente a) almeno uno dei blocchi fissi t_j:

$$\exists j \in \left\{1, \ldots, \frac{k}{2}\right\} \text{ tale che } t_j \subseteq B_i. \tag{7.19}$$

2. I delimitatori s_{i-1} e s_i del secchiello si collocano all'esterno del blocco t_j (rispettivamente a sinistra e a destra di t_j).

3. Per costruzione dell'algoritmo di over-sampling, l'intero secchiello B_i contiene al massimo a campioni strettamente interni, più l'estremo destro s_i. Il numero totale di campioni di A che cadono in B_i è pertanto esattamente $a + 1$.

4. Dato che $t_j \subseteq B_i$ e che il pivot s_i risiede all'esterno di t_j, il blocco fisso t_j può contenere strettamente meno di $a + 1$ campioni di A:

$$|t_j \cap A| < a + 1. \tag{7.20}$$

Vale quindi l'implicazione logica tra eventi:

$$X \implies Y, \tag{7.21}$$

dove:

$$X : \exists B_i \text{ tale che } |B_i| \geq \frac{2n}{k}, \tag{7.22}$$

$$Y : \exists t_j \text{ tale che } t_j \text{ contiene } < a + 1 \text{ campioni di } A. \tag{7.23}$$

Per le proprietà fondamentali della probabilità:

$$X \implies Y \implies \mathbb{P}(X) \leq \mathbb{P}(Y). \tag{7.24}$$

Possiamo dunque maggiorare la probabilità di fallimento analizzando la probabilità che almeno un blocco fisso t_j riceva un numero insufficiente di campioni:

$$\mathbb{P}(\text{Failure}) \leq \mathbb{P}\left(\exists j \in \left\{1, \ldots, \frac{k}{2}\right\} : |t_j \cap A| < a + 1\right). \tag{7.25}$$

7.3.4 Disuguaglianza dell'Unione (Union Bound)

Applicando la disuguaglianza di Boole (*Union Bound*) sui $\frac{k}{2}$ blocchi fissi:

$$\mathbb{P}\left(\bigcup_{j=1}^{k/2} \Big\{|t_j \cap A| < a + 1\Big\}\right) \leq \sum_{j=1}^{k/2} \mathbb{P}\Big(|t_j \cap A| < a + 1\Big). \tag{7.26}$$

Poiché tutti i blocchi t_j hanno la medesima cardinalità fissa $|t_j| = \frac{2n}{k}$, la probabilità che un campione cada in un blocco è identica per ogni j. Fissando un arbitrario blocco di riferimento t^*:

$$\mathbb{P}(\text{Failure}) \leq \frac{k}{2} \cdot \mathbb{P}\Big(t^* \text{ contiene } < a + 1 \text{ campioni di } A\Big). \tag{7.27}$$

7.3.5 Calcolo del Valore Atteso

Fissiamo il blocco t^*. Sia X la variabile aleatoria che conta quanti elementi dell'insieme dei campioni A cadono dentro t^*. Definiamo le variabili indicatrici per ciascun elemento estratto $r \in \{1, \dots, |A|\}$:

$$X_r \coloneqq \begin{cases} 1 & \text{se il } r\text{-esimo campione di } A \text{ appartiene a } t^*, \\ 0 & \text{altrimenti.} \end{cases} \tag{7.28}$$

La variabile aleatoria totale è la somma delle indicatrici:

$$X = \sum_{r=1}^{|A|} X_r. \tag{7.29}$$

La probabilità che un singolo campione uniforme estratto da S cada all'interno di t^* corrisponde al rapporto volumetrico:

$$\mathbb{P}(\text{campione } \in t^*) = \frac{|t^*|}{n} = \frac{\frac{2n}{k}}{n} = \frac{2}{k}. \tag{7.30}$$

Calcoliamo il valore atteso $\mathbb{E}[X]$ sfruttando la linearità dell'attesa:

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{r=1}^{|A|} X_r\right] = \sum_{r=1}^{|A|} \mathbb{E}[X_r] = \sum_{r=1}^{|A|} \mathbb{P}(X_r = 1) = |A| \cdot \frac{2}{k}. \tag{7.31}$$

Sostituendo la dimensione dell'insieme campionario $|A| = (a+1)k - 1$:

$$\begin{aligned} \mathbb{E}[X] &= \Big[(a+1)k - 1\Big] \cdot \frac{2}{k} \\ &= 2(a+1) - \frac{2}{k}. \end{aligned} \tag{7.32}$$

Poiché per ipotesi $k \geq 2$, il termine sottrattivo soddisfa:

$$\frac{2}{k} \leq \frac{2}{2} = 1. \tag{7.33}$$

Dunque possiamo minorare il valore atteso:

$$\mathbb{E}[X] \geq 2(a+1) - 1. \tag{7.34}$$

Poiché $a \geq 1$ (essendo $a + 1 = 12 \ln k \geq 12 \ln 2 \approx 8.31$ per ogni $k \geq 2$), possiamo scomporre algebricamente:

$$\begin{aligned} 2(a+1) - 1 &= \frac{3}{2}(a+1) + \underbrace{\frac{1}{2}(a+1) - 1}_{\geq 0} \\ &\geq \frac{3}{2}(a+1). \end{aligned} \tag{7.35}$$

Otteniamo la disuguaglianza fondamentale che lega il valore atteso alla soglia critica:

$$\mathbb{E}[X] \geq \frac{3}{2}(a+1) \iff a+1 \leq \frac{2}{3}\mathbb{E}[X]. \tag{7.36}$$

7.3.6 Applicazione del Limite di Chernoff (Lower Tail)

Il teorema del Chernoff Bound per la coda inferiore (*lower tail*) stabilisce che, data una somma $X = \sum X_r$ di variabili aleatorie indipendenti limitate in $[0,1]$, per ogni deviazione relativa $\delta \in (0,1)$:

$$\mathbb{P}\Big(X < (1-\delta)\mathbb{E}[X]\Big) \leq e^{-\frac{\delta^2}{2}\mathbb{E}[X]}. \tag{7.37}$$

Proprietà 7.3.2 (Interpretazione Intuitiva del Limite di Chernoff)**.** *La variabile casuale X si concentra esponenzialmente attorno al proprio valore atteso $\mathbb{E}[X]$. La probabilità che X si discosti significativamente dalla media decadendo verso valori bassi decresce a velocità esponenziale rispetto al quadrato della deviazione δ.*

Dobbiamo ricondurre la probabilità $\mathbb{P}(X < a+1)$ alla forma richiesta dal teorema di Chernoff. Dalla disuguaglianza (7.36) sappiamo che $a+1 \leq \frac{2}{3}\mathbb{E}[X]$. Poniamo quindi la deviazione relativa:

$$1-\delta = \frac{2}{3} \implies \delta = 1 - \frac{2}{3} = \frac{1}{3}. \tag{7.38}$$

Di conseguenza:

$$\mathbb{P}(X < a+1) \leq \mathbb{P}\left(X < \frac{2}{3}\mathbb{E}[X]\right) = \mathbb{P}\left(X < \left(1-\frac{1}{3}\right)\mathbb{E}[X]\right). \tag{7.39}$$

Applicando la formula di Chernoff (7.37) con $\delta = \frac{1}{3}$:

$$\mathbb{P}\left(X < \left(1-\frac{1}{3}\right)\mathbb{E}[X]\right) \leq e^{-\frac{(1/3)^2}{2}\mathbb{E}[X]} = e^{-\frac{1/9}{2}\mathbb{E}[X]} = e^{-\frac{1}{18}\mathbb{E}[X]}. \tag{7.40}$$

Sostituendo la minorazione del valore atteso $\mathbb{E}[X] \geq \frac{3}{2}(a+1)$:

$$\begin{aligned} e^{-\frac{1}{18}\mathbb{E}[X]} &\leq e^{-\frac{1}{18}\cdot\frac{3}{2}(a+1)} \\ &= e^{-\frac{3}{36}(a+1)} \\ &= e^{-\frac{1}{12}(a+1)}. \end{aligned} \tag{7.41}$$

7.3.7 Calibrazione Esatta di a e Conclusione della Dimostrazione

Sostituendo il valore scelto per la costante di over-sampling $a + 1 = 12 \ln k$:

$$e^{-\frac{1}{12}(a+1)} = e^{-\frac{1}{12}\cdot(12\ln k)} = e^{-\ln k} = \frac{1}{k}. \tag{7.42}$$

Abbiamo quindi derivato che, per qualsiasi singolo blocco t^*:

$$\mathbb{P}(t^* \text{ contiene } < a+1 \text{ campioni}) \leq \frac{1}{k}. \tag{7.43}$$

Moltiplicando per il fattore $\frac{k}{2}$ derivato dalla disuguaglianza di Boole (7.27):

$$\mathbb{P}(\text{Failure}) \leq \frac{k}{2} \cdot \mathbb{P}\Big(t^* \text{ contiene } < a+1 \text{ campioni}\Big) \leq \frac{k}{2} \cdot \frac{1}{k} = \frac{1}{2}. \tag{7.44}$$

Teorema 7.3.1 (Teorema di Bilanciamento dell'Over-Sampling)**.** *Con probabilità di successo pari ad almeno* $1 - \frac{1}{2} = \frac{1}{2}$*, il campionamento tramite sovracampionamento produce una partizione in cui tutti i secchielli* B_i *hanno dimensione strettamente inferiore a* $\frac{2n}{k}$*:*

$$\mathbb{P}\left(\forall i \in \{1, \ldots, k\} \colon |B_i| < \frac{2n}{k}\right) \geq \frac{1}{2}. \tag{7.45}$$

Questo garantisce un bilanciamento uniforme e completa l'analisi di correttezza ed efficienza I/O dell'External Distribution Sort.

7.4 Introduzione al Random Sampling e Limiti dell'Approccio Non-Streaming

A valle dell'analisi dell'ordinamento esterno, emerge la necessità fondazionale di studiare la primitiva stessa di estrazione casuale impiegata dall'algoritmo: come estrarre formalmente m elementi casuali da un insieme S di cardinalità n?

7.4.1 Definizione Formale del Problema

Dato un array indicizzato $S[1 \ldots n]$ contenente n elementi distinti, si vuole estrarre un sottoinsieme $D \subseteq S$ di cardinalità prefissata m tale da soddisfare rigorosamente le seguenti due proprietà:

- **Uniformità Marginale:** Ogni singolo elemento $i \in S$ ha la medesima probabilità di appartenere al campione:

$$\mathbb{P}(i \in D) = \frac{m}{n}. \tag{7.46}$$

- **Cardinalità Esatta Senza Reinserimento:** Il campione deve contenere esattamente m elementi distinti estratti senza reinserimento:

$$|D| = m. \tag{7.47}$$

Si assume disponibile una primitiva pseudo-casuale elementare:

$$\text{Rand}(a, b) \to r \in [a, b], \qquad \text{con } r \text{ intero distribuito uniformemente.} \tag{7.48}$$

7.4.2 Algoritmo 1: Rejection Sampling (Campionamento per Inserimento)

L'idea più intuitiva consiste nell'estrarre indici casuali continui e inserirli in una struttura dati dinamica per insiemi, scartando i duplicati fino al raggiungimento della cardinalità target m.

Algorithm 7: Rejection Sampling per Inserimento Casuale

Dati: Intervallo di indici $[1 \dots n]$, dimensione del campione desiderata $m \leq n$
Risultato: Insieme ordinato D contenente m indici distinti

```
D ← ∅
while |D| < m do
    p ← Rand(1, n)
    if p ∉ D then
        D ← D ∪ {p}
Sort(D)
return D
```

Analisi delle Collisioni e Probabilità di Fallimento

A ogni iterazione del ciclo `while`, la probabilità che il punto p appena estratto sia già presente nell'insieme D (generando una collisione e un conseguente tentativo a vuoto) è proporzionale al riempimento corrente:

$$\mathbb{P}(\text{collisione}) = \mathbb{P}(p \in D) = \frac{|D|}{n}. \tag{7.49}$$

Durante l'esecuzione, $|D|$ cresce da 0 a $m-1$. Pertanto, la probabilità di fallimento al generico passo è strettamente maggiorata dal valore finale:

$$\mathbb{P}(\text{fallimento}) < \frac{m}{n}. \tag{7.50}$$

Proprietà 7.4.1 (Regimi di Efficienza)**.** *L'efficienza dell'algoritmo dipende fortemente dal rapporto tra m e n:*

- ***Regime Sparso** ($m \leq \frac{n}{2}$)**:** La probabilità di fallimento al generico passo è $\mathbb{P}(\textit{fallimento}) < \frac{1}{2}$. Il numero atteso di estrazioni prima di ottenere un successo è inferiore a 2, rendendo l'algoritmo estremamente rapido. Più m è piccolo rispetto a n ($m \ll n$, come nell'over-sampling), minore è la frequenza di collisione.*
- ***Regime Denso** ($m \to n$)**:** Quando m si avvicina a n, la probabilità di estrarre un elemento non ancora campionato collassa verso zero. Si manifesta il fenomeno del **Coupon Collector's Problem** (problema del collezionista di figurine), che fa impennare il numero atteso complessivo di estrazioni a:*

$$\Theta(n \ln m). \tag{7.51}$$

Impatto delle Strutture Dati

In sede di analisi teorica e implementativa:

- **Gestione dell'insieme D:** Se D è implementato come **tabella hash**, il controllo di appartenenza $p \notin D$ e l'inserimento richiedono tempo atteso $\mathcal{O}(1)$. Se invece D fosse implementato mediante un array non ordinato, la ricerca lineare richiederebbe tempo $\mathcal{O}(|D|) = \mathcal{O}(m)$ per ciascuna estrazione.
- **Ordinamento finale:** La fase Sort(D) richiede tempo $\mathcal{O}(m \log m)$ in CPU per restituire gli indici in ordine crescente.

7.4.3 Elementi a Lunghezza Variabile (Stringhe)

Cosa accade se gli elementi di S sono stringhe o record a lunghezza variabile privi di un passo di memoria uniforme?

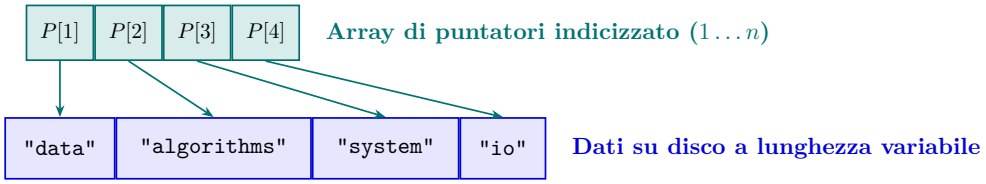


Figura 7.4: Mappatura di record a lunghezza variabile tramite array contiguo di puntatori indicizzato.

In un file sequenziale di stringhe a lunghezza arbitraria, una generica posizione scalare $p \in [1, n]$ generata da $\text{Rand}(1, n)$ non ha un corrispettivo offset in byte calcolabile immediatamente in $\mathcal{O}(1)$.

Per risolvere questo problema, è necessario costruire un **array di puntatori contigui** (*array of pointers*) $P[1 \dots n]$, dove l'i-esimo puntatore indirizza l'esatto offset di inizio dell'i-esima stringa. La generazione pseudo-casuale viene quindi eseguita direttamente sull'indice dell'array di puntatori in tempo $\mathcal{O}(1)$, consentendo l'accesso istantaneo al record associato.

7.4.4 Perché l'Algoritmo 1 Non è Adatto al Modello di Streaming

La trattazione evidenzia una fondamentale osservazione architetturale che fa da ponte diretto con la tematica degli stream:

Proprietà 7.4.2 (Limite Strutturale: Inapplicabilità al Modello di Streaming)**.** *L'Algoritmo 7 presenta limiti strutturali insanabili che ne impediscono l'impiego nel calcolo su flussi:*

1. ***Accesso Casuale Arbitrario (Random Access):*** *Richiede la facoltà di interrogare in qualsiasi momento qualsiasi indice casuale $p \in [1, n]$ dell'intero dataset.*

2. ***Violazione del Vincolo Single-Pass:*** *In uno stream continuo, i dati transitano sequenzialmente; non è consentito tornare indietro o saltare arbitrariamente tra posizioni remote.*

3. ***Dipendenza dalla Conoscenza di n:*** *La primitiva $\text{Rand}(1, n)$ presuppone la conoscenza esatta e preventiva della cardinalità totale n all'inizio della computazione.*

Questa constatazione pone le basi teoriche per gli **Algoritmi di Random Sampling su Data Stream** (analizzati nel Capitolo 8), nei quali la cardinalità n può essere ignota e la decisione di accettare o scartare ciascun elemento deve essere presa *on-the-fly* al suo passaggio.

Capitolo 8

Algoritmi di Random Sampling su Data Stream

8.1 Fondamenti del Modello di Streaming e del Random Sampling

Il trattamento di flussi informativi ad alta velocità e volume potenzialmente illimitato impone di ripensare i modelli di calcolo tradizionali, introducendo vincoli rigidi di memoria e tempo di risposta per singolo record. In questa sezione formalizziamo i pilastri architetturali del Data Streaming Model e definiamo i requisiti algebrico-probabilistici del campionamento casuale uniforme.

8.1.1 Il Modello di Calcolo su Flussi di Dati (Streaming Model)

Nel paradigma convenzionale di elaborazione dati (sia in RAM sia su memoria secondaria), un dataset è interamente disponibile per scansioni ripetute. Nel modello di calcolo a flussi (*Streaming Model*), al contrario, l'algoritmo non può controllare né l'ordine temporale né la frequenza di arrivo dei dati.

Lo scenario streaming è caratterizzato formalmente dai seguenti quattro pilastri architetturali:

1. **Flusso di Dati S:** una sequenza ordinata nel tempo, potenzialmente infinita o di dimensione massiva n ($n \to \infty$ o $n \gg M$), costituita da elementi discreti:
$$S = \langle i_1, i_2, \ldots, i_j, \ldots, i_n \rangle. \tag{8.1}$$

2. **Vincolo di Memoria di Lavoro Limitata ($M \ll n$):** la quantità di memoria rapida di lavoro (*working memory*) disponibile per l'algoritmo soddisfa strettamente la condizione $M \ll n$. Tipicamente, la memoria è confinata a:

$$M = \mathcal{O}(m) \quad \text{oppure} \quad M = \mathcal{O}(\text{polylog}(n)) \text{ parole di memoria,} \tag{8.2}$$

dove m denota la dimensione del campione statistico desiderato. Risulta pertanto strutturalmente impossibile memorizzare l'intero flusso o anche una frazione significativa di esso.

3. **Accesso Sequenziale Single-Pass:** gli elementi transitano uno alla volta in ordine cronologico. Una volta che un elemento i_j è transitato, se l'algoritmo decide di non memorizzarlo esplicitamente nella limitata memoria di lavoro, esso è **perso per sempre** e non potrà mai più essere consultato o recuperato.

4. **Tempo di Elaborazione per Elemento Rigorosamente Limitato:** per evitare colli di bottiglia o saturazione dei buffer di ingresso a fronte di flussi ad alta velocità (ad esempio pacchetti di rete o feed finanziari), il tempo necessario per elaborare ciascun elemento in arrivo i_j deve essere strettamente costante:

$$T(i_j) = \mathcal{O}(1) \quad \text{oppure al massimo} \quad T(i_j) = \mathcal{O}(\log m). \tag{8.3}$$

8.1.2 Definizione Formale del Problema di Campionamento Uniforme

Dato un flusso continuo S, l'obiettivo consiste nel selezionare e mantenere dinamicamente un sottoinsieme di m elementi che costituisca un **campionamento casuale semplice senza reinserimento** (*Simple Random Sampling without Replacement*, SRSWOR).

Formalmente, al generico passo temporale n (o al termine del flusso se n è finito), devono essere rigorosamente soddisfatti due requisiti di uniformità probabilistica:

Definizione 8.1.1 (Uniformità Marginale)**.** Ogni singolo elemento $i_j \in S$ (per $1 \leq j \leq n$) deve possedere la medesima probabilità marginale di appartenere al campione finale:

$$\mathbb{P}(i_j \in \text{Campione}) = \frac{m}{n}. \tag{8.4}$$

Definizione 8.1.2 (Uniformità Congiunta)**.** Tutti i possibili sottoinsiemi $S^* \subseteq S$ aventi cardinalità esattamente pari a m devono possedere la medesima probabilità congiunta di essere estratti:

$$\mathbb{P}(\text{Campione} = S^*) = \frac{1}{\binom{n}{m}}, \qquad \forall S^* \subseteq S \text{ tale che } |S^*| = m. \tag{8.5}$$

A seconda del grado di informazione disponibile a priori sulla cardinalità complessiva n dello stream, gli algoritmi di campionamento si bipartiscono in due categorie fondamentali:

- **Caso A (n noto a priori):** la lunghezza complessiva n del flusso è preventivamente conosciuta prima di iniziare il processo di campionamento (ad esempio la scansione sequenziale di un file di log di n record registrati su nastro o disco non indicizzato).
- **Caso B (n incognito a priori):** la cardinalità n non è nota all'inizio e il flusso può arrestarsi arbitrariamente in qualsiasi momento o proseguire indefinitamente. L'algoritmo deve garantire che, in *qualsiasi istante* $t \geq m$ il flusso venga interrotto, la memoria interna contenga sempre un campione casuale uniforme valido tra i primi t elementi transitati.

8.2 Campionamento con Lunghezza Nota

8.2.1 Selezione di un Singolo Elemento ($m = 1$)

Si consideri il caso elementare in cui si desideri campionare esattamente un singolo elemento ($m = 1$) da uno stream di lunghezza nota n, garantendo che ogni elemento abbia probabilità uniforme $\frac{1}{n}$ di essere scelto, decidendo in modalità puramente online se accettare o scartare l'elemento corrente i_j.

Formulazione dell'Algoritmo

L'algoritmo esamina sequenzialmente gli elementi per $j = 1, 2, \ldots, n$:

1. Ad ogni passo j, si estrae una variabile casuale continua pseudo-uniforme $p \sim \text{Uniform}(0, 1)$, ovvero $p \in [0, 1)$.
2. Si valuta la condizione di arresto e selezione:

$$\text{Se } p \leq \frac{1}{n - j + 1}, \quad \text{allora seleziona } i_j \text{ e termina l'esecuzione.} \tag{8.6}$$

3. Se la condizione non è soddisfatta ($p > \frac{1}{n-j+1}$), l'elemento i_j viene scartato definitivamente e si procede al passo successivo $j+1$.

Algorithm 8: Campionamento Online di un Singolo Elemento ($m = 1$, n noto)

Dati: Stream $S = \langle i_1, \ldots, i_n \rangle$ di lunghezza nota n
Risultato: Un singolo elemento $x \in S$ campionato uniformemente con $\mathbb{P}(x = i_j) = \frac{1}{n}$

```
for j ← 1 to n do
    p ← Uniform(0, 1)
    if p ≤ 1/(n−j+1) then
        return i_j
```

Dimostrazione Rigorosa per Induzione

Teorema 8.2.1. *L'Algoritmo 8 garantisce che ogni elemento $i_j \in S$ ha esattamente probabilità $\frac{1}{n}$ di essere selezionato:*

$$\mathbb{P}(\textit{selezionare } i_j) = \frac{1}{n}, \qquad \forall j \in \{1, 2, \ldots, n\}. \tag{8.7}$$

Dimostrazione. Procediamo per induzione sull'indice j dello stream.

Base dell'induzione ($j = 1$):
Al primo passo, per l'elemento i_1, la soglia di selezione è:

$$\frac{1}{n-1+1} = \frac{1}{n}. \tag{8.8}$$

Poiché $p \sim \text{Uniform}(0, 1)$, la probabilità che $p \leq \frac{1}{n}$ corrisponde direttamente alla misura dell'intervallo:

$$\mathbb{P}(\text{pick } i_1) = \frac{1}{n}. \tag{8.9}$$

La base induttiva è verificata.

Ipotesi induttiva:
Assumiamo che la proprietà sia valida per tutti gli indici precedenti $k \in \{1, 2, \ldots, j-1\}$, ossia che per ciascuno di essi la probabilità di essere stato selezionato sia stata:

$$\mathbb{P}(\text{pick } i_k) = \frac{1}{n}. \tag{8.10}$$

Poiché l'algoritmo termina immediatamente non appena seleziona un elemento, gli eventi $\{\text{pick } i_k\}$ sono mutuamente disgiunti. Pertanto, la probabilità

cumulativa che almeno uno dei primi $j-1$ elementi sia stato selezionato è la somma diretta delle singole probabilità:

$$\mathbb{P}\left(\bigcup_{k=1}^{j-1}\{\text{pick } i_k\}\right) = \sum_{k=1}^{j-1}\mathbb{P}(\text{pick } i_k) = \sum_{k=1}^{j-1}\frac{1}{n} = \frac{j-1}{n}. \tag{8.11}$$

La probabilità complementare che *nessun* elemento tra i primi $j-1$ sia stato selezionato risulta:

$$\begin{aligned}\mathbb{P}(\text{nessun elemento selezionato in } \{i_1,\ldots,i_{j-1}\}) &= 1-\frac{j-1}{n}\\ &= \frac{n-j+1}{n}.\end{aligned} \tag{8.12}$$

Passo induttivo:
Affinché l'elemento i_j venga selezionato dall'algoritmo al passo j, devono verificarsi congiuntamente due eventi:

1. Nessun elemento precedente tra $i_1,\ldots,i_{j-1}$ deve essere stato selezionato.
2. Al passo corrente j, la variabile casuale estratta p deve soddisfare la disuguaglianza $p \leq \frac{1}{n-j+1}$.

$$\begin{aligned}\mathbb{P}(\text{pick } i_j) &= \mathbb{P}\left(p \leq \frac{1}{n-j+1}\,\middle|\,\text{nessun elemento selezionato prima}\right)\\ &\quad\cdot\mathbb{P}(\text{nessun elemento selezionato prima})\\ &= \frac{1}{n-j+1}\cdot\left(\frac{n-j+1}{n}\right) = \frac{1}{n}.\end{aligned} \tag{8.13}$$

I due fattori $(n-j+1)$ si elidono perfettamente, restituendo la costante $\frac{1}{n}$. L'asserto è dimostrato per ogni $j \in \{1,\ldots,n\}$. □

Interpretazione Geometrica

Il meccanismo probabilistico alla base dell'Algoritmo 8 ammette un'elegante interpretazione geometrica visualizzata nella Figura 8.1.

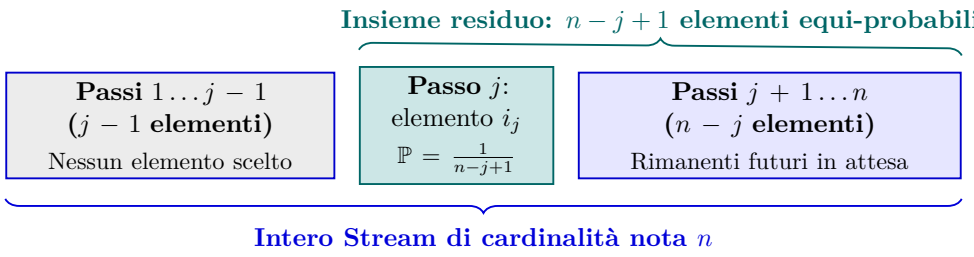


Figura 8.1: Partizione dello spazio dello stream al passo generico j per il campionamento a singolo elemento ($m = 1$).

Se nei primi $j-1$ passi non è stato selezionato alcun elemento, l'elemento target deve necessariamente risiedere nei rimanenti $n-j+1$ elementi. Poiché per uniformità tutti i restanti hanno pari diritto di essere scelti, la probabilità condizionata di eleggere immediatamente i_j è l'inverso del numero di elementi residui: $\frac{1}{n-j+1}$.

8.2.2 Selezione di m Elementi: Algorithm S (Fan, Muller, Rezucha / Knuth)

Quando la taglia richiesta per il campione è $m > 1$ e la cardinalità complessiva n è nota a priori, l'approccio viene generalizzato mediante l'**Algorithm S** (formalizzato da Fan, Muller e Rezucha [7] e reso celebre da Knuth [1]), mantenendo un contatore dinamico degli elementi già selezionati.

Definizione delle Variabili di Stato

L'algoritmo mantiene costantemente aggiornate le seguenti grandezze durante la scansione sequenziale:

- n: numero totale di elementi dello stream;
- m: dimensione del campione target desiderato;
- $j \in \{1, 2, \dots, n\}$: indice temporale dell'elemento corrente esaminato i_j;
- s: numero di elementi già selezionati ed entrati nel campione nei primi $j-1$ passi ($0 \leq s \leq m$);
- $m - s$: numero di elementi che *devono ancora* essere selezionati per completare il campione;
- $n - j + 1$: numero complessivo di elementi dello stream ancora da esaminare (incluso i_j).

Regola di Selezione e Casi Limite

Ad ogni iterazione $j \in \{1, \ldots, n\}$:

1. Si estrae $p \sim \text{Uniform}(0, 1)$.
2. Si confronta p con la soglia dinamica $\frac{m-s}{n-j+1}$:

$$p \leq \frac{m-s}{n-j+1}. \tag{8.14}$$

3. Se la disuguaglianza è soddisfatta: si inserisce i_j nel campione e si incrementa il contatore:

$$s \leftarrow s + 1. \tag{8.15}$$

   Altrimenti, i_j viene scartato definitivamente e s rimane inalterato.

L'algoritmo gestisce automaticamente i seguenti due casi di frontiera:

- **Campionamento Completato ($s = m$):** il numeratore della soglia diventa $m-s = 0$. La probabilità di selezione crolla a zero ($\frac{0}{n-j+1} = 0$), impedendo l'inserimento di ulteriori elementi e consentendo la terminazione anticipata.
- **Selezione Forzata di Tutti i Rimanenti ($m - s = n - j + 1$):** il numero di elementi mancanti coincide con il numero di elementi residui. La soglia diventa $\frac{m-s}{n-j+1} = 1$, forzando deterministicamente l'inclusione di tutti gli elementi restanti poiché $p < 1$ con probabilità 1.

Algorithm 9: Algorithm S (Selection Sampling di Fan, Muller, Rezucha / Knuth)

Dati: Stream $S = \langle i_1, \ldots, i_n \rangle$ di lunghezza nota n, taglia campione $m \leq n$

Risultato: Insieme C contenente m elementi campionati uniformemente

```
C ← ∅
s ← 0                          // Contatore elementi già campionati
for j ← 1 to n do
    if s = m then
        break                  // Campione già completato: terminazione anticipata
    p ← Uniform(0, 1)
    if p ≤ (m − s)/(n − j + 1) then
        C ← C ∪ {i_j}
        s ← s + 1
return C
```

Derivazione Combinatoria della Soglia

La derivazione analitica della soglia $\frac{m-s}{n-j+1}$ discende dai primi principi del calcolo combinatorio e della distribuzione ipergeometrica.

Teorema 8.2.2. *Dati s elementi già selezionati nei primi $j-1$ passi, la probabilità condizionata che l'elemento corrente i_j appartenga a un campione uniforme di cardinalità m estratto dai residui $n-j+1$ elementi è data esattamente da:*

$$\mathbb{P}(i_j \in \textit{Campione} \mid s \textit{ già campionati}) = \frac{m-s}{n-j+1}. \tag{8.16}$$

Dimostrazione. La probabilità condizionata cercata corrisponde al rapporto tra casi favorevoli e casi possibili:

$$\mathbb{P}(\text{pick } i_j \mid s) = \frac{\text{Casi favorevoli (completare il campione con } i_j)}{\text{Casi possibili (estrarre } m-s \text{ elementi dai residui)}}. \tag{8.17}$$

Se vincoliamo l'inclusione di i_j, abbiamo stabilito 1 elemento; restano pertanto da selezionare $m-s-1$ elementi dai successivi $n-j$ elementi disponibili nello stream. Il numero di modi possibili per compiere tale scelta è espresso dal coefficiente binomiale:

$$\binom{n-j}{m-s-1}. \tag{8.18}$$

D'altro canto, il numero totale di configurazioni non vincolate per scegliere $m - s$ elementi tra tutti gli $n - j + 1$ elementi ancora da esaminare è dato da:

$$\binom{n-j+1}{m-s}. \tag{8.19}$$

Rapportando i due coefficienti binomiali e sviluppando i fattoriali:

$$\frac{\binom{n-j}{m-s-1}}{\binom{n-j+1}{m-s}} = \frac{\dfrac{(n-j)!}{(m-s-1)!\,((n-j)-(m-s-1))!}}{\dfrac{(n-j+1)!}{(m-s)!\,((n-j+1)-(m-s))!}}$$

$$= \frac{\dfrac{(n-j)!}{(m-s-1)!\,(n-j-m+s+1)!}}{\dfrac{(n-j+1)!}{(m-s)!\,(n-j-m+s+1)!}}. \tag{8.20}$$

I termini fattoriali $(n-j-m+s+1)!$ a denominatore di entrambe le frazioni sono identici e si semplificano reciprocamente. Riorganizzando i termini:

$$= \frac{(n-j)!}{(n-j+1)!} \cdot \frac{(m-s)!}{(m-s-1)!}. \tag{8.21}$$

Applicando la proprietà fondamentale del fattoriale ($k! = k \cdot (k-1)!$):

$$(n-j+1)! = (n-j+1) \cdot (n-j)!, \tag{8.22}$$

$$(m-s)! = (m-s) \cdot (m-s-1)!. \tag{8.23}$$

Sostituendo nelle rispettive frazioni:

$$= \frac{1}{n-j+1} \cdot (m-s) = \frac{m-s}{n-j+1}. \tag{8.24}$$

La soglia dell'Algoritmo S preserva dunque con assoluta esattezza algebrica la distribuzione ipergeometrica uniforme globale. □

8.2.3 Esecuzione Guidata ed Esempio Pratico dell'Algoritmo S

Per apprezzare il funzionamento deterministico delle decisioni stocastiche, ripercorriamo l'esecuzione guidata descritta negli appunti di lezione.

Esempio 8.2.1 (Traccia di Esecuzione di Algorithm S)**.** Si consideri la seguente istanza di campionamento:

- Stream in ingresso: $S = \langle a, b, c, d, e, f, g, h \rangle \implies n = 8$.
- Dimensione del campione target desiderato: $m = 2$.
- Sequenza di numeri pseudo-casuali estratti $p \in [0, 1)$ associati a ciascun passo:

$$p = \langle 0.5,\ 0.5,\ 0.1,\ 0.5,\ 0.5,\ 0.2,\ 0.9,\ 0.3 \rangle. \tag{8.25}$$

L'evoluzione completa dello stato dell'algoritmo passo dopo passo è dettagliata nella Tabella 8.1.

Tabella 8.1: Traccia dell'esecuzione passo-passo dell'Algorithm S ($n = 8$, $m = 2$).

j	i_j	s	$m - s$	$n - j + 1$	Soglia $\frac{m-s}{n-j+1}$	p	Condizione $p \leq$ Soglia	Esito	Nuovo s
1	a	0	2	8	$\frac{2}{8} = 0.250$	0.5	$0.5 \leq 0.250$ (Falso)	Scartato (×)	0
2	b	0	2	7	$\frac{2}{7} \approx 0.286$	0.5	$0.5 \leq 0.286$ (Falso)	Scartato (×)	0
3	c	0	2	6	$\frac{2}{6} \approx 0.333$	0.1	$0.1 \leq 0.333$ (Vero)	**Selezionato (✓)**	1
4	d	1	1	5	$\frac{1}{5} = 0.200$	0.5	$0.5 \leq 0.200$ (Falso)	Scartato (×)	1
5	e	1	1	4	$\frac{1}{4} = 0.250$	0.5	$0.5 \leq 0.250$ (Falso)	Scartato (×)	1
6	f	1	1	3	$\frac{1}{3} \approx 0.333$	0.2	$0.2 \leq 0.333$ (Vero)	**Selezionato (✓)**	2
7	g	2	0	2	$\frac{0}{2} = 0.000$	0.9	$0.9 \leq 0.000$ (Falso)	Scartato / STOP	2
8	h	2	0	1	$\frac{0}{1} = 0.000$	0.3	$0.3 \leq 0.000$ (Falso)	Scartato / STOP	2

Risultato: Al passo $j = 6$, il contatore ha raggiunto la quota $s = m = 2$, completando il campione con l'insieme $C = \{c, f\}$. Ai passi successivi 7 e 8, la soglia diventa nulla, provocando lo scarto sistematico e immediato dei restanti elementi.

8.3 Campionamento con Lunghezza Incognita

Nelle applicazioni streaming reali su flussi di rete, logging continuo e feed sensoriali, la dimensione totale n non è nota all'inizio e il flusso può terminare arbitrariamente in qualunque istante. In questo scenario, gli algoritmi devono mantenere un campione valido in ogni singolo momento.

8.3.1 Priority Sampling con Min-Heap (Random Keys Sampling)

Un primo approccio per campionare senza conoscere n si fonda sulla tecnica delle **chiavi casuali** (*Random Keys Sampling* o *Priority Sampling*).

Principio Funzionale

A ciascun elemento in arrivo i_j dello stream viene associata una chiave continua pseudo-casuale π_j estratta in modo indipendente e identicamente distribuito (*i.i.d.*):

$$\pi_j \sim \text{Uniform}(0, 1). \tag{8.26}$$

L'obiettivo consiste nel mantenere dinamicamente gli m elementi associati alle m chiavi più grandi (o equivalentemente più piccole) osservate finora. Poiché le variabili casuali π_j sono continue e i.i.d., ogni sottoinsieme di cardinalità m tra i primi j elementi ha esattamente la medesima probabilità di detenere le m chiavi dominanti.

Gestione mediante Min-Heap Bounded

Per implementare l'algoritmo mantenendo l'ingombro di memoria confinato a $\mathcal{O}(m)$, si impiega una coda con priorità strutturata come un **Min-Heap** H di capacità fissa m, che memorizza coppie ordinate $\langle \pi_j, i_j \rangle$ ordinate rispetto alla chiave π_j:

- **Primi m elementi ($j \leq m$):** ciascuna coppia $\langle \pi_j, i_j \rangle$ viene inserita direttamente nell'heap H. La costruzione iniziale richiede tempo $\mathcal{O}(m)$.
- **Elementi successivi ($j > m$):** per ogni nuovo elemento i_j con chiave π_j:
  1. Si esamina la radice del Min-Heap, che custodisce la minima priorità tra le m attualmente nel campione:
  $$\pi_{\min} = \text{min_key}(H). \tag{8.27}$$
  2. **Se $\pi_j \leq \pi_{\min}$:** la nuova chiave non fa parte delle m migliori. L'elemento i_j viene scartato immediatamente con tempo $\mathcal{O}(1)$.
  3. **Se $\pi_j > \pi_{\min}$:** l'elemento che detiene $\pi_{\min}$ viene rimosso dal campione. Si esegue un'operazione combinata di sostituzione della radice (`Replace-Min`) con la nuova coppia $\langle \pi_j, i_j \rangle$ e successivo ripristino dell'invariante di heap (`Heapify-Down`), con costo $\mathcal{O}(\log m)$.

L'architettura del Priority Sampling con Min-Heap è illustrata nella Figura 8.2.

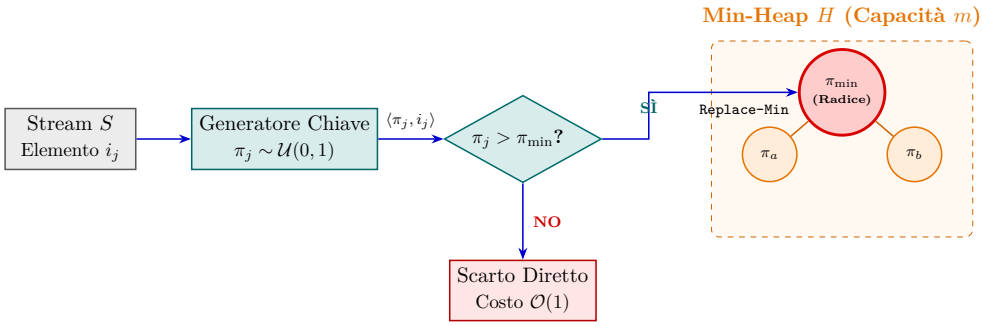


Figura 8.2: Schema funzionale del Priority Sampling su stream mediante Min-Heap di capienza fissa m.

Proprietà 8.3.1 (Analisi di Complessità di Priority Sampling)**.** *L'occupazione di memoria è limitata a $\mathcal{O}(m)$ parole. Nel caso peggiore, l'elaborazione di un elemento richiede tempo $\mathcal{O}(\log m)$. Tuttavia, poiché la probabilità che il j-esimo elemento entri nel campione è pari a $\frac{m}{j}$, il numero atteso di operazioni di inserimento nell'heap su un flusso di n elementi è pari a:*

$$\sum_{j=m+1}^{n} \frac{m}{j} = m \sum_{j=m+1}^{n} \frac{1}{j} = \mathcal{O}(m \log(n/m)). \tag{8.28}$$

Il costo computazionale medio ammortizzato per elemento è dunque:

$$T_{medio} = \mathcal{O}\left(1 + \frac{m \log m \log(n/m)}{n}\right) = \mathcal{O}(1) \quad \textit{per } n \gg m. \tag{8.29}$$

Ciononostante, il Priority Sampling richiede una generazione di numero casuale per ciascuno *degli n elementi e una costante moltiplicativa legata alla gestione dell'heap.*

8.3.2 Reservoir Sampling (Algoritmo R di Waterman / Knuth)

Per ottenere un tempo di elaborazione strettamente $\mathcal{O}(1)$ nel caso peggiore senza l'onere computazionale dell'heap, l'algoritmo di riferimento universale è il **Reservoir Sampling** (originariamente introdotto da Alan G. Waterman e formalizzato come **Algoritmo R** da Knuth [1]).

Meccanismo Operativo

L'algoritmo impiega un semplice array di memoria di lavoro $R[1 \dots m]$, denominato **serbatoio** (*reservoir*). L'esecuzione si articola in due fasi essenziali:

1. **Fase 1 (Inizializzazione Deterministica):** i primi m elementi dello stream vengono inseriti incondizionatamente e direttamente nei rispettivi slot del reservoir:

$$R[j] \leftarrow i_j, \qquad \forall j \in \{1, 2, \ldots, m\}. \tag{8.30}$$

Durante questa fase non viene spesa alcuna chiamata al generatore di numeri casuali.

2. **Fase 2 (Sostituzione Online Competitiva):** per ogni elemento successivo i_j con indice temporale $j = m+1, m+2, \ldots$:

   (a) Si estrae un numero intero pseudo-casuale uniforme discreto k nell'intervallo compreso tra 1 e j:

$$k \sim \text{rand}(1, j). \tag{8.31}$$

   (b) Si valuta la condizione di ammissione:

   - Se $k \leq m$: l'elemento corrente i_j entra a far parte del serbatoio, **rimpiazzando** lo slot k:

$$R[k] \leftarrow i_j. \tag{8.32}$$

     L'elemento precedentemente occupante $R[k]$ viene rimosso definitivamente e scartato.
   - Se $k > m$: l'elemento corrente i_j viene scartato immediatamente senza modificare il reservoir.

Algorithm 10: Reservoir Sampling (Algoritmo R di Waterman / Knuth)

Dati: Stream continuo $S = \langle i_1, i_2, \ldots \rangle$, dimensione serbatoio m
Risultato: Array $R[1 \ldots m]$ contenente il campione casuale uniforme in ogni istante

```
for j ← 1 to m do
    R[j] ← i_j        // Riempi il serbatoio iniziale senza RNG
j ← m + 1
while Stream non terminato ed esiste un nuovo elemento i_j do
    k ← RandomInteger(1, j)
    if k ≤ m then
        R[k] ← i_j    // Sostituisci lo slot k con probabilità m/j
    j ← j + 1
return R
```

L'architettura e il meccanismo di rimpiazzo dinamico sono schematizzati nella Figura 8.3.

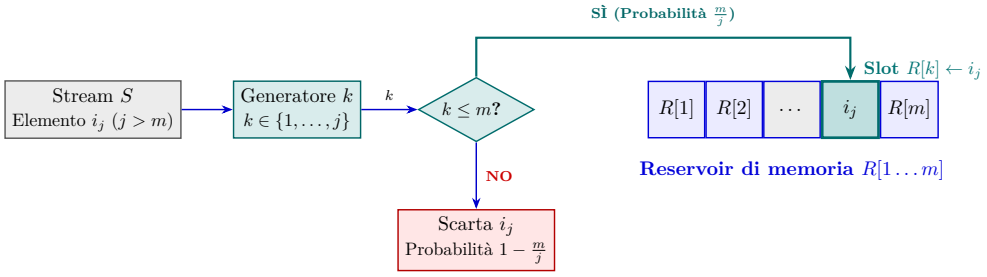


Figura 8.3: Flusso operativo di inserimento e sostituzione del Reservoir Sampling (Algoritmo R).

8.3.3 Esecuzione Guidata ed Esempio Pratico di Reservoir Sampling

Illustriamo la dinamica del Reservoir Sampling ripercorrendo l'esempio didattico su uno stream di cardinalità finita.

Esempio 8.3.1 (Traccia di Esecuzione di Reservoir Sampling)**.** Si consideri il flusso sequenziale:

$$S = \langle a, b, c, d, e, f, g, h, i \rangle, \qquad \text{con taglia serbatoio } m = 3. \tag{8.33}$$

- **Inizializzazione** $(j = 1, 2, 3)$**:** i primi $m = 3$ elementi vengono memorizzati in R:
$$R = [a,\ b,\ c]. \tag{8.34}$$
- **Passi successivi** $(j = 4, 5, 6, 7)$**:** l'evoluzione con i valori casuali di k estratti è riassunta nella Tabella 8.2.

Tabella 8.2: Traccia dell'esecuzione del Reservoir Sampling ($m = 3$).

Passo j	Elemento i_j	Intero Casuale k	$k \leq m$	Azione Eseguita	Stato Reservoir $R[1 \ldots 3]$
$1 \ldots 3$	a, b, c	—	—	Inizializzazione deterministica	$[a, b, c]$
4	d	$k = 2 \in \{1 \ldots 4\}$	Vero $(2 \leq 3)$	Sostituzione nello slot $R[2] \leftarrow d$	$[a, d, c]$
5	e	$k = 5 \in \{1 \ldots 5\}$	Falso $(5 > 3)$	Scarto immediato di e	$[a, d, c]$
6	f	$k = 1 \in \{1 \ldots 6\}$	Vero $(1 \leq 3)$	Sostituzione nello slot $R[1] \leftarrow f$	$[f, d, c]$
7	g	$k = 4 \in \{1 \ldots 7\}$	Falso $(4 > 3)$	Scarto immediato di g	$[f, d, c]$

8.4 Dimostrazione Formale per Induzione del Reservoir Sampling

La correttezza matematica del Reservoir Sampling garantisce che, in qualunque istante si interrompa il flusso, ogni elemento transitato ha pari probabilità di risiedere nel campione. Presentiamo la dimostrazione analitica completa.

Teorema 8.4.1 (Correttezza del Reservoir Sampling)**.** *Per ogni flusso S, al generico passo temporale $n \geq m$, per ciascun elemento i_j transitato fino a quel momento ($1 \leq j \leq n$), la probabilità che esso appartenga al serbatoio R è esattamente:*

$$\mathbb{P}(i_j \in R) = \frac{m}{n}. \tag{8.35}$$

Dimostrazione. Procediamo per induzione matematica sulla lunghezza n del flusso osservato finora, considerando fissata la taglia del serbatoio m.

Base dell'induzione ($n = m$):
Al termine dei primi m passi, tutti gli elementi $i_1, \ldots, i_m$ sono stati inseriti deterministicamente nel serbatoio R:

$$R = [i_1, i_2, \ldots, i_m]. \tag{8.36}$$

Pertanto, per ogni $j \in \{1, \ldots, m\}$:

$$\mathbb{P}(i_j \in R) = 1 = \frac{m}{m}. \tag{8.37}$$

La base induttiva è verificata.

Ipotesi induttiva:
Assumiamo che la proprietà sia verificata al passo $n-1$, ovvero che per ogni elemento i_j con $1 \leq j \leq n-1$:

$$\mathbb{P}(i_j \in R \text{ al termine del passo } n-1) = \frac{m}{n-1}. \tag{8.38}$$

Passo induttivo (transizione dal passo $n-1$ al passo n):
All'arrivo dell'elemento n-esimo i_n, l'algoritmo estrae un indice casuale uniforme discreto:

$$k \sim \text{rand}(1, n). \tag{8.39}$$

Distinguiamo due casi mutuamente esaustivi:

- **Caso 1: Ultimo elemento transitato ($j = n$):**
  L'elemento i_n viene ammesso nel serbatoio R se e solo se l'indice estratto

soddisfa $k \leq m$. Poiché k è scelto uniformemente tra i valori interi $\{1, 2, \ldots, n\}$:

$$\mathbb{P}(i_n \in R) = \mathbb{P}(k \leq m) = \frac{m}{n}. \tag{8.40}$$

L'asserzione è verificata per $j = n$.

- **Caso 2: Qualsiasi elemento precedente ($j < n$):**
  Consideriamo un generico elemento i_j con $1 \leq j \leq n-1$. Affinché i_j si trovi ancora all'interno del serbatoio al termine del passo n, devono realizzarsi due condizioni congiunte:

  1. i_j doveva appartenere al serbatoio R al termine del passo precedente $n-1$;
  2. i_j deve **sopravvivere** al passo n, ossia non deve essere sovrascritto dal nuovo arrivato i_n.

  Formalmente, applicando la regola del prodotto di probabilità condizionata:

$$\begin{aligned}\mathbb{P}(i_j \in R \text{ al passo } n) = \mathbb{P}(i_j \in R \text{ al passo } n-1) \\ \cdot \mathbb{P}(i_j \text{ non sostituito} \mid i_j \in R \text{ al passo } n-1).\end{aligned} \tag{8.41}$$

  Per l'ipotesi induttiva (8.38), il primo fattore è pari a $\frac{m}{n-1}$.

  Valutiamo l'evento di sopravvivenza E_{surv}, ovvero che i_j non sia sovrascritto al passo n. Un elemento presente in R sopravvive in due scenari:

  – *Scenario A:* Il nuovo elemento i_n non entra nel serbatoio (ovvero $k > m$). Questo evento accade con probabilità:

$$\mathbb{P}(k > m) = 1 - \frac{m}{n}. \tag{8.42}$$

    In tale eventualità, nessuno slot viene alterato e i_j si preserva con certezza.

  – *Scenario B:* Il nuovo elemento i_n entra nel serbatoio ($k \leq m$, evento con probabilità $\frac{m}{n}$), **MA** il numero di slot estratto per la sostituzione è diverso da quello attualmente occupato da i_j. Poiché lo slot k viene scelto con distribuzione uniforme tra gli m slot disponibili $\{1, \ldots, m\}$, la probabilità di colpire uno degli altri $m-1$ slot è:

$$\frac{m-1}{m}. \tag{8.43}$$

Sommando le probabilità dei due scenari mutuamente disgiunti, otteniamo la probabilità totale di sopravvivenza:

$$\begin{aligned}\mathbb{P}(E_{\text{surv}}) &= \left(1 - \frac{m}{n}\right) + \left(\frac{m}{n} \cdot \frac{m-1}{m}\right) \\ &= \frac{n-m}{n} + \frac{m-1}{n} \\ &= \frac{n-m+m-1}{n} \\ &= \frac{n-1}{n}. \end{aligned} \tag{8.44}$$

Sostituendo l'espressione di sopravvivenza (8.44) e l'ipotesi induttiva:

$$\begin{aligned}\mathbb{P}(i_j \in R \text{ al passo } n) &= \left(\frac{m}{n-1}\right) \cdot \left(\frac{n-1}{n}\right) \\ &= \frac{m}{n}. \end{aligned} \tag{8.45}$$

Il termine $(n-1)$ si elide algebricamente, dimostrando l'asserto per ogni elemento pregresso $j < n$.

Avendo validato la proprietà sia per $j = n$ sia per tutti i $j < n$, il passo induttivo è completato:

$$\mathbb{P}(i_j \in R) = \frac{m}{n}, \qquad \forall j \in \{1, 2, \ldots, n\}. \tag{8.46}$$

Il teorema di correttezza del Reservoir Sampling è formalmente dimostrato. □

8.5 Sintesi Comparativa

A integrazione dell'analisi teorica, la Tabella 8.3 sintetizza le caratteristiche prestazionali, la memoria di lavoro e i costi computazionali dei tre paradigmi di campionamento esaminati.

Tabella 8.3: Confronto sintetico tra le metodologie di campionamento su flussi di dati.

Algoritmo	**Conoscenza di n**	**Memoria**	**Costo / Elemento**	**Chiamate RNG**	**Struttura Dati**
Algorithm S (Knuth)	Nota a priori	$\mathcal{O}(1)$	$\mathcal{O}(1)$	n (1 per elemento)	Array statico del campione
Priority Sampling	Non nota	$\mathcal{O}(m)$	$\mathcal{O}(\log m)$	n (1 per elemento)	Min-Heap di capienza fissa m
Reservoir Sampling (Alg. R)	Non nota	$\mathcal{O}(m)$	$\mathcal{O}(1)$	$n - m$ (0 nei primi m)	Array serbatoio di taglia m

Nota di Ingegneria degli Algoritmi (Approfondimento Facoltativo: Limite Pratico dell'RNG): Nelle applicazioni reali ad altissimo throughput ($n \gg 10^9$), l'Algoritmo R classico presenta un costo pratico non trascurabile: sebbene richieda tempo $\mathcal{O}(1)$ per elemento, esso interroga il generatore di numeri pseudo-casuali (RNG) per ciascuno degli $n - m$ elementi successivi. Poiché la probabilità di ammissione $\frac{m}{j}$ decresce verso zero, la stragrande maggioranza delle chiamate genera un valore $k > m$, traducendosi in un'inutile perdita di cicli CPU per elementi immediatamente scartati. Per eliminare questo overhead, l'**Algoritmo Z di Vitter** [8] modella il numero di elementi da scartare prima del successivo inserimento come una variabile casuale (*skip distance*). Campionando direttamente tale distanza, l'algoritmo salta interi blocchi di dati senza interrogare l'RNG per ogni elemento ignorato, riducendo le chiamate casuali totali da $\mathcal{O}(n)$ ad $\mathcal{O}(m(1 + \log(n/m)))$.

Capitolo 9

Strutture Dati Randomizzate: Il Problema del Dizionario e Skip List

9.1 Introduzione e Motivazione: Il Problema del Dizionario

La gestione dinamica di insiemi ordinati di chiavi rappresenta un requisito primario nei database e nelle architetture software moderne. In questa sezione formalizziamo le operazioni fondamentali del problema del dizionario e analizziamo i limiti intrinseci delle strutture dati deterministiche tradizionali.

9.1.1 Formalizzazione del Problema del Dizionario (Dictionary Problem)

In informatica teorica e nella progettazione di sistemi ad alte prestazioni, il **problema del dizionario** modella la gestione dinamica di un insieme finito di elementi S prelevati da un universo totalmente ordinato $\mathcal{U}$, ciascuno indicizzato da una chiave univoca $k \in \mathcal{U}$.

Definizione 9.1.1 (Problema del Dizionario Dinamico)**.** Sia $\mathcal{U}$ un universo totalmente ordinato dotato di una relazione d'ordine totale $\leq$, e sia $S \subseteq \mathcal{U}$ un insieme dinamico di cardinalità $|S| = n$. Una struttura dati per il dizionario mantiene S sotto l'esecuzione di una sequenza arbitraria delle seguenti operazioni fondamentali:

1. SEARCH(k): determina se esiste un elemento in S con chiave k. Se presente, restituisce il puntatore/riferimento all'oggetto corrispondente; altrimenti restituisce un valore sentinella nullo (NIL).

2. INSERT(x): aggiunge un nuovo elemento x con chiave $k = \text{Key}(x)$ all'interno dell'insieme, aggiornando $S \leftarrow S \cup \{x\}$.

3. DELETE(x): rimuove l'elemento x da S, aggiornando $S \leftarrow S \setminus \{x\}$.

Oltre a queste operazioni cardine, nei sistemi reali (come DBMS e motori di indicizzazione) è indispensabile supportare in modo efficiente le **query basate sull'ordine** (*order-related queries*):

4. PREDECESSOR(k): restituisce l'elemento in S avente la massima chiave strettamente minore di k, ovvero:

$$\text{pred}(k) = \text{argmax}_{x \in S}\{\text{Key}(x) \mid \text{Key}(x) < k\}. \tag{9.1}$$

5. SUCCESSOR(k): restituisce l'elemento in S avente la minima chiave strettamente maggiore di k, ovvero:

$$\text{succ}(k) = \text{argmin}_{x \in S}\{\text{Key}(x) \mid \text{Key}(x) > k\}. \tag{9.2}$$

9.1.2 Confronto tra Strutture Dati Tradizionali Deterministiche

La progettazione classica offre differenti compromessi spazio-temporali per risolvere il problema del dizionario. La Tabella 9.1 sintetizza le complessità computazionali asintotiche delle principali strutture dati deterministiche.

Struttura Dati	Search	Insert	Delete	Pred/Succ	Note Implementative
Array Ordinato	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	Ricerca binaria ottima su dati statici; inserimenti e cancellazioni richiedono shift di memoria contigua.
Lista Concatenata	$\mathcal{O}(n)$	$\mathcal{O}(1)^*$	$\mathcal{O}(1)^*$	$\mathcal{O}(n)$	Aggiornamenti locali $\mathcal{O}(1)$ solo con puntatore noto; ricerca strettamente sequenziale.
BST Bilanciati (AVL, Red-Black)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	Ottimi nel caso peggiore, ma overhead elevato: metadati per nodo, rotazioni e lock pesanti.

Tabella 9.1: Confronto asintotico tra strutture dati deterministiche tradizionali nel caso peggiore (*costo locale dato il puntatore).

9.1.3 Perché Introdurre la Randomizzazione?

L'approccio deterministico classico presenta due colli di bottiglia architetturali critici nei moderni sistemi di elaborazione ad elevate prestazioni:

1. **Costo di Ristrutturazione e Complessità Implementativa:** Il mantenimento del bilanciamento perfetto o quasi-perfetto negli alberi di ricerca richiede controlli topologici complessi, memorizzazione di metadati per nodo (fattori di bilanciamento negli AVL, bit di colore nei Red-Black) e catene di rotazioni ad albero (singole e doppie) durante le mutazioni.

2. **Collo di Bottiglia nei Sistemi Concorrenti Multi-Thread:** Negli alberi bilanciati deterministici, un singolo inserimento o una cancellazione può innescare una cascata di rotazioni che si propaga dalla foglia fino alla radice dell'albero. Nei moderni ambienti multi-core ad alto parallelismo, ciò richiede l'acquisizione di blocchi esclusivi (*exclusive locks*) su ampie porzioni dell'albero o addirittura sulla radice globale, distruggendo la scalabilità concorrente e creando contesa massiva sui lock.

L'introduzione della **randomizzazione** rivoluziona questo scenario delegando la topologia della struttura dati a scelte casuali generate internamente (simulazioni di lanci di moneta bilanciata o generazione di priorità casuali indipendenti):

- **Indipendenza dalla Distribuzione dei Dati:** Le prestazioni non dipendono in alcun modo dall'ordine di arrivo o dalla distribuzione delle chiavi fornite dall'utente (evitando casi degeneri pilotati da input avversari o chiavi già ordinate), bensì unicamente dalle decisioni casuali del generatore pseudocasuale interno.
- **Garanzie Logaritmiche con Alta Probabilità:** Le complessità operative $\mathcal{O}(\log n)$ non sono soddisfatte solo in valore atteso $\mathbb{E}[T]$, ma sono garantite **con alta probabilità** (*with high probability*, w.h.p.), ossia con probabilità almeno pari a $1 - \mathcal{O}(n^{-c})$ per qualsiasi costante fissata $c \geq 1$.
- **Modularità, Semplicità e Concorrenza Lock-Free:** Le strutture randomizzate sono intrinsecamente basate su puntatori locali (lineari o a cascata), rendendo banale l'implementazione di algoritmi concorrenti privi di blocchi globali (*lock-free* basati su primitive atomiche *Compare-And-Swap*, CAS).

Applicazioni Industriali ad Alte Prestazioni. Nei moderni sistemi di storage distribuiti e motori database, le strutture randomizzate ricoprono un ruolo centrale:

- **Skip List:** Utilizzate come struttura primaria in memoria (*MemTable*) nei motori di memorizzazione LSM-Tree ad altissimo throughput di scrittura, tra cui **LevelDB** (Google) e **RocksDB** (Meta). Sono inoltre impiegate in **Redis** per implementare i *Sorted Sets* (`ZSET`), garantendo tempi logaritmici su inserimento, ranking e range scan concorrenti [9].
- **Treap:** Impiegati nei database geometrici, nei protocolli di routing dinamico su reti ad anello e nei motori di indicizzazione spaziale per la loro duplice natura di albero binario e heap bidimensionale [10].

9.2 Dalla Ricerca Binaria su Lista alla Skip List

Negli anni '90, William Pugh affrontò la sfida teorico-pratica di emulare l'efficienza della ricerca binaria (tipica degli array contigui) direttamente

sopra una lista concatenata, preservando al contempo la flessibilità dinamica dei puntatori [9].

9.2.1 La Lista Concatenata Base (Livello 0)

Consideriamo una lista semplicemente concatenata ordinata, delimitata convenzionalmente da due nodi sentinella con chiavi simboliche $-\infty$ e $+\infty$. Sia $S = \{3, 6, 7, 10, 15, 16\}$ l'insieme delle chiavi memorizzate:



Figura 9.1: Lista semplicemente concatenata base ordinata con sentinelle $-\infty$ e $+\infty$.

In questa struttura, ogni operazione di ricerca SEARCH(k) richiede tempo $\mathcal{O}(n)$, poiché i puntatori consentono di avanzare esclusivamente di un singolo nodo alla volta verso destra. Non è possibile accedere direttamente all'elemento mediano in tempo costante, precludendo l'applicazione della ricerca binaria.

9.2.2 Il Modello Deterministico a Due Livelli

Per accelerare la navigazione, introduciamo una seconda lista sovrapposta denominata **corsia di sorpasso** (*express lane* o Livello L_1), che campiona rigidamente nodi a intervalli regolari dalla lista base (Livello L_0):

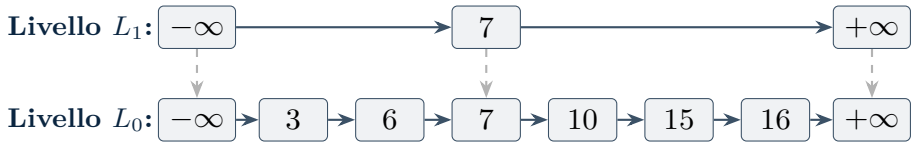


Figura 9.2: Modello deterministico a due livelli: la corsia espressa L_1 scavalca sottoinsiemi di nodi.

La ricerca di una chiave k (ad esempio $k = 15$) si articola in due fasi ordinate:

1. Si naviga orizzontalmente sul livello L_1 a partire da $-\infty$ fino a identificare il massimo elemento minore o uguale a k (nell'esempio, il nodo 7, poiché il successivo $+\infty$ è strettamente maggiore di 15).

2. Si scende verticalmente sul livello base L_0 in corrispondenza del nodo 7 e si prosegue la scansione sequenziale locale ($7 \to 10 \to 15$).

Il Problema Fondamentale dell'Approccio Deterministico. Sebbene questo schema riduca sensibilmente i confronti durante la ricerca, esso collassa in presenza di operazioni di mutazione (INSERT e DELETE). L'inserimento ripetuto di nuove chiavi tra due nodi promossi distrugge la simmetria geometrica dei blocchi. Per preservare le distanze costanti, l'algoritmo dovrebbe effettuare una **riorganizzazione globale** (*reorganization*) spostando i puntatori espressi e ricalcolando gli intervalli, operazione che richiede tempo $\mathcal{O}(n)$ e vanifica ogni beneficio computazionale.

9.2.3 Decomposizione a Blocchi (Square Root Decomposition)

Formalizziamo ora analiticamente il costo di questo schema deterministico a due livelli e ricaviamo la dimensione ottimale dei blocchi per minimizzare il tempo di ricerca.

Supponiamo di partizionare gli n elementi della lista in blocchi contigui:

- Sia n il numero totale di elementi memorizzati;
- Sia k il numero di blocchi (corrispondente al numero di puntatori espressi al livello superiore L_1);
- Ciascun blocco al livello inferiore L_0 conterrà pertanto esattamente $\frac{n}{k}$ elementi.

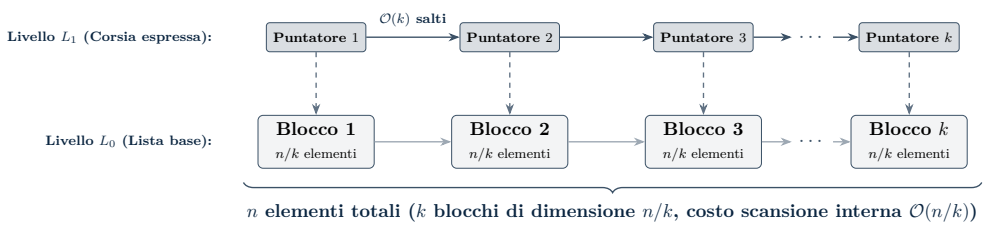


Figura 9.3: Schema della decomposizione a blocchi deterministica con k salti e blocchi di dimensione n/k.

Il costo di ricerca nel caso peggiore $T(k)$ è dato dalla somma dei passi orizzontali percorsi al livello superiore per individuare il blocco corretto, più i passi percorsi all'interno del singolo blocco al livello inferiore:

$$T(k) = \mathcal{O}(k) + \mathcal{O}\left(\frac{n}{k}\right) = \mathcal{O}\left(k + \frac{n}{k}\right). \tag{9.3}$$

Teorema 9.2.1 (Dimensione Ottimale del Blocco)**.** *La funzione di costo* $f(k) = k + \frac{n}{k}$ *definita per* $k > 0$ *ammette un unico punto di minimo globale in:*

$$k^* = \sqrt{n}. \tag{9.4}$$

Per tale valore, la complessità di ricerca nel caso peggiore risulta:

$$T(\sqrt{n}) = \mathcal{O}(\sqrt{n}). \tag{9.5}$$

Dimostrazione. Consideriamo la funzione continua $f(k) = k + \frac{n}{k}$ per $k \in (0, +\infty)$. Calcoliamo la derivata prima rispetto a k:

$$\frac{df}{dk} = 1 - \frac{n}{k^2}. \tag{9.6}$$

Ponendo la derivata prima uguale a zero per determinare i punti stazionari:

$$1 - \frac{n}{k^2} = 0 \iff 1 = \frac{n}{k^2} \iff k^2 = n \iff k = \sqrt{n}. \tag{9.7}$$

Valutiamo la derivata seconda per verificarne la convessità:

$$\frac{d^2 f}{dk^2} = \frac{2n}{k^3} > 0, \quad \forall k > 0. \tag{9.8}$$

Poiché la derivata seconda è strettamente positiva per ogni $k > 0$, la funzione $f(k)$ è strettamente convessa e ammette un minimo globale assoluto in $k^* = \sqrt{n}$. Sostituendo tale valore nell'espressione del costo complessivo otteniamo:

$$f(\sqrt{n}) = \sqrt{n} + \frac{n}{\sqrt{n}} = 2\sqrt{n} = \mathcal{O}(\sqrt{n}). \tag{9.9}$$

□

Estensione Ricorsiva e Limiti Deterministici. Applicando ricorsivamente lo schema di decomposizione a 3 livelli, il costo scenderebbe a $\mathcal{O}(n^{1/3})$; con $\log_2 n$ livelli si raggiungerebbe teoricamente il limite logaritmico $\mathcal{O}(\log n)$. Tuttavia, la rigidità deterministica renderebbe ogni inserimento e cancellazione proibitivamente costoso per via dell'effetto domino di ristrutturazione.

> **L'intuizione chiave della Skip List:** eliminare la rigidità deterministica assegnando l'altezza di ciascun nodo in modo puramente probabilistico e indipendente tramite lanci di moneta!

9.3 Skip List: Struttura e Analisi Probabilistica Formale

L'assegnazione randomizzata dell'altezza dei nodi consente di conciliare la semplicità implementativa delle liste concatenate con le prestazioni logaritmiche tipiche degli alberi bilanciati. In questa sezione formalizziamo la struttura della Skip List e ne deriviamo l'analisi probabilistica rigorosa dell'altezza e della complessità di ricerca.

9.3.1 Definizione Formale della Skip List

La Skip List è una struttura dati probabilistica costituita da una gerarchia di liste concatenate ordinate sovrapposte $\{L_0, L_1, \ldots, L_h\}$ governata dalle seguenti proprietà strutturali:

Definizione 9.3.1 (Skip List)**.** Sia S un insieme di n chiavi distinte. Una Skip List per S soddisfa:

1. Il livello base L_0 contiene tutti gli n elementi di S ordinati in modo crescente, preceduti dalla sentinella $-\infty$ e seguiti dalla sentinella $+\infty$.
2. Per ogni livello $i \geq 0$, la lista L_{i+1} è un sottoinsieme proprio della lista L_i, ovvero:
$$L_h \subset L_{h-1} \subset \cdots \subset L_1 \subset L_0. \tag{9.10}$$
3. **Crescita Geometrica Indipendente:** L'appartenenza di ciascun elemento al livello successivo L_{i+1} è determinata indipendentemente mediante il lancio di una moneta bilanciata con probabilità di successo $p = \frac{1}{2}$. Se l'esito è **Testa** (Head), il nodo viene promosso al livello $i + 1$; se l'esito è **Croce** (Tail), la sua crescita verticale si arresta definitivamente.

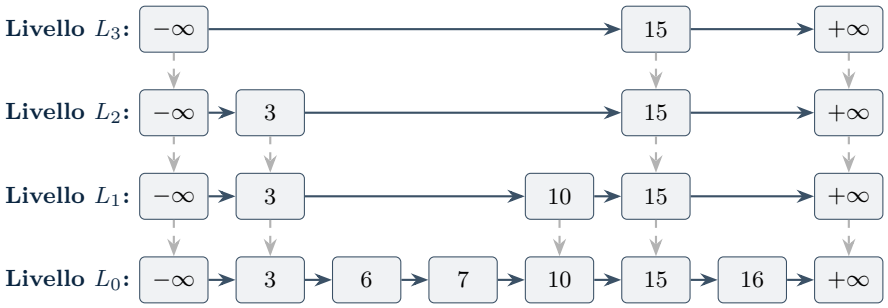


Figura 9.4: Rappresentazione di una Skip List multilivello a 4 piani (L_0 fino a L_3).

9.3.2 Algoritmi Fondamentali

1. Ricerca (Search(k))

L'algoritmo di ricerca (riportato in Algoritmo 11) inizia dal livello più alto h in corrispondenza del puntatore di testa ($-\infty$). Ad ogni passo, se la chiave successiva al livello corrente è minore o uguale a k, l'algoritmo avanza orizzontalmente verso destra. Non appena la chiave successiva supera strettamente k, l'algoritmo scende al livello sottostante. Il procedimento termina quando si scende al di sotto del livello base L_0.

Algorithm 11: Ricerca in una Skip List (SEARCH(k))

Dati: Skip List SL con altezza massima corrente h, chiave cercata $k \in \mathcal{U}$
Risultato: Riferimento al nodo contenente k, oppure NIL se assente

```
curr ← testa della Skip List al livello h
for i ← h to 0 do
    while curr.next[i] ≠ NIL and curr.next[i].key ≤ k do
        curr ← curr.next[i]
if curr.key == k then
    return curr
return NIL
```

2. Inserimento (Insert(x))

Per inserire un nuovo elemento x con chiave k:

1. Si esegue una ricerca preliminare per localizzare la posizione, memorizzando i predecessori nell'array update$[0 \dots h]$.
2. Si genera l'altezza l simulando lanci di moneta indipendenti:
$$l \leftarrow 0; \quad \textbf{while } \text{random}() < 0.5 \textbf{ do } l \leftarrow l + 1.$$
3. Se $l > h$, si incrementa l'altezza massima dell'intera Skip List aggiornando l'array update.
4. Si alloca il nuovo nodo di altezza l e, per ogni livello $i \in [0, l]$, si ricollegano i puntatori orizzontali:
$$\text{nodo.next}[i] \leftarrow \text{update}[i].\text{next}[i], \qquad \text{update}[i].\text{next}[i] \leftarrow \text{nodo}. \tag{9.11}$$

3. Cancellazione (Delete(x))

L'operazione individua i predecessori del nodo da eliminare su tutti i livelli in cui esso è presente (tramite l'array update). Successivamente, scavalca il nodo aggiornando i riferimenti update[i].next[i] $\leftarrow x$.next[i] e dealloca la memoria associata.

9.3.3 Formulazione dei Costi Operativi

I costi computazionali delle operazioni fondamentali sono governati dal numero di puntatori attraversati durante la navigazione:

$$\text{Cost}(\textsc{Search}) = \#\text{puntatori verticali} + \#\text{puntatori orizzontali}, \tag{9.12}$$

$$\text{Cost}(\textsc{Insert}) = \text{Cost}(\textsc{Search}) + \text{altezza(chiave)}, \tag{9.13}$$

$$\text{Cost}(\textsc{Delete}) = \text{Cost}(\textsc{Search}) + \#\text{modifiche puntatori}. \tag{9.14}$$

Poiché il numero di puntatori verticali è pari all'altezza totale della Skip List e l'altezza della singola chiave è dominata dall'altezza massima, il costo complessivo è asintoticamente determinato dall'altezza globale della struttura e dal numero medio di passi orizzontali per livello.

9.3.4 Dimostrazione: Altezza Massima della Skip List

Vogliamo dimostrare che l'altezza massima H di una Skip List contenente n elementi rimane confinata entro $\mathcal{O}(\log n)$ *with high probability.*

Teorema 9.3.1 (Altezza Massima Logaritmica)**.** *Sia S un insieme di n elementi inseriti in una Skip List con parametro di promozione $p = \frac{1}{2}$, e sia $H = \max_{k \in S}\{\text{level}(k)\}$ l'altezza massima raggiunta dalla struttura. Per qualsiasi costante $c > 1$:*

$$\mathbb{P}(H \geq c \log_2 n) \leq \frac{1}{n^{c-1}} = n^{-(c-1)}. \tag{9.15}$$

Pertanto, l'altezza della Skip List è $\mathcal{O}(\log n)$ con alta probabilità (w.h.p.).

Dimostrazione. La dimostrazione procede in tre passaggi analitici rigorosi:

Passo 1: Probabilità di altezza per un singolo elemento. Sia level(k) l'altezza assegnata all'elemento con chiave k. Un elemento raggiunge un livello maggiore o uguale a l se e solo se ottiene almeno l successi consecutivi ("Testa") nel processo di lancio della moneta:

$$\mathbb{P}(\text{level}(k) \geq l) = \left(\frac{1}{2}\right)^l. \tag{9.16}$$

Passo 2: Altezza globale dell'intera struttura e Union Bound. L'altezza globale della Skip List supera o eguaglia l se e solo se esiste almeno una chiave $k \in S$ il cui livello è $\geq l$:

$$\mathbb{P}(H \geq l) = \mathbb{P}\left(\exists k \in S : \text{level}(k) \geq l\right) = \mathbb{P}\left(\bigcup_{k \in S} \{\text{level}(k) \geq l\}\right). \tag{9.17}$$

Applicando la **disuguaglianza di Boole** (*Union Bound*), la probabilità dell'unione è maggiorata dalla somma delle probabilità individuali:

$$\mathbb{P}(H \geq l) \leq \sum_{k \in S} \mathbb{P}(\text{level}(k) \geq l) = \sum_{k=1}^{n} \left(\frac{1}{2}\right)^{l} = n \cdot \left(\frac{1}{2}\right)^{l} = \frac{n}{2^l}. \tag{9.18}$$

Passo 3: Scelta di un'altezza logaritmica. Fissiamo la soglia d'altezza $l = c \log_2 n$, con costante reale arbitraria $c > 1$. Sostituendo tale valore nella disuguaglianza precedente otteniamo:

$$\mathbb{P}(H \geq c \log_2 n) \leq \frac{n}{2^{c \log_2 n}} = \frac{n}{\left(2^{\log_2 n}\right)^c} = \frac{n}{n^c} = \frac{1}{n^{c-1}} = n^{-(c-1)}. \tag{9.19}$$

□

Esempio 9.3.1 (Interpretazione Asintotica dell'Union Bound)**.** Scegliendo una costante moderata come $c = 3$, la probabilità che l'altezza massima superi $3 \log_2 n$ è inferiore a $\frac{1}{n^2}$. Per un dizionario contenente un milione di chiavi ($n = 10^6$), tale probabilità di sforamento è minore di 10^{-12}, rendendo l'evento di un albero sbilanciato praticamente impossibile nella prassi ingegneristica.

9.3.5 Analisi del Costo di Ricerca: Analisi all'Indietro (Backward Analysis)

Per stimare il numero atteso di puntatori orizzontali attraversati durante la ricerca, analizzare il percorso dall'alto verso il basso risulta complicato, poiché i passi orizzontali dipendono dalla configurazione dei nodi sottostanti.

Applichiamo la tecnica fondamentale della **Backward Analysis** (Analisi all'Indietro): tracciamo il cammino di ricerca a ritroso, partendo dal nodo di destinazione al livello base L_0 e risalendo verso la testa della Skip List al livello massimo.

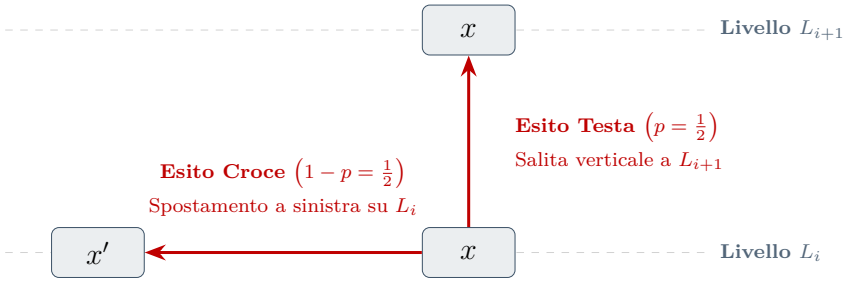


Figura 9.5: Transizioni elementari nell'analisi all'indietro: salita verticale (Testa) vs spostamento a sinistra (Croce).

Ad ogni passo del percorso all'indietro a partire da un nodo x al livello i:

1. Se la chiave x è stata promossa al livello superiore L_{i+1} (evento che si verifica con probabilità $p = \frac{1}{2}$, corrispondente all'esito **Testa**), il percorso di ricerca a ritroso deve necessariamente risalire verticalmente al livello $i + 1$.

2. Se la chiave x non è stata promossa a L_{i+1} (evento che si verifica con probabilità $1 - p = \frac{1}{2}$, corrispondente all'esito **Croce**), il percorso di ricerca a ritroso deve necessariamente provenire dal nodo immediatamente precedente a sinistra sullo stesso livello i.

Teorema 9.3.2 (Valore Atteso dei Passi Orizzontali per Livello)**.** *Il numero atteso di spostamenti orizzontali verso sinistra compiuti ad ogni livello prima di risalire verticalmente al livello superiore è esattamente pari a* 2*:*

$$\mathbb{E}[\#\textit{passi orizzontali per livello}] = 2. \tag{9.20}$$

Dimostrazione. Il numero di spostamenti orizzontali a sinistra prima di salire di livello è una variabile aleatoria discreta X che conta il numero di insuccessi (*Croce*) prima di ottenere il primo successo (*Testa*). Poiché ogni lancio è indipendente con probabilità di successo $p = \frac{1}{2}$, la variabile aleatoria segue una distribuzione geometrica con probabilità:

$$\mathbb{P}(X = h) = \left(\frac{1}{2}\right)^h \cdot \frac{1}{2} = \left(\frac{1}{2}\right)^{h+1}, \quad \text{per } h \geq 0. \tag{9.21}$$

Negli appunti universitari il valore atteso è formalizzato attraverso la serie dei prodotti:

$$\mathbb{E}[\#\text{tails}] = \sum_{h=0}^{\infty} h \cdot \left(\frac{1}{2}\right)^h = 2. \tag{9.22}$$

Ricostruiamo la derivazione matematica analitica completa. Consideriamo la serie geometrica fondamentale per $|x| < 1$:

$$\sum_{h=0}^{\infty} x^h = \frac{1}{1-x}. \tag{9.23}$$

Derivando termine a termine ambo i membri rispetto a x:

$$\frac{d}{dx}\left(\sum_{h=0}^{\infty} x^h\right) = \sum_{h=1}^{\infty} h x^{h-1} = \frac{d}{dx}\left(\frac{1}{1-x}\right) = \frac{1}{(1-x)^2}. \tag{9.24}$$

Moltiplicando l'intera uguaglianza per x:

$$\sum_{h=0}^{\infty} h x^h = \frac{x}{(1-x)^2}. \tag{9.25}$$

Valutando l'equazione (9.25) nel punto $x = \frac{1}{2}$:

$$\sum_{h=0}^{\infty} h \left(\frac{1}{2}\right)^h = \frac{\frac{1}{2}}{\left(1-\frac{1}{2}\right)^2} = \frac{\frac{1}{2}}{\left(\frac{1}{2}\right)^2} = \frac{\frac{1}{2}}{\frac{1}{4}} = 2. \tag{9.26}$$

□

Teorema 9.3.3 (Costo Totale Atteso della Ricerca)**.** *Dato che il numero totale di livelli attraversati è limitato dall'altezza massima $\mathcal{O}(\log n)$ e il numero atteso di passi orizzontali per livello è al più 2, il costo atteso complessivo della ricerca soddisfa:*

$$\mathbb{E}[\textit{Costo Ricerca}] \leq 2 \cdot \mathcal{O}(\log n) + \mathcal{O}(\log n) = \mathcal{O}(\log n). \tag{9.27}$$

9.3.6 Nota Applicativa Avanzata: Vector Database e Grafi HNSW

La filosofia gerarchica a corsie multiple della Skip List ha trovato una straordinaria estensione moderna nei **Vector Database** impiegati per il retrieval semantico di vettori di embedding nei modelli di intelligenza artificiale.

L'algoritmo **HNSW** (*Hierarchical Navigable Small World*) modella la ricerca del vicino più prossimo (*Approximate Nearest Neighbor Search*, ANNS) strutturando i vettori su livelli gerarchici di grafi di prossimità [11]:

- Al livello base L_0 risiede il grafo denso completo di tutti i vettori multidimensionali;

- Nei livelli superiori ($L_1, L_2, \dots$) vengono mantenuti grafi progressivamente più radi dotati di collegamenti a lungo raggio (*long-range edges*), esattamente analoghi ai puntatori espressi della Skip List.

La navigazione parte dal livello più alto con passi ampi e scende progressivamente nei livelli inferiori per affinare localmente la ricerca, risolvendo il problema dei minimi locali e consentendo ricerche sublineari $\mathcal{O}(\log n)$ su miliardi di vettori in spazi ad altissima dimensionalità.

9.4 Conclusioni e Prospettive: Verso gli Alberi di Ricerca Randomizzati

In questa trattazione abbiamo esplorato come la randomizzazione consenta di superare la rigidità e i vincoli delle strutture dati deterministiche tradizionali:

- Partendo dall'inefficienza intrinseca dell'accesso sequenziale su liste collegate, abbiamo progressivamente introdotto gerarchie a puntatori espressi (dalla decomposizione deterministica in blocchi di dimensione $\sqrt{n}$ fino a torri a densità geometricamente decrescente con probabilità $p = 1/2$);
- La **Skip List** emula probabilisticamente una ricerca binaria mantenendo una complessità spaziale attesa lineare $\mathcal{O}(n)$ e garantendo operazioni di ricerca, inserimento e cancellazione in tempo atteso $\mathcal{O}(\log n)$ e con alta probabilità;
- La tecnica della **Backward Analysis** rovescia la prospettiva analitica seguendo il cammino a ritroso dalla chiave cercata verso la radice sentinella, formalizzando come il numero atteso di passi orizzontali per ciascun livello sia rigorosamente limitato da $1/p = 2$.

Mentre la Skip List organizza i dati mediante una gerarchia di liste collegate multilivello, sorge naturale una questione architetturale: *è possibile applicare la medesima eleganza probabilistica direttamente agli alberi binari di ricerca, evitando le costose rotazioni deterministiche e i vincoli di bilanciamento degli alberi AVL e Red-Black?* A questo interrogativo fondamentale risponde il **Treap** (Randomized Search Tree), la cui teoria, meccanica delle rotazioni, analisi di Seidel-Aragon e connessioni con la geometria computazionale costituiscono l'oggetto monografico del Capitolo 10.

Capitolo 10

Alberi di Ricerca Randomizzati: Treap, Operazioni Dinamiche e Applicazioni Geometriche

10.1 Introduzione e Definizione Formale del Treap

La combinazione delle proprietà di un albero binario di ricerca con quelle di una coda di priorità heap permette di realizzare una struttura dati elegantemente bilanciata per via probabilistica. In questa sezione formalizziamo la nozione di Treap e ne dimostriamo il teorema fondamentale di unicità topologica.

10.1.1 Motivazione e Concetto Fondamentale

Negli alberi binari di ricerca classici (BST), l'efficienza delle operazioni elementari di ricerca, inserimento e cancellazione è strettamente vincolata all'altezza h dell'albero:

$$\text{Cost}(\textsc{Operation}) = \mathcal{O}(h). \tag{10.1}$$

Nel caso migliore, un albero perfettamente bilanciato garantisce $h = \lfloor \log_2 n \rfloor$. Tuttavia, sequenze sfortunate di inserimento (ad esempio chiavi inserite in ordine strettamente crescente o decrescente) degradano la topologia dell'albero in una lista monodirezionale, portando l'altezza al caso pessimo degenere $h = \mathcal{O}(n)$.

Le soluzioni deterministiche tradizionali (come gli alberi AVL [1] o gli alberi Red-Black) ripristinano forzatamente il bilanciamento imponendo vincoli strutturali rigidi:

- Richiedono bit ausiliari di memoria per ciascun nodo (bit di colore o altezze relative dei sottoalberi);
- Comportano complesse casistiche di rotazione singola e doppia durante gli aggiornamenti;
- Presentano gravi colli di bottiglia nei contesti concorrenti e multi-thread, poiché i ribilanciamenti possono propagarsi fino alla radice, richiedendo lock esclusivi estesi lungo l'intero cammino.

Il **Treap** risolve questa criticità introducendo una separazione netta tra la dimensione logica dei dati (gestita dalle **chiavi**) e la dimensione strutturale dell'albero (governata dalle **priorità**).

Definizione 10.1.1 (Nodo del Treap)**.** Ogni nodo u di un Treap memorizza una coppia ordinata di attributi:

$$u = \langle k(u), p(u) \rangle, \tag{10.2}$$

dove:

- $k(u) \in \mathcal{U}$ è la **chiave** appartenente a un dominio totalmente ordinato (ordinamento orizzontale);
- $p(u) \in \mathbb{R}$ è la **priorità** appartenente a un dominio numerico continuo totalmente ordinato (ordinamento verticale).

10.1.2 Invarianti Strutturali Fondamentali

Un albero binario T è un Treap valido se e solo se soddisfa congiuntamente due proprietà ortogonali per ogni nodo $u \in T$:

1. **Proprietà di Binary Search Tree (BST) rispetto alle Chiavi:** L'ordinamento orizzontale riflette l'ordine delle chiavi:

$$\forall v \in \text{left}(u) \implies k(v) < k(u), \tag{10.3}$$
$$\forall w \in \text{right}(u) \implies k(w) > k(u). \tag{10.4}$$

   Ne consegue che una visita simmetrica in-order restituisce l'insieme delle chiavi in ordine strettamente crescente.

2. **Proprietà di Max-Heap rispetto alle Priorità:** L'ordinamento verticale gerarchico riflette l'ordine decrescente delle priorità:

$$\forall v \in \{\text{left}(u), \text{right}(u)\}, \quad v \neq \text{NIL} \implies p(u) \geq p(v). \tag{10.5}$$

   Ne consegue che la radice dell'albero possiede sempre la priorità massima assoluta di tutto il Treap.

10.1.3 Teorema di Unicità della Struttura

Un risultato teorico di primaria importanza stabilisce che la forma dell'albero non dipende dalla sequenza temporale con cui le coppie sono state inserite.

Teorema 10.1.1 (Unicità del Treap)**.** *Sia $S = \{\langle x_1, p_1\rangle, \ldots, \langle x_n, p_n\rangle\}$ un insieme di n coppie distinte aventi chiavi mutualmente distinte ($x_i \neq x_j$) e priorità mutualmente distinte ($p_i \neq p_j$). Allora esiste* ***uno ed un solo*** *Treap T che memorizza l'insieme S.*

Dimostrazione. Procediamo per induzione matematica sulla cardinalità $n = |S|$.

- **Caso base ($n = 0$ o $n = 1$):**
  Se $n = 0$, l'unico albero possibile è l'albero vuoto (NIL). Se $n = 1$, l'unico albero ammissibile è costituito da un singolo nodo isolato contenente l'unica coppia $\langle x_1, p_1\rangle$. In entrambi i casi, la topologia è banale ed univoca.

- **Passo induttivo ($n > 1$):**
  Assumiamo che il teorema sia valido per ogni insieme di cardinalità strettamente inferiore a n. Consideriamo l'insieme S di n elementi.

  Poiché le priorità $\{p_1, \ldots, p_n\}$ sono tutte distinte, esiste un unico indice $r \in \{1, \ldots, n\}$ tale che:
  $$p_r = \max_{1 \leq i \leq n} \{p_i\}. \tag{10.6}$$
  Affinché l'invariante di Max-Heap sia soddisfatto su tutto l'albero, la radice di T deve necessariamente essere il nodo $\langle x_r, p_r\rangle$. Non esiste alcuna altra scelta ammissibile per la radice.

  Fissata la radice $\langle x_r, p_r\rangle$, la proprietà di Binary Search Tree impone in modo deterministico e categorico l'assegnazione di tutti i rimanenti $n - 1$ elementi ai sottoalberi sinistro e destro:
  $$S_L = \{\langle x_i, p_i\rangle \in S \setminus \{\langle x_r, p_r\rangle\} \mid x_i < x_r\}, \tag{10.7}$$
  $$S_R = \{\langle x_i, p_i\rangle \in S \setminus \{\langle x_r, p_r\rangle\} \mid x_i > x_r\}. \tag{10.8}$$
  Poiché le chiavi sono distinte, la partizione $S_L \cup S_R = S \setminus \{\langle x_r, p_r\rangle\}$ con $S_L \cap S_R = \emptyset$ è univoca.

  Inoltre, $|S_L| < n$ e $|S_R| < n$. Per ipotesi induttiva, esiste un unico Treap valido T_L costruito su S_L e un unico Treap valido T_R costruito su S_R. Assegnando T_L come sottoalbero sinistro della radice e T_R come sottoalbero destro, otteniamo l'unico albero binario che soddisfa simultaneamente la proprietà di BST e la proprietà di Heap. L'unicità della struttura è pertanto dimostrata per ogni $n \geq 0$.

□

Corollario 10.1.1 (Isomorfismo con il Random BST)**.** *Se le priorità p_i sono estratte in modo indipendente e identicamente distribuito (i.i.d.) da una variabile aleatoria continua uniforme $\mathcal{U}[0,1]$, l'ordinamento decrescente delle priorità induce una permutazione uniforme casuale $\pi \in \mathcal{S}_n$ sull'insieme delle chiavi. Pertanto, il Treap generato è* ***topologicamente indistinguibile da un albero binario di ricerca costruito inserendo le chiavi nell'ordine della permutazione casuale*** *π.*

10.2 Operazioni Elementari: Rotazioni nel Treap

Il mantenimento dell'invariante di heap in seguito a modifiche dinamiche si basa su trasformazioni topologiche locali che non alterano l'ordinamento simmetrico delle chiavi. In questa sezione formalizziamo la cinematica delle rotazioni destra e sinistra e ne verifichiamo la correttezza strutturale.

10.2.1 Meccanica delle Rotazioni Destra e Sinistra

Per mantenere l'invariante di Max-Heap a seguito di inserimenti o cancellazioni senza alterare l'ordinamento BST delle chiavi, il Treap impiega le **rotazioni locali ad albero**. Una rotazione scambia la relazione padre-figlio tra due nodi adiacenti modificandone le altezze relative, ma preservando rigorosamente la sequenza in-order dei sottoalberi.

Consideriamo due nodi adiacenti u e v:

- **Rotazione Destra (Rotate-Right(v)):**
  Si applica quando u è il figlio sinistro di v. Il nodo u sale al posto di v, v diventa il figlio destro di u, e il sottoalbero destro di u (B) viene riassegnato come sottoalbero sinistro di v.

- **Rotazione Sinistra (Rotate-Left(u)):**
  Operazione duale e speculare della rotazione destra. Si applica quando v è il figlio destro di u. Il nodo v sale al posto di u, u diventa il figlio sinistro di v, e il sottoalbero sinistro di v (B) viene riassegnato come sottoalbero destro di u.

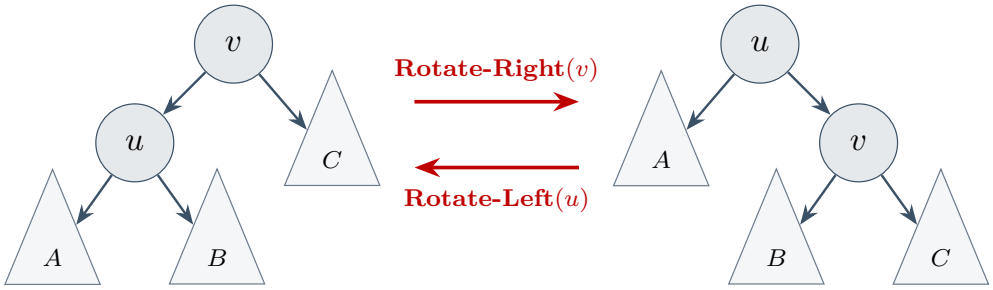


Figura 10.1: Meccanica algebrica delle rotazioni nel Treap. In entrambe le configurazioni, la sequenza ordinata in-order dei sottoalberi e delle chiavi è invariante: $A < u < B < v < C$.

10.2.2 Conservazione Rigorosa degli Invarianti

La dimostrazione di correttezza delle rotazioni si fonda sulla verifica dei due invarianti:

Proprietà 10.2.1 (Invarianza dell'Ordinamento BST)**.** *Nella configurazione prima della rotazione destra:*

$$\forall a \in A,\ b \in B,\ c \in C \implies k(a) < k(u) < k(b) < k(v) < k(c). \tag{10.9}$$

Nella configurazione dopo la rotazione destra:

- A *è il sottoalbero sinistro di* $u \implies \forall a \in A,\ k(a) < k(u)$*;*
- v *è il figlio destro di* $u \implies k(u) < k(v)$*;*
- B *è il sottoalbero sinistro di* $v \implies \forall b \in B,\ k(u) < k(b) < k(v)$*;*
- C *è il sottoalbero destro di* $v \implies \forall c \in C,\ k(v) < k(c)$*.*

L'invariante di BST è dunque rigorosamente preservato. Poiché ogni rotazione coinvolge solo 3 puntatori locali, il costo computazionale di una rotazione è esattamente $\mathcal{O}(1)$ *nel tempo deterministico.*

10.3 Algoritmi Fondamentali e Operazioni Dinamiche

Le operazioni di manipolazione del Treap orchestrano la ricerca su BST con il ripristino prioritario delle proprietà di heap lungo il cammino di accesso. In questa sezione sviluppiamo gli algoritmi completi per la ricerca, l'inserimento, la cancellazione, la scissione (Split) e la fusione (Merge).

10.3.1 Ricerca (Search)

La procedura di ricerca ignora completamente le priorità dei nodi e coincide esattamente con la ricerca binaria standard in un BST:

Algorithm 12: Ricerca di una chiave nel Treap (SEARCH(u, k))

Input: Riferimento alla radice u del sottoalbero corrente, chiave cercata $k \in \mathcal{U}$

Output: Riferimento al nodo contenente k, oppure NIL se non presente

```
1 if u = NIL or k = k(u) then
2     return u
3 if k < k(u) then
4     return SEARCH(left(u), k)
5 else
6     return SEARCH(right(u), k)
```

La complessità temporale è proporzionale all'altezza del cammino percorso: $\mathcal{O}(h)$. Come dimostreremo nella Sezione 10.4, l'altezza attesa è $\mathcal{O}(\log n)$.

10.3.2 Inserimento (Insert) con Risalita a Rotazioni (Bubble-Up)

L'inserimento di una nuova chiave k procede in due fasi distinte:

1. **Inserimento BST standard a foglia:** Si genera una priorità casuale indipendente $p \sim \mathcal{U}[0, 1]$. Si individua la posizione della foglia seguendo la classica discesa binaria BST e si alloca il nuovo nodo $z = \langle k, p \rangle$.

2. **Ripristino dell'invariante di Max-Heap (Bubble-Up):** Fintanto che il nodo z non è la radice e la sua priorità supera quella del padre ($p(z) > p(\text{parent}(z))$), si esegue una rotazione locale per far salire z:

   - Se z è figlio sinistro, si applica ROTATE-RIGHT(parent(z));
   - Se z è figlio destro del padre, si applica ROTATE-LEFT(parent(z)).

Algorithm 13: Inserimento di una nuova coppia nel Treap (INSERT(T, k))

Input: Radice T del Treap, chiave k da inserire
Output: Nuova radice del Treap contenente k

```
1 p ← RandomUniform(0, 1)
2 z ← CreateNode(k, p)
3 return InsertNode(T, z)
```

Algorithm 14: Inserimento ricorsivo con rotazioni di risalita (INSERTNODE(u, z))

Input: Nodo radice del sottoalbero corrente u, nuovo nodo da inserire z
Output: Radice aggiornata del sottoalbero

```
1  if u = NIL then
2      return z
3  if k(z) < k(u) then
4      left(u) ← InsertNode(left(u), z)
5      if p(left(u)) > p(u) then
6          u ← Rotate-Right(u)
7  else if k(z) > k(u) then
8      right(u) ← InsertNode(right(u), z)
9      if p(right(u)) > p(u) then
10         u ← Rotate-Left(u)
11 return u
```

Esempio di Traccia: Inserimento con Risalita Bubble-Up

Per comprendere concretamente la dinamica dell'inserimento, consideriamo il Treap contenente l'insieme di coppie:

$$S = \{(G, 3), (H, 8), (I, 6), (M, 9), (P, 2), (R, 5), (T, 7)\} \tag{10.10}$$

e supponiamo di voler inserire la nuova coppia:

$$z = \langle K, 7.5 \rangle. \tag{10.11}$$

L'esecuzione si articola in due fasi sequenziali distinte:

1. **Fase 1 (Discesa BST a foglia):**

- Confronto con la radice M: poiché $K < M$, la discesa prosegue a sinistra verso H.
- Confronto con H: poiché $K > H$, si scende a destra verso I.
- Confronto con I: poiché $K > I$ e il puntatore destro di I è NIL, il nodo $z = \langle K, 7.5 \rangle$ viene agganciato come foglia figlia destra di I.

In questa configurazione provvisoria (Figura 10.2(a)), l'ordinamento orizzontale BST è intatto, ma si manifesta una violazione locale dell'invariante verticale di Max-Heap tra il padre I e il figlio K:

$$p(K) = 7.5 > p(I) = 6. \tag{10.12}$$

2. **Fase 2 (Risalita con rotazione sinistra su I):** Poiché K è figlio destro di I, l'invariante di Max-Heap viene ripristinato invocando Rotate-Left(I):
   - Il nodo K viene promosso al posto di I, diventando il nuovo figlio destro di H.
   - Il nodo I retrocede diventando figlio sinistro di K.
   - L'eventuale sottoalbero sinistro di K (in questo caso vuoto) verrebbe riassegnato come figlio destro di I.
3. **Verifica della terminazione al livello superiore:** Il nuovo genitore di K è H. Verifichiamo la condizione di heap:

$$p(H) = 8 \geq p(K) = 7.5. \tag{10.13}$$

Poiché la priorità del genitore è maggiore di quella del nodo risalito, la proprietà di Max-Heap è pienamente soddisfatta su tutti i livelli superiori. La procedura di inserimento termina con successo (Figura 10.2(b)).

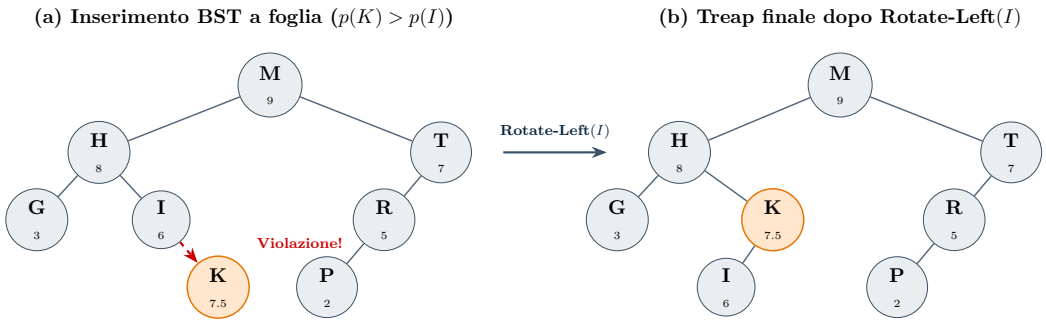


Figura 10.2: Traccia dell'inserimento della coppia $\langle K, 7.5 \rangle$: (a) allocazione iniziale come foglia con violazione dell'heap su $I(6)$; (b) configurazione finale dopo la rotazione Rotate-Left(I).

10.3.3 Cancellazione (Delete) con Discesa a Rotazioni (Trickle-Down)

Per eliminare una chiave k senza violare le proprietà del Treap, si adotta una strategia elegante che spinge il nodo da eliminare verso il basso:

1. Si individua il nodo u avente $k(u) = k$ tramite discesa BST.
2. Fintanto che u non è una foglia, si seleziona il figlio avente **priorità massima**:
   - Se $p(\text{left}(u)) > p(\text{right}(u))$, si applica Rotate-Right(u);
   - Viceversa, si applica Rotate-Left(u).

   Questa rotazione preserva l'invariante di Max-Heap su tutti i nodi sovrastanti e sposta u un livello più in basso.
3. Quando u diventa foglia (entrambi i figli sono NIL), viene deallocato e il puntatore del padre viene impostato a NIL in tempo $\mathcal{O}(1)$.

Algorithm 15: Cancellazione di una chiave dal Treap (DELETE(u, k))

Input: Radice u del sottoalbero, chiave k da rimuovere
Output: Nuova radice del sottoalbero privo di k

```
1  if u = NIL then
2      return NIL
3  if k < k(u) then
4      left(u) ← DELETE(left(u), k)
5  else if k > k(u) then
6      right(u) ← DELETE(right(u), k)
7  else
       // Nodo u trovato
8      if left(u) = NIL and right(u) = NIL then
9          DEALLOCATE(u)
10         return NIL
11     else if left(u) ≠ NIL and (right(u) = NIL or
        p(left(u)) > p(right(u))) then
12         u ← ROTATE-RIGHT(u)
13         right(u) ← DELETE(right(u), k)
14     else
15         u ← ROTATE-LEFT(u)
16         left(u) ← DELETE(left(u), k)
17 return u
```

Esempio di Traccia: Cancellazione con Discesa Trickle-Down

Supponiamo ora di voler rimuovere il nodo interno H dal Treap ottenuto al termine dell'inserimento precedente:

$$\text{DELETE}(T, H). \tag{10.14}$$

La cancellazione di un nodo con due figli in un Treap non ricorre alla sostituzione del valore con il predecessore o successore (come avverrebbe in un BST standard), bensì spinge il nodo bersaglio verso il basso simulando un processo di "galleggiamento inverso":

1. **Stato iniziale di H:** Il nodo H possiede due figli: G a sinistra con $p(G) = 3$, e K a destra con $p(K) = 7.5$. L'algoritmo confronta le

priorità dei due figli:

$$\max(p(G), p(K)) = \max(3, 7.5) = p(K) = 7.5. \tag{10.15}$$

Poiché il figlio con priorità maggiore è il figlio destro (K), l'algoritmo esegue ROTATE-LEFT(H):

- K viene promosso a nuovo figlio sinistro della radice globale M.
- H scende diventando figlio sinistro di K.
- Il sottoalbero sinistro di K (il nodo $I(6)$) diventa il nuovo figlio destro di H.

I figli correnti di H sono adesso $G(3)$ (sinistro) e $I(6)$ (destro).

2. **Secondo passo di discesa per H:** Il nodo H possiede ancora due figli. L'algoritmo seleziona nuovamente il figlio con priorità massima:

$$\max(p(G), p(I)) = \max(3, 6) = p(I) = 6. \tag{10.16}$$

Poiché la priorità maggiore risiede a destra in I, si esegue una seconda rotazione ROTATE-LEFT(H):

- I sale diventando figlio sinistro di K.
- H scende diventando figlio sinistro di I.
- Il sottoalbero sinistro originario di I (NIL) diventa figlio destro di H.

Ora H ha come unico figlio $G(3)$ a sinistra, mentre il puntatore destro è NIL.

3. **Terzo passo di discesa (rotazione a destra):** Avendo unicamente il figlio sinistro G, l'algoritmo esegue ROTATE-RIGHT(H):

- G sale diventando figlio sinistro di I.
- H retrocede diventando figlio destro di G.

Ora entrambi i puntatori di H sono nulli ($\text{left}(H) = \text{NIL}, \text{right}(H) = \text{NIL}$): H è diventato una **foglia**!

4. **Deallocazione finale $\mathcal{O}(1)$:** Trovandosi a foglia, il puntatore del genitore viene azzerato ($\text{right}(G) \leftarrow \text{NIL}$) e il nodo H viene deallocato. L'albero risultante (Figura 10.3) soddisfa sia l'ordinamento BST ($G < I < K < M < P < R < T$) sia l'invariante di Max-Heap ad ogni livello.

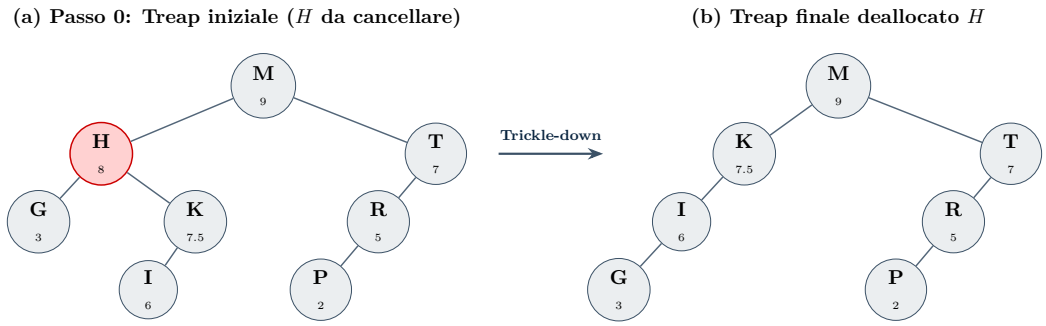


Figura 10.3: Traccia della cancellazione del nodo H: (a) configurazione iniziale (Passo 0) con il nodo bersaglio H evidenziato; (b) Treap finale dopo la discesa a rotazioni (trickle-down) e la deallocazione a foglia di H in tempo $\mathcal{O}(1)$.

10.3.4 Operazioni Avanzate: Split e Merge

Il Treap supporta con eccezionale naturalezza due operazioni fondamentali per strutture dati persistenti e manipolazione di insiemi:

- **Scissione (Split(T, k)):**
  Dato un Treap T e una chiave di soglia k, partiziona T in due Treap indipendenti T_1 e T_2 tali che:

  $$\forall x \in T_1,\ k(x) \leq k \quad \text{e} \quad \forall y \in T_2,\ k(y) > k. \tag{10.17}$$

  Implementazione in tempo logaritmico $\mathcal{O}(\log n)$: Si inserisce temporaneamente nel Treap una chiave fittizia $\langle k, +\infty \rangle$. Poiché la sua priorità è $+\infty$, le rotazioni di bubble-up la porteranno categoricamente alla **radice dell'albero**. A questo punto, il sottoalbero sinistro della radice è esattamente T_1, mentre il sottoalbero destro è esattamente T_2. Rimuovendo il nodo fittizio si ottengono i due Treap voluti in tempo $\mathcal{O}(h) = \mathcal{O}(\log n)$ (Figura 10.4(a)).

- **Fusione (Merge(T_1, T_2)):**
  Dati due Treap disgiunti T_1 e T_2 tali che ogni chiave in T_1 sia strettamente minore di ogni chiave in T_2 ($\max(T_1) < \min(T_2)$), restituisce un unico Treap $T = T_1 \cup T_2$.
  Implementazione in tempo logaritmico $\mathcal{O}(\log n)$: Si alloca una radice fittizia $z = \langle k^*, -\infty \rangle$ avente come figlio sinistro T_1 e come figlio destro T_2. Si applica poi l'algoritmo di cancellazione DELETE su z: le rotazioni di trickle-down spingeranno z verso il basso rispettando le priorità di T_1 e T_2, fondendoli in un unico Treap bilanciato in tempo $\mathcal{O}(\log n)$ (Figura 10.4(b)).

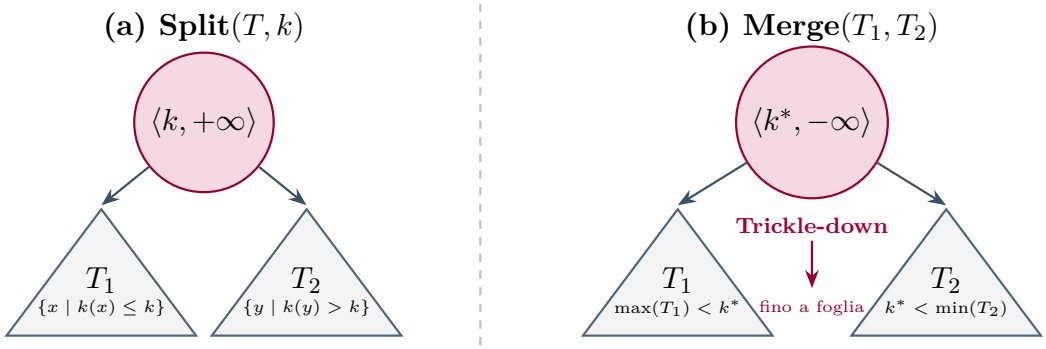


Figura 10.4: Principio costruttivo di Split e Merge: l'assegnazione temporanea di priorità estreme ($+\infty$ e $-\infty$) sfrutta direttamente le rotazioni del Treap per sezionare o fondere collezioni in tempo $\mathcal{O}(\log n)$.

10.4 Analisi Probabilistica Formale (Seidel e Aragon)

Presentiamo ora la celebre analisi probabilistica introdotta da Raimund Seidel e Cecilia Aragon [10], che fornisce le garanzie asintotiche sulle prestazioni del Treap.

10.4.1 Spazio di Probabilità e Variabili Indicatrici

Sia $S = \{x_1, x_2, \ldots, x_n\}$ l'insieme delle chiavi ordinate in modo strettamente crescente:

$$x_1 < x_2 < \cdots < x_n. \tag{10.18}$$

A ciascuna chiave x_i è associata una priorità indipendente $p_i \sim \mathcal{U}[0, 1]$. Definiamo la variabile aleatoria indicatrice $A_{i,j}$:

$$A_{i,j} = \begin{cases} 1 & \text{se il nodo } x_i \text{ è un antenato (\textit{ancestor}) del nodo } x_j \text{ nel Treap,} \\ 0 & \text{altrimenti.} \end{cases} \tag{10.19}$$

Convenzionalmente, consideriamo ogni nodo come antenato di se stesso: $A_{i,i} = 1$.

10.4.2 Teorema Fondamentale della Relazione di Parentela

Teorema 10.4.1 (Condizione di Antenato nel Treap)**.** *Per ogni coppia di indici $i, j \in \{1, \dots, n\}$, il nodo x_i è un antenato del nodo x_j se e solo se x_i possiede la priorità massima tra tutte le chiavi comprese tra x_i e x_j inclusi:*

$$A_{i,j} = 1 \iff p_i = \max\{p_k \mid k \in [\min(i,j), \max(i,j)]\}. \tag{10.20}$$

Dimostrazione. Consideriamo l'intervallo di chiavi $S_{i,j} = \{x_k \mid \min(i,j) \leq k \leq \max(i,j)\}$.

- **Necessità ($\implies$):** Supponiamo che x_i sia un antenato di x_j. Per la proprietà di BST, tutte le chiavi strettamente comprese tra x_i e x_j devono trovarsi nel sottoalbero radicato in x_i (dal lato in cui risiede x_j). Poiché l'intero intervallo $S_{i,j}$ appartiene al sottoalbero di x_i, per la proprietà di Max-Heap il nodo radice x_i deve possedere una priorità strettamente superiore a qualsiasi altro nodo presente nel suo sottoalbero: $p_i > p_k$ per ogni $k \neq i$ tale che $x_k \in S_{i,j}$.
- **Sufficienza ($\impliedby$):** Supponiamo che p_i sia la priorità massima in $S_{i,j}$. Esaminiamo il cammino dalla radice assoluta del Treap verso il basso. Il primo nodo dell'insieme $S_{i,j}$ incontrato lungo la discesa deve necessariamente essere il nodo con la priorità più elevata dell'insieme, ovvero x_i. Nessun altro nodo di $S_{i,j}$ può precedere x_i, altrimenti violerebbe l'invariante di Max-Heap. Di conseguenza, tutti gli altri elementi di $S_{i,j}$ (compreso x_j) risiedono nel sottoalbero radicato in x_i. Dunque x_i è un antenato di x_j.

□

Lemma 10.4.1 (Probabilità di Parentela)**.** *La probabilità che x_i sia antenato di x_j è inversamente proporzionale alla loro distanza nell'ordinamento delle chiavi:*

$$\mathbb{P}(A_{i,j} = 1) = \frac{1}{|j - i| + 1}. \tag{10.21}$$

Dimostrazione. L'intervallo $S_{i,j}$ contiene esattamente $|j - i| + 1$ elementi distinti. Poiché le priorità sono estratte in modo indipendente da una distribuzione continua uniforme, tutte le $(|j - i| + 1)!$ possibili permutazioni delle priorità all'interno di questo sottoinsieme sono equiprobabili per simmetria. La probabilità che uno specifico elemento (x_i) possieda la priorità massima tra gli $|j - i| + 1$ elementi è esattamente $\frac{1}{|j-i|+1}$. □

10.4.3 Profondità Attesa di un Nodo

La profondità $\text{Depth}(x_i)$ di un nodo x_i nel Treap è definita come il numero di spigoli sul cammino dalla radice a x_i, che equivale al numero di antenati propri di x_i:

$$\text{Depth}(x_i) = \sum_{j=1}^{n} A_{j,i} - 1 = \sum_{j \neq i} A_{j,i}. \tag{10.22}$$

Teorema 10.4.2 (Valore Atteso della Profondità)**.** *Il valore atteso della profondità del nodo x_i in un Treap di dimensione n è:*

$$\mathbb{E}[\text{Depth}(x_i)] = H_i + H_{n-i+1} - 2 = \mathcal{O}(\log n), \tag{10.23}$$

dove $H_k = \sum_{m=1}^{k} \frac{1}{m}$ denota il k-esimo numero armonico ($H_k = \ln k + \gamma + \mathcal{O}(1/k)$).

Dimostrazione. Per la linearità del valore atteso:

$$\begin{aligned} \mathbb{E}[\text{Depth}(x_i)] &= \sum_{j \neq i} \mathbb{E}[A_{j,i}] = \sum_{j \neq i} \mathbb{P}(A_{j,i} = 1) \\ &= \sum_{j=1}^{i-1} \frac{1}{i-j+1} + \sum_{j=i+1}^{n} \frac{1}{j-i+1}. \end{aligned} \tag{10.24}$$

Operiamo il cambio di variabile $k = i - j + 1$ nella prima sommatoria (k varia da 2 a i):

$$\sum_{j=1}^{i-1} \frac{1}{i-j+1} = \sum_{k=2}^{i} \frac{1}{k} = H_i - 1. \tag{10.25}$$

Operiamo il cambio di variabile $m = j - i + 1$ nella seconda sommatoria (m varia da 2 a $n - i + 1$):

$$\sum_{j=i+1}^{n} \frac{1}{j-i+1} = \sum_{m=2}^{n-i+1} \frac{1}{m} = H_{n-i+1} - 1. \tag{10.26}$$

Sommando i due contributi:

$$\mathbb{E}[\text{Depth}(x_i)] = (H_i - 1) + (H_{n-i+1} - 1) = H_i + H_{n-i+1} - 2. \tag{10.27}$$

Poiché $H_k \leq \ln k + 1$, per ogni $i \in \{1, \ldots, n\}$ si ha:

$$\mathbb{E}[\text{Depth}(x_i)] \leq \ln i + \ln(n - i + 1) \leq 2 \ln n = \mathcal{O}(\log n). \tag{10.28}$$

□

10.4.4 Numero Atteso di Rotazioni per Aggiornamento: $\mathcal{O}(1)$

Un risultato straordinario dimostrato da Seidel e Aragon riguarda il costo di ristrutturazione strutturale:

Teorema 10.4.3 (Costo Atteso di Ribilanciamento)**.** *Il numero atteso di rotazioni eseguite durante un'operazione di inserimento o cancellazione nel Treap è limitato da una costante universale:*

$$\mathbb{E}[\textit{rotazioni per inserimento}] \leq 2, \qquad \mathbb{E}[\textit{rotazioni per rimozione}] \leq 2. \tag{10.29}$$

Traccia della dimostrazione. Durante l'inserimento di una chiave z, ogni rotazione destra fa salire z lungo la colonna sinistra del suo sottoalbero, mentre ogni rotazione sinistra lo fa salire lungo la colonna destra. La lunghezza attesa della spina (*spine*) sinistra e della spina destra del sottoalbero radicato nel nuovo nodo è pari a $1 - \frac{1}{k}$, la cui somma complessiva converge a meno di 2 rotazioni in valore atteso. □

10.5 Isomorfismo Geometrico e Applicazioni Multidimensionali

Associando la chiave x all'ascissa e la priorità y all'ordinata di un punto bidimensionale, il Treap rivela un isomorfismo diretto con gli **Alberi Cartesiani** [12] e i **Priority Search Trees** [13]. Tale corrispondenza geometrica consente di estendere la struttura oltre la ricerca monodimensionale: dalla costruzione lineare deterministica $\mathcal{O}(n)$ e la risoluzione in tempo $\mathcal{O}(1)$ delle *Range Minimum Queries* (RMQ), fino alle interrogazioni ortogonali a tre lati (*3-sided range queries*) con complessità ottima $\mathcal{O}(\log n + k)$.

10.5.1 Alberi Cartesiani (Cartesian Tree) e Mappatura 2D

Definizione 10.5.1 (Albero Cartesiano)**.** Sia $P = \{(x_1, y_1), \ldots, (x_n, y_n)\} \subset \mathbb{R}^2$ un insieme di n punti nel piano cartesiano con coordinate mutualmente distinte. L'albero cartesiano associato a P è un albero binario in cui:

- Le ascisse x_i definiscono le **chiavi BST** (ordinamento orizzontale);
- Le ordinate y_i definiscono le **priorità Max-Heap** (ordinamento verticale).

Il punto con ordinata massima coincide con la radice dell'albero; la sua ascissa partiziona il piano nei sottoalberi sinistro e destro (Figura 10.5).

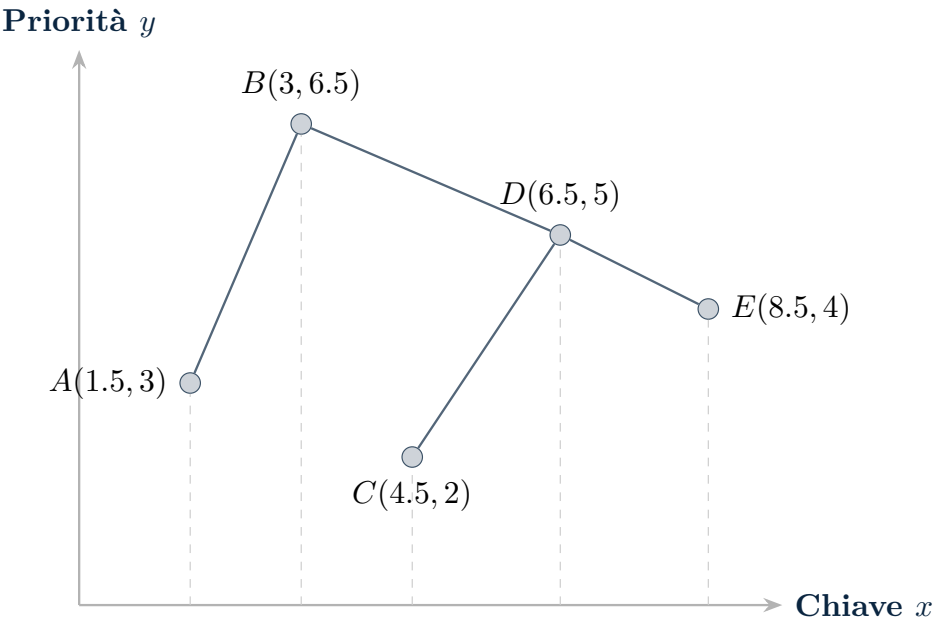


Figura 10.5: Isomorfismo geometrico dell'Albero Cartesiano: B possiede l'ordinata massima ed è radice. A sinistra di B risiede A ($x_A < x_B$). A destra di B risiedono $\{C, D, E\}$, con D come radice locale avente ordinata massima nel sottoinsieme destro.

10.5.2 Costruzione Deterministica Lineare $\mathcal{O}(n)$ tramite Stack Monotono

Se i punti sono preordinati per ascissa crescente ($x_1 < x_2 < \cdots < x_n$), l'albero cartesiano può essere costruito in tempo deterministico strettamente lineare $\mathcal{O}(n)$ mediante un algoritmo incrementale a **stack monotono** (Algoritmo 16):

Algorithm 16: Costruzione Lineare dell'Albero Cartesiano (BUILD-CARTESIAN($A[1 \dots n]$))

Input: Array di punti $A[1 \dots n]$ preordinati per chiave x crescente
Output: Radice dell'albero cartesiano

```
1  Inizializza uno stack vuoto S
2  for i ← 1 to n do
3      last ← NIL
4      while S non è vuoto and y(S.TOP()) < y(A[i]) do
5          last ← S.POP()
6      left(A[i]) ← last
7      if S non è vuoto then
8          right(S.TOP()) ← A[i]
9      S.PUSH(A[i])
10 return elemento più profondo nello stack (la radice globale)
```

Analisi di Complessità Ammortizzata: L'algoritmo mantiene nello stack la cosiddetta *spina destra* (*right spine*) dell'albero corrente. A ogni iterazione, l'elemento $A[i]$ rimuove dallo stack tutti i nodi con ordinata inferiore alla propria; l'ultimo nodo estratto diventa il suo figlio sinistro, mentre $A[i]$ diviene il figlio destro del nuovo elemento in cima allo stack. Poiché ciascuno degli n nodi viene inserito nello stack esattamente una volta (PUSH) ed estratto al massimo una volta (POP), il numero complessivo di operazioni sullo stack è al più $2n$. Il tempo di esecuzione complessivo è pertanto strettamente deterministico e lineare: $\mathcal{O}(n)$.

10.5.3 Risoluzione Ottimale di Range Minimum Query (RMQ) in $\mathcal{O}(1)$

L'albero cartesiano costituisce il ponte teorico fondamentale per collegare la ricerca del minimo in un intervallo di un array al problema del minimo antenato comune in un albero:

$$\mathrm{RMQ}_A(i, j) = \arg\min_{i \le k \le j} A[k]. \tag{10.30}$$

Costruendo l'albero cartesiano (versione Min-Heap) sulle coppie $(k, A[k])$:

$$\mathrm{RMQ}_A(i, j) = \mathrm{LCA}_{\mathrm{CartesianTree}}(A[i], A[j]), \tag{10.31}$$

dove LCA denota il *Lowest Common Ancestor* (Minimo Antenato Comune). Poiché l'LCA su alberi statici può essere risolto in tempo $\mathcal{O}(1)$ con pre-

processing lineare $\mathcal{O}(n)$ (mediante la tecnica di Bender e Farach-Colton [14]), l'albero cartesiano consente di ottenere RMQ in tempo $\mathcal{O}(1)$ ottimale.

10.5.4 Query di Intervallo a Tre Lati (3-Sided Range Queries)

Nelle applicazioni geospaziali, nei database temporali e nella geometria computazionale, una classe fondamentale di interrogazioni multidimensionali è costituita dalle **query di intervallo a tre lati** (o regioni semi-aperte superiormente):

$$Q = [x_{\min}, x_{\max}] \times [y_{\min}, +\infty). \tag{10.32}$$

Una tale query richiede di identificare tutti i punti $p = (x, y)$ che soddisfano simultaneamente:

$$x_{\min} \leq x \leq x_{\max} \quad \wedge \quad y \geq y_{\min}. \tag{10.33}$$

Nel contesto dei Treap, l'asse delle ascisse esprime l'ordinamento alfabetico o lessicografico delle chiavi (definendo la query come un intervallo di lettere $[x_{\min}, x_{\max}]$), mentre l'asse delle ordinate misura le priorità numeriche (esprimendo il vincolo verticale come un intervallo di priorità $[y_{\min}, +\infty)$), come illustrato in Figura 10.6.

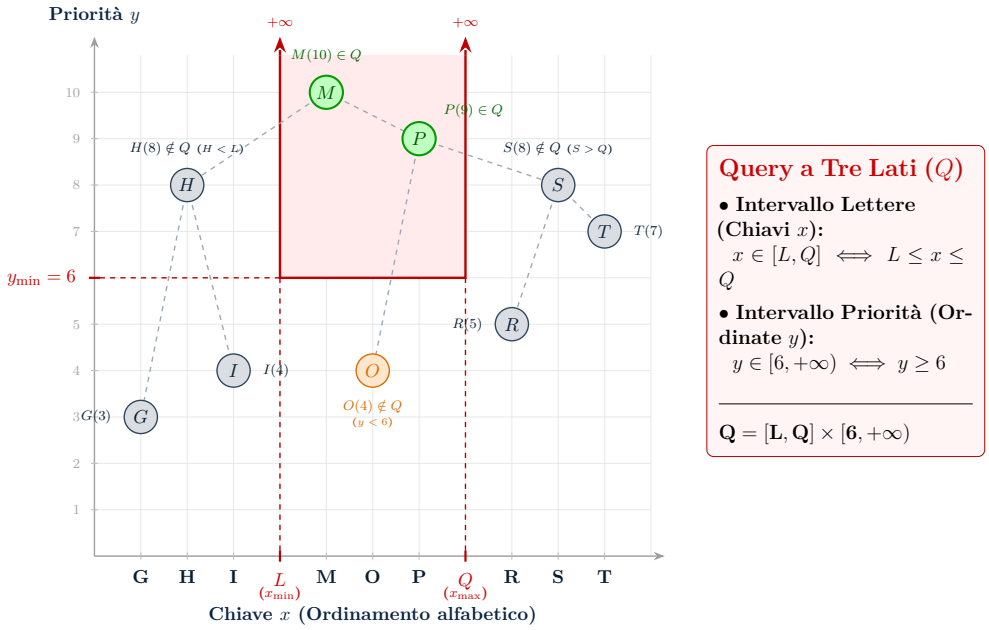


Figura 10.6: Query a tre lati $Q = [L, Q] \times [6, +\infty)$ nel piano cartesiano: l'asse x segue l'ordine alfabetico delle chiavi e l'asse y le priorità. I punti nella regione sono M e P. I punti G, H, I e R, S, T cadono fuori dall'intervallo di lettere, mentre O viola la priorità minima ($y < 6$). Le linee tratteggiate indicano la struttura del Treap.

Il Principio di Potatura Istantanea ($\mathcal{O}(1)$ Pruning): La combinazione tra ordinamento orizzontale e proprietà di Max-Heap offre un meccanismo di potatura insuperabile per filtrare i punti fuori regione (Figura 10.6):

- **Filtro orizzontale (BST sulle chiavi x):** consente di delimitare la ricerca esclusivamente ai rami che si intersecano con $[x_{\min}, x_{\max}]$ discendendo dal Lowest Common Ancestor (LCA) di $x_{\min}$ e $x_{\max}$.
- **Filtro verticale (Max-Heap sulle priorità y):** per qualsiasi nodo u, se la sua priorità viola la soglia minima ($p(u) < y_{\min}$), allora per la proprietà di Max-Heap **nessun discendente di u potrà mai avere priorità $\geq y_{\min}$**. L'esplorazione del sottoalbero radicato in u viene interrotta all'istante in tempo $\mathcal{O}(1)$.

Due Paradigmi di Risoluzione: Nell'ingegneria algoritmica, l'interrogazione può essere condotta secondo due approcci complementari:

1. **Visita ricorsiva guidata:** si discendono i cammini di confine per $x_{\min}$ e $x_{\max}$, visitando i sottoalberi intermedi e potando istantaneamente ogni ramo la cui radice locale presenta $y < y_{\min}$.

2. **Isolamento mediante Split a sentinella:** si eseguono due operazioni di Split inserendo due nodi sentinella con priorità $+\infty$ corrispondenti a $x_{\min}$ e $x_{\max}$. Tale procedura isola un sottoalbero contenente esattamente i nodi candidati con $x \in [x_{\min}, x_{\max}]$, sul quale la visita si riduce a un puro filtraggio heap guidato dalla soglia $y_{\min}$.

Teorema 10.5.1 (Complessità Output-Sensitive della 3-Sided Query)**.** *L'esecuzione di una 3-sided range query su un Treap di n nodi contenente k_{out} punti soddisfacenti richiede tempo:*

$$T(n, k_{out}) = \mathcal{O}(\log n + k_{out}) \quad \textit{in valore atteso.} \tag{10.34}$$

Dimostrazione. L'insieme dei nodi esplorati dall'algoritmo si partiziona in due classi disgiunte:

1. **Nodi di confine:** sono i nodi incontrati lungo i cammini di ricerca BST per delimitare le soglie $x_{\min}$ e $x_{\max}$. Poiché l'altezza attesa del Treap è $\mathcal{O}(\log n)$, il numero atteso di tali nodi è $\mathcal{O}(\log n)$. Nel paradigma basato su split, le due rotazioni di risalita delle sentinelle impiegano parimenti tempo $\mathcal{O}(\log n)$.

2. **Nodi interni alla regione e foglie di potatura:** ogni nodo visitato che supera la soglia di priorità $p(u) \geq y_{\min}$ appartiene alla soluzione e viene emesso in output, contribuendo al termine $+k_{\text{out}}$. Ogni ramo infruttuoso viene troncato non appena viene incontrato il primo nodo con priorità $< y_{\min}$; pertanto, il numero di nodi scartati visitati è al più pari al numero di foglie di potatura, limitato da $2k_{\text{out}} + 2 = \mathcal{O}(k_{\text{out}})$.

La complessità totale è dunque ottimale ed *output-sensitive*: $\mathcal{O}(\log n + k_{\text{out}})$. □

10.5.5 Esempio Passo-Passo: Risoluzione di una 3-Sided Query con Split e Potatura

Per illustrare in dettaglio la dinamica operativa della struttura, esaminiamo l'esecuzione completa di una query a tre lati mediante la tecnica dell'isolamento con nodi sentinella e successiva potatura heap.

Definizione dell'Istanza di Query: Si consideri la query bidimensionale:

$$Q = [x_1, x_2] \times [y_1, +\infty) = [L, Q] \times [6, +\infty), \tag{10.35}$$

definita dai vincoli:

- **Intervallo orizzontale:** chiavi alfabetiche $x \in [L, Q]$, ovvero $L \leq x \leq Q$ secondo l'ordine lessicografico;
- **Soglia verticale:** priorità numeriche $y \geq 6$.

Topologia del Treap Iniziale: I nodi memorizzano coppie $\frac{\text{Key}}{\text{Priority}} = \frac{x}{y}$, ordinati per chiave alfabetica ($G < H < I < L < M < O < P < Q < R < S < T$) e vincolati dalla proprietà di Max-Heap sulle ordinate. L'albero di partenza conta 9 elementi (Figura 10.7(a)):

- Radice globale: $M/10$;
- Sottoalbero sinistro: $H/8$, con figli $G/3$ (sinistro) e $I/4$ (destro);
- Sottoalbero destro: $P/9$, con figlio sinistro $O/4$ e figlio destro $S/8$ (a sua volta genitore di $R/5$ a sinistra e $T/7$ a destra).

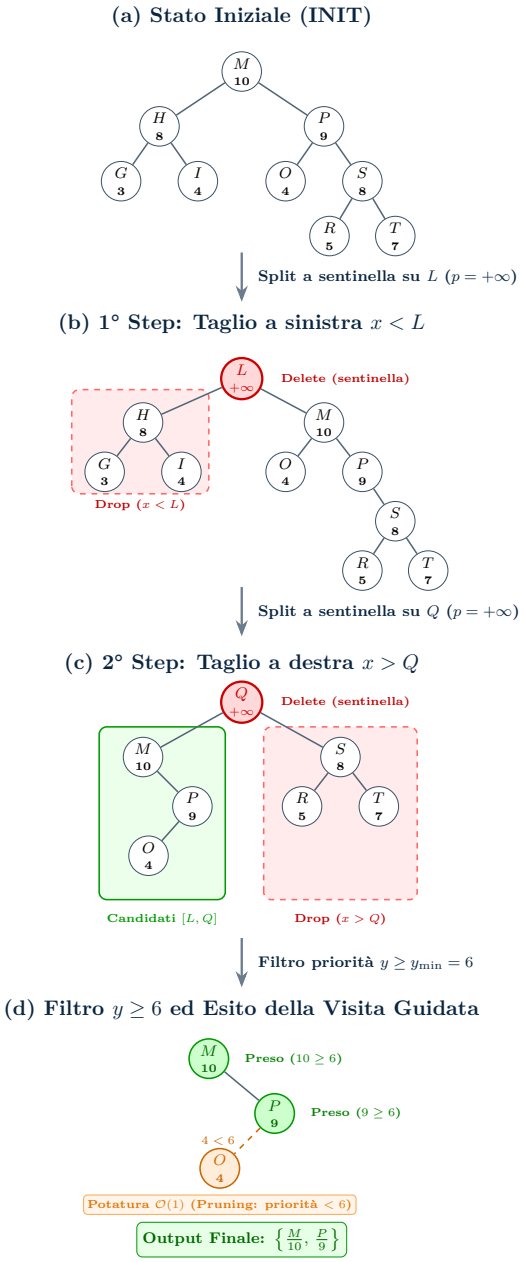


Figura 10.7: Esecuzione passo-passo della 3-sided range query $[L, Q] \times [6, +\infty)$ su Treap, rappresentata verticalmente: (a) Stato iniziale del Treap; (b) 1° Step: Taglio sinistro con sentinella $(L, +\infty)$ e DROP di $\{G, H, I\}$ $(x < L)$; (c) 2° Step: Taglio destro con sentinella $(Q, +\infty)$ e DROP di $\{R, S, T\}$ $(x > Q)$, isolando il sottoalbero dei candidati $[L, Q]$; (d) Visita guidata del sottoalbero candidati con filtro $y \geq 6$ e potatura immediata $\mathcal{O}(1)$ del ramo radicato in $(O/4)$.

Fase 1: Isolamento del Confine Inferiore ($x < L$): Per scartare in blocco tutti gli elementi con chiave inferiore a L, si inserisce la chiave sentinella $(L, +\infty)$:

- **Bubble-up alla radice:** La sentinella entra come foglia (figlio destro di $I/4$). Avendo priorità infinita, una catena di rotazioni la promuove a radice dell'intero albero in tempo atteso $\mathcal{O}(\log n)$ (Figura 10.7(b)).
- **Scarto del sottoalbero sinistro (DROP):** Per la proprietà BST, il sottoalbero sinistro di L racchiude l'insieme $\{G, H, I\}$. Poiché ciascuna di queste chiavi verifica $x < L$, esse cadono interamente fuori da $[L, Q]$ e vengono eliminate all'istante in tempo $\mathcal{O}(1)$.
- **Deallocazione della sentinella:** Il nodo fittizio $(L, +\infty)$ viene rimosso, lasciando il sottoalbero destro $\{M, O, P, R, S, T\}$ come insieme attivo per la fase successiva.

Fase 2: Isolamento del Confine Superiore ($x > Q$): Sul sottoalbero residuo si delimita l'estremo superiore inserendo la seconda sentinella $(Q, +\infty)$:

- **Bubble-up di Q alla radice:** La priorità massima costringe Q a scalare fino alla radice del sottoalbero tramite rotazioni in tempo $\mathcal{O}(\log n)$ (Figura 10.7(c)).
- **Scarto del sottoalbero destro (DROP):** Il sottoalbero destro di Q include $\{R, S, T\}$. Poiché $x > Q$, tutti questi punti violano il vincolo orizzontale e vengono scartati in tempo $\mathcal{O}(1)$.
- **Sottoalbero dei candidati:** Eliminata la sentinella $(Q, +\infty)$, il sottoalbero sinistro isola precisamente l'insieme dei nodi con $x \in [L, Q]$:

$$\text{Candidati } [L, Q] = \left\{ \frac{M}{10}, \frac{P}{9}, \frac{O}{4} \right\}. \tag{10.36}$$

  Tale sottoalbero mantiene rigorosamente valida la proprietà di Max-Heap sulle ordinate.

Fase 3: Esplorazione Guidata e Potatura Immediata ($y \geq 6$): L'estrazione dei punti validi procede con una visita ricorsiva a partire dalla radice dei candidati ($M/10$) applicando il filtro $y \geq 6$ (Figura 10.7(d)):

1. **Nodo** $(M/10)$**:** Priorità $y = 10 \geq 6$. Il punto soddisfa la condizione e viene emesso in output. Si prosegue sui figli (figlio destro $P/9$).

2. **Nodo** $(P/9)$**:** Priorità $y = 9 \geq 6$. Il punto viene emesso in output. Si prosegue sui figli (figlio sinistro $O/4$).

3. **Nodo** $(O/4)$**:** Priorità $y = 4 < 6$. Il punto viola la soglia minima.

    - **Potatura** $\mathcal{O}(1)$ **(*Pruning*):** Per la proprietà di Max-Heap, nessun discendente nel sottoalbero di $(O/4)$ potrà mai avere priorità superiore a 4, e di conseguenza nessuno potrà soddisfare la soglia $y \geq 6$. La visita di questo intero ramo viene arrestata immediatamente in tempo $\mathcal{O}(1)$.

Esito Computazionale e Analisi dei Costi: L'insieme soluzione risultante è:

$$\text{Output} = \left\{ \frac{M}{10}, \frac{P}{9} \right\}. \tag{10.37}$$

L'intero processo si articola in:

- **Fase di Split / Isolamento:** due inserimenti con risalita di sentinelle alla radice, con costo atteso $\mathcal{O}(\log n)$;
- **Fase di Traversal ed emissione:** visita dei soli nodi appartenenti all'output (M e P) più una singola foglia di potatura (O), con costo proporzionale all'output: $\mathcal{O}(k)$ con $k = 2$;
- **Tempo complessivo:**

$$T(n) = \mathcal{O}(\log n + k), \tag{10.38}$$

  a riprova dell'eccellente efficienza output-sensitive garantita dalla sinergia tra albero di ricerca binario ed heap.

Capitolo 11

Algoritmi di Intersezione per Posting Lists nell'Information Retrieval

11.1 Architettura di un Motore di Ricerca e Contesto del Problema

L'efficienza della valutazione delle query nei moderni sistemi di reperimento dell'informazione dipende strettamente dall'organizzazione modulare dell'infrastruttura e dalla rappresentazione compressa e ordinata dei dati testuali. In questa sezione formalizziamo i componenti cardine di un motore di ricerca e introduciamo il problema dell'intersezione di insiemi ordinati.

11.1.1 I Moduli Fondamentali di un Web Search Engine

Un moderno motore di ricerca web elabora collezioni composte da decine di miliardi di documenti testuali eterogenei, rispondendo a query complesse con tempi di latenza inferiori a poche decine di millisecondi. Per raggiungere questo obiettivo, l'infrastruttura di retrieval viene strutturata in cinque macro-moduli cooperanti, illustrati schematicamente in Figura 11.1:

1. **Web Crawler (24/7):** Modulo autonomo sempre attivo che scandisce continuativamente il World Wide Web navigando la struttura ad ipertesto. Scopre nuovi URL, preleva pagine HTML aggiornate e archivia i dati grezzi in un repository locale distribuito.

2. **Parser & Indexer:** Elabora i flussi di testo grezzo scaricati. Esegue la pulizia del codice di markup, la tokenizzazione (*tokenization*), la

normalizzazione linguistica (conversione in minuscolo, rimozione dei segni di punteggiatura), l'eliminazione dei termini ad altissima frequenza privi di contenuto informativo discriminante (*stop-words elimination*) e la riduzione alla radice morfologica (*stemming* o *lemmatization*). Al termine dell'analisi, costruisce la struttura dati cardine per il retrieval: l'**Indice Invertito**.

3. **Inverted Index (Indice Invertito):** È la struttura dati primaria per l'indicizzazione del testo. Consiste di due componenti collegate:
   - **Dizionario (Dictionary / Vocabulary):** Mantiene l'elenco dei termini unici $\mathcal{V} = \{t_1, t_2, \ldots, t_{|\mathcal{V}|}\}$ estratti dall'intera collezione di documenti. Per ciascun termine memorizza metadati globali, quali la frequenza nei documenti (df_t, *document frequency*) e un puntatore alla rispettiva lista di inversione. Il dizionario risiede tipicamente interamente in memoria RAM centrale (tramite tabelle hash, trie o B-Tree).
   - **Posting Lists (Liste di inversione):** Per ogni termine $t \in \mathcal{V}$, una sequenza strettamente ordinata di identificatori interi univoci dei documenti (`DocID`) in cui il termine compare almeno una volta.

4. **Retriever:** Riceve le query sottomesse dagli utenti (nella forma più elementare, query booleane congiuntive quali $t_1 \wedge t_2$). Consulta il dizionario, recupera le posting lists associate ai termini interrogati ed esegue l'**intersezione dei set** per identificare l'insieme dei documenti che contengono contemporaneamente tutti i termini richiesti.

5. **Ranker:** Riceve l'insieme dei documenti candidati generato dal Retriever. Calcola per ciascuno di essi un punteggio di pertinenza numerico (*relevance score*) impiegando modelli statistici (es. TF-IDF, Okapi BM25) o modelli avanzati di apprendimento automatico (*Learning to Rank*). Ordina i risultati in modo decrescente rispetto al punteggio e restituisce la Top-k all'utente.

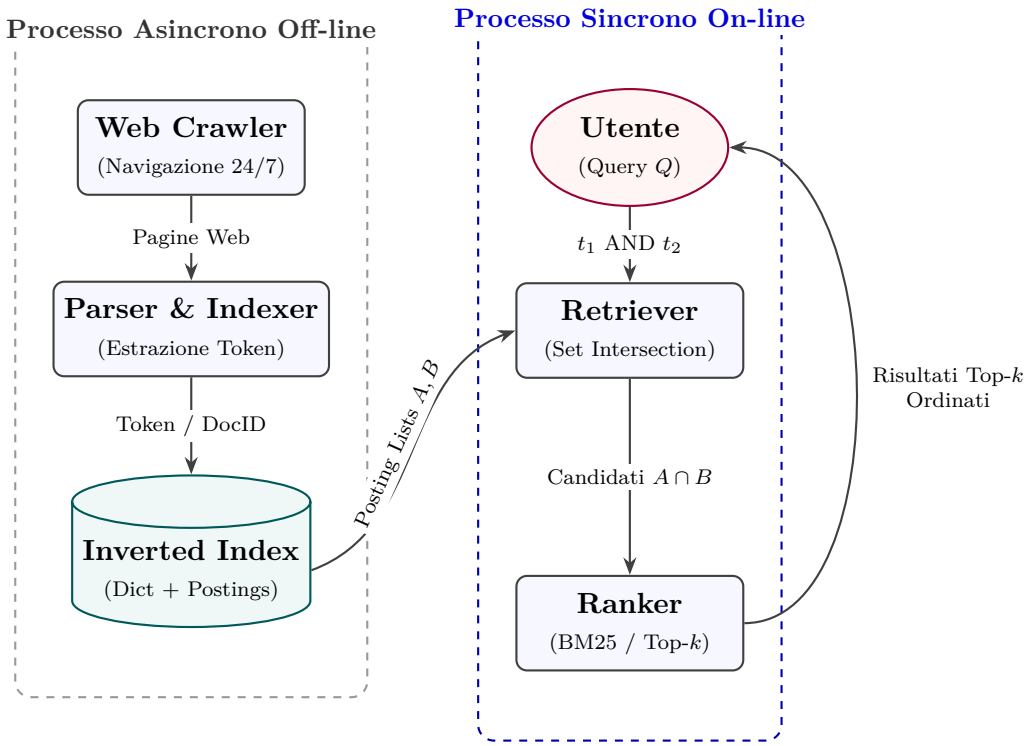


Figura 11.1: Architettura a moduli coordinati di un motore di ricerca web: flusso off-line di crawling e indicizzazione vs pipeline on-line di retrieval e ranking.

11.1.2 Formulazione Formale del Problema (Set Intersection)

Data una query congiuntiva formulata da un utente nella forma:

$$Q = t_1 \ \wedge \ t_2, \tag{11.1}$$

il modulo Retriever interroga l'indice invertito ed estrae le posting lists corrispondenti ai termini t_1 e t_2, denotate rispettivamente con A e B:

$$A = [a_1, a_2, \ldots, a_n], \quad |A| = n, \tag{11.2}$$
$$B = [b_1, b_2, \ldots, b_m], \quad |B| = m. \tag{11.3}$$

Proprietà 11.1.1 (Proprietà degli Insiemi di Inversione)**.** *Le due liste di identificatori soddisfano i seguenti assiomi operativi:*

1. ***Ordinamento Stretto:*** *Gli elementi di ciascuna lista sono numeri interi positivi strettamente crescenti, rappresentanti i* `DocID` *univoci*

assegnati ai documenti durante la scansione:

$$a_1 < a_2 < \cdots < a_n \quad e \quad b_1 < b_2 < \cdots < b_m. \tag{11.4}$$

2. ***Assunzione di Asimmetria (Senza Perdita di Generalità):*** *Senza alcuna perdita di generalità, assumiamo convenzionalmente che la lista B sia la più corta delle due:*

$$m \leq n. \tag{11.5}$$

Nei casi reali, il rapporto n/m può risultare marcatamente sbilanciato (ad esempio quando t_1 è un termine frequente come "computer" con $n \approx 10^7$, e t_2 è un termine specialistico come "quicksort" con $m \approx 10^3$).

Obiettivo Algoritmico: Calcolare l'insieme intersezione:

$$A \cap B = \{x \in \mathbb{N} \mid x \in A \wedge x \in B\} \tag{11.6}$$

nel minor tempo computazionale possibile, minimizzando i confronti effettuati e massimizzando l'efficienza degli accessi alla memoria.

11.2 Algoritmi Fondamentali e Analisi Comparativa

Per calcolare l'intersezione tra due insiemi ordinati di cardinalità sbilanciata, la letteratura classica propone strategie basate sulla scansione sequenziale coordinata o sulla ricerca binaria mirata. In questa sezione analizziamo la progressione da approcci elementari a soluzioni ottimali per coppie di liste, derivandone il punto di pareggio computazionale.

11.2.1 Algoritmo Brute Force

L'approccio ingenuo prescinde dalla struttura ordinata delle liste:

- Per ciascuno degli m elementi $b_j \in B$, si scandisce linearmente l'intera sequenza A dall'indice 1 fino ad n, arrestandosi non appena si incontra una corrispondenza $a_i == b_j$ o un elemento $a_i > b_j$.
- Nel caso peggiore, ogni elemento di B comporta la verifica completa di tutti gli n elementi di A.
- **Complessità Temporale:** $\mathcal{O}(n \cdot m)$.
- **Valutazione Ingegneristica:** Assolutamente inapplicabile su indici reali: con $n = 10^7$ e $m = 10^4$, comporterebbe fino a 10^{11} operazioni per una singola query.

11.2.2 Merge-Based: Scansione Lineare a Due Puntatori

L'algoritmo **Merge-based** sfrutta sistematicamente l'ordinamento monotono crescente di entrambe le sequenze. Mantenendo due indici di scansione i (per A) e j (per B) inizializzati all'inizio delle rispettive liste, la procedura emula il passo di unione (*merge step*) del classico MergeSort (algorithm 17):

Algorithm 17: Intersezione Merge-Based a Due Puntatori

Dati: Due liste ordinate $A = [a_1, \ldots, a_n]$ e $B = [b_1, \ldots, b_m]$ con $m \leq n$
Risultato: Lista ordinata dei documenti comuni $C = A \cap B$

```
1  i ← 1; j ← 1
2  C ← ∅
3  while i ≤ n and j ≤ m do
4  |  if A[i] == B[j] then
5  |  |  C.APPEND(A[i])
6  |  |  i ← i + 1
7  |  |  j ← j + 1
8  |  end
9  |  else if A[i] < B[j] then
10 |  |  i ← i + 1
11 |  end
12 |  else
   |  |  // A[i] > B[j]
13 |  |  j ← j + 1
14 |  end
15 end
16 return C
```

Teorema 11.2.1 (Complessità della Scansione Merge-Based)**.** *L'algoritmo di intersezione a due puntatori esegue al più $n + m - 1$ confronti tra elementi e richiede tempo:*

$$T_{merge}(n, m) = \mathcal{O}(n + m). \tag{11.7}$$

Dimostrazione. Ad ogni iterazione del ciclo **while**, viene effettuato almeno un confronto tra $A[i]$ e $B[j]$. Indipendentemente dall'esito (uguaglianza, minoranza o maggioranza), almeno uno dei due indici (i oppure j) viene incrementato rigidamente di un'unità. Poiché l'algoritmo termina non appena $i > n$ o $j > m$, il numero totale di iterazioni non può superare la somma delle lunghezze delle liste diminuita di uno, ossia $(n + m - 1)$. □

Vantaggi e Limiti della Scansione Lineare.

- **Pregi:** L'algoritmo accede agli elementi di A e B in modo puramente **sequenziale**. Questo schema garantisce una perfetta località spaziale in cache, massimizza l'efficacia del meccanismo di *hardware prefetching* dei processori moderni e consente la lettura in streaming (*pipeline streaming*) direttamente da disco senza seek casuali.
- **Difetti:** Quando $m \ll n$, l'algoritmo esegue un lavoro ridondante sproporzionato: scandisce comunque l'intera lista lunga A elemento per elemento, ignorando la possibilità di compiere ampi salti in avanti per scartare blocchi di documenti non rilevanti.

11.2.3 Binary Search Intersection

Per superare la lentezza della scansione lineare quando la lista B è molto più corta di A, si applica la ricerca binaria:

- Si itera unicamente sopra gli m elementi della lista corta B.
- Per ciascun elemento $b_j \in B$, si esegue una ricerca dicotomica classica (BinarySearch) sull'intero array A.
- Poiché una ricerca binaria su un array di dimensione n richiede al più $\lfloor \log_2 n \rfloor + 1$ confronti:

$$T_{\text{binary}}(n, m) = \mathcal{O}(m \log n). \tag{11.8}$$

11.2.4 Analisi Comparativa del Trade-off: Merge-Based vs Binary Search

Determiniamo analiticamente la soglia di convenienza (*breakeven threshold*) confrontando le due complessità:

$$T_{\text{binary}}(n, m) < T_{\text{merge}}(n, m) \iff m \log n < n + m. \tag{11.9}$$

Risolvendo rispetto alla cardinalità m:

$$\begin{aligned} m \log n - m &< n \\ m(\log n - 1) &< n \\ m &< \frac{n}{\log n - 1} \approx \frac{n}{\log n}. \end{aligned} \tag{11.10}$$

Corollario 11.2.1 (Regioni di Efficienza Asintotica)**.** *Dalla relazione* (11.10) *discendono due regimi operativi distinti:*

1. ***Regime Sparso*** *($m < \frac{n}{\log n}$):* *La lista B è sufficientemente rada rispetto ad A. La ricerca binaria risulta asintoticamente più veloce della scansione lineare, poiché il costo totale dei salti dicotomici è strettamente sublineare rispetto alla dimensione di A.*

2. ***Regime Denso*** *($m \geq \frac{n}{\log n}$):* *La lista B possiede una densità comparabile con A. La scansione lineare a due puntatori risulta asintoticamente e praticamente superiore, poiché il fattore moltiplicativo costante delle letture sequenziali in cache batte i continui salti logaritmici non locali.*

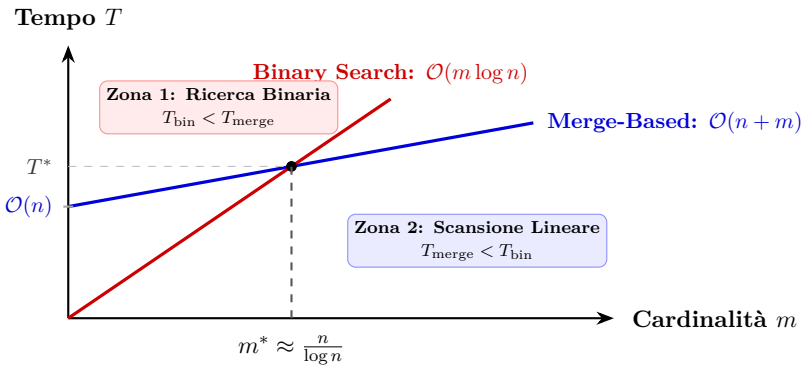


Figura 11.2: Comportamento qualitativo e punto di pareggio computazionale $m^* \approx \frac{n}{\log n}$ tra scansione lineare a due puntatori e ricerca binaria al variare della cardinalità m della lista corta (con dimensione n della lista lunga fissata).

11.3 Mutual Partition (Divide et Impera)

La combinazione del paradigma Divide et Impera con il partizionamento sincronizzato consente di superare le inefficienze delle ricerche binarie ripetute su indici non bilanciati. In questa sezione esaminiamo l'algoritmo Mutual Partition, dimostrandone l'ottimalità teorica rispetto al limite inferiore basato su confronti.

11.3.1 Principio di Funzionamento Algoritmico

L'algoritmo **Mutual Partition** (partizionamento mutuale) supera la rigidità delle ricerche binarie indipendenti applicando una suddivisione bilanciata simultanea basata sul paradigma **Divide et Impera** [15]:

1. **Scelta del Pivot su A:** Si seleziona l'elemento mediano della lista lunga A:
$$pivot = A\left[\left\lfloor \frac{n}{2} \right\rfloor\right]. \tag{11.11}$$

2. **Localizzazione del Pivot su B:** Tramite una singola ricerca binaria sulla lista corta B, si determina la posizione esatta in cui il $pivot$ si colloca o dovrebbe collocarsi in B.

3. **Decomposizione in Sottoproblemi Disgiunti:** L'elemento $pivot$ suddivide naturalmente entrambi gli insiemi in due partizioni ordinate:
$$A = A_1 \cup \{pivot\} \cup A_2, \quad \text{con } A_1 < pivot \text{ e } A_2 > pivot, \tag{11.12}$$
$$B = B_1 \cup (\{pivot\} \cap B) \cup B_2, \quad \text{con } B_1 < pivot \text{ e } B_2 > pivot. \tag{11.13}$$
Se la ricerca binaria individua il $pivot$ in B alla posizione p_B ($B[p_B] = pivot$), esso appartiene all'intersezione ($pivot \in A \cap B$) e viene inserito immediatamente nell'insieme risultato C. In caso contrario ($pivot \notin B$), p_B individua l'indice di split tale che $B[p_B - 1] < pivot < B[p_B]$.

4. **Proprietà Distributiva:** Poiché tutti gli elementi di A_1 e B_1 sono strettamente minori di $pivot$, e tutti gli elementi di A_2 e B_2 sono strettamente maggiori di $pivot$, nessun elemento di A_1 potrà mai intersecare B_2, e viceversa. L'intersezione globale si decompone esattamente nell'unione disgiunta di due intersezioni indipendenti:
$$A \cap B = (A_1 \cap B_1) \cup (\{pivot\} \cap B) \cup (A_2 \cap B_2). \tag{11.14}$$
Si richiamano ricorsivamente le due istanze disgiunte (A_1, B_1) e (A_2, B_2), senza reinserire il $pivot$ al loro interno.

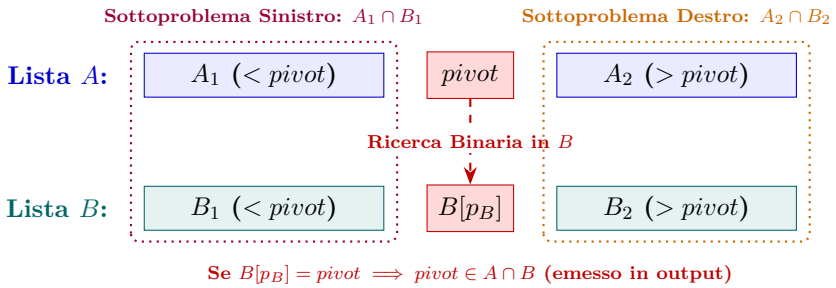


Figura 11.3: Schema concettuale di Mutual Partition: il mediano $pivot = A[n/2]$ viene ricercato in B tramite ricerca binaria; se presente alla posizione p_B, entra nell'intersezione. I due insiemi si decompongono nei sottoproblemi indipendenti $A_1 \cap B_1$ e $A_2 \cap B_2$, risolvibili ricorsivamente in parallelo.

11.3.2 Complessità e Limite Teorico dell'Albero di Decisione

Teorema 11.3.1 (Complessità di Mutual Partition)**.** *L'algoritmo Mutual Partition calcola l'intersezione tra la lista A di dimensione n e la lista B di dimensione m ($m \leq n$) in tempo asintotico:*

$$T(n, m) = \Theta\left(m\left(1 + \log\frac{n}{m}\right)\right). \tag{11.15}$$

Verifica dei Casi Limite Estremi. La formula (11.15) interpola in modo ideale l'intero spettro di dimensioni possibili:

- **Caso Bilanciato ($n = m$):**

  $$T(m, m) = \Theta\left(m\left(1 + \log\frac{m}{m}\right)\right) = \Theta(m(1 + 0)) = \mathcal{O}(m) = \mathcal{O}(n + m). \tag{11.16}$$

  La complessità degenera naturalmente in quella ottimale della scansione lineare merge-based.

- **Caso Fortemente Asimmetrico ($m = \mathcal{O}(1)$):**

  $$T(n, 1) = \Theta\left(1 \cdot \left(1 + \log\frac{n}{1}\right)\right) = \mathcal{O}(\log n). \tag{11.17}$$

  La complessità corrisponde a quella della singola ricerca binaria.

11.3.3 Lower Bound Informativo basato su Confronti

Dimostriamo che la complessità raggiunta da Mutual Partition è asintoticamente imbattibile da qualunque algoritmo basato su confronti.

Teorema 11.3.2 (Lower Bound Informativo per Set Intersection)**.** *Qualsiasi algoritmo basato su confronti che calcoli l'intersezione tra due sequenze ordinate di dimensioni n ed m ($m \leq n$) richiede nel caso peggiore:*

$$\#CMP \geq \Omega\left(m \log\frac{n}{m}\right) \quad \textit{confronti.} \tag{11.18}$$

Dimostrazione. Consideriamo il problema di determinare la posizione relativa degli m elementi ordinati di B all'interno della sequenza ordinata degli n elementi di A. Il numero di modi distinti in cui m elementi possono essere posizionati tra gli n slot ordinati è pari al coefficiente binomiale:

$$\#\text{Configurazioni Possibili} = \binom{n}{m} = \frac{n!}{m!\,(n-m)!}. \tag{11.19}$$

In un modello computazionale astratto basato su **albero di decisione ternario** (in cui ogni nodo interno rappresenta un confronto tricotomico $<, =, >$ e ciascuna foglia rappresenta una configurazione di output consistente), l'altezza minima h dell'albero necessaria per distinguere tra tutte le foglie valide rispetta il vincolo informativo fondamentale:

$$\#\text{CMP} \geq \log_2(\#\text{Configurazioni}) \geq \log_2 \binom{n}{m}. \tag{11.20}$$

Sfruttando la ben nota disuguaglianza combinatoria per il coefficiente binomiale:

$$\binom{n}{m} \geq \left(\frac{n}{m}\right)^m, \tag{11.21}$$

applicando il logaritmo in base 2 ad entrambi i membri otteniamo:

$$\log_2 \binom{n}{m} \geq \log_2 \left(\left(\frac{n}{m}\right)^m\right) = m \log_2 \left(\frac{n}{m}\right). \tag{11.22}$$

Pertanto, nessun algoritmo basato su confronti può avere una complessità asintotica inferiore a $\Omega\left(m \log \frac{n}{m}\right)$. L'algoritmo Mutual Partition raggiunge esattamente questo limite teorico, risultando asintoticamente ottimo. □

11.3.4 Limitazione Pratica di Mutual Partition

Nonostante la sua eleganza matematica e ottimalità asintotica, l'algoritmo Mutual Partition presenta una seria criticità nell'ingegneria dei sistemi reali:

"This is good, but it is recursive!"

La natura intrinsecamente ricorsiva comporta una continua allocazione di frame sullo stack di sistema. Inoltre, a causa del partizionamento mediano a salti, gli accessi alla memoria fisica risultano non sequenziali e irregolari, vanificando la cache line locality e provocando un continuo decadimento del throughput computazionale su vettori di grandi dimensioni.

11.4 Doubling Search (Exponential Search / Galloping Search)

L'eliminazione della ricorsione e il miglioramento della località spaziale nei salti guidano la progettazione di algoritmi iterativi basati su incrementi esponenziali del passo di ricerca. In questa sezione introduciamo il Doubling Search, dimostrandone rigorosamente la complessità ottima mediante la disuguaglianza di Jensen.

11.4.1 Motivazione e Intuizione dei Salti Esponenziali

L'algoritmo **Doubling Search** (denominato nella letteratura classica anche **Exponential Search** o **Galloping Search**, formalizzato da Bentley e Yao [16]) nasce per conseguire la medesima complessità ottima $\mathcal{O}(m \log \frac{n}{m})$ di Mutual Partition, ma attraverso una procedura **completamente iterativa e non ricorsiva**.

L'intuizione fondamentale risiede nello sfruttare la correlazione spaziale tra elementi successivi di B. Supponiamo di aver appena localizzato l'elemento $b_{j-1} \in B$ in corrispondenza dell'indice i_{j-1} nell'array A:

- Per individuare la posizione del successivo elemento $b_j \in B$, non è necessario ripartire dall'indice 1 né condurre una ricerca binaria estesa a tutto il prefisso rimanente di A.
- Poiché B è ordinato ($b_j > b_{j-1}$), sappiamo a priori che la posizione di b_j in A, indicata con i_j, deve necessariamente soddisfare $i_j \geq i_{j-1}$.
- Invece di avanzare di un solo passo alla volta (come nella scansione lineare) o di dimezzare l'intero array residuo, si avanza a partire da i_{j-1} compiendo salti di ampiezza esponenziale pari a **potenze crescenti di 2**:
$$+2^0,\ +2^1,\ +2^2,\ +2^3,\ \ldots,\ +2^k. \tag{11.23}$$
- Non appena si individua il primo esponente k tale che $A[i_{j-1} + 2^k] \geq b_j$, si restringe la ricerca binaria unicamente all'intervallo compreso tra il penultimo salto e l'ultimo salto:
$$\text{Intervallo di Ricerca Binaria} = \left[i_{j-1} + 2^{k-1},\ i_{j-1} + 2^k \right]. \tag{11.24}$$

Esempio 11.4.1 (Traccia di Esecuzione di Doubling Search)**.** Sia A la posting list memorizzata e supponiamo di aver trovato il match precedente per $b_{j-1} = 12$ all'indice i_{j-1}. Dobbiamo ora ricercare il valore $b_j = 41$:

- Si interrogano gli indici con salti potenze di 2:
$$\begin{aligned} A[i_{j-1} + 1] &= 14 < 41, \\ A[i_{j-1} + 2] &= 16 < 41, \\ A[i_{j-1} + 4] &= 20 < 41, \\ A[i_{j-1} + 8] &= 34 < 41, \\ A[i_{j-1} + 16] &= 60 \geq 41. \end{aligned}$$

- Poiché $34 < 41 \leq 60$, l'elemento 41 può trovarsi esclusivamente all'interno del sotto-intervallo $[i_{j-1} + 8,\ i_{j-1} + 16]$.
- Si esegue una classica ricerca binaria ristretta a questa finestra locale di ampiezza $16 - 8 = 8$, localizzando 41 con pochissimi confronti.

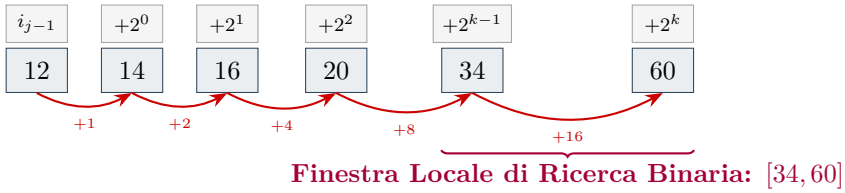


Figura 11.4: Visualizzazione della progressione dei salti esponenziali in Doubling Search (Galloping): raddoppio progressivo dell'ampiezza di salto fino a delimitare la finestra locale di ricerca binaria.

11.4.2 Derivazione Matematica Rigorosa Passo-Passo

Conduciamo l'analisi della complessità computazionale di Doubling Search articolandola formalmente in quattro passi.

Passo 1: Determinazione del Numero di Salti k. Sia i_j la posizione finale in cui si colloca l'elemento b_j in A. La distanza di avanzamento tra due corrispondenze successive è data da $(i_j - i_{j-1})$. La scansione esponenziale a potenze di 2 prosegue finché il salto corrente non oltrepassa la distanza richiesta. Dunque, l'esponente di arresto k soddisfa rigorosamente la doppia disuguaglianza:

$$2^{k-1} < i_j - i_{j-1} \leq 2^k. \tag{11.25}$$

Esaminiamo i due vincoli separatamente:

1. Dal vincolo inferiore:

$$2^{k-1} < i_j - i_{j-1} \implies k-1 < \log_2(i_j - i_{j-1}) \implies k_1 = \mathcal{O}(\log_2(i_j - i_{j-1})). \tag{11.26}$$

2. Dal vincolo superiore:

$$i_j - i_{j-1} \leq 2^k \implies 2^k \geq i_j - i_{j-1} \implies k_2 = \Omega(\log_2(i_j - i_{j-1})). \tag{11.27}$$

Pertanto, il numero totale di passi di salto compiuti per localizzare l'intervallo è strettamente proporzionale al logaritmo della distanza percorsa:

$$k = k_1 + k_2 = \Theta(\log_2(i_j - i_{j-1})). \tag{11.28}$$

Passo 2: Costo della Ricerca Binaria Locale. Una volta determinato k, si esegue la ricerca binaria nell'intervallo $[i_{j-1} + 2^{k-1},\ i_{j-1} + 2^k]$. L'ampiezza di questo intervallo è:

$$\Delta = 2^k - 2^{k-1} = 2^{k-1}(2-1) = 2^{k-1}. \tag{11.29}$$

Il costo computazionale per eseguire la ricerca binaria su un intervallo di dimensione 2^{k-1} è:

$$T_{\text{local-BS}} = \mathcal{O}(\log_2(2^{k-1})) = \mathcal{O}(k-1) = \mathcal{O}(k) = \mathcal{O}(\log_2(i_j - i_{j-1})). \tag{11.30}$$

Notiamo con grande eleganza che il costo della ricerca binaria locale coincide asintoticamente con il costo speso nella fase di salto esponenziale.

Passo 3: Costo Complessivo su Tutti gli m Elementi. Il tempo richiesto per processare il singolo elemento $b_j \in B$ è la somma del costo dei salti e del costo della ricerca binaria:

$$T_j = \mathcal{O}(k) + \mathcal{O}(k) = \mathcal{O}(\log_2(i_j - i_{j-1})). \tag{11.31}$$

Sommando il tempo impiegato su tutti gli m elementi della lista B:

$$T_{\text{totale}} = \sum_{j=1}^{m} T_j = \sum_{j=1}^{m} \mathcal{O}(\log_2(i_j - i_{j-1})) = \mathcal{O}\left(\sum_{j=1}^{m} \log_2(i_j - i_{j-1})\right). \tag{11.32}$$

Passo 4: Applicazione della Disuguaglianza di Jensen. Per massimizzare la sommatoria (11.32) e determinare la complessità nel caso peggiore, osserviamo che la funzione logaritmica $f(x) = \log_2 x$ è **strettamente concava** su tutto il suo dominio positivo $\mathbb{R}^+$. Infatti, la derivata seconda è costantemente negativa:

$$f'(x) = \frac{1}{x \ln 2}, \qquad f''(x) = -\frac{1}{x^2 \ln 2} < 0 \quad \forall x > 0. \tag{11.33}$$

Teorema 11.4.1 (Disuguaglianza di Jensen per Funzioni Concave)**.** *Data una funzione $f : \mathbb{R}^+ \to \mathbb{R}$ concava e un insieme finito di m punti $x_1, \ldots, x_m$, vale la disuguaglianza:*

$$\frac{1}{m} \sum_{j=1}^{m} f(x_j) \leq f\left(\frac{1}{m} \sum_{j=1}^{m} x_j\right). \tag{11.34}$$

Ponendo $f(x) = \log_2 x$ e considerando le distanze $x_j = i_j - i_{j-1}$:

$$\sum_{j=1}^{m} \log_2(i_j - i_{j-1}) \leq m \cdot \log_2 \left(\frac{1}{m} \sum_{j=1}^{m} (i_j - i_{j-1}) \right). \tag{11.35}$$

Valutiamo ora la sommatoria delle distanze all'interno dell'argomento del logaritmo. Poiché gli indici avanzano monotonicamente lungo la lista A, la somma forma una **serie telescopica**:

$$\sum_{j=1}^{m} (i_j - i_{j-1}) = (i_1 - i_0) + (i_2 - i_1) + \cdots + (i_m - i_{m-1}) = i_m - i_0. \tag{11.36}$$

Poiché $i_0 = 1$ e l'indice massimo raggiungibile i_m è vincolato superiormente dalla lunghezza totale n dell'array A, abbiamo:

$$\sum_{j=1}^{m} (i_j - i_{j-1}) \leq n. \tag{11.37}$$

Sostituendo la maggiorazione telescopica nella disuguaglianza di Jensen (11.35), otteniamo la stima superiore definitiva:

$$\sum_{j=1}^{m} \log_2(i_j - i_{j-1}) \leq m \log_2 \left(\frac{n}{m} \right). \tag{11.38}$$

Concludiamo quindi formalmente che:

$$T_{\text{Doubling}}(n, m) = \mathcal{O}\left(m \log_2 \frac{n}{m} \right). \tag{11.39}$$

L'algoritmo Doubling Search eguaglia asintoticamente la complessità teorica ottima di Mutual Partition, ma elimina totalmente la ricorsione ed esegue una scansione puramente in avanti (*forward scan*).

11.4.3 Limiti Pratici di Doubling Search nell'Hardware Reale

Sebbene Doubling Search risolva il problema della ricorsione, nei motori di ricerca operanti su memorie gerarchiche e dischi magnetici o SSD ad alta capacità, **non viene impiegato direttamente nella sua formulazione elementare**:

- I salti esponenziali $(+1, +2, +4, +8, \dots)$ e le successive ricerche binarie locali provocano pattern di accesso non sequenziali alla memoria fisica.

- Su dischi tradizionali, ogni salto casuale comporta un'operazione meccanica di riposizionamento della testina (*disk seek latency*), il cui costo (circa 5-10 ms) è di ordini di grandezza superiore a una scansione sequenziale di interi blocchi.
- Anche su architetture basate su RAM o memorie flash, i salti generano frequenti *cache line misses* e invalidazioni nel translation lookaside buffer (TLB).

11.5 Block Partitioning / Skip Pointers (Approccio Pratico a Blocchi)

Nel contesto delle memorie secondarie e gerarchiche, l'accesso casuale a singoli elementi penalizza severamente le prestazioni rispetto al trasferimento sequenziale a blocchi. In questa sezione presentiamo la tecnica del Block Partitioning con puntatori di salto, derivando analiticamente la dimensione ottima dei blocchi per minimizzare il costo complessivo.

11.5.1 Architettura dell'Indice a Blocchi e Skip Pointers

Per coniugare l'efficienza asintotica dei salti con la velocità massima della scansione sequenziale in memoria primaria e secondaria, i moderni motori di ricerca (tra cui Apache Lucene ed Elasticsearch) adottano la tecnica del **Block Partitioning** con **Skip Pointers** (puntatori di salto) [17, 18].

Definizione 11.5.1 (Ripartizione a Blocchi di una Posting List)**.** Sia A una posting list di dimensione n. Fissato un parametro intero $L \geq 1$ (dimensione del blocco):

1. La sequenza A viene suddivisa in $\frac{n}{L}$ blocchi contigui disgiunti, indicati con $A_1, A_2, \ldots, A_{n/L}$, ciascuno composto da esattamente L elementi (tranne eventualmente l'ultimo blocco di cardinalità $\leq L$):

$$A_k = [a_{(k-1)L+1},\ a_{(k-1)L+2},\ \ldots,\ a_{kL}]. \tag{11.40}$$

2. Si costruisce una lista ausiliaria ridotta A' di cardinalità pari a $\frac{n}{L}$, denominata **Skip List dei Massimi** (o lista delle sentinelle):

$$A' = \Big[\max(A_1),\ \max(A_2),\ \ldots,\ \max(A_{n/L})\Big] = [a_L,\ a_{2L},\ \ldots,\ a_n]\,. \tag{11.41}$$

Ciascun elemento di A' memorizza il valore massimo presente nel rispettivo blocco e un puntatore di inizio blocco (*skip pointer*).

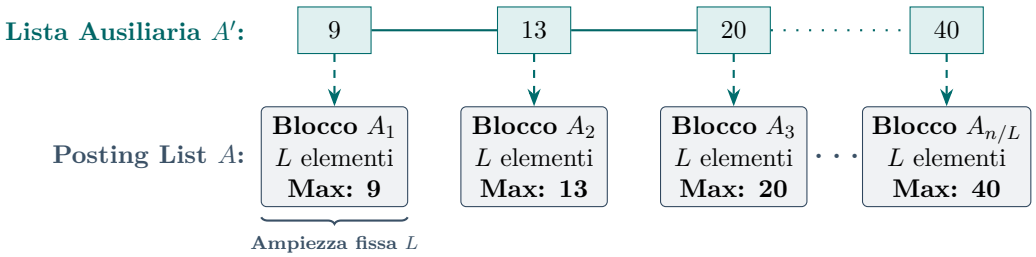


Figura 11.5: Organizzazione della struttura Block Partitioning con Skip Pointers: la lista ausiliaria A' memorizza i massimi di ciascun blocco e consente di scavalcare interi blocchi non rilevanti.

11.5.2 Procedura Algoritmica e Analisi dei Costi

L'intersezione tra la lista breve B e la lista strutturata a blocchi A si articola in due fasi rigorose:

Fase 1: Distribuzione degli Elementi di B nei Blocchi. Si confronta linearmente la lista B (di dimensione m) con la lista ausiliaria dei massimi A' (di dimensione $\frac{n}{L}$). Poiché entrambe sono ordinate, si esegue una scansione merge a due puntatori per instradare ciascun elemento $b \in B$ nel rispettivo blocco di competenza:

- L'elemento b appartiene al blocco A_k se e solo se $\max(A_{k-1}) < b \leq \max(A_k)$.
- Al termine della fusione lineare, la lista B risulta partizionata in $\frac{n}{L}$ sottoinsiemi disgiunti:

$$B = B_1 \cup B_2 \cup \cdots \cup B_{n/L}, \quad \text{con } B_k = \{b \in B \mid b \in \text{intervallo}(A_k)\}. \tag{11.42}$$

- Il costo temporale della Fase 1 è proporzionale alla somma delle dimensioni delle due sequenze scansionate:

$$T_{\text{fase 1}} = \mathcal{O}\left(m + \frac{n}{L}\right). \tag{11.43}$$

Fase 2: Calcolo delle Intersezioni Locali nei Blocchi Attivi. Si itera sopra i blocchi della lista A.

- Se per un blocco k-esimo il corrispondente sottoinsieme B_k è vuoto ($B_k = \emptyset$), il blocco A_k **non viene neppure caricato né scansionato**, compiendo uno salto effettivo (*skip*).
- Se invece $B_k \neq \emptyset$ (blocco attivo), si esegue l'intersezione lineare tra il blocco A_k (di dimensione L) e il sottoinsieme B_k (di cardinalità $|B_k|$). Il tempo richiesto per questo blocco è:

$$T_{\text{blocco } k} = \mathcal{O}(|A_k| + |B_k|) = \mathcal{O}(L + |B_k|). \tag{11.44}$$

- Quanti possono essere al massimo i blocchi attivi ($B_k \neq \emptyset$)? Poiché l'insieme B possiede in totale m elementi, ogni elemento può attivare al più un solo blocco. Di conseguenza, il numero di blocchi non vuoti è superiormente limitato da m:

$$|\{k \in \{1, \ldots, n/L\} \mid B_k \neq \emptyset\}| \leq m. \tag{11.45}$$

- Sommando il costo di scansione su tutti i blocchi attivi:

$$T_{\text{fase 2}} = \sum_{k:B_k \neq \emptyset} (L + |B_k|) = \sum_{k:B_k \neq \emptyset} L + \sum_{k:B_k \neq \emptyset} |B_k| \leq L \cdot m + m. \tag{11.46}$$

Costo Computazionale Complessivo. Sommando i costi analizzati per le due fasi, il tempo computazionale complessivo dell'algoritmo di Block Partitioning in funzione della dimensione del blocco L risulta:

$$\begin{aligned} T_{\text{totale}}(L) &= T_{\text{fase 1}} + T_{\text{fase 2}} \\ &= \mathcal{O}\left(m + \frac{n}{L}\right) + \mathcal{O}(L \cdot m + m) \\ &= \mathcal{O}\left(\frac{n}{L} + m \cdot L\right). \end{aligned} \tag{11.47}$$

Il termine $\mathcal{O}(m)$ viene assorbito poiché $m \leq m \cdot L$ per qualsiasi parametro intero $L \geq 1$.

Capitolo 12

Ricerca per Interpolazione su Bucket e Ordinamento di Stringhe

12.1 Ricerca per Interpolazione Basata su Bucket

La ricerca per interpolazione estende il principio della ricerca intuitiva impiegata nei dizionari o negli elenchi telefonici: conoscendo gli estremi numerici dei dati e assumendo una certa regolarità distribuzionale, la posizione di un valore può essere stimata proiettando direttamente la chiave nell'intervallo dell'universo anziché dividere ciecamente lo spazio a metà come avviene nella ricerca binaria standard.

12.1.1 Definizione Formale del Modello e Partizione dell'Universo

Sia dato un vettore ordinato $X[1 \dots n]$ composto da n elementi interi distinti e disposti in ordine strettamente crescente:

$$X = [x_1, x_2, \dots, x_n] \quad \text{con } x_i < x_{i+1} \quad \forall i \in \{1, \dots, n-1\},\ x_i \in \mathbb{N} \tag{12.1}$$

L'intervallo continuo discreto racchiuso tra l'elemento minimo e l'elemento massimo definisce l'**universo dei valori** $U = [x_1, x_n]$, la cui estensione complessiva (*span*) è pari a:

$$|U| = x_n - x_1 + 1 \tag{12.2}$$

Mentre la ricerca binaria standard garantisce un costo di $\mathcal{O}(\log_2 n)$ confronti operando unicamente sulla struttura degli indici, la **ricerca per interpolazione basata su bucket** sfrutta la cardinalità dell'universo numerico per raggiungere un accesso diretto in tempo $\mathcal{O}(1)$ a una ristretta regione dell'array.

A tal fine, l'universo U viene suddiviso in n partizioni contigue e disgiunte, denominate **bucket** $(B_1, B_2, \ldots, B_n)$, ciascuna avente un'ampiezza costante $b \in \mathbb{R}^+$ pari a:

$$b = \frac{x_n - x_1 + 1}{n} = \frac{|U|}{n} \tag{12.3}$$

Ciascun bucket generico B_J (per $J \in \{1, \ldots, n\}$) corrisponde all'intervallo semi-aperto a destra dei valori dell'universo:

$$B_J = [x_1 + (J-1)b,\ x_1 + Jb) \tag{12.4}$$

con l'ultimo bucket $B_n = [x_1 + (n-1)b,\ x_1 + nb) = [x_1 + (n-1)b,\ x_n + 1)$, il quale copre e racchiude superiormente il valore massimo x_n.

12.1.2 Struttura Dati Ausiliaria: Array degli Intervalli $I[J]$

Poiché i valori di X possono presentare concentrazioni irregolari o lacune, la quantità di elementi dell'array X che ricadono all'interno di ciascun bucket B_J non è a priori uniforme: alcuni bucket conterranno molteplici elementi di X, mentre altri risulteranno completamente vuoti.

Per consentire l'accesso immediato in tempo $\mathcal{O}(1)$ agli elementi di X appartenenti a ciascun bucket, si costruisce durante la fase di pre-elaborazione un array ausiliario di delimitatori $I[1 \ldots n]$:

- Per ciascun bucket **non vuoto** B_J, la cella $I[J]$ memorizza una coppia di indici interi $\langle j_{\min}, j_{\max} \rangle$ che identificano il segmento contiguo in X i cui elementi appartengono all'intervallo di B_J:

$$I[J] = \langle j_{\min}, j_{\max} \rangle \iff \forall k \in [j_{\min}, j_{\max}],\ X[k] \in B_J \tag{12.5}$$

- Se il bucket B_J è **vuoto** (nessun elemento dell'array X cade nell'intervallo $[x_1 + (J-1)b, x_1 + Jb)$), la voce viene contrassegnata con una coppia di valori sentinella:

$$I[J] = \langle -1, -1 \rangle \tag{12.6}$$

Lo spazio ausiliario richiesto dall'array I è esattamente pari a $2n$ interi, ovvero $\mathcal{O}(n)$ parole di memoria, risultando asintoticamente lineare rispetto alla dimensione del vettore originario.

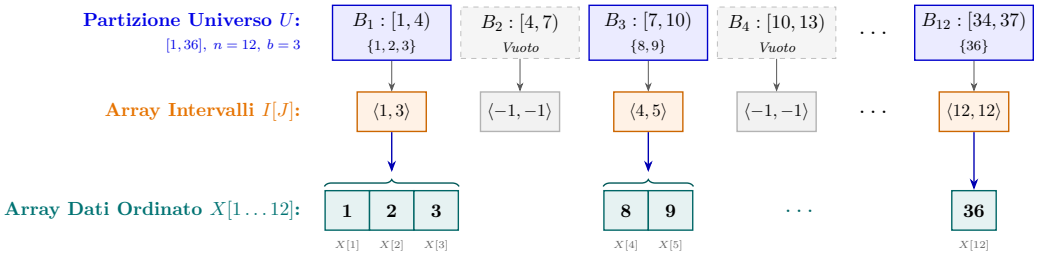


Figura 12.1: Architettura della Bucket Interpolation Search: partizione dell'universo numerico in bucket B_J di ampiezza uniforme $b = 3$, array ausiliario degli intervalli $I[J]$ con puntamento a graffa sui segmenti contigui del vettore ordinato dei dati X.

12.1.3 Esempio Applicativo Completo

Si consideri un array ordinato composto da $n = 12$ elementi interi:

Indice i:	1	2	3	4	5	6	7	8	9	10	11	12
$X[i]$**:**	**1**	**2**	**3**	**8**	**9**	**17**	**19**	**20**	**28**	**30**	**32**	**36**

Tabella 12.1: Istanza didattica del vettore ordinato X di $n = 12$ elementi con distribuzione non uniforme.

Calcolo dei Parametri Strutturali:

- Estremi: $x_1 = 1$, $x_n = x_{12} = 36$.
- Estensione dell'universo: $|U| = 36 - 1 + 1 = 36$.
- Dimensione di ciascun bucket:

$$b = \frac{x_n - x_1 + 1}{n} = \frac{36 - 1 + 1}{12} = \frac{36}{12} = 3 \tag{12.7}$$

I 12 bucket partizionano l'intervallo continuo $[1, 36]$ a intervalli regolari di ampiezza $b = 3$. La Tabella 12.2 illustra la corrispondenza completa tra intervalli, elementi contenuti e voci dell'array $I[J]$.

Bucket	**Intervallo Numerico**	**Elementi in X**	**Indici in X**	**Array $I[J]$**
B_1	$[1, 4)$	$\{1, 2, 3\}$	Da $X[1]$ a $X[3]$	$\langle 1, 3\rangle$
B_2	$[4, 7)$	$\emptyset$	Nessuno	$\langle -1, -1\rangle$
B_3	$[7, 10)$	$\{8, 9\}$	Da $X[4]$ a $X[5]$	$\langle 4, 5\rangle$
B_4	$[10, 13)$	$\emptyset$	Nessuno	$\langle -1, -1\rangle$
B_5	$[13, 16)$	$\emptyset$	Nessuno	$\langle -1, -1\rangle$
B_6	$[16, 19)$	$\{17\}$	Solo $X[6]$	$\langle 6, 6\rangle$
B_7	$[19, 22)$	$\{19, 20\}$	Da $X[7]$ a $X[8]$	$\langle 7, 8\rangle$
B_8	$[22, 25)$	$\emptyset$	Nessuno	$\langle -1, -1\rangle$
B_9	$[25, 28)$	$\emptyset$	Nessuno	$\langle -1, -1\rangle$
B_{10}	$[28, 31)$	$\{28, 30\}$	Da $X[9]$ a $X[10]$	$\langle 9, 10\rangle$
B_{11}	$[31, 34)$	$\{32\}$	Solo $X[11]$	$\langle 11, 11\rangle$
B_{12}	$[34, 37)$	$\{36\}$	Solo $X[12]$	$\langle 12, 12\rangle$

Tabella 12.2: Mappatura esaustiva dell'universo numerico per l'array didattico in esame.

12.1.4 Algoritmo di Ricerca e Tracciamento delle Operazioni

Data una chiave interrogata $y \in [x_1, x_n]$, l'algoritmo di ricerca esegue la localizzazione in due stadi distinti:

1. **Identificazione Diretta del Bucket in Tempo $\mathcal{O}(1)$:** Applicando la funzione di proiezione per interpolazione aritmetica, si determina l'indice J del bucket di competenza:

$$J = \left\lfloor \frac{y - x_1}{b} \right\rfloor + 1 \tag{12.8}$$

2. **Verifica e Localizzazione Intra-Bucket:** Si consulta la voce $I[J]$ dell'array ausiliario:

   - Se $I[J] = \langle -1, -1\rangle$, il bucket è vuoto; si conclude immediatamente che $y \notin X$ in tempo $\mathcal{O}(1)$, senza compiere alcun accesso alla memoria dell'array X.
   - Se $I[J] = \langle j_{\min}, j_{\max}\rangle \neq \langle -1, -1\rangle$, l'elemento y può trovarsi esclusivamente all'interno del segmento $X[j_{\min} \dots j_{\max}]$. Si esegue una ricerca binaria ristretta unicamente a questo intervallo locale di dimensione $|B_J| = j_{\max} - j_{\min} + 1$.

Algorithm 18: Bucket Interpolation Search

Dati: Vettore ordinato $X[1 \dots n]$, array intervalli $I[1 \dots n]$, estremo inferiore x_1, ampiezza bucket b, chiave cercata y

Risultato: Indice pos tale che $X[pos] = y$, oppure -1 se assente

```
// Verifica confini dell'universo
if y < x_1 oppure y > x_n then
    return −1

// Stadio 1: Proiezione aritmetica in tempo O(1)
J ← min(n, ⌊(y − x_1)/b⌋ + 1)

// Stadio 2: Ispezione della voce ausiliaria I[J]
if I[J] = ⟨−1, −1⟩ then
    return −1        // Terminazione immediata: bucket vuoto
⟨j_min, j_max⟩ ← I[J]
return BinarySearch(X, key = y, low = j_min, high = j_max)
```

Esempio di Traccia 1: Ricerca di Chiave Presente ($y = 9$):

1. Calcolo del bucket:

$$J = \left\lfloor \frac{9-1}{3} \right\rfloor + 1 = \left\lfloor \frac{8}{3} \right\rfloor + 1 = 2 + 1 = 3 \tag{12.9}$$

   Costo: un'operazione aritmetica elementare in tempo $\mathcal{O}(1)$.

2. Ispezione di $I[3]$: si riscontra $I[3] = \langle 4, 5 \rangle \neq \langle -1, -1 \rangle$.

3. Ricerca binaria ristretta al sotto-array $X[4 \dots 5] = [8, 9]$:

$$\text{Costo: } \mathcal{O}(\log_2 |B_3|) = \mathcal{O}(\log_2 2) = 1 \text{ confronto} \tag{12.10}$$

   L'elemento viene identificato alla posizione $X[5] = 9$.

Esempio di Traccia 2: Ricerca di Chiave Assente ($y = 5$):

1. Calcolo del bucket:

$$J = \left\lfloor \frac{5-1}{3} \right\rfloor + 1 = \left\lfloor \frac{4}{3} \right\rfloor + 1 = 1 + 1 = 2 \tag{12.11}$$

2. Ispezione di $I[2]$: si riscontra $I[2] = \langle -1, -1 \rangle$.

3. L'algoritmo termina immediatamente restituendo -1 in tempo $\mathcal{O}(1)$, senza effettuare alcuna comparazione nell'array dei dati X.

12.1.5 Analisi della Complessità Data-Aware e Parametro Δ

La complessità temporale dell'algoritmo di ricerca dipende dalla cardinalità $|B_J|$ del bucket indirizzato dalla chiave y:

$$T(n) = \mathcal{O}(1) + \mathcal{O}(\log_2 |B_J|) \quad \text{con } 0 \leq |B_J| \leq b \tag{12.12}$$

Nel caso peggiore degenere (tutti gli n elementi concentrati in un unico intervallo di ampiezza b), si avrebbe $|B_J| = n$, riconducendosi al costo della ricerca binaria standard $\mathcal{O}(\log_2 n)$. Tuttavia, sviluppando un'analisi **Data-Aware** (sensibile alla struttura intrinseca dei dati), è possibile esprimere la complessità in funzione della dispersione dei divari tra elementi contigui.

Definizione 12.1.1 (Gap tra Elementi Consecutivi)**.** Dato il vettore ordinato $X[1 \dots n]$, si definisce **gap** i-esimo la differenza scalare tra due elementi consecutivi:

$$g_i = x_i - x_{i-1} \quad \forall i \in \{2, \dots, n\} \tag{12.13}$$

Lemma 12.1.1 (Limite Inferiore per il Gap Massimo)**.** *Dato un vettore ordinato di n elementi con universo $U = [x_1, x_n]$ e ampiezza di bucket $b = \frac{x_n - x_1 + 1}{n}$, il massimo divario tra elementi adiacenti soddisfa la disuguaglianza:*

$$\max_{i=2\dots n} g_i \geq b \tag{12.14}$$

Dimostrazione. La somma di tutti gli $n-1$ divari contigui costituisce una somma telescopica la cui risultante è la dispersione complessiva dell'array:

$$\sum_{i=2}^{n} g_i = \sum_{i=2}^{n} (x_i - x_{i-1}) = x_n - x_1 \tag{12.15}$$

Il divario medio aritmetico tra elementi consecutivi è pertanto:

$$\bar{g} = \frac{\sum_{i=2}^{n} g_i}{n-1} = \frac{x_n - x_1}{n-1} \tag{12.16}$$

Poiché il valore massimo di un insieme finito di numeri reali non può mai essere inferiore alla rispettiva media aritmetica, si ha necessariamente:

$$\max_{i=2\dots n} g_i \geq \bar{g} = \frac{x_n - x_1}{n-1} \tag{12.17}$$

Osservando che $n - 1 < n$ e che $x_n - x_1 < x_n - x_1 + 1$, vale la catena di disuguaglianze:

$$\frac{x_n - x_1}{n-1} \geq \frac{x_n - x_1 + 1}{n} = b \tag{12.18}$$

Confrontando i due membri estremi si ottiene la tesi:

$$\max_{i=2\dots n} g_i \geq b \tag{12.19}$$

□

Teorema 12.1.1 (Limite Superiore alla Dimensione del Bucket e Complessità Data-Aware)**.** *Sia* Δ *il* ***rapporto di dispersione dei gap*** *(gap ratio) definito da:*

$$\Delta = \frac{\max_{i=2\dots n}(x_i - x_{i-1})}{\min_{i=2\dots n}(x_i - x_{i-1})} = \frac{\max g}{\min g} \tag{12.20}$$

La cardinalità di qualsiasi bucket B_J *è limitata superiormente da* $|B_J| \leq \Delta$*, e la complessità temporale della ricerca soddisfa:*

$$T(n) = \mathcal{O}(\log_2 \Delta) \tag{12.21}$$

Dimostrazione. Ciascun bucket B_J copre un intervallo di ampiezza b. Due elementi distinti dell'array X che ricadono all'interno del medesimo bucket B_J devono distare tra loro almeno una quantità pari al divario minimo presente nell'intera sequenza, ovvero $\min_{i=2\dots n} g_i$.

Di conseguenza, il numero massimo di elementi distinti che possono coesistere in un intervallo continuo di ampiezza b è limitato superiormente dal rapporto:

$$|B_J| \leq \frac{b}{\min_{i=2\dots n} g_i} \tag{12.22}$$

Applicando il Lemma 12.1.1, per cui $b \leq \max_{i=2\dots n} g_i$, si ottiene direttamente:

$$|B_J| \leq \frac{\max_{i=2\dots n} g_i}{\min_{i=2\dots n} g_i} = \Delta \tag{12.23}$$

Poiché la ricerca locale all'interno del bucket viene eseguita mediante ricerca binaria sul segmento di dimensione $|B_J|$, il costo temporale complessivo ammonta a:

$$T(n) = \mathcal{O}(1) + \mathcal{O}(\log_2 |B_J|) \leq \mathcal{O}(\log_2 \Delta) \tag{12.24}$$

□

Interpretazione Ingegneristica e Casi Notevoli:

- **Distribuzione Uniforme ($\Delta = \mathcal{O}(1)$):** Se le chiavi sono distanziate in modo omogeneo, il rapporto tra il divario massimo e il divario minimo è limitato da una costante. Ne consegue che ogni bucket contiene al più un numero costante di elementi ($|B_J| \leq c$), e la ricerca termina in tempo ottimo costante:

$$T(n) = \mathcal{O}(\log_2 \mathcal{O}(1)) = \mathcal{O}(1) \tag{12.25}$$

- **Resilienza Logaritmica:** All'aumentare dell'asimmetria distribuzionale, il parametro Δ cresce. Tuttavia, poiché il parametro entra sotto l'operatore logaritmo, l'impatto sul tempo di ricerca risulta fortemente smorzato: persino con un rapporto di sbilanciamento enorme $\Delta = 10^6$, sono necessari meno di 20 confronti locali.

- **Comportamento Probabilistico su Dati Casuali:** Se le n chiavi sono generate come variabili aleatorie indipendenti e identicamente distribuite (i.i.d.) secondo una distribuzione uniforme continua sull'intervallo $[x_1, x_n]$, la teoria probabilistica dell'occupancy problem dimostra che la cardinalità massima di un bucket soddisfa:

$$|B_J| = \mathcal{O}\left(\log_2^2 n\right) \quad \text{con alta probabilità (w.h.p.)} \tag{12.26}$$

  Sostituendo tale valore nell'espressione della complessità locale, si ottiene un costo atteso eccezionalmente basso:

$$T(n) = \mathcal{O}\left(\log_2\left(\log_2^2 n\right)\right) = \mathcal{O}\left(2\log_2\log_2 n\right) = \mathcal{O}(\log_2\log_2 n) \tag{12.27}$$

  risultato in perfetto accordo con la teoria classica della ricerca per interpolazione di Perl, Itai e Avni [19].

12.2 Ordinamento di Stringhe e Analisi del Limite Inferiore

Quando le chiavi da ordinare non sono scalari compatti ma sequenze di caratteri a lunghezza variabile, il costo del confronto tra elementi cessa di essere unitario ($\mathcal{O}(1)$). In questa sezione formalizziamo il modello computazionale per le stringhe, evidenziamo i colli di bottiglia del Quicksort convenzionale e deriviamo il limite inferiore teorico del problema.

12.2.1 Modello di Calcolo e Notazione

Sia dato un insieme S composto da n stringhe definite sopra un alfabeto finito e totalmente ordinato Σ:

$$S = \{s_1, s_2, \ldots, s_n\} \tag{12.28}$$

I parametri dimensionali che governano il problema sono:

- $n = |S|$: numero totale di stringhe appartenenti all'insieme.

- $N = \sum_{j=1}^{n} |s_j|$: lunghezza cumulativa (numero totale di caratteri) di tutte le stringhe in S.
- $\sigma = |\Sigma|$: dimensione (cardinalità) dell'alfabeto dei simboli.
- $\bar{L} = \frac{N}{n}$: lunghezza media delle stringhe nella collezione.

Nel modello di memoria reale, l'insieme S è rappresentato mediante un **array di puntatori** $S[1 \dots n]$. Ciascuna cella $S[k]$ memorizza un indirizzo di memoria (a 64 bit) che punta all'area contigua nell'heap dove risiedono i caratteri della stringa s_k, terminata dal carattere nullo speciale (`'\0'`), come illustrato in Figura 12.2.

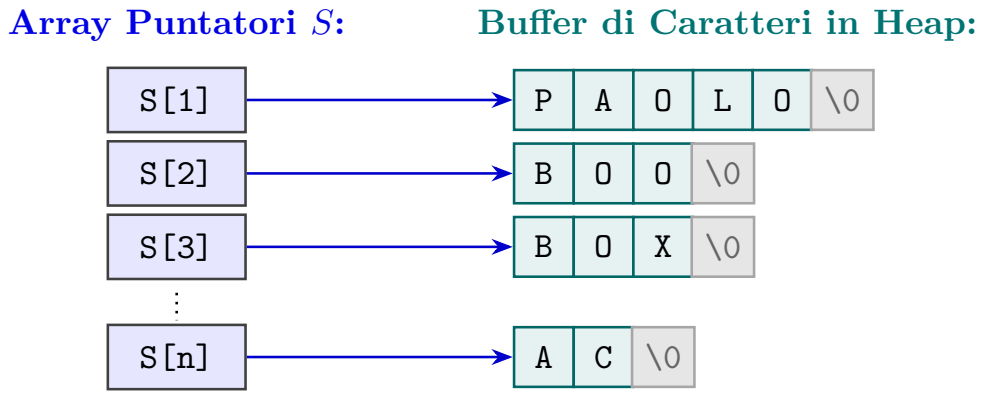


Figura 12.2: Rappresentazione standard in memoria per collezioni di stringhe: array di puntatori contiguo e buffer di caratteri a lunghezza variabile sparsi nell'heap.

12.2.2 Limiti del Quicksort Classico su Stringhe

Un approccio ingenuo al problema consiste nell'invocare l'algoritmo Quicksort tradizionale sull'array di puntatori S, delegando il confronto d'ordine a una funzione di comparazione lessicografica a due vie (quale `strcmp` in linguaggio C):

$$\text{qSORT}(S,\ 1,\ n,\ \text{cmp} = \texttt{strcmp}) \tag{12.29}$$

Questa architettura manifesta due gravi inefficienze strutturali:

1. **Costo Computazionale Super-Lineare dei Confronti:** Il Quicksort standard esegue $\mathcal{O}(n \log_2 n)$ confronti tra elementi. Mentre per i tipi primitivi ciascuna comparazione costa $\mathcal{O}(1)$ cicli macchina, la funzione `strcmp` tra due stringhe s_a, s_b deve scorrere i caratteri in avanti partendo

dalla prima posizione sino a incontrare il primo simbolo disuguale. Nel caso medio, il confronto richiede un numero di operazioni proporzionale alla lunghezza del prefisso comune, che può estendersi fino all'intera lunghezza media $\bar{L} = \frac{N}{n}$.

Moltiplicando il numero di confronti per il costo medio di ciascuna comparazione si ottiene:

$$T(n, N) = \mathcal{O}(n \log_2 n) \cdot \mathcal{O}\left(\frac{N}{n}\right) = \mathcal{O}(N \log_2 n) \tag{12.30}$$

Tale costo è intrinsecamente ridondante: se un gruppo di stringhe condivide un lungo prefisso comune (ad esempio 50 caratteri identici), tali primi 50 caratteri vengono ri-confrontati da capo a ogni singolo livello di profondità dell'albero di ricorsione del Quicksort.

2. **Assenza di Località di Riferimento (*Poor Cache Locality*):** Sebbene l'array dei puntatori S sia allocato in un blocco contiguo di memoria, i buffer contenenti i caratteri delle stringhe risiedono in regioni arbitrarie dell'heap. A ogni confronto eseguito da `strcmp`, la CPU deve dereferenziare due puntatori distanti, generando continui salti casuali nella memoria virtuale. Ciò causa un numero elevatissimo di **cache miss** sia nella gerarchia L1/L2/L3 sia nel TLB (*Translation Lookaside Buffer*), degradando pesantemente il throughput effettivo del processore.

12.2.3 Analisi del Limite Inferiore: Lower Bound per lo String Sorting

Per determinare l'efficienza teorica di qualsiasi algoritmo di ordinamento basato su confronti di singoli caratteri (*atomic character comparisons*), decomponiamo l'attività computazionale necessaria in due componenti indipendenti e non sovrapposte.

Componente 1: Ordinamento Combinatorio delle Permutazioni. Anche nell'ipotesi ideale in cui tutte le n stringhe dell'insieme S possiedano caratteri iniziali reciprocamente distinti (distinguibili al passo 1), l'algoritmo deve comunque determinare l'unica permutazione ordinata tra le $n!$ possibili disposizioni dell'insieme.

In base al classico modello dell'albero di decisione basato su confronti binari:

$$\text{Altezza dell'albero} \geq \log_2(n!) = \Omega(n \log_2 n) \tag{12.31}$$

Sono quindi strettamente necessari almeno $\Omega(n \log_2 n)$ confronti atomici tra caratteri.

Componente 2: Ispezione dei Prefissi Distinguibili (d_s) e LCP. In presenza di stringhe che condividono prefissi identici, non è sufficiente ordinare le stringhe come entità astratte: l'algoritmo deve necessariamente scendere lungo i caratteri condivisi per identificare il primo simbolo discriminante.

Definizione 12.2.1 (Longest Common Prefix — LCP)**.** Date due stringhe $s_a, s_b \in \Sigma^*$, si definisce $\mathrm{LCP}(s_a, s_b)$ la lunghezza del più lungo prefisso comune identico condiviso da entrambe:

$$\mathrm{LCP}(s_a, s_b) = \max\{\ell \geq 0 \mid s_a[1 \ldots \ell] = s_b[1 \ldots \ell]\} \tag{12.32}$$

Definizione 12.2.2 (Prefisso Distinguibile d_s)**.** Data una stringa $s \in S$ e considerato l'ordinamento lessicografico globale di S, si definisce **prefisso distinguibile** d_s il numero minimo di caratteri di s che qualsiasi algoritmo corretto deve obbligatoriamente esaminare per distinguerla in modo univoco sia dal suo predecessore immediato ($\mathrm{pred}(s)$) sia dal suo successore immediato ($\mathrm{succ}(s)$) nella sequenza ordinata:

$$d_s = 1 + \max\left(\mathrm{LCP}(s,\ \mathrm{pred}(s)),\ \mathrm{LCP}(s,\ \mathrm{succ}(s))\right) \tag{12.33}$$

ponendo per convenzione $\mathrm{LCP}(s_1, \mathrm{pred}(s_1)) = 0$ per la prima stringa e $\mathrm{LCP}(s_n, \mathrm{succ}(s_n)) = 0$ per l'ultima.

Si definisce quindi la somma dei prefissi distinguibili su tutte le n stringhe dell'insieme:

$$d = \sum_{s \in S} d_s \tag{12.34}$$

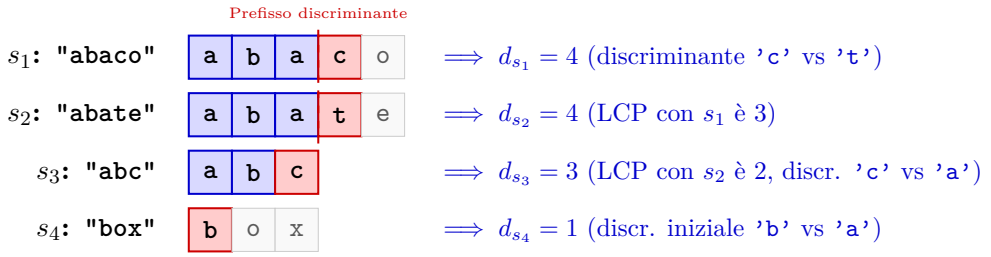


Figura 12.3: Visualizzazione dei prefissi distinguibili d_s su stringhe ordinate lessicograficamente: le celle azzurre rappresentano i caratteri condivisi (LCP), le celle rosse denotano il carattere discriminante obbligatorio (d_s), mentre le celle grigie non necessitano di ispezione.

Nessun algoritmo di ordinamento può esimersi dall'ispezionare ciascuno dei d_s caratteri per ogni stringa $s \in S$: qualora un carattere compreso nel prefisso

distinguibile non venisse letto, l'ordinamento relativo tra s e i suoi vicini lessicografici rimarrebbe indecidibile. Ne consegue che sono strettamente necessarie almeno $\Omega(d)$ operazioni atomiche di lettura e confronto.

Teorema 12.2.1 (Limite Inferiore per lo String Sorting)**.** *Dato un insieme di n stringhe S con somma dei prefissi distinguibili $d = \sum_{s \in S} d_s$, la complessità di qualsiasi algoritmo di ordinamento basato su confronti atomici tra caratteri soddisfa il limite inferiore:*

$$\Omega(d + n \log_2 n) \quad \textit{confronti di caratteri} \tag{12.35}$$

Dimostrazione. La dimostrazione discende direttamente dall'indipendenza delle due componenti necessarie:

1. Per distinguere ogni stringa dai suoi vicini immediati, l'algoritmo deve ispezionare tutti i caratteri del prefisso distinguibile, imponendo un numero di letture pari ad almeno $\Omega(d)$.

2. Per identificare la corretta permutazione tra le n stringhe indipendentemente dalla lunghezza dei caratteri discriminanti, l'albero di decisione impone almeno $\Omega(\log_2(n!)) = \Omega(n \log_2 n)$ diramazioni binarie.

Poiché le due richieste computazionali devono essere soddisfatte contemporaneamente, la complessità complessiva è dominata dal termine congiunto $\Omega(d + n \log_2 n)$. □

12.3 Multi-Key Quicksort (3-Way Radix Sort)

L'algoritmo **Multi-Key Quicksort** (noto anche come algoritmo di Bentley-Sedgewick o 3-Way Radix Quicksort [20]) risolve in modo elegante le inefficienze del Quicksort classico combinando il partizionamento a tre vie di Dijkstra per chiavi duplicate con la scansione progressiva per singoli caratteri tipica del Radix Sort.

12.3.1 Principio di Funzionamento e Invariante Fondamentale

L'algoritmo opera ricorsivamente su una coppia (R, i):

- $R \subseteq S$: sottoinsieme corrente di stringhe da ordinare.
- $i \in \mathbb{N}^+$: indice posizionale (1-based) del carattere corrente da confrontare.

Invariante Fondamentale del Multi-Key Quicksort

All'atto della chiamata ricorsiva MKQ(R, i), tutte le stringhe appartenenti al sottoinsieme R condividono un prefisso comune identico di lunghezza esattamente pari a $i-1$ caratteri:

$$\forall s_a, s_b \in R, \quad s_a[1 \dots i-1] = s_b[1 \dots i-1] \tag{12.36}$$

Questo invariante garantisce che i primi $i-1$ caratteri non debbano **mai più essere ri-confrontati**, abbattendo alla radice la ridondanza di `strcmp`.

12.3.2 Formalizzazione dell'Algoritmo ed Esecuzione Ricorsiva

Dato il sottoinsieme di stringhe R e l'indice i:

1. **Caso Base:** Se $|R| \leq 1$, l'insieme è banalmente ordinato: restituisci R.
2. **Selezione del Pivot:** Si sceglie una stringa di pivot $p \in R$ (ad esempio casualmente, o mediante la mediana di un piccolo campione) e si estrae il suo i-esimo carattere $p[i]$.
3. **Partizionamento a Tre Vie (3-Way Partitioning):** L'insieme R viene partizionato rispetto al carattere $p[i]$ in tre sottoinsiemi disgiunti:

$$R_< = \{s \in R \mid s[i] < p[i]\} \tag{12.37}$$
$$R_= = \{s \in R \mid s[i] = p[i]\} \tag{12.38}$$
$$R_> = \{s \in R \mid s[i] > p[i]\} \tag{12.39}$$

4. **Chiamate Ricorsive e Regola di Avanzamento dell'Indice i:**
   - $A = \text{MKQ}(R_<,\ i)$: le stringhe in $R_<$ hanno $s[i] < p[i]$, ma possono differire tra loro alla posizione i. L'indice i **rimane invariato**.
   - $B = \text{MKQ}(R_=,\ i+1)$: tutte le stringhe in $R_=$ possiedono esattamente lo stesso carattere $p[i]$ alla posizione i. Avendo validato la concordanza su tale simbolo, l'invariante si estende a i caratteri e l'indice avanza: $i \leftarrow i+1$.
   - $C = \text{MKQ}(R_>,\ i)$: le stringhe in $R_>$ hanno $s[i] > p[i]$; l'indice i **rimane invariato**.
5. **Composizione del Risultato:** Si restituisce la concatenazione ordinata:

$$\text{MKQ}(R, i) = A \cdot B \cdot C \tag{12.40}$$

Algorithm 19: Multi-Key Quicksort ($\mathrm{MKQ}(R, i)$)

Dati: Insieme di stringhe $R \subseteq S$, indice del carattere corrente i
Risultato: Insieme R ordinato lessicograficamente

if $|R| \leq 1$ **then**
  return R

`// Selezione del pivot e del carattere discriminante`
Scegli una stringa pivot $p \in R$
$c \leftarrow p[i]$

`// Partizionamento ternario in-place`
$R_< \leftarrow \{s \in R \mid s[i] < c\}$
$R_= \leftarrow \{s \in R \mid s[i] = c\}$
$R_> \leftarrow \{s \in R \mid s[i] > c\}$

`// Rami ricorsivi: solo il ramo centrale avanza l'indice` i
$A \leftarrow \mathrm{MKQ}(R_<,\ i)$
$B \leftarrow \mathrm{MKQ}(R_=,\ i+1)$
$C \leftarrow \mathrm{MKQ}(R_>,\ i)$
return $A \cdot B \cdot C$

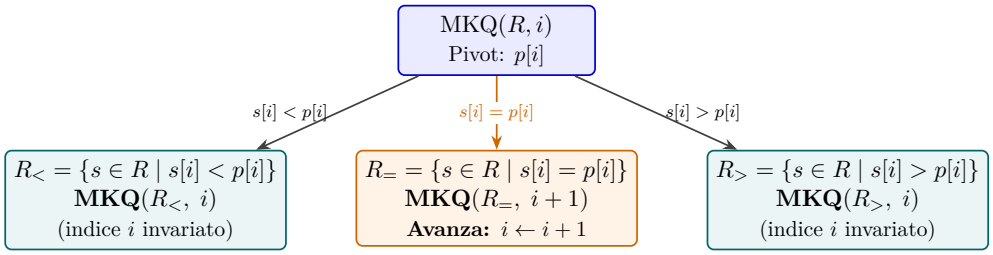


Figura 12.4: Diagramma di partizionamento ternario del Multi-Key Quicksort: le partizioni laterali riducono la dimensione del set mantenendo fermo l'indice i, mentre la partizione centrale convalida il carattere ed estende l'invariante incrementando i.

12.3.3 Tracciamento Numerico e Analitico dell'Esempio Didattico

Si consideri la chiamata ricorsiva $\mathrm{MKQ}(R, 3)$ sul seguente insieme R composto da $n = 7$ stringhe, le quali condividono già il prefisso di 2 caratteri `"ab"` (da

cui $i = 3$):

$$R = \{\text{"abaco"},\ \text{"abate"},\ \text{"abc"},\ \text{"abba"},\ \text{"abbins"},\ \text{"abi"},\ \text{"abnormal"}\} \tag{12.41}$$

Passo 1: Selezione del Pivot e Carattere Discriminante. Viene selezionata come pivot la stringa:

$$p = \text{"abi"} \implies p[3] = \text{'i'} \tag{12.42}$$

Passo 2: Ispezione del Carattere in Posizione $i = 3$. La Tabella 12.3 riporta l'analisi dettagliata per ciascuna stringa di R rispetto al carattere pivot $p[3] = \text{'i'}$.

Stringa s	**Carattere** $s[3]$	**Confronto con** $p[3] =$ **'i'**	**Partizione**	**Chiamata Ricorsiva**
`"abaco"`	`'a'`	`'a' < 'i'`	$s \in R_<$	MKQ($R_<, 3$)
`"abate"`	`'a'`	`'a' < 'i'`	$s \in R_<$	MKQ($R_<, 3$)
`"abc"`	`'c'`	`'c' < 'i'`	$s \in R_<$	MKQ($R_<, 3$)
`"abba"`	`'b'`	`'b' < 'i'`	$s \in R_<$	MKQ($R_<, 3$)
`"abbins"`	`'b'`	`'b' < 'i'`	$s \in R_<$	MKQ($R_<, 3$)
`"abi"` (pivot)	`'i'`	`'i' == 'i'`	$s \in R_=$	MKQ($R_=, 4$)
`"abnormal"`	`'n'`	`'n' > 'i'`	$s \in R_>$	MKQ($R_>, 3$)

Tabella 12.3: Tracciamento analitico del partizionamento a tre vie per la chiamata MKQ($R, 3$).

Passo 3: Composizione dei Rami Ricorsivi: Al termine della scansione ternaria, l'insieme R risulta decomposto in tre sotto-problemi indipendenti:

- $R_< = \{\text{"abaco"}, \text{"abate"}, \text{"abc"}, \text{"abba"}, \text{"abbins"}\}$ con cardinalità $|R_<| = 5 < |R| = 7$. Si invoca la chiamata ricorsiva MKQ($R_<, 3$): la dimensione del problema si è contratta, consentendo di ordinare le stringhe sul terzo carattere.
- $R_= = \{\text{"abi"}\}$ con cardinalità $|R_=| = 1$. Poiché il carattere 3 è identico per tutti i membri del gruppo, la ricorsione avanza a $i = 4$ con MKQ($R_=, 4$), raggiungendo immediatamente il caso base $|R| \leq 1$.
- $R_> = \{\text{"abnormal"}\}$ con cardinalità $|R_>| = 1$. Essendo un singleton, il caso base restituisce immediatamente la stringa senza ulteriori elaborazioni.

12.3.4 Dimostrazione Rigorosa della Complessità mediante Token Accounting

Per analizzare il tempo di esecuzione complessivo dell'algoritmo MKQ, adottiamo la tecnica formale di **contabilità per stringa** (*amortized token accounting*): quantifichiamo il numero massimo di confronti a cui una singola stringa $s \in S$ può essere sottoposta durante l'intera esecuzione dell'algoritmo, per poi sommare i contributi su tutte le n stringhe dell'insieme.

Teorema 12.3.1 (Complessità Ottima del Multi-Key Quicksort)**.** *Dato un insieme di n stringhe S con somma dei prefissi distinguibili $d = \sum_{s \in S} d_s$, l'algoritmo Multi-Key Quicksort ordina l'insieme in tempo medio:*

$$T(n, d) = \Theta(d + n \log_2 n) \tag{12.43}$$

raggiungendo esattamente il limite inferiore teorico per lo String Sorting.

Dimostrazione. Si consideri il percorso di una generica stringa $s \in S$ lungo l'albero delle chiamate ricorsive di MKQ(R, i). Ad ogni nodo in cui s partecipa a un partizionamento ternario, si verifica mutuamente uno e uno solo dei due seguenti eventi:

Caso 1: La stringa s cade in $R_<$ oppure in $R_>$. In questo scenario, la stringa s possiede un carattere $s[i] \neq p[i]$ e viene smistata in una partizione strettamente più piccola dell'insieme corrente:

$$|R_<| < |R| \quad \text{oppure} \quad |R_>| < |R| \tag{12.44}$$

Assumendo una selezione bilanciata del pivot (garantita ad esempio scegliendo il pivot in modo pseudo-casuale o calcolando la mediana di tre elementi), le partizioni laterali si contraggono in media di un fattore costante:

$$\mathbb{E}[|R_<|],\ \mathbb{E}[|R_>|] \leq \alpha |R| \quad \text{con } 0 < \alpha < 1 \tag{12.45}$$

Poiché a ogni occorrenza di questo evento la cardinalità del sottoinsieme si riduce geometricamente, la stringa s può partecipare a partizioni di tipo "inferiore/superiore" al più un numero di volte proporzionale alla profondità media di un albero di Quicksort bilanciato:

$$\text{Numero di eventi } (s \in R_< \cup R_>) = \mathcal{O}(\log_2 n) \tag{12.46}$$

Caso 2: La stringa s cade in $R_=$. In questo scenario, il carattere $s[i]$ coincide con il carattere pivot $p[i]$. Di conseguenza, l'invariante si estende e l'indice del carattere da confrontare viene incrementato:

$$i \leftarrow i + 1 \tag{12.47}$$

L'indice i per la specifica stringa s parte da 1 e può avanzare al massimo fino a raggiungere la lunghezza del prefisso distinguibile d_s. Infatti, per definizione di d_s, non appena $i = d_s$ la stringa s si differenzia univocamente da qualsiasi altra stringa dell'insieme; essa isolerà se stessa in un sottoinsieme di cardinalità $|R| = 1$, innescando il caso base di arresto.

Ne consegue che l'evento $s \in R_=$ può verificarsi al massimo d_s volte:

$$\text{Numero di eventi } (s \in R_=) \leq d_s \tag{12.48}$$

Costo Globale Cumulativo: Sommando i due contributi indipendenti per la singola stringa s, il numero totale di operazioni a cui s partecipa durante l'intero algoritmo è:

$$\text{Costo}(s) = d_s + \mathcal{O}(\log_2 n) \tag{12.49}$$

Sommando tale quantità su tutte le n stringhe appartenenti all'insieme S:

$$\begin{aligned} T(n, d) &= \sum_{s \in S} \text{Costo}(s) \\ &= \sum_{s \in S} (d_s + \mathcal{O}(\log_2 n)) \\ &= \sum_{s \in S} d_s + \sum_{s \in S} \mathcal{O}(\log_2 n) \\ &= d + \mathcal{O}(n \log_2 n) \end{aligned} \tag{12.50}$$

Poiché per il Teorema 12.2.1 qualsiasi algoritmo basato su confronti atomici richiede almeno $\Omega(d + n \log_2 n)$, la complessità del Multi-Key Quicksort è asintoticamente ottima:

$$T(n, d) = \Theta(d + n \log_2 n) \tag{12.51}$$

□

12.3.5 Quadro di Sintesi Comparativa

La Tabella 12.4 sintetizza le caratteristiche computazionali, architetturali e di memoria dei tre paradigmi fondamentali per l'ordinamento di stringhe.

Proprietà	Quicksort Standard su Stringhe	Radix Sort (MSD) Classico	Multi-Key Quicksort (3-Way)
Complessità Temporale	$\mathcal{O}(N \log_2 n)$ (ridondanza su prefissi)	$\mathcal{O}(N + \sigma \cdot \text{profondità})$ (costoso per grandi alfabeti)	$\mathbf{\Theta(d + n \log_2 n)}$ **(Ottima al Lower Bound)**
Località della Memoria	Scarsa: continui accessi casuali all'heap	Scarsa: richiede array ausiliari di conteggio per bucket	**Elevata: scansione sequenziale** in-place sull'array puntatori
Consumo di Spazio	$\mathcal{O}(\log_2 n)$ stack frames	$\mathcal{O}(n+\sigma)$ ausiliario per ciascun livello ricorsivo	$\mathcal{O}(\log_2 \mathbf{n})$ **stack frames** (in-place)
Sensibilità all'Alfabeto	Indipendente dalla dimensione σ	Fortemente penalizzato da σ grande (es. Unicode a 16/32 bit)	**Completamente indipendente da** σ (partiziona a 3 vie sul carattere)
Gestione Prefissi Lunghi	Pessima: riesamina i caratteri comuni ad ogni livello	Ottima: avanza sui singoli caratteri	**Ottima: avanza l'indice** i **solo nel ramo centrale**

Tabella 12.4: Quadro sinottico comparativo tra gli algoritmi cardine per l'ordinamento di stringhe.

Perché il Multi-Key Quicksort Vince nella Pratica Ingegneristica?

Il Multi-Key Quicksort combina i punti di forza del Quicksort e del Radix Sort, eliminandone i rispettivi difetti critici:

1. Rispetto al **Quicksort standard**, non ricomincia mai la scansione dei prefissi comuni dall'inizio: non appena un carattere è validato nel ramo centrale ($R_=$), l'indice avanza definitivamente a $i + 1$.

2. Rispetto al **Radix Sort MSD**, non alloca array di bucket di ampiezza σ. Quando l'alfabeto è vasto (come l'UTF-8 o l'UTF-16), il Radix Sort convenzionale alloca decine di migliaia di celle quasi sempre vuote ad ogni livello di ricorsione; il Multi-Key Quicksort esegue invece un partizionamento ternario compatto che si adatta dinamicamente a qualunque alfabeto senza alcuno spreco di memoria.

Capitolo 13

Strutture Dati e Algoritmi per Stringhe: TST e Radix Sort

13.1 Introduzione e Modello Computazionale per Stringhe

Nel modello computazionale convenzionale adottato per l'analisi degli algoritmi di ordinamento e ricerca su tipi atomici (come interi a precisione prefissata a 32 o 64 bit o valori in virgola mobile), l'operazione elementare di confronto tra due elementi viene considerata atomica e caratterizzata da un costo unitario $\mathcal{O}(1)$. Quando si estende l'orizzonte applicativo alla manipolazione di collezioni di **stringhe** o chiavi a lunghezza arbitraria, tale assunzione teorica decade completamente, rendendo necessario un modello computazionale dedicato.

Sia dato un insieme S costituito da n stringhe definite su un alfabeto finito ed ordinato Σ di cardinalità finita $\sigma = |\Sigma|$:

$$S = \{s_1, s_2, \ldots, s_n\}, \quad s_j \in \Sigma^* \quad \forall j \in \{1, \ldots, n\} \tag{13.1}$$

Il confronto lessicografico tra due stringhe arbitrarie $s_1, s_2 \in S$ non può avvenire in tempo unitario: esso richiede la scansione carattere per carattere a partire dalla posizione iniziale, arrestandosi al primo indice in cui i caratteri differiscono oppure al termine della stringa più corta. Tale costo è intrinsecamente proporzionale alla lunghezza del più lungo prefisso comune (*Longest Common Prefix*, LCP):

$$\text{Costo}(s_1, s_2) = \mathcal{O}(\text{LCP}(s_1, s_2) + 1) \leq \mathcal{O}(\min(|s_1|, |s_2|)) \tag{13.2}$$

Se si gestissero insiemi di stringhe mediante algoritmi generici di ordinamento basati su comparazioni globali (quali ad esempio il *MergeSort* o il *Quicksort*

convenzionale) applicati a n stringhe di lunghezza uniforme o massima pari a L, nel caso peggiore il costo computazionale risulterebbe:

$$T_{\text{generico}}(n, L) = \mathcal{O}(L \cdot n \log_2 n) \tag{13.3}$$

Tale limite riflette la ridondanza tipica degli approcci a confronto completo: le coppie di stringhe che condividono prefissi identici vengono confrontate ripetutamente dall'inizio ad ogni passo ricorsivo, riesaminando caratteri già validati e dissipando risorse di calcolo.

Per superare queste inefficienze, l'Algorithm Engineering introduce strutture dati e algoritmi specializzati che operano **carattere per carattere** o su blocchi discreti di bit:

1. **Ternary Search Trees (TST)** [20]: strutture ad albero che combinano l'efficienza temporale dei Trie con il limitato consumo spaziale degli alberi binari di ricerca (BST). La loro architettura è strettamente interconnessa all'algoritmo di partizionamento ternario *Multi-Key Quicksort* (o *3-Way Radix Quicksort*) introdotto da Jon Bentley e Robert Sedgewick [20].

2. **Radix Sort** [21, 22]: una classe di algoritmi di ordinamento non comparativi che distribuiscono le stringhe esaminandone le singole componenti elementari (cifre decimali, caratteri ASCII o byte), declinati nelle varianti **LSD** (*Least Significant Digit*) e **MSD** (*Most Significant Digit*).

13.2 Ternary Search Trees: Modello Teorico e Struttura del Nodo

I **Ternary Search Trees (TST)**, formulati da Bentley e Sedgewick [20], nascono per risolvere il fondamentale compromesso fra consumo spaziale e tempo di risposta nei dizionari dinamici per stringhe:

- **Trie Standard (ad arità σ):** garantisce ricerche in tempo ottimo $\mathcal{O}(|s|)$, indipendente dal numero di chiavi n. Tuttavia, ciascun nodo interno deve allocare un vettore di puntatori di dimensione $\sigma = |\Sigma|$. Quando l'alfabeto è vasto (come l'ASCII esteso con $\sigma = 256$ o lo standard Unicode) e l'albero è poco denso, la stragrande maggioranza dei puntatori rimane nulla, causando un'enorme dispersione di memoria e compromettendo la località dei riferimenti nella cache.

- **Binary Search Tree (BST) convenzionale:** memorizza intere stringhe all'interno di ciascun nodo, mantenendo l'occupazione spaziale limitata a $\mathcal{O}(n)$. Nondimeno, a ogni biforcazione dell'albero viene eseguito un confronto lessicografico completo fra l'intera chiave cercata e la stringa memorizzata nel nodo, comportando una riesecuzione ridondante dei test sui caratteri del prefisso comune.

13.2.1 Definizione Formale del Nodo di un TST

Un Ternary Search Tree organizza i caratteri delle chiavi in una gerarchia ternaria. Ciascun nodo non rappresenta una stringa intera, bensì un singolo carattere discriminante associato a uno specifico indice di scansione.

Definizione 13.2.1 (Nodo di un Ternary Search Tree)**.** Un nodo u di un TST è definito formalmente dalla tupla:

$$u = \langle X,\ i,\ \text{left}[u],\ \text{mid}[u],\ \text{right}[u] \rangle \tag{13.4}$$

dove:

- $X = \text{char}(u) \in \Sigma$ è il **carattere discriminante** associato al nodo.
- $i \in \{1, 2, \ldots, |s|\}$ è l'**indice di ispezione**, che indica quale carattere della stringa in ingresso viene confrontato con X nel nodo corrente.
- left$[u]$ (**Ramo Minore**, $<$): punta al sottoalbero contenente le chiavi il cui i-esimo carattere è strettamente minore di X nell'ordine dell'alfabeto (chiave$[i] < X$). Se la navigazione imbocca questo ramo, **l'indice** i **rimane invariato**: la ricerca proseguirà confrontando la medesima posizione i con il carattere del nodo figlio.
- mid$[u]$ (**Ramo Uguale**, $=$): punta al sottoalbero contenente le chiavi il cui i-esimo carattere coincide esattamente con X (chiave$[i] == X$). Poiché il carattere corrente è stato validato con successo, **l'indice avanza di una posizione**:

$$i \leftarrow i + 1 \tag{13.5}$$

- right$[u]$ (**Ramo Maggiore**, $>$): punta al sottoalbero contenente le chiavi il cui i-esimo carattere è strettamente maggiore di X (chiave$[i] > X$). Analogamente al ramo sinistro, **l'indice** i **rimane invariato**.

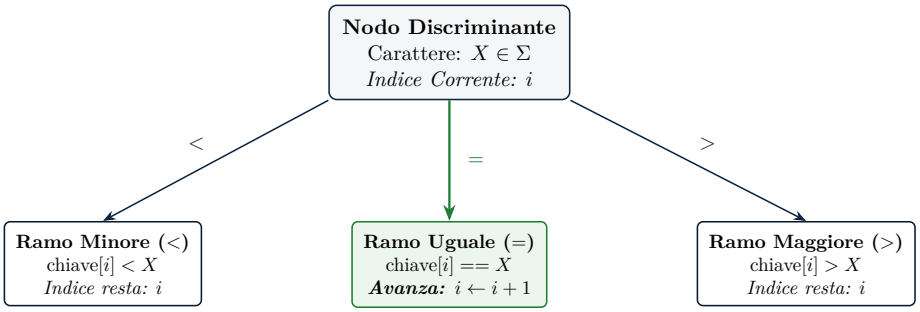


Figura 13.1: Rappresentazione concettuale del nodo elementare di un TST e della regola fondamentale di avanzamento dell'indice: la transizione lungo il ramo centrale (=) valida il carattere corrente e incrementa l'indice di scansione ($i \leftarrow i + 1$), mentre le transizioni laterali ($<$ e $>$) mantengono inalterato l'indice i.

13.2.2 Gestione dei Terminatori di Stringa e Sentinelle

All'interno di un dizionario per stringhe, può verificarsi la condizione in cui una chiave costituisca prefisso proprio di un'altra chiave già inserita nell'albero (si consideri ad esempio la coppia di parole "CAT" e "CAST"). Se l'albero si limitasse a memorizzare i soli caratteri della parola, la chiave più corta verrebbe completamente oscurata dal percorso della chiave più estesa.

Per garantire la separabilità e l'univocità di ciascuna chiave, si adotta una convenzione basata su **caratteri sentinella**:

- Si appende al termine di ciascuna stringa un carattere speciale convenzionale di fine stringa, denotato con il simbolo \$ oppure con il byte nullo `\0`.
- Il terminatore possiede un valore convenzionale strettamente inferiore a qualsiasi carattere alfanumerico o simbolico dell'alfabeto:

$$\$ < a \quad \forall a \in \Sigma \tag{13.6}$$

- In questo schema, la condizione di terminazione e corrispondenza esatta coincide formalmente con il transito attraverso un collegamento centrale (=) che valida il carattere sentinella \$.
- Negli appunti si fa inoltre riferimento all'impiego coordinato di sentinelle inferiori e superiori (quali \$ e #) per delimitare gli estremi lessicografici dell'universo delle chiavi, garantendo che qualsiasi ricerca per intervallo o ricerca di predecessore e successore operi su domini compatti e ben definiti.

13.3 Costruzione del TST e Partizionamento a 3 Vie

La topologia di un TST riflette in modo diretto l'albero delle chiamate ricorsive generato dall'algoritmo di **Multi-Key Quicksort** (o partizionamento ternario di Bentley-Sedgewick) applicato sull'insieme delle stringhe [20].

Dato un insieme corrente di stringhe $R \subseteq S$ e un indice di carattere attivo i:

1. Si seleziona una stringa pivot $P \in R$, estraendone il carattere discriminante $X = P[i]$.
2. L'insieme R viene partizionato in tre sottoinsiemi mutualmente disgiunti:

$$R_< = \{s \in R \mid s[i] < X\} \tag{13.7}$$
$$R_= = \{s \in R \mid s[i] = X\} \tag{13.8}$$
$$R_> = \{s \in R \mid s[i] > X\} \tag{13.9}$$

3. Si applica la ricorsione sui tre rami:
   - Sul sottoinsieme $R_<$, mantenendo l'indice invariato (i).
   - Sul sottoinsieme $R_=$, avanzando l'indice al carattere successivo ($i + 1$).
   - Sul sottoinsieme $R_>$, mantenendo l'indice invariato (i).

13.3.1 Tracciamento Analitico dell'Esempio Didattico

Si consideri l'insieme di $n = 5$ stringhe fornito negli appunti:

$$S = \{\text{"CAT"}, \text{"AB"}, \text{"CAST"}, \text{"CAR"}, \text{"AT"}\} \tag{13.10}$$

Passo 1: Radice Globale (Livello 1)

- **Indice di partenza:** $i = 1$.
- **Selezione del Pivot:** si seleziona come pivot la stringa $P =$ "CAST", per cui il carattere discriminante alla radice è $X = P[1] =$ 'C'.
- **Valutazione dei caratteri a $i = 1$:**
  - "AB"$[1] =$ 'A' $<$ 'C' $\implies$ "AB" $\in R_<$
  - "AT"$[1] =$ 'A' $<$ 'C' $\implies$ "AT" $\in R_<$

- "CAT"[1] = 'C' == 'C' $\implies$ "CAT" $\in R_=$
- "CAST"[1] = 'C' == 'C' $\implies$ "CAST" $\in R_=$
- "CAR"[1] = 'C' == 'C' $\implies$ "CAR" $\in R_=$

- **Partizioni risultanti:**

$$R_< = \{\text{"AB"},\ \text{"AT"}\}, \quad R_= = \{\text{"CAT"},\ \text{"CAST"},\ \text{"CAR"}\}, \quad R_> = \emptyset \tag{13.11}$$

Passo 2A: Sottoalbero Sinistro ($R_<$) Si analizza il sottoinsieme $R_< =$ {"AB", "AT"} a partire dall'indice iniziale $i = 1$:

- **Pivot selezionato:** $P =$ "AT", con $i = 1 \implies P[1] =$ 'A'.
- Sia "AB"[1] = 'A' che "AT"[1] = 'A'. Poiché entrambi gli elementi condividono il carattere 'A', non si generano rami laterali ($R_< = \emptyset$, $R_> = \emptyset$) e l'insieme confluisce interamente nel ramo centrale $R_=$.
- Si avanza pertanto l'indice di scansione sul ramo centrale a $i = 2$:
  - Carattere di pivot a $i = 2$: $P[2] =$ 'T'.
  - Confronto a $i = 2$:

$$\text{"AB"}[2] = \text{'B'} < \text{'T'} \implies \text{"AB"} \in R_< \tag{13.12}$$
$$\text{"AT"}[2] = \text{'T'} == \text{'T'} \implies \text{"AT"} \in R_= \tag{13.13}$$

    mentre $R_> = \emptyset$.

- Ne discende che la stringa "AB" devia sul ramo minore del nodo associato a 'T', mentre "AT" prosegue e conclude il percorso lungo il ramo centrale.

Passo 2B: Sottoalbero Centrale ($R_=$) Si elabora il sottoinsieme $R_= =$ {"CAT", "CAST", "CAR"} derivante dalla radice:

- Poiché l'insieme scaturisce dalla transizione centrale della radice ($X =$ 'C'), l'ispezione si sposta su $i = 2$.
- **Pivot selezionato:** $P =$ "CAST", con $i = 2 \implies P[2] =$ 'A'.
- Tutte e tre le stringhe presentano la lettera 'A' in seconda posizione ("CAT"[2] = "CAST"[2] = "CAR"[2] = 'A').
- Di conseguenza, $R_< = \emptyset$, $R_> = \emptyset$, e si avanza ricorsivamente lungo il ramo uguale a $i = 3$.

- **Nuovo pivot a $i = 3$:** $P =$ "CAST", con $X = P[3] =$ 'S'.
- **Confronto dei terzi caratteri ($i = 3$):**
  - "CAR"[3] = 'R'. Poiché nel codice alfabetico 'R' < 'S', si ha "CAR" $\in R_<$.
  - "CAST"[3] = 'S'. Corrispondenza esatta 'S' == 'S', per cui "CAST" $\in R_=$.
  - "CAT"[3] = 'T'. Poiché 'T' > 'S', si ha "CAT" $\in R_>$.
- **Partizioni a $i = 3$:**

$$R_< = \{\text{"CAR"}\}, \quad R_= = \{\text{"CAST"}\}, \quad R_> = \{\text{"CAT"}\} \tag{13.14}$$

  Tutte le tre partizioni risultano composte da un singolo elemento (singleton), esaurendo le biforcazioni necessarie per disambiguare l'intero insieme.

Livello/Nodo	**Indice i**	**Carattere X**	**Ramo <**	**Ramo =**	**Ramo >**
Radice	$i = 1$	`'C'`	{AB, AT}	{CAT, CAST, CAR}	$\emptyset$
Figlio Sx	$i = 1$	`'A'`	$\emptyset$	{AB, AT} (avanza a $i = 2$)	$\emptyset$
Sub Sx	$i = 2$	`'T'`	{AB}	{AT}	$\emptyset$
Figlio Mid	$i = 2$	`'A'`	$\emptyset$	{CAT, CAST, CAR} (avanza a $i = 3$)	$\emptyset$
Sub Mid	$i = 3$	`'S'`	{CAR}	{CAST}	{CAT}

Tabella 13.1: Quadro riassuntivo dei passi di partizionamento ternario e delle ramificazioni ricorsive per l'insieme didattico $S = \{\text{CAT}, \text{AB}, \text{CAST}, \text{CAR}, \text{AT}\}$.

13.3.2 Topologia Vettoriale dell'Albero Ricostruito

La Figura 13.2 illustra la conformazione geometrica esatta del TST corrispondente ai passaggi sopra derivati.

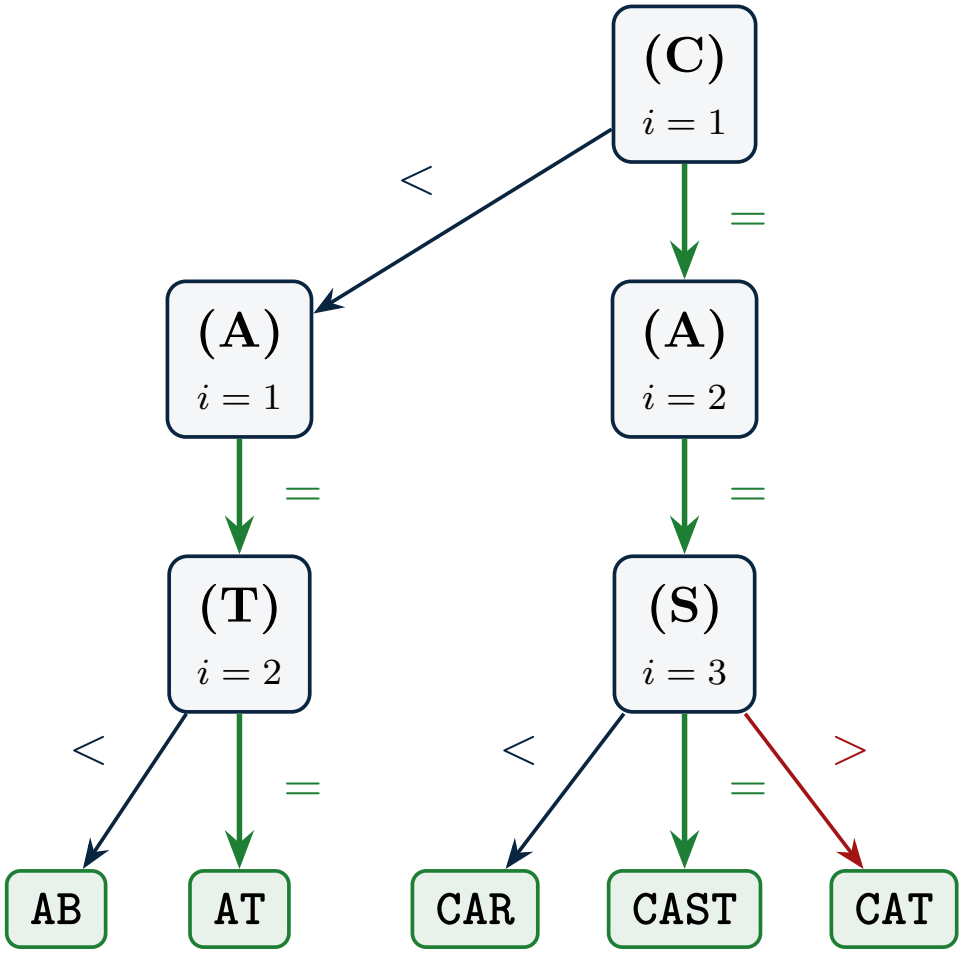


Figura 13.2: Diagramma topologico del Ternary Search Tree ottenuto dal partizionamento dell'insieme $S = \{\text{CAT}, \text{AB}, \text{CAST}, \text{CAR}, \text{AT}\}$, inteso come albero di ricorsione del Multi-Key Quicksort (con compressione alle foglie non appena $|R| = 1$). La geometria ternaria è rigorosamente uniforme: le transizioni centrali ($=$) scendono verticalmente verso il basso avanzando l'indice ($i \leftarrow i + 1$), mentre le transizioni laterali ($<$ e $>$) divergono diagonalmente a sinistra e a destra mantenendo invariato l'indice i.

13.4 Operazioni Fondamentali e Analisi di Complessità nei TST

13.4.1 Ricerca Esatta (Exact Match)

Data una chiave interrogante $s \in \Sigma^*$ di lunghezza $|s|$ da localizzare all'interno del dizionario S contenente n stringhe:

1. La procedura ha inizio presso il nodo radice impostando l'indice di scansione a $i = 1$.

2. Presso ciascun nodo corrente u caratterizzato dal discriminante $X = \text{char}(u)$ e indice i:

   - Se $s[i] < X$, la ricerca prosegue sul figlio sinistro left$[u]$ mantenendo i invariato.
   - Se $s[i] > X$, la ricerca prosegue sul figlio destro right$[u]$ mantenendo i invariato.
   - Se $s[i] == X$, la corrispondenza del carattere corrente è verificata: la ricerca prosegue sul figlio centrale mid$[u]$ incrementando l'indice di scansione:
   $$i \leftarrow i + 1 \tag{13.15}$$

3. La ricerca si conclude con successo se e solo se, dopo aver consumato l'intero vettore dei caratteri di s, si perviene alla validazione del carattere sentinella di fine stringa \$ (o a un marcatore di nodo terminale attivo). Qualora si raggiunga un puntatore nullo prima del termine di s, la chiave non è presente nel dizionario.

Analisi Formale della Complessità: Il tempo di esecuzione di una ricerca esatta si scompone nella somma di due contributi computazionali nettamente distinti:

1. Il numero di archi centrali (=) attraversati, che corrisponde esattamente al numero di caratteri validati con successo, pari a $|s|$.

2. Il numero di biforcazioni binarie interne ($<$ e $>$) percorse per individuare il carattere corretto a ciascun livello.

Teorema 13.4.1 (Complessità della Ricerca in un TST)**.** *Sia dato un TST contenente n chiavi.*

1. *Nel **caso peggiore teorico** (albero completamente sbilanciato o catene lineari lungo i branch laterali), il costo di ricerca è delimitato superiormente da:*

$$T_{worst}(s) = \mathcal{O}(|s| \cdot \sigma) \quad \textit{oppure} \quad \mathcal{O}(|s| \log_2 n) \tag{13.16}$$

2. *Nel **caso medio** (sotto l'ipotesi di inserimento delle chiavi in ordine casuale, assimilabile all'albero di partizionamento di un Quicksort bilanciato), il costo computazionale per una ricerca con successo è:*

$$T_{medio}(s) = \mathcal{O}(|s| + \ln n) = \mathcal{O}(|s| + \log_2 n) \tag{13.17}$$

Dimostrazione. La componente dovuta ai collegamenti uguali è vincolata deterministicamente dal numero di caratteri della chiave: ogni transizione centrale consuma esattamente un carattere di s, comportando al più $|s|$ passi. Per la componente laterale, lungo qualsiasi cammino dalla radice a una foglia, i nodi con il medesimo indice i formano un albero binario di ricerca sui caratteri dell'alfabeto presenti a quel livello. Quando le chiavi sono inserite in ordine casuale, la profondità cumulata attraversata lungo i rami minori e maggiori sull'intero percorso è asintoticamente governata dall'altezza media di un BST su n elementi, pari a $\mathcal{O}(\log_2 n)$ confronti. Sommando i due termini, si ottiene la complessità media $\mathcal{O}(|s| + \log_2 n)$. □

13.4.2 Ricerca per Prefisso (Prefix Search)

Dato un prefisso arbitrario $\hat{s}$ di lunghezza $|\hat{s}|$, determinare se esiste almeno una chiave $s \in S$ che ammetta $\hat{s}$ come prefisso iniziale e individuare l'insieme di tutte le chiavi che lo estendono:

- Si discende l'albero guidati dai caratteri del prefisso $\hat{s}$ applicando le medesime regole della ricerca esatta.
- Se il cammino si interrompe su un puntatore nullo prima di aver esaurito tutti i caratteri di $\hat{s}$, nessuna chiave nel dizionario possiede tale prefisso.
- Se tutti i caratteri di $\hat{s}$ vengono validati con successo pervenendo al nodo figlio centrale dell'ultimo carattere, **l'intero sottoalbero radicato in quel nodo contiene tutte e sole le chiavi che ammettono $\hat{s}$ come prefisso**.
- L'elenco completo delle stringhe associate si ricava mediante una visita simmetrica ordinata (in-order traversal) del sottoalbero, senza dover riesaminare i caratteri di $\hat{s}$.

- Il costo temporale medio per localizzare il nodo radice del prefisso è:

$$T_{\text{prefix}}(\hat{s}) = \mathcal{O}(|\hat{s}| + \log_2 n) \tag{13.18}$$

13.4.3 Operazioni Lessicografiche Avanzate

A differenza delle tabelle hash (che disperdono l'ordine dei dati per massimizzare l'uniformità di distribuzione), i Ternary Search Trees preservano nativamente l'**ordinamento lessicografico** delle chiavi grazie alla separazione ordinata tra sottoalberi sinistri ($<$) e destri ($>$). Ciò rende i TST eccezionalmente adatti all'esecuzione di operazioni ordinate complesse:

- **Ricerca del Predecessore e Successore Lessicografico:** data una stringa arbitraria s, individua la chiave immediatamente precedente o successiva appartenente a S. La ricerca scende lungo l'albero tenendo traccia dell'ultimo ramo laterale imboccato, consentendo di identificare la chiave più vicina anche se $s \notin S$.
- **Rango o Posizione Lessicografica (Rank(s, S)):** calcola in tempo logaritmico quante stringhe memorizzate nel dizionario precedono lessicograficamente la chiave s. Arricchendo ciascun nodo con un campo aggiuntivo che memorizza la cardinalità (*size*) del sottoalbero, il calcolo del rango avviene in tempo $\mathcal{O}(|s| + \log_2 n)$ accumulando le dimensioni dei rami minori scartati durante la discesa.
- **Ricerca per Intervallo (Range Search(s_1, s_2)):** estrae ordinatamente tutte le chiavi $s \in S$ tali che $s_1 \leq s \leq s_2$. La visita attraversa l'albero potando tempestivamente qualsiasi sottoalbero le cui chiavi risultino strettamente minori di s_1 o strettamente maggiori di s_2.

13.5 Radix Sort: Tassonomia, Funzionamento e Complessità

Il **Radix Sort** è un algoritmo di ordinamento lineare che non si basa sul confronto diretto tra coppie di elementi, bensì sull'estrazione e classificazione ordinata delle singole **componenti elementari** o "cifre" che costituiscono la chiave (come bit, nibble, byte o caratteri) [21].

13.5.1 Tassonomia Fondamentale: LSD vs MSD

La letteratura algoritmica classifica il Radix Sort in due filosofie operative distinte in base alla direzione di scansione delle cifre:

1. **LSD (Least Significant Digit):**
    - **Direzione:** Scansione da destra verso sinistra ($\leftarrow$), partendo dalla cifra meno significativa fino a raggiungere la più significativa.
    - **Requisito Vincente:** Richiede **tassativamente** che la routine di ordinamento ausiliaria impiegata ad ogni passata sia **stabile** (ovvero preservi rigorosamente l'ordine relativo preesistente tra elementi che possiedono la medesima cifra).
    - **Caratteristiche di Memoria:** Algoritmo prettamente **iterativo**, privo di stack di ricorsione profonda e caratterizzato da un ciclo compatto ad accessi sequenziali alla memoria.
2. **MSD (Most Significant Digit):**
    - **Direzione:** Scansione da sinistra verso destra ($\rightarrow$), partendo dal carattere più significativo.
    - **Meccanismo:** Partiziona ricorsivamente le chiavi in sottoinsiemi distinti (*bucket*) caratterizzati dal medesimo prefisso iniziale, operando in modo concettualmente identico alla costruzione di un Trie o alle partizioni del Multi-Key Quicksort.
    - **Caratteristiche di Memoria:** Non richiede la stabilità della routine ausiliaria, ma richiede la gestione di stack ricorsivi e allocazioni dinamiche di vettori d'appoggio per i bucket intermedi.

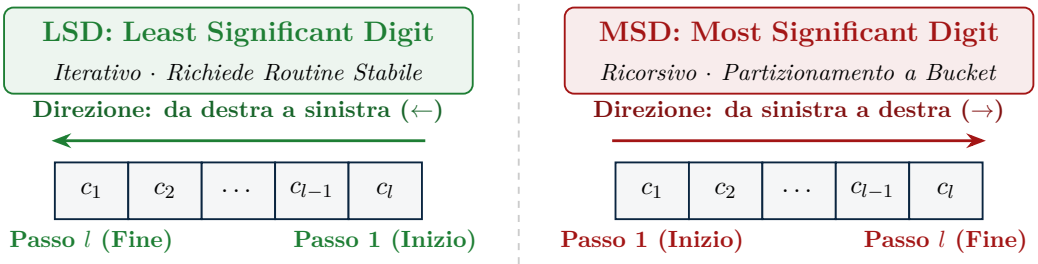


Figura 13.3: Confronto schematico sulla direzione di elaborazione tra l'approccio LSD (a passate iterative stabili dalla cifra meno significativa) e l'approccio MSD (a partizioni ricorsive a partire dal prefisso più significativo).

13.5.2 Analisi dell'Algoritmo LSD Radix Sort

Si consideri un insieme composto da n stringhe di lunghezza uniforme prefissata pari a l caratteri (o cifre), definite su un alfabeto di cardinalità σ:

- **Struttura a Fasi:** L'algoritmo esegue esattamente l **passate successive**, scansionando le posizioni a ritroso dall'indice l (ultima componente a destra) fino all'indice 1 (prima componente a sinistra).
- **Routine di Ordinamento Stabile:** In ciascuna passata $j \in \{l, l-1, \ldots, 1\}$, l'insieme delle n stringhe viene riordinato rispetto alla sola componente $s[j]$ mediante un ordinamento stabile. La procedura canonica d'elezione è il **Counting Sort** [21].
- **Complessità del Counting Sort per Singola Fase:** Dati n elementi le cui chiavi assumono valori discreti nell'intervallo $[0, \sigma - 1]$:
  1. Calcolo dell'istogramma delle frequenze: tempo $\mathcal{O}(n)$ e memoria ausiliaria $\mathcal{O}(\sigma)$.
  2. Calcolo delle somme prefisse (*prefix sums*) e ridistribuzione stabile in un array d'appoggio: tempo $\mathcal{O}(n + \sigma)$.

  Il costo di una singola fase risulta quindi:
$$T_{\text{fase}} = \mathcal{O}(n + \sigma) \tag{13.19}$$
- **Complessità Temporale Complessiva dell'LSD Radix Sort:** Poiché l'algoritmo esegue esattamente l passate indipendenti:
$$T_{\text{totale}}(n, l, \sigma) = l \cdot \mathcal{O}(n + \sigma) = \mathcal{O}(l(n + \sigma)) \tag{13.20}$$

13.5.3 Esecuzione Dettagliata dell'Esempio Numerico Didattico

Si traccia l'esecuzione rigorosa dell'algoritmo LSD Radix Sort sull'insieme di $n = 5$ chiavi numeriche di lunghezza $l = 3$ su alfabeto decimale ($\Sigma = \{0, 1, \ldots, 9\}$, con $\sigma = 10$) riportato negli appunti:

$$S_1 = \text{"111"}, \quad S_2 = \text{"042"}, \quad S_3 = \text{"665"}, \quad S_4 = \text{"077"}, \quad S_5 = \text{"007"} \tag{13.21}$$

Elemento	Cifra 1 (MSD)	Cifra 2 (Mid)	Cifra 3 (LSD)	Valore Completo
S_1	'1'	'1'	'1'	111
S_2	'0'	'4'	'2'	042
S_3	'6'	'6'	'5'	665
S_4	'0'	'7'	'7'	077
S_5	'0'	'0'	'7'	007

Tabella 13.2: Configurazione tabellare iniziale delle chiavi numeriche per l'algoritmo LSD Radix Sort.

Fase 1: Ordinamento Stabile sulla Cifra 3 (Posizione Meno Significativa / Destra)

- Si estraggono le terze cifre delle stringhe nell'ordine originario:

$$S_1 \to \texttt{'1'}, \quad S_2 \to \texttt{'2'}, \quad S_3 \to \texttt{'5'}, \quad S_4 \to \texttt{'7'}, \quad S_5 \to \texttt{'7'} \tag{13.22}$$

- Si applica il Counting Sort rispetto a tali chiavi. Si osservi che sia S_4 che S_5 presentano la medesima cifra `'7'`. In virtù della **stabilità del Counting Sort**, l'ordine relativo originario viene rigidamente conservato: S_4 precede S_5 nel vettore prodotto.

- **Vettore al termine della Fase 1:**

$$[S_1,\ S_2,\ S_3,\ S_4,\ S_5] = [\texttt{"111"},\ \texttt{"042"},\ \texttt{"665"},\ \texttt{"077"},\ \texttt{"007"}] \tag{13.23}$$

Fase 2: Ordinamento Stabile sulla Cifra 2 (Posizione Centrale)

- Si estraggono le seconde cifre rispettando l'ordine scaturito dalla Fase 1:
  - `"111"` $\implies$ cifra `'1'`
  - `"042"` $\implies$ cifra `'4'`
  - `"665"` $\implies$ cifra `'6'`
  - `"077"` $\implies$ cifra `'7'`
  - `"007"` $\implies$ cifra `'0'`

- Chiavi di ordinamento estratte: $\{\texttt{'1'}, \texttt{'4'}, \texttt{'6'}, \texttt{'7'}, \texttt{'0'}\}$.

- L'ordinamento stabile riorganizza il vettore secondo i valori crescenti delle cifre:
  1. `'0'` $\implies$ `"007"` (diviene 1°)
  2. `'1'` $\implies$ `"111"` (diviene 2°)
  3. `'4'` $\implies$ `"042"` (diviene 3°)
  4. `'6'` $\implies$ `"665"` (diviene 4°)
  5. `'7'` $\implies$ `"077"` (diviene 5°)

- **Vettore al termine della Fase 2:**

$$[\texttt{"007"},\ \texttt{"111"},\ \texttt{"042"},\ \texttt{"665"},\ \texttt{"077"}] \tag{13.24}$$

Fase 3: Ordinamento Stabile sulla Cifra 1 (Posizione Più Significativa / Sinistra)

- Si estraggono le prime cifre dalla sequenza ordinata al termine della Fase 2:
  - `"007"` $\Longrightarrow$ cifra `'0'`
  - `"111"` $\Longrightarrow$ cifra `'1'`
  - `"042"` $\Longrightarrow$ cifra `'0'`
  - `"665"` $\Longrightarrow$ cifra `'6'`
  - `"077"` $\Longrightarrow$ cifra `'0'`
- Chiavi di ordinamento estratte: {`'0'`, `'1'`, `'0'`, `'6'`, `'0'`}.
- Notiamo che vi sono tre elementi aventi prima cifra identica pari a `'0'`: `"007"`, `"042"` e `"077"`. Poiché il Counting Sort è rigorosamente stabile, l'ordine relativo maturato alla fine della Fase 2 (`"007"`, seguito da `"042"`, seguito da `"077"`) **viene interamente preservato**!
- **Vettore finale (Fase 3 ultimata):**

$$[\texttt{"007"},\ \texttt{"042"},\ \texttt{"077"},\ \texttt{"111"},\ \texttt{"665"}] \tag{13.25}$$

Il vettore risulta perfettamente ordinato sia numericamente sia lessicograficamente in senso strettamente crescente.

13.6 Ottimizzazione di LSD Radix Sort per Stringhe Binarie

Un problema cardinale nell'ingegneria dei sistemi software (gestione di indici di database, elaborazione di hash crittografici SHA-256 a 256 bit o ordinamento di indirizzi di memoria fisica a 64 bit) consiste nell'ordinare grandi collezioni di **chiavi binarie**:

$$S = \{s_1, s_2, \ldots, s_n\}, \quad s_j \in \{0,1\}^L \quad \forall j \in \{1, \ldots, n\} \tag{13.26}$$

ovvero n stringhe su alfabeto binario $\Sigma = \{0, 1\}$ di lunghezza uniforme pari a L bit.

Se si applicasse ingenuamente l'LSD Radix Sort considerando la singola cifra elementare a livello di singolo bit ($R = 1$ bit), l'alfabeto avrebbe

cardinalità minima $\sigma = 2^1 = 2$, ma l'algoritmo richiederebbe un numero proibitivo di passate, esattamente pari a L, comportando un tempo di esecuzione complessivo pari a:

$$T(n, L) = L \cdot \mathcal{O}(n + 2) = \mathcal{O}(L \cdot n) \tag{13.27}$$

Per abbattere il numero di passate, si raggruppano i bit adiacenti in blocchi omogenei (**chunk**) di dimensione fissa R bit ciascuno.

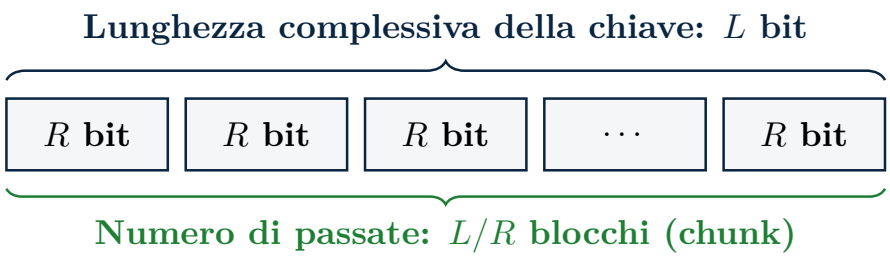


Figura 13.4: Scomposizione di una chiave binaria di lunghezza L bit in blocchi elementari di dimensione fissa R bit.

13.6.1 Formalizzazione del Modello a Blocchi

- **Dimensione del singolo blocco:** R bit.
- **Alfabeto virtuale per ciascun blocco:** Ciascun blocco di R bit esprime uno tra 2^R valori interi distinti nell'intervallo $[0, 2^R - 1]$; l'alfabeto equivalente ha pertanto cardinalità:

$$\sigma = 2^R \tag{13.28}$$

- **Numero di passate complessive:**

$$\text{Passate} = \frac{L}{R} \tag{13.29}$$

- **Costo di una singola passata di Counting Sort:**

$$T_{\text{passata}} = \mathcal{O}(n + \sigma) = \mathcal{O}(n + 2^R) \tag{13.30}$$

- **Funzione di Complessità Globale:** Moltiplicando il numero di passate per il costo temporale della singola passata, si ottiene:

$$T(n, L, R) = \mathcal{O}\left(\frac{L}{R}(n + 2^R)\right) \tag{13.31}$$

13.6.2 Derivazione Matematica del Valore Ottimo di R

La funzione di costo evidenzia un evidente **trade-off**:

- All'aumentare di R, il fattore $\frac{L}{R}$ decresce proporzionalmente, riducendo il numero totale di iterazioni richieste.
- Al contempo, il costo di allocazione e inizializzazione dei contatori del Counting Sort $\mathcal{O}(2^R)$ cresce in modo esponenziale.

Per identificare il minimo globale, analizziamo il comportamento asintotico della funzione dividendo il dominio di R in due regimi distinti rispetto a $\log_2 n$:

Regime 1: $1 \leq R < \log_2 n$ In questo intervallo si ha $2^R < n$. Nel binomio $(n + 2^R)$, la cardinalità dell'insieme n domina strettamente la dimensione della tabella di conteggio 2^R:

$$n + 2^R = \Theta(n) \tag{13.32}$$

La funzione di complessità asintotica si riduce a:

$$T(n, L, R) = \mathcal{O}\left(\frac{L}{R} \cdot n\right) \tag{13.33}$$

Poiché la funzione è **strettamente decrescente** rispetto a R in tutto l'intervallo $R < \log_2 n$, risulta vantaggioso incrementare R per comprimere il numero di passate $\frac{L}{R}$ senza pagare penalizzazioni significative nella dimensione dell'alfabeto.

Regime 2: $R \geq \log_2 n$ In questo intervallo si ha $2^R \geq n$. Nel binomio $(n + 2^R)$, il termine esponenziale 2^R sovrasta il numero di stringhe n:

$$n + 2^R = \Theta(2^R) \tag{13.34}$$

La funzione di complessità asintotica diventa:

$$T(n, L, R) = \mathcal{O}\left(\frac{L \cdot 2^R}{R}\right) \tag{13.35}$$

Poiché al numeratore compare la funzione esponenziale 2^R e al denominatore compare la funzione lineare R, per $R \geq \log_2 n$ la funzione è **strettamente crescente** rispetto a R.

Punto di Minimo Globale ed Efficienza Ottima: Il minimo della funzione di complessità temporale coincide con il punto di pareggio (*break-even*) in cui i due contributi risultano perfettamente bilanciati:

$$2^R \approx n \implies R_{\text{ottimo}} = \log_2 n \tag{13.36}$$

Sostituendo il valore ottimo $R_{\text{ottimo}} = \log_2 n$ nell'espressione generale di costo:

$$\begin{aligned} T(n, L, \log_2 n) &= \mathcal{O}\left(\frac{L}{\log_2 n}(n + 2^{\log_2 n})\right) \\ &= \mathcal{O}\left(\frac{L}{\log_2 n}(n + n)\right) \\ &= \mathcal{O}\left(\frac{2n \cdot L}{\log_2 n}\right) \end{aligned} \tag{13.37}$$

Trascurando le costanti moltiplicative, si ottiene il teorema di ottimalità per l'ordinamento di stringhe binarie:

Teorema 13.6.1 (Complessità Ottima di LSD Radix Sort su Stringhe Binarie)**.** *Dato un insieme di n stringhe binarie di lunghezza uniforme L bit, l'algoritmo LSD Radix Sort configurato con blocchi di ampiezza $R = \log_2 n$ bit raggiunge una complessità temporale ottima pari a:*

$$T_{min} = \mathcal{O}\left(\frac{n \cdot L}{\log_2 n}\right) \tag{13.38}$$

Nota 13.6.1 (Verifica Analitica con Calcolo Differenziale)**.** Considerando la funzione continua $f(R) = \frac{n+2^R}{R}$ definita per $R > 0$ e calcolandone la derivata prima rispetto a R:

$$f'(R) = \frac{\frac{d}{dR}(n + 2^R) \cdot R - (n + 2^R) \cdot 1}{R^2} = \frac{(2^R \ln 2)R - n - 2^R}{R^2} \tag{13.39}$$

Ponendo la derivata uguale a zero per ricercare i punti stazionari:

$$2^R(R \ln 2 - 1) = n \tag{13.40}$$

Poiché asintoticamente $(R \ln 2 - 1)$ cresce in modo logaritmico rispetto a 2^R, la soluzione esatta soddisfa $R = \Theta(\log_2 n)$, confermando rigorosamente la derivazione discreta.

13.7 Analisi Comparativa Critica e Scelta Architetturale

Negli appunti del corso viene formulata una fondamentale osservazione comparativa conclusiva:

> *"Radix sort is good when prefix is close to L, otherwise multikey quicksort is better."*

Questo commento del docente racchiude un principio architetturale di primaria importanza per l'Algorithm Engineering, chiarendo le motivazioni teoriche e pratiche che guidano la scelta tra algoritmi basati su partizionamento (*Multi-Key Quicksort*) e algoritmi a scansione lineare (*Radix Sort*).

13.7.1 Perché il Radix Sort è penalizzato da prefissi corti?

L'algoritmo LSD Radix Sort è per sua natura **"cieco"** rispetto alla variabilità intrinseca dei prefissi delle stringhe e alla loro distribuzione:

- Anche qualora due stringhe differiscano macroscopicamente fin dal primissimo bit o carattere (si pensi a due chiavi che iniziano rispettivamente per `"0..."` e `"1..."`), l'LSD Radix Sort non dispone di alcun meccanismo di arresto anticipato (*early exit*).
- Ogni singola stringa dell'insieme deve obbligatoriamente transitare attraverso tutte le $\frac{L}{R}$ passate del Counting Sort, ispezionando l'intera estensione di L bit fino all'ultima fase.
- Quando le stringhe si distinguono quasi immediatamente (ovvero la lunghezza media del prefisso comune prima della separazione, denominato **prefisso distinguibile** o *Distinguishing Prefix* D_i, è molto piccola rispetto a L: $D_i \ll L$), l'ispezione forzata di tutti gli L bit genera una mole ingente di operazioni di conteggio ridondanti.

13.7.2 Perché il Multi-Key Quicksort eccelle con prefissi corti?

Il **Multi-Key Quicksort** (così come l'inserimento nei TST o l'MSD Radix Sort con potatura) sfrutta attivamente l'informazione estratta dai caratteri più significativi:

- Non appena una stringa viene instradata in un sottoinsieme laterale ($R_<$ o $R_>$), essa viene isolata rispetto alle altre partizioni e non verrà mai più confrontata con esse sui caratteri successivi.
- L'algoritmo ispeziona ciascuna stringa s_i unicamente fino al raggiungimento del suo prefisso distinguibile $D_i \leq L$.
- Come dimostrato nel Teorema 12.3.1, il costo computazionale totale del Multi-Key Quicksort è:

$$T_{\text{MKQ}} = \Theta\left(n \log_2 n + \sum_{i=1}^{n} D_i\right) \tag{13.41}$$

- Se i prefissi distinguibili sono brevi ($D_i \ll L$), la sommatoria $\sum_{i=1}^{n} D_i$ risulta trascurabile rispetto a $n \cdot L$, rendendo il Multi-Key Quicksort incomparabilmente più veloce dell'LSD Radix Sort.

13.7.3 Il Caso Favorevole al Radix Sort: Prefissi Prossimi a L

Al contrario, si consideri uno scenario in cui l'insieme di dati sia composto da stringhe densamente collegate che condividono lunghissimi prefissi comuni (come nel caso di URL web con domini e percorsi identici, firme genomiche o serie di timestamp ad altissima frequenza):

- In questo contesto, il prefisso distinguibile si avvicina alla lunghezza massima della chiave: $D_i \approx L$.
- Anche gli algoritmi top-down come il Multi-Key Quicksort o i TST sono costretti a scendere in profondità lungo quasi tutti gli L caratteri per disambiguare le chiavi; nessun algoritmo basato su confronti può eludere l'ispezione di $\Omega(n \cdot L)$ caratteri.
- Poiché l'ispezione completa dell'intera stringa diviene inevitabile per qualsiasi metodo, il **Radix Sort (LSD)** manifesta appieno la propria superiorità architetturale:
    1. **Accessi Sequenziali alla Memoria:** il Counting Sort opera mediante scansioni lineari contigue su array vettoriali, massimizzando il throughput della cache hardware e azzerando i *cache miss*.
    2. **Assenza di Branch Misprediction:** l'esecuzione è rigidamente iterativa, priva dei salti condizionali imprevedibili caratteristici dei confronti del Quicksort.

3. **Zero Overhead Ricorsivo:** non vi sono chiamate ricorsive né dereferenziazioni di puntatori a nodi dinamici.

13.7.4 Quadro Sinottico Comparativo delle Metodologie

La Tabella 13.3 sintetizza le proprietà teoriche, operative ed architetturali delle metodologie per stringhe analizzate.

Proprietà / Criterio	**Ternary Search Tree (TST)**	**LSD Radix Sort (Chunking)**	**Multi-Key Quicksort (3-Way)**
Paradigma Computazionale	Albero di ricerca dinamico (ibrido Trie/BST)	Ordinamento non comparativo a passate di Counting Sort	Divide et Impera ternario sui singoli caratteri
Complessità Temporale	Ricerca: $\mathcal{O}(\|s\| + \log_2 n)$ Costruzione: $\mathcal{O}(n \log_2 n + \sum D_i)$	Ordinamento: $\mathcal{O}\left(\frac{n \cdot L}{\log_2 n}\right)$ (su chiavi binarie di L bit)	Ordinamento: $\boldsymbol{\Theta(\mathbf{n} \log_2 \mathbf{n} + \sum \mathbf{D_i})}$ (ottimo al Lower Bound)
Occupazione Spaziale	$\mathcal{O}(\text{nodi}) \approx 3$ puntatori per nodo	$\mathcal{O}(n + 2^R)$ ausiliario per i bucket di conteggio	$\mathcal{O}(\log_2 n)$ stack di ricorsione in-place
Sensibilità al Prefisso (D_i)	**Massima:** naviga unicamente i caratteri necessari	**Nulla:** ispeziona obbligatoriamente tutti gli L bit	**Massima:** potatura immediata delle partizioni disgiunte
Operazioni Supportate	Ricerca esatta, prefissi, predecessore, rango, range query	Esclusivamente ordinamento statico di massa	Esclusivamente ordinamento dell'array di stringhe
Scenario Ideale d'Uso	Dizionari dinamici, autocompletamento e ricerche lessicografiche	Grandi moli di chiavi binarie o dense con prefissi $D_i \approx L$	Ordinamento generale di stringhe con prefissi eterogenei o corti

Tabella 13.3: Quadro sinottico comparativo tra Ternary Search Trees, LSD Radix Sort a blocchi e Multi-Key Quicksort.

Regola Decisionale per l'Ingegnere del Software

Nella progettazione di sistemi di elaborazione ad elevate prestazioni:

- Se la collezione di stringhe è dinamica e richiede frequenti interrogazioni di tipo *prefix match*, *autocompletion* o ricerche per intervallo ordinato, il **Ternary Search Tree** rappresenta la struttura d'elezione per via dell'eccellente compromesso tra tempo $\mathcal{O}(|s| + \log_2 n)$ e compattezza di memoria.
- Se il problema consiste nell'ordinamento statico di grandi vettori di stringhe con prefissi corti ed eterogenei, il **Multi-Key Quicksort** è preferibile poiché ispeziona solo i caratteri strettamente necessari ($\sum D_i$), operando in-place con memoria ausiliaria minima.
- Se le chiavi sono binarie a lunghezza fissa L bit e fortemente correlate (prefissi lunghi $D_i \approx L$), l'**LSD Radix Sort** con chunking a $R = \log_2 n$ bit supera qualsiasi altro algoritmo grazie alla località sequenziale dei trasferimenti nella gerarchia di memoria.

Capitolo 14

Alberi Digitali per Stringhe: Trie, MSD Radix Sort e Trie Compatti

14.1 Relazione Fondamentale tra MSD Radix Sort e Trie

Nel contesto dell'Algorithm Engineering applicato alle sequenze di simboli, emerge un principio unificante di fondamentale eleganza formale: la **dualità biunivoca** tra l'esecuzione top-down dell'algoritmo di ordinamento *MSD Radix Sort* (*Most Significant Digit*) e la topologia logica dell'albero digitale dei prefissi, noto in letteratura come **Trie** [23, 21].

14.1.1 Dualità tra Algoritmo e Struttura Dati

L'algoritmo MSD Radix Sort ordina una collezione di chiavi alfanumeriche scansionando i caratteri progressivamente da sinistra verso destra ($\rightarrow$), a partire dalla componente più significativa. A ogni passo ricorsivo su una specifica posizione i:

- L'insieme corrente delle chiavi viene ripartito in un insieme di sotto-secchielli (*bucket*), ciascuno associato a un simbolo distinto dell'alfabeto finito ed ordinato Σ di dimensione $\sigma = |\Sigma|$.
- Tutti gli elementi che condividono il medesimo carattere in posizione i confluiscono nello stesso bucket.

- Ciascun bucket contenente almeno due elementi innesca una chiamata ricorsiva indipendente per ordinare il sottoinsieme rispetto alla posizione successiva $i + 1$.

Questo processo ricorsivo traccia nello spazio di computazione un albero delle chiamate che coincide strutturalmente con la definizione di un **Trie**:

1. **Radice dell'Albero:** rappresenta l'insieme complessivo iniziale di tutte le chiavi S.
2. **Nodi Interni:** incarnano i prefissi comuni condivisi da due o più stringhe della collezione. La profondità del nodo corrisponde alla lunghezza del prefisso validato.
3. **Archi Orientati:** ogni arco uscente da un nodo è contrassegnato dal simbolo discriminante dell'alfabeto Σ che determina la transizione al carattere successivo.
4. **Foglie Terminali:** contengono le singole chiavi dell'insieme. Poiché gli archi uscenti da ciascun nodo sono naturalmente ordinati secondo l'ordine totale di Σ, una visita simmetrica anticipata (*in-order traversal*) del Trie produce le chiavi nell'esatto ordine lessicografico.

14.1.2 Esempio Guidato e Corrispondenza Grafica

Si consideri un insieme didattico composto da $n = 5$ chiavi numeriche a lunghezza fissa uniforme $l = 3$ cifre, definite sull'alfabeto decimale $\Sigma = \{0, 1, 2, \ldots, 9\}$ con $\sigma = |\Sigma| = 10$:

$$S = \{\text{"007"},\ \text{"017"},\ \text{"042"},\ \text{"911"},\ \text{"999"}\} \tag{14.1}$$

Traccia di Esecuzione dell'MSD Radix Sort:

1. **Livello 1 (Ispezione della prima cifra, $i = 1$):**
   - Il bucket per il simbolo `'0'` riceve:

     $$\{\text{"007"},\ \text{"017"},\ \text{"042"}\}$$

   - Il bucket per il simbolo `'9'` riceve:

     $$\{\text{"911"},\ \text{"999"}\}$$

   - I restanti bucket (da `'1'` a `'8'`) rimangono vuoti ($\emptyset$).

2. **Livello 2 (Ispezione della seconda cifra, $i = 2$):**
   - Il sottoinsieme del bucket `'0'` si suddivide ricorsivamente:
     - Bucket `'0'` $\rightarrow$ {"007"} (singleton, isolato).
     - Bucket `'1'` $\rightarrow$ {"017"} (singleton, isolato).
     - Bucket `'4'` $\rightarrow$ {"042"} (singleton, isolato).
   - Il sottoinsieme del bucket `'9'` si suddivide ricorsivamente:
     - Bucket `'1'` $\rightarrow$ {"911"} (singleton, isolato).
     - Bucket `'9'` $\rightarrow$ {"999"} (singleton, isolato).
3. **Livello 3 (Ispezione della terza cifra, $i = 3$):** Tutte le chiavi raggiungono nodi foglia individuali, determinando l'ordinamento conclusivo:

$$\text{["007", "017", "042", "911", "999"]} \tag{14.2}$$

La Figura 14.1 visualizza la topologia dell'albero risultante, evidenziando come l'albero di ricorsione dell'MSD Radix Sort generi un Trie non compatto.

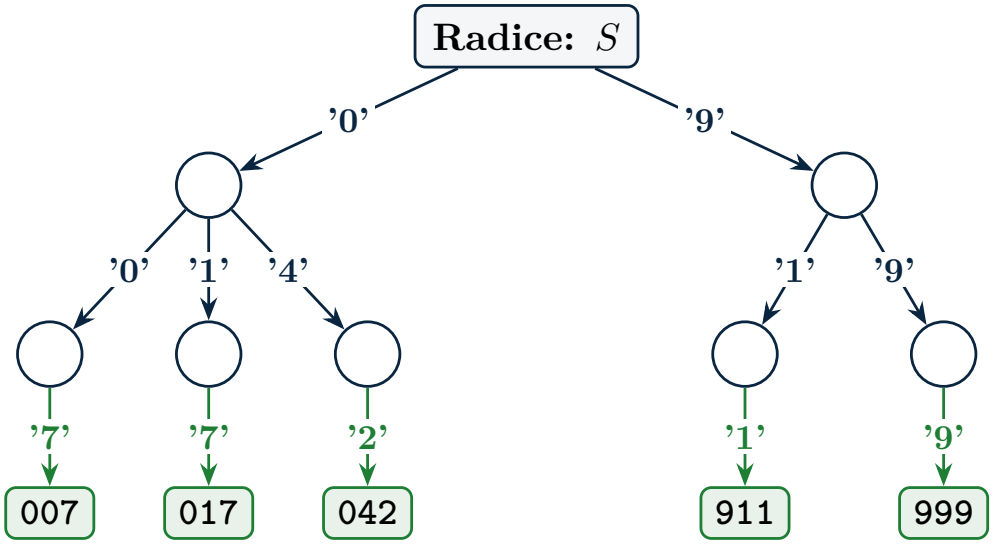


Figura 14.1: Rappresentazione del Trie non compatto corrispondente all'albero delle chiamate ricorsive dell'MSD Radix Sort per l'insieme $S = \{"007", "017", "042", "911", "999"\}$. Ciascun livello di profondità corrisponde alla cifra esaminata; una visita anticipata da sinistra verso destra restituisce le foglie perfettamente ordinate.

14.2 Implementazione Standard tramite Vettori (Array-Based Trie)

14.2.1 Architettura dei Nodi e Tabella dei Puntatori

Nella classica implementazione canonica di un Trie per stringhe, ciascun nodo interno u viene modellato come un array statico di puntatori di dimensione prefissata e pari all'ampiezza dell'alfabeto $\sigma = |\Sigma|$:

$$\text{Node}[u] = \langle \text{child}[0],\ \text{child}[1],\ \ldots,\ \text{child}[\sigma - 1] \rangle \tag{14.3}$$

Ciascuna cella $c \in \{0, \ldots, \sigma - 1\}$ contiene:

- Il puntatore al nodo figlio raggiungibile tramite la transizione etichettata con il simbolo c.
- Il puntatore nullo (`NULL`) qualora non esista alcuna chiave nell'insieme che prosegua con tale carattere da quel nodo.

Il pregio architetturale primario di questa rappresentazione risiede nella velocità di lookup: l'accesso al nodo figlio dato un carattere c avviene mediante una singola operazione di indicizzazione vettoriale diretta (*direct table addressing*):

$$\text{next_node} = u.\text{child}[\text{rank}(c)] \tag{14.4}$$

garantendo un tempo di transizione rigidamente deterministico $\mathcal{O}(1)$.

14.2.2 Problema dello Spreco di Memoria (Waste of Space)

Nonostante l'eccellenza temporale delle transizioni, l'approccio basato su array soffre di una severa inefficienza di occupazione della memoria quando l'alfabeto è esteso (es. ASCII a 256 caratteri o lo spazio Unicode UTF-8/UTF-16) e l'insieme delle chiavi è sparso:

- Se un nodo possiede un grado effettivo di uscita (*out-degree*) $k_u \ll \sigma$, la quasi totalità delle celle dell'array viene allocata inutilmente, rimanendo valorizzata a `NULL`.
- Con riferimento all'esempio didattico precedente su cifre decimali ($\sigma = 10$): - Al nodo radice risultano attive unicamente le transizioni per `'0'` e `'9'`. - Le celle relative a `'1'`, `'2'`, `'3'`, `'4'`, `'5'`, `'6'`, `'7'`, `'8'` (l'80% della tabella) rimangono azzerate, ma richiedono comunque 8 parole di memoria (64 bit ciascuna su architetture moderne).

14.2.3 Analisi della Complessità: Spazio e Tempo di Ordinamento

Sia dato un insieme S composto da n stringhe, dove $d_s = |s|$ rappresenta la lunghezza della generica stringa $s \in S$. Si definisca la **lunghezza complessiva cumulativa** di tutte le stringhe dell'insieme come:

$$d = \sum_{s \in S} d_s = \sum_{s \in S} |s| \tag{14.5}$$

Teorema 14.2.1 (Complessità dell'Array-Based Trie)**.** *La rappresentazione di un Trie basata su array statici soddisfa i seguenti limiti computazionali:*

1. ***Occupazione Spaziale (Worst Case):***

$$\textit{Spazio} \leq \mathcal{O}(d \cdot \sigma) = \left(\sum_{s \in S} d_s \right) \cdot \sigma \tag{14.6}$$

2. ***Tempo di Costruzione e Ordinamento (Sorting Time):***

$$\textit{Tempo} = \mathcal{O}(d \cdot \sigma) \tag{14.7}$$

Dimostrazione. Nel caso peggiore in cui le stringhe non condividano alcun prefisso comune significativo, ogni carattere di ogni stringa introduce un nodo interno distinto lungo il rispettivo percorso di discesa. Poiché il numero totale di caratteri è d, l'albero può contenere fino a $\mathcal{O}(d)$ nodi. Ciascun nodo interno alloca inderogabilmente un vettore di puntatori di dimensione σ, determinando un'occupazione complessiva di memoria pari a $\mathcal{O}(d \cdot \sigma)$ puntatori.

Per quanto concerne il tempo di costruzione e ordinamento: l'allocazione e l'inizializzazione a `NULL` di ciascun array di dimensione σ richiede tempo proporzionale a σ. Inoltre, per estrarre le chiavi ordinate tramite una visita in-order, l'algoritmo è costretto a iterare su tutte le σ posizioni di ciascun nodo interno per verificare quali puntatori siano attivi e quali nulli. Sommando su tutti gli $\mathcal{O}(d)$ nodi, il costo temporale globale è $\mathcal{O}(d \cdot \sigma)$. □

Quando σ è elevato (si pensi a un dizionario su un alfabeto Unicode dove $\sigma \approx 65536$), il fattore moltiplicativo σ degrada l'efficienza a livelli insostenibili, rendendo necessaria una progettazione data-aware più evoluta.

14.3 Ottimizzazione della Memoria: Trie Basato su Tabelle Hash

14.3.1 Progettazione con Tabella Hash Locale

Per eliminare il fattore di spreco moltiplicativo σ, la soluzione ingegneristica d'elezione consiste nel sostituire l'array statico sovradimensionato di ciascun nodo con una **tabella hash locale** [21]:

- **Chiave:** il singolo carattere di transizione $c \in \Sigma$.
- **Valore:** il puntatore al sottoalbero figlio corrispondente.

In questa configurazione (*Hash-Based Trie*), ogni nodo interno memorizza esclusivamente le transizioni **effettivamente esistenti** ($k_u = \text{out-degree}(u)$), senza riservare alcuno spazio per i simboli assenti.

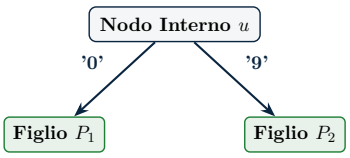


Tabella Hash Locale di u

Chiave (Carattere)	Valore (Puntatore)
'0'	P_1
'9'	P_2

Figura 14.2: Struttura di un nodo in un Hash-Based Trie: l'array statico di dimensione σ è sostituito da una mappa hash locale che indicizza solo le transizioni attive (nell'esempio, unicamente i caratteri `'0'` e `'9'`).

14.3.2 Analisi di Complessità: Spazio e Ricerca

Dimensione della Struttura Dati (Space): La tabella hash T_u associata al nodo generico u impegna una quantità di memoria proporzionale al numero di archi uscenti dal nodo stesso:

$$\text{Size}(T_u) = \Theta(k_u) = \Theta(\#\text{outgoing edges}) \tag{14.8}$$

Sommando lo spazio occupato su tutti i nodi interni u che compongono l'albero:

$$\text{Spazio Totale} = \sum_u \mathcal{O}(k_u) = \mathcal{O}\left(\sum_{s \in S} d_s\right) = \mathcal{O}(d) \tag{14.9}$$

Il consumo spaziale diviene pertanto **strettamente lineare** rispetto alla lunghezza totale dei caratteri d, risultando totalmente svincolato dalla dimensione dell'alfabeto σ.

Tempo di Ricerca (Search Time): Sia data una chiave interrogante di lunghezza ℓ. A ciascun livello dell'albero viene effettuato un lookup nella tabella hash del nodo corrente rispetto al carattere corrispondente:

- Sotto l'ipotesi di una funzione hash uniforme (e un fattore di carico controllato mediante chaining o open addressing), il lookup medio in tabella hash richiede tempo unitario $\mathcal{O}(1)$.
- La discesa lungo il cammino di ℓ livelli comporta dunque:

$$\text{Tempo di Ricerca Medio} = \mathcal{O}(\ell) \tag{14.10}$$

14.3.3 Tempo di Costruzione e Ordinamento (MSD Radix Sort con Hash)

Quando si impiega l'Hash-Based Trie per completare l'ordinamento delle chiavi (secondo il paradigma dell'MSD Radix Sort), emergono due fasi operative distinte:

Fase 1: Inserimento delle Chiavi nel Trie L'inserimento di ogni singolo carattere richiede una ricerca ed eventuale inserimento nella tabella hash locale del nodo, con costo medio $\mathcal{O}(1)$. La costruzione completa di tutte le stringhe richiede pertanto tempo:

$$T_{\text{costruzione}} = \mathcal{O}(d) \tag{14.11}$$

Fase 2: Ordinamento degli Archi a Ciascun Nodo Mentre nell'array statico i figli sono naturalmente memorizzati in ordine crescente di indice da 0 a $\sigma - 1$, **la tabella hash non conserva alcun ordinamento intrinseco tra le proprie chiavi**. Di conseguenza, per estrarre le stringhe ordinate mediante visita anticipata, è indispensabile estrarre ed ordinare esplicitamente gli archi uscenti da ciascun nodo interno.

Teorema 14.3.1 (Costo di Ordinamento degli Archi nell'Hash Trie)**.** *L'ordinamento totale delle transizioni su tutti i nodi interni dell'Hash-Based Trie richiede complessivamente tempo:*

$$T_{\textit{ordinamento archi}} = \mathcal{O}(d \log_2 \sigma) \tag{14.12}$$

Dimostrazione. Sia u un generico nodo interno avente grado uscente $k_u \leq \sigma$. L'estrazione e l'ordinamento degli archi di u tramite un algoritmo di

comparazione standard (come un introsort o mergesort compatto) richiede tempo:

$$T(u) = \mathcal{O}(k_u \log_2 k_u) \tag{14.13}$$

Poiché il grado uscente è superiormente delimitato dalla cardinalità dell'alfabeto ($k_u \leq \sigma$), si ha la maggiorazione monotona $\log_2 k_u \leq \log_2 \sigma$. Sommando su tutti i nodi u dell'albero:

$$\begin{aligned} \sum_u k_u \log_2 k_u &\leq \sum_u k_u \log_2 \sigma \\ &= \log_2 \sigma \sum_u k_u \\ &= \mathcal{O}(d \log_2 \sigma) \end{aligned} \tag{14.14}$$

in quanto la somma cumulativa dei gradi uscenti $\sum_u k_u$ coincide con il numero totale di archi del Trie, delimitato superiormente da $\mathcal{O}(d)$. □

14.3.4 Quadro Comparativo delle Due Implementazioni

La Tabella 14.1 formalizza il compromesso computazionale fra l'implementazione su vettore statico e quella su mappa hash locale.

Metodologia di Rappresentazione	Complessità Spaziale	Tempo di Ordinamento (Sort)
Array-Based Trie	$\mathcal{O}(d \cdot \sigma)$	$\mathcal{O}(d \cdot \sigma)$
Hash-Based Trie	$\mathcal{O}(\mathbf{d})$	$\mathcal{O}(\mathbf{d} \log_{\mathbf{2}} \sigma)$

Tabella 14.1: Confronto di efficienza asintotica tra l'implementazione su array statico e l'implementazione su tabella hash locale per l'MSD Radix Sort.

14.4 Trie Compatti (Compacted Trie o Patricia Trie)

14.4.1 Motivazione: Eliminazione dei Nodi Unari (Unary Nodes)

Nel Trie convenzionale non compatto, l'esistenza di sequenze di caratteri prive di diramazioni alternative produce catene verticali allungate di nodi aventi un solo figlio:

Definizione 14.4.1 (Nodo Unario / Unary Node)**.** Un nodo interno u di un albero si dice **unario** se il suo grado uscente è esattamente pari a uno:

$$k_u = \text{out-degree}(u) = 1 \tag{14.15}$$

Questi nodi non svolgono alcuna funzione di partizionamento o discriminazione tra chiavi distinte: la loro unica funzione è quella di consumare sequenzialmente i caratteri del prefisso condiviso. L'architettura del **Trie Compatto** (introdotta da Donald R. Morrison con l'algoritmo **PATRICIA** [24]) risolve questa ridondanza attraverso il principio del *"dropping unary nodes"*:

- Ogni cammino lineare non ramificato composto da soli nodi unari viene contratto (*path compression*).
- Il cammino viene collassato in un **singolo arco composito**, la cui etichetta coincide con la concatenazione dei caratteri attraversati lungo la catena.

14.4.2 Esempio di Compattazione Passo per Passo

Si consideri un insieme didattico composto da $n = 4$ stringhe definite sull'alfabeto a 4 lettere $\Sigma = \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}\}$:

$$S = \{S_1 : \text{"ABA"},\ S_2 : \text{"ABBB"},\ S_3 : \text{"ABBD"},\ S_4 : \text{"BA"}\} \tag{14.16}$$

1. Struttura del Trie Non Compatto: Analizzando le diramazioni carattere per carattere si riscontrano plurimi nodi unari:

- La radice ramifica sui caratteri `'A'` e `'B'`.
- Dal ramo `'B'`, si genera un nodo unario che transita obbligatoriamente su `'A'` per concludere la foglia $S_4 = \text{"BA"}$.
- Dal ramo `'A'`, si genera un nodo unario che prosegue su `'B'`; da questo secondo nodo si dipartono due transizioni:
  - Il ramo `'A'` conclude la stringa $S_1 = \text{"ABA"}$.
  - Il ramo `'B'` conduce a un ulteriore nodo che infine discrimina tra `'B'` (stringa $S_2 = \text{"ABBB"}$) e `'D'` (stringa $S_3 = \text{"ABBD"}$).

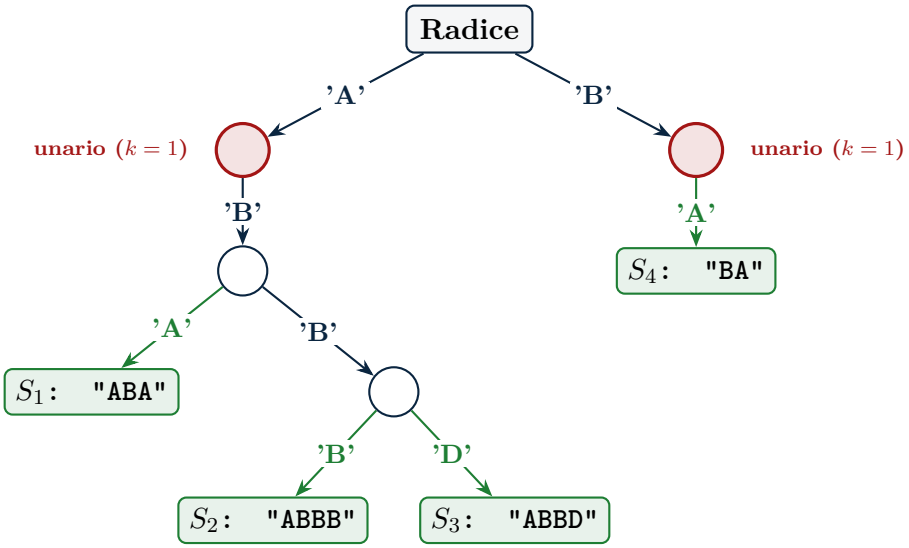


Figura 14.3: Trie non compatto per $S = \{\text{"ABA"}, \text{"ABBB"}, \text{"ABBD"}, \text{"BA"}\}$. I nodi evidenziati in rosso sono nodi unari privi di ramificazione ($k_u = 1$), destinati a essere eliminati nella variante compatta.

2. Contrazione e Collasso dei Nodi Unari: Eliminando i nodi unari intermedi ed etichettando ciascun arco con la concatenazione delle stringhe contratte:

- L'arco diretto dalla Radice alla foglia S_4 riceve l'etichetta contratta `"BA"`.
- L'arco dalla Radice al primo nodo ramificante (Nodo 2) assume l'etichetta `"AB"`.
- Dal Nodo 2:
  - Il ramo con etichetta `"A"` conduce direttamente a S_1.
  - Il ramo con etichetta `"B"` conduce al Nodo 3.
- Dal Nodo 3:
  - Il ramo con etichetta `"B"` conduce alla foglia S_2.
  - Il ramo con etichetta `"D"` conduce alla foglia S_3.

14.4.3 Rappresentazione ad Invarianza Spaziale tramite Indici

Memorizzare esplicitamente le stringhe contratte (`"AB"`, `"BA"`, ecc.) all'interno degli archi causerebbe un consumo addizionale di memoria e richiederebbe allocazioni dinamiche non predicibili. Per preservare un consumo di memoria rigidamente lineare, la letteratura algoritmica impiega una rappresentazione compatta basata su **puntatori al testo originario**: ciascun arco memorizza una tupla di tre valori interi:

$$\text{Arco} = \langle S_i,\ \text{start_index},\ \text{end_index} \rangle \tag{14.17}$$

dove:

- S_i è l'indice (o puntatore) a una qualsiasi stringa della collezione che attraversa quell'arco.
- start_index ed end_index delimitano la sottostringa lungo S_i che definisce l'etichetta dell'arco.

Con indicizzazione posizionale 1-based, gli archi del Trie compatto didattico sono specificati in modo analitico:

- Radice $\xrightarrow{\langle S_4,1,2 \rangle}$ S_4: identifica la sottostringa $S_4[1 \dots 2] =$ "BA".
- Radice $\xrightarrow{\langle S_1,1,2 \rangle}$ Nodo 2: identifica la sottostringa $S_1[1 \dots 2] =$ "AB".
- Nodo 2 $\xrightarrow{\langle S_1,3,3 \rangle}$ S_1: identifica la sottostringa $S_1[3 \dots 3] =$ "A".
- Nodo 2 $\xrightarrow{\langle S_2,3,3 \rangle}$ Nodo 3: identifica la sottostringa $S_2[3 \dots 3] =$ "B".
- Nodo 3 $\xrightarrow{\langle S_2,4,4 \rangle}$ S_2: identifica la sottostringa $S_2[4 \dots 4] =$ "B".
- Nodo 3 $\xrightarrow{\langle S_3,4,4 \rangle}$ S_3: identifica la sottostringa $S_3[4 \dots 4] =$ "D".

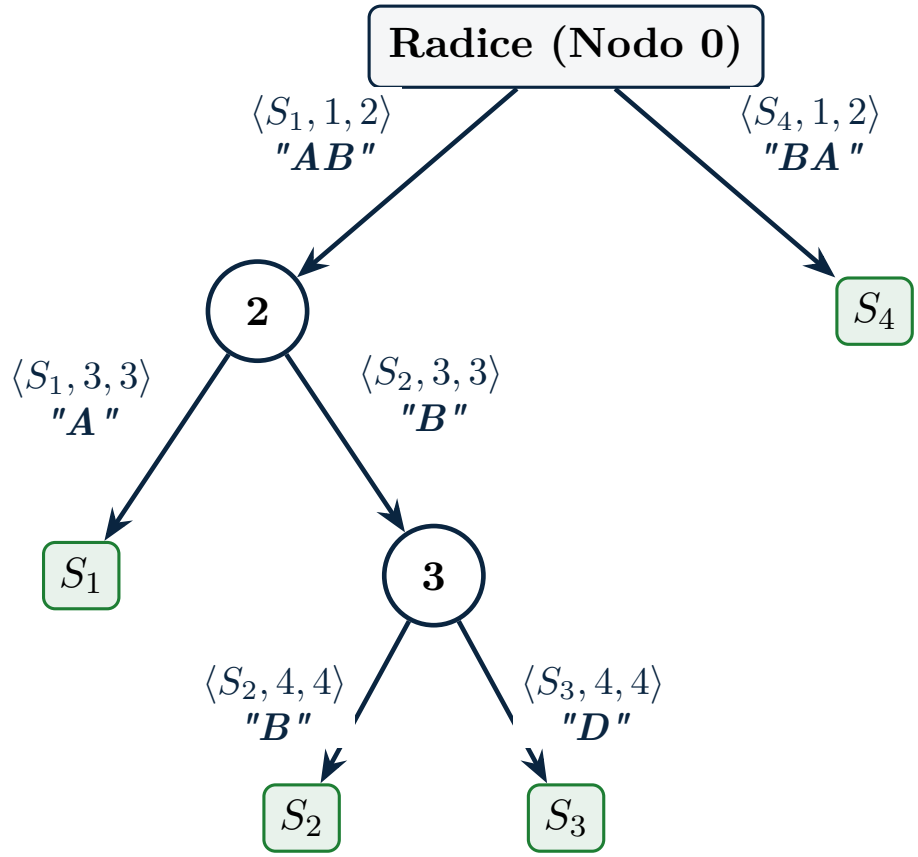


Figura 14.4: Topologia finale del Compacted Trie (Patricia Trie) con indicizzazione esatta delle sottostringhe tramite tuple $\langle S_i, \text{start}, \text{end} \rangle$. I nodi interni unari sono completamente rimossi, garantendo che ciascun nodo interno abbia grado ≥ 2.

14.4.4 Proprietà Strutturali e Analisi Asintotica

L'eliminazione dei nodi unari conferisce al Trie compatto proprietà combinatorie straordinariamente potenti:

Teorema 14.4.1 (Proprietà Topologiche del Compacted Trie)**.** *Sia data una collezione di $n = |S|$ stringhe distinte. Nel Trie compatto valgono rigorosamente le seguenti relazioni:*

1. ***Numero di Foglie:***
$$\#foglie = n \tag{14.18}$$
2. ***Numero di Nodi Interni:***
$$\#nodi\ interni \leq n - 1 \tag{14.19}$$

3. ***Numero Totale di Nodi:***

$$\#nodi\ totali \leq 2n - 1 \tag{14.20}$$

4. ***Numero Totale di Archi:***

$$\#archi\ totali \leq 2n - 2 \tag{14.21}$$

Dimostrazione. Ogni stringa distinta della collezione identifica in modo univoco una foglia dell'albero, per cui #foglie $= n$. Poiché tutti i nodi unari sono stati eliminati, ogni nodo interno u possiede obbligatoriamente un grado di ramificazione minimo pari a due ($k_u \geq 2$).

In qualunque albero ad arità almeno 2, il numero massimo di nodi interni si registra nella configurazione massimamente profonda, ovvero nell'albero strettamente binario (in cui ciascun nodo interno possiede esattamente 2 figli). In un albero binario con n foglie, il numero di nodi interni è noto essere esattamente pari a $n - 1$. Pertanto, in generale:

$$\#\text{nodi interni} \leq n - 1 \tag{14.22}$$

Sommando foglie e nodi interni:

$$\#\text{nodi totali} = \#\text{foglie} + \#\text{nodi interni} \leq n + (n - 1) = 2n - 1 \tag{14.23}$$

Infine, poiché in ogni grafo ad albero connesso con V vertici il numero di archi è esattamente $E = V - 1$:

$$\#\text{archi} = \#\text{nodi totali} - 1 \leq (2n - 1) - 1 = 2n - 2 \tag{14.24}$$

□

Corollario Fondamentale sull'Occupazione Spaziale: Poiché ciascun arco memorizza una tupla numerica di tre costanti intere $\langle S_i, \text{start}, \text{end} \rangle$ di dimensione fissa $\mathcal{O}(1)$:

$$\text{Spazio Globale}(n) = \mathcal{O}(n) \quad \text{parole di memoria} \tag{14.25}$$

La quantità di memoria impegnata da un Compacted Trie dipende **esclusivamente dal numero di stringhe indicizzate (n)**, risultando completamente indipendente dalla loro lunghezza cumulativa $d = \sum |s|$ e dalla dimensione dell'alfabeto σ.

Complessità della Ricerca di Pattern (Pattern Search): Dato un pattern arbitrario di interrogazione P:

- La navigazione scende lungo l'albero comparando i caratteri di P con i caratteri del testo referenziati dalla tupla dell'arco corrente.
- A ogni confronto positivo, l'indice di scansione sul pattern P avanza in avanti di una posizione.
- Pertanto, il tempo per validare la presenza o l'assenza del pattern è dominato linearmente dalla lunghezza del pattern stesso:

$$\text{Tempo di Ricerca} = \mathcal{O}(|P|) \tag{14.26}$$

14.5 Estensioni per Prefix Search con Frequenze (Autocompletamento)

14.5.1 Definizione Formale del Problema Top-1 Prefix Query

Nei moderni motori di ricerca, nelle piattaforme di e-commerce e negli ambienti di sviluppo integrati (IDE), il problema cardinale dell'autocompletamento algoritmico consiste nel suggerire all'utente la query o il simbolo più probabile durante la digitazione.

Formalmente, si consideri una collezione di n stringhe a ciascuna delle quali è associata una frequenza empirica d'uso o popolarità:

$$S = \{S_1, S_2, \ldots, S_n\}, \quad S_i \longleftrightarrow f_i \in \mathbb{R}^+ \quad \forall i \in \{1, \ldots, n\} \tag{14.27}$$

Definizione 14.5.1 (Query di Autocompletamento / Top-1 Prefix Query)**.** Data una sequenza di input parziale $P[1 \ldots q]$ (prefisso di interrogazione di lunghezza q), identificare in modo efficiente la stringa $S^* \in S$ che massimizza la frequenza di popolarità tra tutte quelle che ammettono P come prefisso iniziale:

$$S^* = \arg \max_{\substack{S_i \in S \\ P[1 \ldots q] \sqsubseteq S_i}} f_i \tag{14.28}$$

dove $P \sqsubseteq S_i$ denota che P è prefisso proprio o improprio di S_i.

Una soluzione ingenua che localizzi il sottoalbero associato al prefisso P per poi eseguire una visita esaustiva di tutte le foglie sottostanti richiederebbe tempo proporzionale al numero di elementi nel sottoalbero, introducendo una latenza inaccettabile per scenari interattivi in tempo reale.

14.5.2 Strategie Risolutive Accademiche

La teoria degli algoritmi introduce due paradigmi strutturali fondamentali per risolvere la Top-1 Prefix Query con latenza ottima:

1. Trie Arricchito con Massimi Precalcolati nei Nodi Interni

- **Modello dei Dati:** Ciascun nodo interno u del Trie viene arricchito con due campi informativi ausiliari precalcolati in fase di indicizzazione:

$$\text{max_freq}[u] = \max\{f_i \mid S_i \text{ è una foglia discendente di } u\} \quad (14.29)$$

  unitamente a un **puntatore diretto alla foglia** best_leaf$[u] = S^*$ che realizza tale valore massimo.

- **Elaborazione della Query:**

  1. Si effettua la consueta discesa lungo il Trie guidati dai q caratteri del prefisso P, con costo deterministico $\mathcal{O}(q) = \mathcal{O}(|P|)$.
  2. Raggiunto il nodo u_P corrispondente all'ultimo carattere di P, non è richiesta alcuna esplorazione del sottoalbero: **la risposta ottima S^* viene letta in tempo $\mathcal{O}(1)$** dereferenziando il puntatore precalcolato u_P.best_leaf.

- **Complessità Complessiva:** $\mathcal{O}(|P|)$ tempo per interrogazione e un overhead di memoria costante per nodo $\mathcal{O}(1)$ puntatori.

2. Treap (Albero Ibrido Trie-Heap)

- **Modello dei Dati:** Il Treap per stringhe [10] unifica la semantica lessicografica di un albero digitale di prefissi (chiavi primarie) con la proprietà strutturale di **Max-Heap** governata dalle frequenze f_i (chiavi di priorità).

- **Proprietà Strutturale:** Le chiavi caratterizzate da elevata frequenza d'accesso risalgono naturalmente verso i livelli più alti dell'albero mediante rotazioni bilanciate.

- **Ottimizzazione della Query:** Quando si naviga l'albero per soddisfare una query di autocompletamento (o per estrarre le prime k opzioni più frequenti, *Top-k Prefix Queries*), le priorità dello heap consentono un **pruning anticipato**: qualsiasi sottoalbero la cui priorità massima sia inferiore al miglior candidato già rilevato viene immediatamente potato senza essere visitato.

14.5.3 Quadro Sinottico Comparativo delle Strutture Dati per Stringhe

La Tabella 14.2 sintetizza le proprietà teoriche e pratiche delle principali architetture ad albero per la memorizzazione, ricerca e indicizzazione di collezioni di stringhe.

Struttura Dati	**Occupazione Spaziale**	**Tempo di Ricerca**	**Caratteristiche Distintive e Uso**
Standard Trie (Array)	$\mathcal{O}(d \cdot \sigma)$ (severo spreco per σ elevato)	$\mathcal{O}(\|P\|)$ (transizione deterministica $\mathcal{O}(1)$)	Alfabeti molto piccoli (es. DNA $\sigma = 4$) e insiemi densi
Hash-Based Trie	$\mathcal{O}(d)$ (svincolato da σ)	$\mathcal{O}(\|P\|)$ (tempo medio atteso)	Dizionari generici su alfabeti estesi; ordinamento archi in $\mathcal{O}(d \log_2 \sigma)$
Compacted Trie (Patricia)	$\mathcal{O}(\mathbf{n})$ (indipendente dalla lunghezza d)	$\mathcal{O}(\|P\|)$ (confronto su tuple $\langle S_i, s, e \rangle$)	Indicizzazione di testi lunghi, suffix trees e collezioni con lunghi prefissi
Ternary Search Tree (TST)	$\mathcal{O}(n)$ nodi (≈ 3 puntatori per nodo)	$\mathcal{O}(\|P\| + \log_2 n)$ medio ($\mathcal{O}(\|P\| \cdot \sigma)$ worst case)	Nessun overhead di hash, preserva l'ordine lessicografico totale nativo
Augmented Trie / Treap	$\mathcal{O}(n)$ oppure $\mathcal{O}(d)$	Top-1 Query: $\mathcal{O}(\|\mathbf{P}\|)$ (lookup $\mathcal{O}(1)$ alla fine del prefisso)	Motori di autocompletamento in tempo reale e query per frequenza

Tabella 14.2: Quadro sinottico comparativo delle strutture dati ad albero per la ricerca e indicizzazione di stringhe.

Regola di Selezione Architetturale per Stringhe

Nella progettazione ingegneristica di motori di ricerca e sistemi ad alte prestazioni:

1. Se la priorità è comprimere drasticamente il consumo di memoria indipendentemente dalla lunghezza delle stringhe (garantendo al contempo ricerche lineari in $\mathcal{O}(|P|)$), il **Compacted Trie** con indicizzazione a tuple rappresenta la soluzione teoricamente e praticamente ottimale.

2. Se il sistema richiede risposte immediate a query interattive di completamento su stringhe pesate, il **Trie arricchito con puntatori al massimo** risolve la Top-1 Prefix Query con costo $\mathcal{O}(|P|)$ senza dover esplorare i sottoalberi.

3. Se si richiede il supporto congiunto a ricerche lessicografiche per intervallo (*range queries*) senza le complicazioni e l'overhead delle tabelle hash, il **Ternary Search Tree (TST)** costituisce l'alternativa compatta preferibile.

4. Se infine la dimensione della collezione eccede la memoria primaria (modello a due livelli RAM–Disco), il Compacted Trie viene esteso nella variante lossy **Patricia / Blind Trie** combinata con la compressione su disco **Front-Coding** (Capitolo 15), garantendo ricerche ottimali con esattamente 1 I/O e occupazione in RAM pari a $\mathcal{O}(N/B)$ parole.

Capitolo 15

Algoritmi e Strutture Dati per la Ricerca di Stringhe in Memoria Gerarchica

15.1 Introduzione e Modelli di Calcolo per la Ricerca di Stringhe

Nel Capitolo 13 e nel Capitolo 14 sono state esaminate le strutture dati fondamentali per la gestione di collezioni di stringhe in memoria centrale (Ternary Search Tree, Trie standard e Trie compatti). In questo capitolo si affronta il problema della ricerca di stringhe quando la dimensione della collezione oltrepassa la capacità della memoria primaria (RAM), imponendo l'adozione di architetture a memoria gerarchica (modello a due livelli RAM–Disco). Si formalizzano la compressione su disco tramite **Front-Coding**, la progettazione di indici ultracompatti tramite **Patricia Trie (Blind Trie)** e l'algoritmo di ricerca ottima **Three-Phase Blind Search**, capace di risolvere qualsiasi interrogazione con esattamente 1 I/O su memoria secondaria [2, 25].

15.1.1 Il Problema del Dizionario di Stringhe

La gestione efficiente di un dizionario di stringhe rappresenta uno dei pilastri teorici e applicativi dell'Algorithm Engineering e dell'Information Retrieval [18].

Definizione 15.1.1 (Dizionario di Stringhe)**.** Sia dato un alfabeto finito e totalmente ordinato Σ di cardinalità $\sigma = |\Sigma|$, le cui lettere sono disposte secondo un ordinamento lessicografico convenzionale $<$. Si definisce **Dizionario**

di Stringhe un insieme finito:

$$\mathcal{D} = \{S_1, S_2, \ldots, S_N\} \subset \Sigma^* \tag{15.1}$$

composto da N sequenze distinte di caratteri (dette *chiavi*).

La dimensione totale della collezione in termini di caratteri complessivi memorizzati è definita come:

$$n = \sum_{i=1}^{N} |S_i| \tag{15.2}$$

dove $|S_i|$ indica la lunghezza della stringa i-esima.

Dato un generico pattern di ricerca $P \in \Sigma^*$ di lunghezza $p = |P|$, un'architettura di indicizzazione per dizionari deve supportare in modo efficiente le seguenti quattro classi di interrogazioni:

1. **Exact Search (Ricerca Esatta):** Determinare se la stringa P appartiene alla collezione ($P \in \mathcal{D}$). In caso affermativo, restituire il puntatore ai dati satellite o l'identificativo documentale (`docID`) associato a P.

2. **Prefix Search (Ricerca per Prefisso):** Individuare l'insieme di tutte le chiavi del dizionario $\mathcal{D}$ che ammettono il pattern P come prefisso:

   $$\text{Prefix}(P) = \{S \in \mathcal{D} \mid S[1 \ldots p] = P\} \tag{15.3}$$

   Come approfondito nel sezione 14.5, questa primitiva costituisce la base logica dei motori di autocompletamento in tempo reale.

3. **Lexicographic Search (Ricerca del Predecessore e Successore):** Nell'ipotesi in cui $\mathcal{D}$ sia ordinato lessicograficamente, individuare l'esatta posizione di collocazione di P rispetto alle chiavi memorizzate:

   $$\text{Pred}(P) = \max\{S \in \mathcal{D} \mid S < P\}, \quad \text{Succ}(P) = \min\{S \in \mathcal{D} \mid S \geq P\} \tag{15.4}$$

   Tale operazione consente l'inserimento dinamico coerente e la gestione di interrogazioni approssimate.

4. **Range Search (Ricerca per Intervallo):** Dati due pattern delimitatori $P_1, P_2 \in \Sigma^*$ tali che $P_1 \leq P_2$, identificare tutte le stringhe di $\mathcal{D}$ lessicograficamente comprese nell'intervallo chiuso:

   $$\text{Range}(P_1, P_2) = \{S \in \mathcal{D} \mid P_1 \leq S \leq P_2\} \tag{15.5}$$

Proprietà 15.1.1 (Equivalenza tra Prefix Search e Range Search)**.** *Poiché tutte le stringhe aventi un prefisso comune P occupano un segmento contiguo nell'ordinamento lessicografico di $\mathcal{D}$, l'operazione di Prefix Search coincide con una Range Search compresa tra P e la stringa estesa $P \circ \#$:*

$$Prefix(P) = Range(P,\ P \circ \#) \tag{15.6}$$

dove $\#$ è un carattere sentinella convenzionale posizionato strettamente oltre l'ultimo carattere dell'alfabeto ($\forall c \in \Sigma,\ c <_{lex} \#$).

15.1.2 Limiti del Modello a Memoria Singola

Nel modello di calcolo standard di Von Neumann (la macchina RAM analizzata nel sezione 1.3), si assume che l'intero dizionario $\mathcal{D}$ risieda in una memoria centrale omogenea ad accesso casuale con costo unitario $\mathcal{O}(1)$ per parola macchina.

Come documentato nel Capitolo 13 e nel Capitolo 14, le strutture ad albero progettate per questo paradigma mostrano caratteristiche distinte:

- **Ternary Search Tree (TST):** Coniuga la flessibilità degli alberi binari di ricerca con la scomposizione per caratteri dei trie (sezione 13.2), raggiungendo un tempo medio di ricerca pari a $\mathcal{O}(p+\ln N)$ con consumo spaziale $\mathcal{O}(n)$.
- **Trie Standard (Non Compatto):** Offre una ricerca ottima nel caso peggiore in tempo $\mathcal{O}(p)$, ma soffre di un'elevata dispersione di memoria dovuta alle catene di nodi unari privi di ramificazioni, richiedendo uno spazio complessivo pari a $\Theta(n \cdot \sigma)$ o $\Theta(n)$ (sezione 14.2 e sezione 14.3).
- **Compacted Trie (Trie Compatto):** Elimina i nodi unari comprimendo i cammini lineari in archi compositi (sezione 14.4). Il numero di nodi interni scende rigorosamente sotto N, garantendo tempo di discesa $\mathcal{O}(p)$ e spazio per puntatori limitato a $\mathcal{O}(N)$.

Il Collasso Prestazionale del 1-Memory Model

Quando il numero di chiavi N o la dimensione lessicale n oltrepassano la capacità fisica della RAM (M), l'astrazione a memoria singola collassa. La navigazione su alberi di puntatori digitali (pointer chasing) costringe la CPU ad accedere a indirizzi non contigui e casuali nello spazio di memoria virtuale. Come dimostrato analiticamente nel sottosezione 2.4.1 e nel sottosezione 2.4.2, ogni dereferenziazione di puntatore in un albero allocato su disco genera un **page fault**, con tempi di latenza nell'ordine dei millisecondi. Il sistema entra in una condizione di *thrashing*: la CPU trascorre oltre il 99% del tempo nell'attesa dell'I/O meccanico o a stato solido.

15.1.3 Il Modello a Due Livelli di Memoria (Two-Level Memory Hierarchy)

Per neutralizzare il costo dei continui trasferimenti disco–RAM, si adotta il modello **External Memory** formalizzato da Aggarwal e Vitter [2] (introdotto nel sezione 1.4):

- M: dimensione della memoria interna (RAM) misurata in byte o parole macchina.
- B: dimensione del blocco di trasferimento del disco (capacità di una pagina fisica I/O).
- N: numero totale di chiavi nel dizionario $\mathcal{D}$.

Il dizionario viene partizionato su disco in blocchi ordinati lessicograficamente. Il numero di blocchi risultante è:

$$K = \left\lceil \frac{N}{B} \right\rceil \tag{15.7}$$

Nel paradigma a due livelli (**Two-Level Approach**):

1. **Su Disco (Memoria Secondaria):** Le stringhe appartenenti a ciascun blocco sono ordinate alfabeticamente e compresse tramite la tecnica del **Front-Coding**, eliminando la ridondanza dei prefissi comuni e massimizzando il fattore di riempimento B.
2. **In Memoria Primaria (RAM):** Risiede un indice ultracompatto costruito unicamente sulle **stringhe rappresentative** di ciascun blocco (la prima stringa di ogni pagina disco, $R_1, R_2, \ldots, R_K$). L'indice

identifica in memoria centrale l'unico blocco candidato su disco con costo computazionale $\mathcal{O}(p)$.

3. **Costo di I/O Ottimale:** La ricerca del dato su disco richiede **esattamente 1 I/O**, poiché la navigazione avviene interamente in RAM e richiede un solo accesso mirato alla pagina candidata.

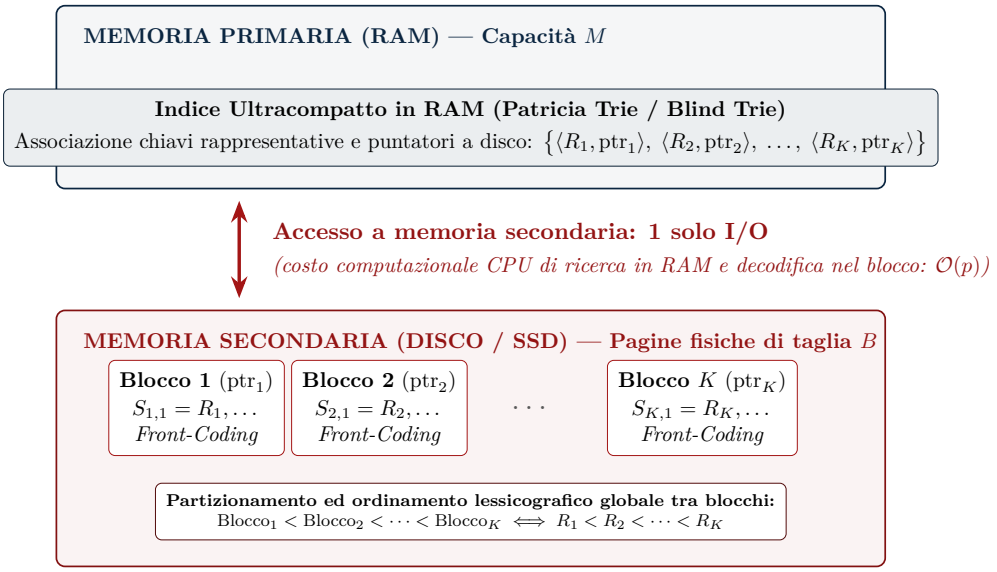


Figura 15.1: Architettura a due livelli per la ricerca di stringhe: l'indice compatto in RAM associa a ciascuna chiave rappresentativa il puntatore al rispettivo blocco su disco ($\langle R_i, \mathrm{ptr}_i\rangle$). La ricerca instrada la query con costo computazionale CPU $\mathcal{O}(p)$ ed effettua un unico accesso I/O su memoria secondaria verso il blocco compresso con Front-Coding.

15.2 Organizzazione e Compressione su Disco: Front-Coding

Nei sistemi su memoria secondaria, il collo di bottiglia primario è determinato dal numero di byte trasferiti attraverso il canale I/O. L'applicazione della compressione incrementale sui blocchi disco consente di compattare drasticamente le chiavi testuali.

15.2.1 Definizione Formale e Algoritmo di Codifica

Il **Front-Coding** (denominato anche *differential prefix compression* o compressione per prefissi incrementali) è una tecnica di compressione *lossless* specificamente ideata per insiemi di stringhe disposti in ordine lessicografico. Essa si fonda sull'osservazione empirica e analitica che stringhe alfabeticamente consecutive condividono lunghi prefissi comuni.

Definizione 15.2.1 (Longest Common Prefix (LCP))**.** Date due stringhe $A, B \in \Sigma^*$, si definisce la funzione **LCP** come la lunghezza del più lungo prefisso identico condiviso:

$$\mathrm{LCP}(A, B) = \max\left\{k \geq 0 \;\middle|\; A[1 \ldots k] = B[1 \ldots k]\right\} \tag{15.8}$$

Sia $D_{\text{block}} = \{S_1, S_2, \ldots, S_m\}$ l'insieme delle m stringhe assegnate a un singolo blocco fisico di disco, con $S_1 < S_2 < \cdots < S_m$. La procedura di codifica trasforma ogni stringa S_i in una coppia ordinata:

$$C(S_i) = \langle \ell_i,\ \hat{S}_i \rangle \tag{15.9}$$

secondo le seguenti regole:

- **Stringa di ancoraggio (S_1):** La prima stringa del blocco funge da riferimento assoluto. Non avendo predecessori nel blocco, la lunghezza del prefisso condiviso è posta convenzionalmente a zero, e la stringa viene registrata integralmente:

$$C(S_1) = \langle 0,\ S_1 \rangle \tag{15.10}$$

- **Stringhe successive (S_i per $i = 2, \ldots, m$):** Per ciascuna stringa S_i, si calcola la lunghezza del massimo prefisso comune rispetto alla stringa immediatamente precedente S_{i-1}:

$$\ell_i = \mathrm{LCP}(S_{i-1}, S_i) \tag{15.11}$$

  La rappresentazione memorizza unicamente l'intero scalare ℓ_i seguito dal suffisso residuo $\hat{S}_i$, privato dei primi ℓ_i caratteri:

$$\hat{S}_i = S_i[\ell_i + 1 \ldots |S_i|] \tag{15.12}$$

Procedura di Ricostruzione Sequenziale. Data la sequenza compressa $\left[\langle \ell_1, \hat{S}_1 \rangle, \ldots, \langle \ell_m, \hat{S}_m \rangle\right]$, la ricostruzione della generica stringa S_i in memoria avviene ricorsivamente:

$$S_i = S_{i-1}[1 \ldots \ell_i] \circ \hat{S}_i \tag{15.13}$$

dove $\circ$ denota l'operatore di concatenazione di stringhe.

15.2.2 Trade-off Ingegneristico: Compressione vs. Accesso Sequenziale

L'adozione del Front-Coding introduce un classico compromesso architetturale:

- **Vantaggi Strutturali:** La rimozione dei prefissi ridondanti comprime il volume delle stringhe fino al $70\% - 80\%$ della taglia originale. Ciò incrementa il numero di elementi m ospitabili all'interno di una pagina da B byte e riduce direttamente il numero totale di blocchi $K = \lceil N/B \rceil$, minimizzando l'impronta su disco.
- **Dipendenza Sequenziale:** Per ottenere S_i non è possibile effettuare un accesso casuale diretto alla posizione i-esima nel blocco, poiché ciascuna chiave dipende dai prefissi delle stringhe precedenti fino all'ancoraggio S_1.

Risoluzione del Trade-off nel Modello I/O

Sebbene l'accesso casuale intra-blocco sia precluso, nel modello gerarchico l'intero blocco di taglia B (es. $4\text{ KB} - 64\text{ KB}$) viene caricato in RAM con **1 solo I/O**. La decodifica sequenziale di alcune decine o centinaia di stringhe compresse all'interno della memoria primaria richiede pochi microsecondi di calcolo della CPU, un costo infinitamente inferiore rispetto alla latenza meccanica o di bus dell'I/O su disco (nell'ordine dei millisecondi). Pertanto, il Front-Coding massimizza l'efficienza complessiva del sistema.

15.2.3 Esempio Guidato di Front-Coding

Si consideri il sottoinsieme ordinato di quattro chiavi binarie:

$$S_1 = 0001100, \quad S_2 = 0001110, \quad S_3 = 010000, \quad S_4 = 100\ldots \tag{15.14}$$

Eseguiamo la derivazione analitica della sequenza compressa:

1. **Codifica di $S_1 = 0001100$:**
   - Prima stringa del blocco: $\ell_1 = 0$.
   - Suffisso residuo: $\hat{S}_1 = S_1 = 0001100$.
   - Tupla registrata: $\langle 0,\ 0001100 \rangle$.
2. **Codifica di $S_2 = 0001110$:**

- Confronto tra $S_1 = \mathbf{0001}00$ e $S_2 = \mathbf{0001}10$.
- I primi 5 bit coincidono (00011), il sesto diverge ($0 \neq 1$): $\ell_2 = 5$.
- Suffisso residuo: $\hat{S}_2 = S_2[6 \dots 7] = 10$.
- Tupla registrata: $\langle 5,\ 10 \rangle$.

3. **Codifica di $S_3 = 010000$:**
   - Confronto tra $S_2 = \mathbf{0}001110$ e $S_3 = \mathbf{0}10000$.
   - Solamente il primo bit coincide (0): $\ell_3 = 1$.
   - Suffisso residuo: $\hat{S}_3 = S_3[2 \dots 6] = 10000$.
   - Tupla registrata: $\langle 1,\ 10000 \rangle$.
4. **Codifica di $S_4 = 100 \dots$:**
   - Confronto tra $S_3 = 010000$ e $S_4 = 100 \dots$.
   - Il primo bit diverge immediatamente ($0 \neq 1$): $\ell_4 = 0$.
   - Suffisso residuo: $\hat{S}_4 = 100 \dots$.
   - Tupla registrata: $\langle 0,\ 100 \dots \rangle$.

Il blocco su memoria secondaria registra compattamente la sequenza di tuple:

$$\mathcal{B}_{\text{disk}} = \Big[\langle 0,\ 0001100 \rangle,\ \langle 5,\ 10 \rangle,\ \langle 1,\ 10000 \rangle,\ \langle 0,\ 100 \dots \rangle \Big] \tag{15.15}$$

15.3 L'Architettura a Due Livelli e il Compacted Trie come Indice Primario

15.3.1 Selezione dei Rappresentanti e Instradamento delle Query

Sia data una collezione $\mathcal{D}$ partizionata su disco in $K = \lceil N/B \rceil$ blocchi. Per ciascun blocco j ($j = 1, \dots, K$), si estrae la prima stringa $R_j = S_{j,1}$, definita **stringa rappresentativa** del blocco j. Poiché le chiavi sono ordinate globalmente tra i blocchi:

$$R_1 < R_2 < \cdots < R_K \tag{15.16}$$

Proposizione 15.3.1 (Instradamento Univoco tramite Predecessore)**.** *Dato un generico pattern di ricerca $P \in \Sigma^*$, se $P \in \mathcal{D}$ allora P deve appartenere necessariamente all'unico blocco j per cui vale la condizione di contenimento:*

$$R_j \leq P < R_{j+1} \tag{15.17}$$

(ponendo convenzionalmente $R_{K+1} = \infty$). Di conseguenza, l'operazione di instradamento in RAM equivale alla ricerca del ***predecessore lessicografico*** *di P all'interno dell'insieme ridotto dei rappresentanti $\mathcal{R} = \{R_1, R_2, \ldots, R_K\}$.*

15.3.2 Analisi della Complessità con Compacted Trie Standard e Limiti di Memoria

Se implementiamo l'indice in memoria primaria mediante un **Compacted Trie** convenzionale (come descritto nel sezione 14.4):

- **Costo di Navigazione in RAM:** La ricerca del predecessore richiede al più $\mathcal{O}(p)$ confronti di caratteri, scendendo lungo l'albero digitale.
- **Costo di I/O:** Esattamente **1 I/O**, necessario per caricare il blocco disco individuato dalla foglia raggiunta.
- **Occupazione Spaziale in RAM:**
  - Numero di foglie: esattamente $K = \lceil N/B \rceil$.
  - Numero di nodi interni: al più $K - 1 = \mathcal{O}(N/B)$.
  - Spazio per etichette degli archi: nel Compacted Trie ordinario, ogni arco memorizza la sottostringa contratta o una tupla di indici $\langle S_i, \text{start}, \text{end} \rangle$ (sottosezione 14.4.3). Per poter confrontare i caratteri senza effettuare accessi a disco durante la discesa, tali sottostringhe devono essere conservate o duplicate in memoria primaria.

Lo spazio complessivo occupato in RAM dal Compacted Trie standard risulta:

$$\text{Spazio}_{\text{RAM}} = \mathcal{O}\left(\frac{N}{B}\right) \text{ puntatori} + \mathcal{O}\left(\sum_{j=1}^{K} |R_j|\right) \text{ caratteri} \tag{15.18}$$

Il Collo di Bottiglia delle Chiavi Lunghe

Se le stringhe del dizionario sono lunghe (come nel caso di URL completi, percorsi di file di sistema, annotazioni semantiche o sequenze genomiche), il termine $\sum_{j=1}^{K} |R_j|$ cresce arbitrariamente. L'indice dei rappresentanti finisce per saturare la memoria primaria RAM, rendendo insostenibile l'approccio per dizionari massivi su hardware convenzionale.

15.4 La Patricia Trie (Blind Trie) e la Compressione Lossy

Per abbattere l'occupazione in RAM a una quantità rigorosamente proporzionale al numero di blocchi $\mathcal{O}(N/B)$, svincolandola completamente dalla lunghezza delle stringhe e del pattern, Donald Morrison [24] e successivamente Ferragina e Grossi [25] hanno introdotto la variante denominata **Blind Trie** (o *Patricia Trie* a compressione lossy).

15.4.1 Definizione Formale e Compressione Lossy

La Patricia Trie applica una **compressione con perdita (lossy)** alla topologia dell'albero digitale:

1. **Eliminazione Totale delle Sottostringhe:** Sugli archi non viene memorizzata alcuna sottostringa contratta, né viene memorizzato alcun puntatore a testo in RAM.

2. **Carattere Singolo di Biforcazione:** Ciascun arco uscente da un nodo interno memorizza esclusivamente il **singolo carattere** dell'alfabeto che discrimina la diramazione.

3. **Valore di Salto (*Skip Value*):** Ogni nodo interno u memorizza unicamente un singolo valore scalare intero:

$$\text{skip}(u) \in \{1, 2, \dots\} \tag{15.19}$$

   che specifica la **posizione assoluta** (1-based) all'interno del pattern da testare per scegliere quale arco uscente intraprendere.

4. **Cecità Rispetto ai Caratteri Intermedi:** La memoria primaria non conserva alcun carattere intermedio compreso tra due nodi di salto consecutivi. L'albero "ignora" cosa sia presente nelle posizioni intermedie, da cui la denominazione di *Blind Trie* (Trie Cieco).

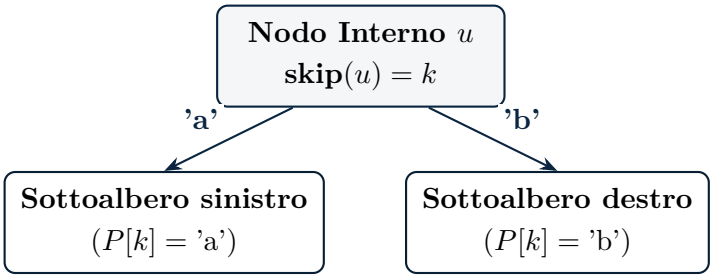


Figura 15.2: Anatomia di un nodo interno nella Patricia Trie (Blind Trie): la decisione di instradamento si fonda unicamente sull'ispezione del singolo carattere alla posizione $k = \text{skip}(u)$, senza consultare né memorizzare il testo intermedio.

15.4.2 Analisi Spaziale Rigorosa in Memoria Primaria

La struttura combinatoria della Patricia Trie assicura proprietà di compattezza estreme:

Teorema 15.4.1 (Invarianza Spaziale della Patricia Trie)**.** *Sia dato un dizionario partizionato in $K = \lceil N/B \rceil$ blocchi. La Patricia Trie costruita sulle stringhe rappresentative soddisfa i seguenti vincoli di complessità:*

1. *Il numero di nodi foglia è esattamente pari a K:*
$$N_{foglie} = K = \left\lceil \frac{N}{B} \right\rceil \tag{15.20}$$
2. *Poiché ogni nodo interno è almeno binario (out-degree ≥ 2), il numero totale di nodi interni è strettamente limitato da:*
$$N_{interni} \leq K - 1 = \left\lceil \frac{N}{B} \right\rceil - 1 \tag{15.21}$$
3. *Lo spazio totale occupato in memoria primaria (RAM) è pari a:*
$$Spazio_{RAM} = \mathcal{O}(N_{interni} + N_{foglie}) = \mathcal{O}\left(\frac{N}{B}\right) \text{ parole macchina} \tag{15.22}$$

*Tale consumo è rigorosamente **indipendente** dalla lunghezza delle stringhe rappresentative $|R_j|$ e dalla lunghezza del pattern di ricerca $p = |P|$.*

Poiché la compressione è lossy, le informazioni contenute nell'indice in RAM non sono sufficienti per certificare l'appartenenza o la corretta posizione di P senza consultare la stringa reale archiviata su disco. Per risolvere questa ambiguità si rende necessario uno specifico algoritmo di discesa e verifica.

15.5 L'Algoritmo di Ricerca a Tre Fasi (Three-Phase Blind Search)

A causa dell'assenza dei caratteri saltati, la ricerca non può operare come in un trie standard. L'algoritmo **Three-Phase Blind Search** si articola rigorosamente in tre fasi sequenziali coordinate, dettagliate nei paragrafi seguenti e formalizzate nell'algorithm 20:

15.5.1 Fase 1: Discesa Cieca (Downward Traversal)

La prima fase opera interamente all'interno della memoria centrale (RAM) partendo dalla radice della Patricia Trie:

1. Per ogni nodo interno incontrato u, si legge l'indice di salto $k = \text{skip}(u)$.
2. Se $k \leq |P|$, si estrae il carattere $c = P[k]$.
3. Se esiste un arco uscente da u etichettato con il carattere c, la navigazione scende lungo tale arco verso il nodo figlio corrispondente.

Condizioni di Arresto della Discesa. La discesa in RAM si interrompe quando si verifica una delle seguenti tre condizioni:

1. $k > |P|$: il pattern P è più corto dell'indice di salto memorizzato nel nodo (il carattere $P[k]$ non esiste).
2. **Mismatch di ramo (Branching Mismatch):** il nodo u non possiede alcun arco uscente etichettato con il carattere $c = P[k]$.
3. **Raggiungimento di una foglia:** la navigazione giunge con successo a un nodo foglia dell'albero.

Definizione 15.5.1 (Regola di Selezione della Foglia Arbitraria)**.** Nel momento in cui la discesa cieca si arresta (per esaurimento della stringa o per branching mismatch presso un nodo interno), l'algoritmo seleziona una **qualsiasi foglia discendente** appartenente al sottoalbero radicato nel punto di arresto (per convenzione implementativa, la foglia più a sinistra raggiungibile seguendo sempre il primo arco disponibile). Sia L la foglia individuata e sia R la stringa rappresentativa associata.

Costo di I/O: 0 I/O su memoria secondaria.

15.5.2 Fase 2: Verifica Esterna e Calcolo dell'LCP

Poiché la discesa ha saltato il controllo di tutti i caratteri intermedi, la stringa R associata a L è unicamente una chiave candidata. Non vi è alcuna garanzia a priori che P condivida con R i caratteri posizionati tra gli indici di salto ispezionati.

Esecuzione dell'Accesso su Disco.

1. Si esegue un accesso I/O a memoria secondaria per caricare il blocco disco puntato da L.
2. Si estrae in RAM la stringa rappresentativa completa R.
3. Si esegue un confronto esaustivo carattere a carattere da sinistra verso destra tra P ed R, calcolando la lunghezza del massimo prefisso comune effettivo:
$$\ell = \mathrm{LCP}(P, R) \tag{15.23}$$

Esiti del Confronto:

- Se $\ell = |P| = |R|$: abbiamo identificato un **match esatto** ($P = R$).
- Se $\ell = |P| < |R|$: il pattern P è un **prefisso proprio** di R.
- Se $\ell < |P|$: si manifesta una **divergenza reale** alla posizione $\ell + 1$, tale che:
$$P[\ell + 1] \neq R[\ell + 1] \tag{15.24}$$

Costo di I/O: Esattamente **1** I/O. Il calcolo dell'LCP in memoria richiede $\mathcal{O}(p)$ operazioni elementari.

15.5.3 Fase 3: Risalita e Localizzazione Corretta (Upward Traversal)

Qualora nella Fase 2 si riscontri $\ell < |P|$ (oppure si stia ricercando il predecessore lessicografico o il range), la foglia L identificata dalla discesa cieca potrebbe trovarsi in un sottoalbero errato. Tuttavia, l'indice del massimo prefisso comune reale ℓ fornisce l'informazione matematica esatta sul punto in cui il pattern P diverge dall'insieme delle stringhe memorizzate.

Procedura di Risalita in RAM. Partendo dalla foglia L, l'algoritmo risale l'albero verso la radice utilizzando i puntatori al nodo genitore, arrestandosi al primo nodo antenato v che soddisfa la disuguaglianza:

$$\text{skip}(v) \leq \ell + 1 \tag{15.25}$$

Il nodo v così individuato rappresenta il **locus di divergenza reale**:

- **Caso 1: skip$(v) = \ell + 1$.**
  Il nodo v possiede uno skip che discrimina esattamente il carattere divergente. L'algoritmo esamina gli archi uscenti da v:
  - Se esiste un arco etichettato con il carattere $P[\ell + 1]$, il pattern P deve essere ricercato nel sottoalbero radicato lungo tale arco.
  - Se non esiste alcun arco etichettato con $P[\ell+1]$, la posizione ordinale di $P[\ell+1]$ rispetto alle etichette degli archi uscenti da v individua inequivocabilmente il predecessore e il successore lessicografico di P.
- **Caso 2: skip$(v) \leq \ell$.**
  Ciò implica che nella topologia dell'albero non esiste alcun nodo interno con indice di salto pari a $\ell + 1$. Il punto di separazione ricade lungo l'arco virtualmente compresso tra v e il suo figlio lungo il cammino verso L. In tal caso:
  - Se $P[\ell+1] < R[\ell+1]$, tutte le chiavi del sottoalbero radicato sotto l'arco verso L sono strettamente maggiori di P. Di conseguenza, P precede l'intero blocco di R.
  - Se $P[\ell + 1] > R[\ell + 1]$, P segue lessicograficamente tutte le chiavi che condividono il prefisso fino ad ℓ.

Costo di I/O: 0 I/O (la risalita si svolge interamente nella struttura ad albero memorizzata in RAM).

15.5.4 Formalizzazione Algoritmica Completa

L'intera logica di ricerca è formalizzata nell'Algoritmo 20.

Algorithm 20: Three-Phase Blind Search

Dati: Radice della Patricia Trie T_{root} in RAM, Pattern di ricerca $P[1 \dots p]$

Risultato: Blocco disco contenente P oppure coppia $\langle \text{Pred}(P), \text{Succ}(P) \rangle$

`// FASE 1: Discesa Cieca in RAM (0 I/O)`

$u \leftarrow T_{\text{root}}$
while u **è un nodo interno do**
  $k \leftarrow \text{skip}(u)$
  if $k > p$ **oppure** non esiste arco uscente da u etichettato con $P[k]$ **then**
    break
    `// Arresto per fine stringa o mismatch di ramo`
  $u \leftarrow$ figlio di u corrispondente all'arco con etichetta $P[k]$
$L \leftarrow \text{SelezionaFogliaArbitraria}(u)$
$R \leftarrow$ Rappresentante associato alla foglia L

`// FASE 2: Verifica Esterna su Disco (1 I/O)`

$\text{CaricaBloccoDaDisco}(L)$
`// Unico accesso I/O su memoria secondaria`
$\ell \leftarrow \text{LCP}(P, R)$
if $\ell = p = |R|$ **then**
  return Match Esatto nel Blocco di R

`// FASE 3: Risalita e Localizzazione in RAM (0 I/O)`

$v \leftarrow L$
while $v \neq T_{root}$ **e** $skip(padre(v)) > \ell + 1$ **do**
  $v \leftarrow \text{padre}(v)$
$v_{\text{div}} \leftarrow \text{padre}(v)$
if $skip(v_{div}) = \ell + 1$ **then**
  if esiste arco uscente da v_{div} con etichetta $P[\ell + 1]$ **then**
    return Sottoalbero identificato dall'arco $P[\ell + 1]$
return Predecessore/Successore determinato dalla posizione di $P[\ell + 1]$

15.6 Esempi Applicativi Completi e Simulazione Passo-Passo

15.6.1 Esempio 1: Dizionario Binario a 3 Blocchi

Si consideri un dizionario binario $\mathcal{D}$ composto da $N = 6$ chiavi con capacità di blocco disco $B = 2$:

$$\mathcal{D} = \{0001100,\ 0001110,\ 0010,\ 0011,\ 101,\ 111\} \tag{15.26}$$

Il numero di blocchi risultante è $K = \frac{N}{B} = \frac{6}{2} = 3$. I blocchi su disco contengono:

- **Blocco 1:** $\{0001100, 0001110\}$
  - Front-Coding: $\langle 0,\ 0001100\rangle,\ \langle 5,\ 10\rangle$.
  - Rappresentante: $R_1 = \mathbf{0001100}$.
- **Blocco 2:** $\{0010, 0011\}$
  - Front-Coding: $\langle 0,\ 0010\rangle,\ \langle 3,\ 1\rangle$.
  - Rappresentante: $R_2 = \mathbf{0010}$.
- **Blocco 3:** $\{101, 111\}$
  - Front-Coding: $\langle 0,\ 101\rangle,\ \langle 1,\ 11\rangle$.
  - Rappresentante: $R_3 = \mathbf{101}$.

Topologia della Patricia Trie sui Rappresentanti. L'indice residente in RAM indicizza le tre stringhe rappresentative $\{R_1, R_2, R_3\}$ e memorizza i puntatori ai corrispondenti blocchi disco:

1. R_1 ed R_2 iniziano per '0', mentre R_3 inizia per '1'. La prima divergenza si manifesta alla posizione 1:

$$\text{skip}(\text{Radice}) = 1 \tag{15.27}$$

   L'arco etichettato '0' punta al nodo interno u; l'arco etichettato '1' conduce direttamente alla foglia del Blocco 3 ($\langle R_3, \text{ptr}_3\rangle$, con primo bit discriminante 1).

2. Tra $R_1 = \texttt{0001100}$ ed $R_2 = \texttt{0010}$, i primi 2 bit coincidono (00), mentre il primo bit discriminante è il terzo:

$$\text{skip}(u) = 3 \tag{15.28}$$

L'arco etichettato '0' conduce alla foglia del Blocco 1 ($\langle R_1, \text{ptr}_1 \rangle$, con terzo bit discriminante pari a **0**); l'arco '1' conduce alla foglia del Blocco 2 ($\langle R_2, \text{ptr}_2 \rangle$, con terzo bit discriminante pari a **1**).

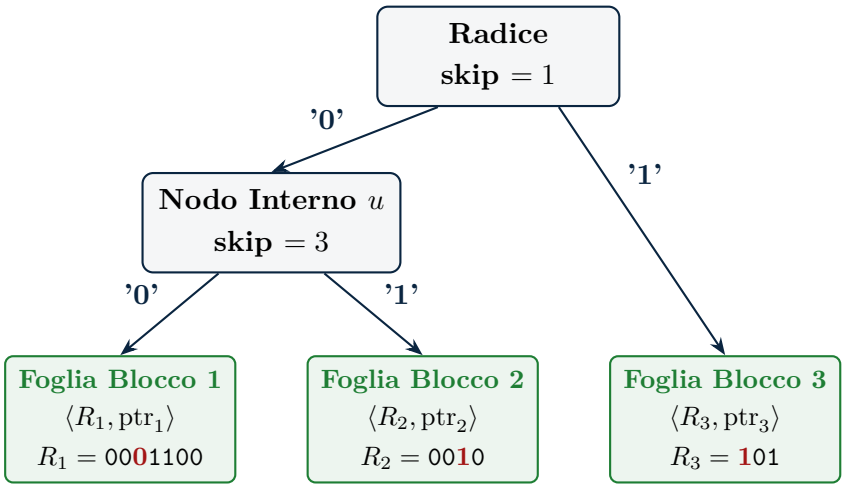


Figura 15.3: Patricia Trie (Blind Trie) costruita sui rappresentanti $\langle R_i, \text{ptr}_i \rangle$: i nodi interni memorizzano unicamente gli indici di salto (skip $\in \{1, 3\}$), mentre tutte le foglie sono distribuite in modo perfettamente uniforme e allineate sullo strato di base con il rispettivo bit discriminante evidenziato in rosso (**1** al 1° bit per R_3 testato dalla radice; **0** e **1** al 3° bit per R_1 e R_2 testati dal nodo u).

Tracciamento della Query: Ricerca di $P_1 = 01000$.

1. **Fase 1 (Discesa Cieca in RAM):**
   - Radice (skip = 1): si controlla $P_1[1] = 0$. Esiste l'arco '0', si scende verso il nodo u.
   - Nodo u (skip = 3): si controlla $P_1[3] = 0$. Esiste l'arco '0', si scende verso la **Foglia per Blocco 1**.
   - **Osservazione analitica:** Durante la discesa sono stati ispezionati unicamente i caratteri alle posizioni 1 e 3. Il carattere $P_1[2] = 1$ è stato ignorato dalla navigazione cieca!

2. **Fase 2 (Verifica Esterna su Disco):**

   - Si effettua **1 I/O** caricando da disco il Blocco 1 ed estraendo la stringa rappresentativa $R_1 = 0001100$.
   - Confronto esaustivo:

     $$P_1 = 0\,\mathbf{1}\,000$$
     $$R_1 = 0\,\mathbf{0}\,01100$$

     Posizione 1: $0 == 0$ (Match). Posizione 2: $1 \neq 0$ (**Mismatch**).
   - Si ottiene:

     $$\ell = \mathrm{LCP}(P_1, R_1) = 1 \tag{15.29}$$

     Il primo carattere divergente reale è situato alla posizione $\ell + 1 = 2$.

3. **Fase 3 (Risalita e Risoluzione in RAM):**

   - Dalla Foglia del Blocco 1 si risale verso l'alto.
   - Il nodo padre u possiede $\mathrm{skip}(u) = 3$. Poiché $3 > \ell + 1$ $(3 > 2)$, il punto di divergenza reale è situato superiormente rispetto a u.
   - Si risale alla Radice, che ha skip $= 1 \leq 2$.
   - La separazione reale si colloca lungo l'arco compresso compreso tra la Radice e il nodo u.
   - Poiché alla posizione 2 il pattern presenta $P_1[2] = 1$, mentre tutte le stringhe nel sottoalbero di u presentano bit 2 pari a 0 $(R_1[2] = 0, R_2[2] = 0)$, deduciamo lessicograficamente che:

     $$R_1 < R_2 < P_1 < R_3 \tag{15.30}$$

   - La ricerca conclude con successo che $P_1 \notin \mathcal{D}$ e colloca il predecessore nel Blocco 2 e il successore nel Blocco 3, con un costo globale di **esattamente 1 I/O**.

15.6.2 Esempio 2: Dizionario Alfabetico a 5 Blocchi

Si consideri un dizionario lessicale partizionato in $K = 5$ blocchi con capacità $B = 3$ stringhe per blocco, come riassunto nella Tabella 15.1.

Blocco	Stringhe Contenute	Rappresentante Estratto (S_i)
Blocco 1	*aabaa*, *aabab*, *aabb*	$S_1 =$ **aabaa**
Blocco 2	*abbaaa*, *abbaab*, *abbaba*	$S_2 =$ **abbaaa**
Blocco 3	*bbaba*, *bbabba*, *bbb*	$S_3 =$ **bbaba**
Blocco 4	*bbabb*, *bbabbbb*, *bbb*	$S_4 =$ **bbabb**
Blocco 5	*bbba*, *bbbaa*, *bbbb*	$S_5 =$ **bbba**

Tabella 15.1: Configurazione del dizionario alfabetico a 5 blocchi ordinati lessicograficamente e stringhe rappresentative estratte.

Costruzione della Patricia Trie sui Rappresentanti. I rappresentanti da indicizzare sono:

$$S_1 = aabaa, \quad S_2 = abbaaa, \quad S_3 = bbaba, \quad S_4 = bbabb, \quad S_5 = bbba \tag{15.31}$$

1. **Posizione 1:** Separa $\{S_1, S_2\}$ (iniziano con 'a') da $\{S_3, S_4, S_5\}$ (iniziano con 'b'):
$$\text{skip}(\text{Radice}) = 1 \tag{15.32}$$

2. **Sottoalbero sinistro ('a'):** Tra $S_1 = a\mathbf{a}baa$ ed $S_2 = a\mathbf{b}baaa$, il carattere 1 è comune ('a'), mentre la divergenza occorre alla posizione 2:
$$\text{skip}(u) = 2 \tag{15.33}$$
L'arco etichettato 'a' conduce direttamente alla foglia del Blocco 1 ($\langle S_1, \text{ptr}_1 \rangle$), mentre l'arco 'b' conduce direttamente alla foglia del Blocco 2 ($\langle S_2, \text{ptr}_2 \rangle$).

3. **Sottoalbero destro ('b'):** Tutte le chiavi S_3, S_4, S_5 iniziano con 'bb'. La prima divergenza si presenta alla posizione 3 tra $\{S_3, S_4\}$ (che presentano $bb\mathbf{a}\ldots$) ed $S_5 = bb\mathbf{b}a$:
$$\text{skip}(v_1) = 3 \tag{15.34}$$
L'arco 'b' isola direttamente S_5 (Blocco 5). L'arco 'a' conduce al nodo v_2, che discrimina $S_3 = bbab\mathbf{a}$ ed $S_4 = bbab\mathbf{b}$ alla posizione 5:
$$\text{skip}(v_2) = 5 \tag{15.35}$$
L'arco 'a' conduce alla foglia del Blocco 3 ($\langle S_3, \text{ptr}_3 \rangle$), mentre l'arco 'b' conduce alla foglia del Blocco 4 ($\langle S_4, \text{ptr}_4 \rangle$).

In ciascuna foglia è evidenziato in rosso l'**ultimo carattere discriminante del cammino**, ossia il carattere testato dal nodo genitore immediato:

- 2° carattere per $S_1 = a\mathbf{a}baa$ ed $S_2 = a\mathbf{b}baaa$ (testato da u con skip $= 2$);
- 3° carattere per $S_5 = bb\mathbf{b}a$ (testato da v_1 con skip $= 3$);
- 5° carattere per $S_3 = bbab\mathbf{a}$ ed $S_4 = bbab\mathbf{b}$ (testato da v_2 con skip $= 5$).

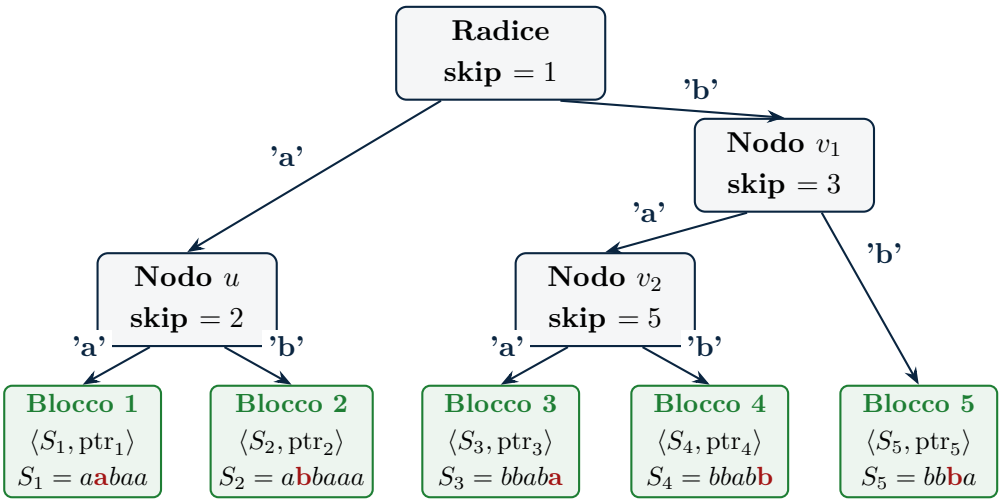


Figura 15.4: Topologia completa della Patricia Trie per il dizionario a 5 blocchi: ogni nodo interno possiede grado 2 e tutte le 5 foglie sono distribuite in modo uniforme sullo strato inferiore dei blocchi con spaziatura regolare (7 mm di gap tra foglie contigue). In ciascuna foglia è evidenziato in rosso l'ultimo carattere discriminante testato dal rispettivo genitore immediato (2° carattere per S_1 ed S_2; 3° per S_5; 5° per S_3 ed S_4).

Simulazione Query 1: Ricerca di $P_1 = abbab$.

1. **Fase 1 (Discesa Cieca):**
   - Radice (skip $= 1$): $P_1[1] =' a' \implies$ segue arco ''a''.
   - Nodo u (skip $= 2$): $P_1[2] =' b' \implies$ segue arco ''b''.
   - Viene raggiunta direttamente la **Foglia del Blocco 2** ($\langle S_2, \text{ptr}_2 \rangle$, con $S_2 = abbaaa$). Costo: 0 I/O.
2. **Fase 2 (Verifica Esterna su Disco):**

- Caricamento del Blocco 2 con **1 I/O** ed estrazione del rappresentante $S_2 = abbaaa$.
- Confronto esaustivo:

$$P_1 = \mathbf{abba}\, b$$
$$S_2 = \mathbf{abba}\, aa$$

  I primi 4 caratteri coincidono: $\ell = \mathrm{LCP}(P_1, S_2) = 4$.
- Alla posizione 5 si rileva la divergenza: $P_1[5] =' b' > S_2[5] =' a'$.

3. **Fase 3 (Risalita e Risoluzione):**
   - La divergenza reale è a posizione 5 ($\ell + 1 = 5$), mentre il prefisso comune condiviso ha lunghezza 4 ($\ell = 4$).
   - Poiché P_1 condivide l'intero prefisso $S_2[1 \ldots 4] =$ "abba" del Blocco 2 e risulta $P_1 > S_2$, la ricerca non deve risalire oltre il Blocco 2.
   - Si effettua la scansione sequenziale in memoria del Blocco 2 decodificando il Front-Coding:

$$[abbaaa,\ abbaab,\ abbaba] \tag{15.36}$$

     Dato che $P_1 = abbab$, non si rileva un match esatto, ma si individuano con precisione matematica il predecessore ($abbaab$) e il successore ($abbaba$).
   - Costo complessivo: **1 I/O** e $\mathcal{O}(p)$ tempo CPU.

Simulazione Query 2: Ricerca con Branching Mismatch ($P_2 = acba$).

1. **Fase 1 (Discesa Cieca):**
   - Radice (skip = 1): $P_2[1] =' a' \implies$ segue arco ''a''.
   - Nodo u (skip = 2): nel pattern abbiamo $P_2[2] = '\mathbf{c}'$.
   - **Branching Mismatch:** Il nodo u possiede unicamente archi uscenti per ''a'' e per ''b''. Non esiste alcun arco etichettato ''c''.
   - **Regola della Foglia Arbitraria:** La discesa si arresta presso il nodo u e seleziona una qualsiasi foglia discendente del sottoalbero radicato in u. Seguendo la convenzione implementativa (foglia più a sinistra), seleziona la **Foglia del Blocco 1** ($\langle S_1, \mathrm{ptr}_1 \rangle$, con $S_1 = aabaa$). Costo: 0 I/O.

2. **Fase 2 (Verifica Esterna su Disco):**

- Caricamento del Blocco 1 con **1 I/O** ed estrazione di $S_1 = aabaa$.
- Confronto esaustivo tra P_2 ed S_1:

$$P_2 = a\,\mathbf{c}\,ba$$
$$S_1 = a\,\mathbf{a}\,baa$$

Solamente il primo carattere coincide, mentre al secondo carattere emerge la divergenza:

$$\ell = \mathrm{LCP}(P_2, S_1) = 1, \qquad P_2[2] =' c', \quad S_1[2] =' a' \tag{15.37}$$

3. **Fase 3 (Risalita e Localizzazione Corretta):**
   - Poiché $\ell = 1$, la vera divergenza tra P_2 e l'albero si trova alla posizione $\ell + 1 = 2$.
   - Dalla Foglia del Blocco 1 si risale verso la radice: il nodo genitore è u, avente $\mathrm{skip}(u) = 2$. Poiché $\mathrm{skip}(u) \leq \ell + 1$ ($2 \leq 2$), la risalita si arresta esattamente su u (locus di diramazione reale).
   - Il nodo u discrimina il carattere 2. Nel pattern abbiamo $P_2[2] =' c'$, mentre gli archi esistenti sono ''a'' e ''b''.
   - Poiché $'c' >' b' >' a'$, deduciamo che P_2 è strettamente maggiore di tutte le chiavi nel sottoalbero di u (ovvero maggiore di tutte le stringhe dei Blocchi 1 e 2 aventi prefisso 'a').
   - Avendo verificato nella Fase 2 che P_2 inizia per ''a'' ($\ell = 1$), P_2 si colloca lessicograficamente subito dopo le chiavi del Blocco 2 e immediatamente prima di quelle del Blocco 3:

$$S_1 < S_2 < P_2 < S_3 \tag{15.38}$$

   - La ricerca conclude con successo che $P_2 \notin \mathcal{D}$ e identifica il predecessore nel Blocco 2 e il successore nel Blocco 3.
   - Costo complessivo: **esattamente 1 I/O** e $\mathcal{O}(p)$ tempo CPU.

15.7 Quadro Comparativo e Considerazioni Conclusive

15.7.1 Tabella Sinottica delle Prestazioni

La Tabella 15.2 sintetizza il confronto asintotico tra l'architettura a due livelli basata su **Patricia Trie (Blind Trie) + Front-Coding** e le soluzioni algoritmiche alternative per la gestione di dizionari di stringhe.

Struttura / Approccio	Tempo CPU Ricerca	Complessità I/O Disco	Spazio Memoria Primaria (RAM)	Spazio Memoria Secondaria (Disco)
Scansione Lineare Non Indicizzata	$\mathcal{O}(N \cdot p)$	$\mathcal{O}\left(\frac{n}{B}\right)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$ (testo piano)
B-Tree Standard su Stringhe	$\mathcal{O}(p \log_B N)$	$\mathcal{O}\left(\log_B N\right)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Compacted Trie Standard in RAM	$\mathcal{O}(p)$	$\mathcal{O}(1)$	$\mathcal{O}\left(\frac{N}{B} + \sum \lvert R_j \rvert\right)$	$\mathcal{O}(n)$
Two-Level: Patricia Trie + Front-Coding	$\mathcal{O}(\mathbf{p})$	**1 I/O** (Ottimo)	$\mathcal{O}\left(\frac{\mathbf{N}}{\mathbf{B}}\right)$ (Indipendente da $\lvert R_j \rvert$)	$\ll \mathbf{n}$ (Compresso Front-Coding)

Tabella 15.2: Quadro comparativo delle metodologie di indicizzazione e ricerca di stringhe in memoria gerarchica.

15.7.2 Conclusioni Ingegneristiche e Linee Guida

L'architettura analizzata in questo capitolo risolve in modo rigoroso e ottimale il trade-off fondamentale tra tempo di calcolo della CPU, capacità della memoria centrale e latenza di accesso al disco:

1. **Scalabilità Spaziale in RAM:** Grazie alla compressione lossy della Patricia Trie (eliminazione totale dei puntatori a testo e memorizzazione dei soli indici scalari di salto), l'occupazione in RAM è rigorosamente limitata a $\mathcal{O}(N/B)$ parole macchina. L'indice scala linearmente solo rispetto al numero di pagine fisiche, consentendo l'indicizzazione di miliardi di chiavi lunghe anche su macchine con memoria RAM modesta.

2. **Minimalità Assoluta di I/O:** A differenza dei B-Tree convenzionali (che richiedono $\mathcal{O}(\log_B N)$ accessi a disco, vedi sottosezione 2.3.2) e delle scansioni non indicizzate, l'algoritmo Three-Phase Blind Search garantisce il limite teorico invalicabile di **esattamente 1 I/O** per qualunque interrogazione di ricerca esatta o di predecessore.

3. **Efficacia della Compressione Front-Coding:** La rimozione incrementale dei prefissi sui blocchi disco massimizza la densità di informazione per pagina fisica senza penalizzare i tempi di latenza, poiché il costo computazionale della decodifica in RAM è completamente dominato dal tempo di trasferimento dell'unico blocco disco.

Regola Architetturale per la Ricerca di Stringhe su Larga Scala

Nei sistemi industriali e nei motori di ricerca moderni (es. moduli di indexing per motori Web e database a chiavi alfanumeriche):

- Per collezioni di stringhe che risiedono interamente in memoria RAM, il **Compacted Trie** con tuple (sezione 14.4) o il **TST** (Capitolo 13) rappresentano la scelta d'elezione.
- Quando la collezione scala su memoria secondaria o storage distribuito, l'accoppiamento tra **Patricia Trie / Blind Trie** (in memoria centrale per instradare le query) e **Front-Coding** (su disco per comprimere i blocchi) costituisce lo standard de facto per garantire interrogazioni ad alto throughput con garanzia di 1 solo I/O.
- Quando il problema computazionale non verte su un dizionario di parole isolate, bensì sulla ricerca di sottostringhe arbitrarie in un testo continuo non strutturato $T[1 \dots m]$ (full-text indexing), il problema viene ricondotto alla ricerca per prefisso sull'insieme di tutti gli m suffissi di T, introducendo le strutture cardine **Suffix Array** e **Suffix Tree** (Capitolo 16).

Capitolo 16

Indicizzazione e Ricerca di Sottostringhe: Suffix Array, Suffix Tree e Array LCP

16.1 Dal Problema del Dizionario alla Ricerca di Sottostringhe

16.1.1 Il Problema del Dizionario su Stringhe e sue Limitazioni

Nei capitoli precedenti (Capitolo 12, Capitolo 13, Capitolo 14 e Capitolo 15) è stato affrontato il problema canonico del **dizionario su stringhe**, in cui una collezione finita di n chiavi distinte:

$$\mathcal{D} = \{S_1, S_2, \ldots, S_n\} \subset \Sigma^* \tag{16.1}$$

definita su un alfabeto ordinato Σ, viene organizzata in memoria centrale o gerarchica per supportare operazioni mirate:

- **Exact Match (Ricerca Esatta):** Verificare se un pattern P appartiene alla collezione ($P \in \mathcal{D}$).
- **Prefix Search (Ricerca per Prefisso):** Identificare tutte le chiavi $S \in \mathcal{D}$ che ammettono P come prefisso iniziale ($S[1 \ldots |P|] = P$).
- **Range Search e Ricerca Lessicografica:** Individuare le chiavi appartenenti a un intervallo $[P_1, P_2]$, oppure localizzare il predecessore e successore lessicografico.

Le strutture canoniche introdotte per risolvere tali query comprendono:

1. **Trie Standard (Uncompacted Trie):** Albero digitale ad archi a carattere singolo (sezione 14.2), ottimale per chiavi corte ma soggetto a degenerazione lineare in profondità.

2. **Compacted Trie (Patricia Tree / Radix Tree):** Compressione dei cammini privi di diramazioni mediante sottostringhe o tuple di indici (sezione 14.4), con tempo di ricerca $\mathcal{O}(|P|)$ e spazio $\mathcal{O}(n)$ nodi.

3. **Ternary Search Tree (TST):** Albero a tre vie con nodi bilanciati (sezione 13.2), che sostituisce i vettori di puntatori con ramificazioni inferiori, uguali e superiori.

4. **Blind Trie (Patricia Trie Lossy):** Rappresentazione a compressione con perdita orientata alla memoria esterna (sezione 15.4), che conserva unicamente interi di salto skip(u) ed etichette a singolo carattere per eliminare l'overhead del testo in RAM.

Tutte queste strutture si fondano tuttavia su un'assunzione rigida: **la collezione è partizionata in parole predeterminate e isolate**. Se l'obiettivo dell'applicazione è individuare le occorrenze di una sequenza arbitraria all'interno di un testo continuo privo di delimitatori lessicali, il modello del dizionario canonico risulta inadeguato.

16.1.2 La Ricerca di Sottostringhe in un Testo Generale

Nel problema fondamentale del **Substring Search** (o ricerca a testo pieno, *full-text pattern matching*), lo scenario applicativo muta radicalmente: non si dispone di un vocabolario di termini predefiniti, bensì di un unico testo continuo e non strutturato:

$$T[1 \dots n] = T[1]T[2] \dots T[n] \in \Sigma^n \tag{16.2}$$

di lunghezza $|T| = n$. Dato un generico pattern di interrogazione:

$$P[1 \dots m] = P[1]P[2] \dots P[m] \in \Sigma^m \tag{16.3}$$

di lunghezza $|P| = m \leq n$, l'obiettivo consiste nel determinare:

1. **Query di Esistenza / Conteggio:** Stabilire se P compare all'interno di T e contare il numero complessivo t_{occ} di occorrenze:

$$\text{Occ}(P, T) = \{j \in \{1, \dots, n - m + 1\} \mid T[j \dots j + m - 1] = P\} \tag{16.4}$$

2. **Query di Localizzazione (Reporting):** Restituire l'elenco esaustivo degli indici posizionali $\{j_1, j_2, \ldots, j_{t_{\text{occ}}}\}$ in cui P ha inizio all'interno di T.

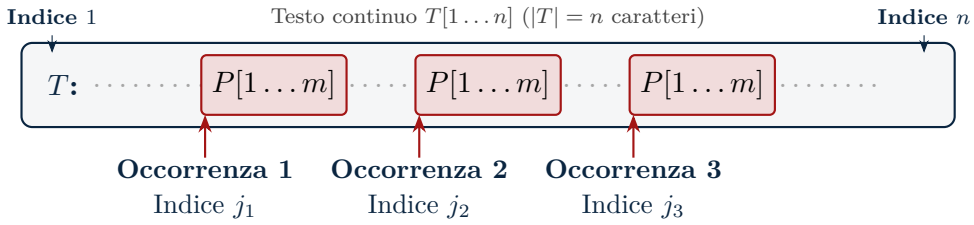


Figura 16.1: Visualizzazione concettuale del problema di Substring Search: localizzazione di tutte le occorrenze arbitrarie di un pattern P di lunghezza m all'interno di un testo continuo $T[1 \ldots n]$. Ciascuna freccia identifica l'indice di partenza j_i del rispettivo intervallo continuo $T[j_i \ldots j_i + m - 1] = P$.

Domini Applicativi Fondamentali. La necessità di ricercare pattern posizionati arbitrariamente all'interno di collezioni massive emerge in ambiti tecnologici critici:

- **Bioinformatica e Analisi Genomica:** Sequenze di DNA, RNA e proteine non presentano separatori naturali tra geni o motivi funzionali. Con alfabeti biologici compatti (es. nucleotidi $\Sigma = \{A, C, G, T\}$ o 20 amminoacidi), le interrogazioni mirano a localizzare mutazioni, marcatori genetici o sottostringhe comuni tra organismi su testi composti da centinaia di milioni o miliardi di basi.
- **Motori di Ricerca Web e Testi Non Strutturati:** Indicizzazione a testo pieno (*full-text search*) su documenti digitali, codici sorgente o lingue naturali prive di spaziatori convenzionali tra vocaboli (come il cinese o il giapponese).
- **Cybersecurity e Network Intrusion Detection:** Monitoraggio in tempo reale dei flussi di rete e ispezione profonda dei pacchetti (*Deep Packet Inspection*, DPI), in cui si scansionano flussi continui di byte per rilevare signature binarie malevole o shellcode.

16.1.3 Limite della Scansione Sequenziale (Brute-Force)

L'algoritmo ingenuo di scansione sequenziale confronta il pattern P scorrendolo lungo tutte le possibili posizioni di allineamento $i \in \{1, 2, \ldots, n - m + 1\}$

del testo T:

$$\text{Per ciascun offset } i, \quad \text{confronta } T[i \dots i+m-1] \text{ con } P[1 \dots m]. \tag{16.5}$$

Analisi della Complessità nel Caso Peggiore. Per ogni indice i, la verifica può richiedere fino a m confronti di caratteri prima di identificare una corrispondenza o rilevare un mismatch. Ne consegue che:

$$\text{Tempo}_{\text{Brute-Force}} = \mathcal{O}(n \cdot m) \tag{16.6}$$

In scenari reali altamente ripetitivi (ad esempio nel genoma umano con sequenze di basi identiche $T = AAAAA\dots$ e pattern $P = AAAB$), il caso peggiore viene raggiunto con regolarità. Se $n \approx 3 \times 10^9$ (la taglia del genoma umano) e si eseguono milioni di query con pattern lunghi $m = 1000$, la scansione lineare richiederebbe svariati minuti o ore per singola interrogazione, risultando totalmente inapplicabile.

Sebbene algoritmi di scansione lineare senza pre-elaborazione del testo (come Knuth-Morris-Pratt, Boyer-Moore o Rabin-Karp) riducano il tempo di ricerca a $\mathcal{O}(n+m)$, essi richiedono comunque di rileggere l'intero testo T a ogni nuova interrogazione:

$$\text{Costo per } Q \text{ interrogazioni indipendenti} = \mathcal{O}(Q \cdot n) \tag{16.7}$$

Per ottenere tempi di risposta interattivi (*sub-lineari* o indipendenti da n), è indispensabile pre-elaborare e indicizzare il testo una volta per tutte in una struttura dati specializzata.

16.1.4 La Riduzione Chiave: Ricondurre Substring Search a Prefix Search

La svolta teorica che permette di superare la scansione sequenziale risiede in una riduzione concettuale universale: **trasformare la ricerca di una sottostringa generica in una ricerca di prefisso su un opportuno dizionario di suffissi**.

Teorema 16.1.1 (Principio di Riduzione ai Suffissi)**.** *Sia dato un testo $T[1 \dots n]$ e un pattern arbitrario $P[1 \dots m]$. La stringa P è una **sottostringa** di T se e solo se P è un **prefisso** di almeno uno dei suffissi di T.*

Dimostrazione. La dimostrazione segue direttamente dalle definizioni formali:

1. ($\Longrightarrow$) Sia P una sottostringa di T. Allora esiste un indice posizionale $j \in \{1, \ldots, n-m+1\}$ tale che $T[j \ldots j+m-1] = P$. Si consideri il suffisso di T che ha inizio all'indice j, denotato con $S_j = T[j \ldots n]$. I primi m caratteri di S_j sono identici a $T[j \ldots j+m-1]$, che per ipotesi coincide con P. Pertanto, P è un prefisso di S_j.

2. ($\Longleftarrow$) Sia P un prefisso del suffisso $S_j = T[j \ldots n]$. Poiché P coincide con i primi m caratteri di S_j, abbiamo $T[j \ldots j+m-1] = P$, il che prova che P occorre come sottostringa continua in T all'indice j.

□

Il Dizionario Completo dei Suffissi. Definiamo formalmente il dizionario dei suffissi di T come l'insieme esaustivo di tutti gli n suffissi generabili dal testo:

$$\mathcal{D}_{\text{suffissi}} = \{S_1, S_2, \ldots, S_n\} = \{T[1 \ldots n],\ T[2 \ldots n],\ \ldots,\ T[n \ldots n]\} \quad (16.8)$$

Grazie al Teorema 16.1.1, la ricerca di tutte le occorrenze di P in T si riduce esattamente a una **Prefix Search** per P all'interno del dizionario $\mathcal{D}_{\text{suffissi}}$.

L'Ostacolo della Rappresentazione Esplicita: Esplosione Quadratica $\Theta(n^2)$. Se tentassimo di memorizzare il dizionario $\mathcal{D}_{\text{suffissi}}$ allocando esplicitamente ciascun suffisso come stringa indipendente, il numero totale di caratteri da registrare in memoria risulterebbe:

$$\sum_{i=1}^{n} |S_i| = \sum_{i=1}^{n} (n-i+1) = \sum_{k=1}^{n} k = \frac{n(n+1)}{2} = \Theta(n^2) \text{ caratteri} \quad (16.9)$$

Insostenibilità dell'Approccio Esplicito

Un consumo spaziale pari a $\Theta(n^2)$ è proibitivo per qualunque collezione di medie o grandi dimensioni. Per un genoma umano ($n \approx 3 \times 10^9$ caratteri), $\Theta(n^2)$ byte richiederebbe circa 4.5×10^{18} byte (4.5 Exabyte), un volume inconcepibile per qualsiasi infrastruttura computazionale moderna.

La sfida centrale dell'Algorithm Engineering consiste pertanto nell'indicizzare l'intero dizionario dei suffissi $\mathcal{D}_{\text{suffissi}}$ garantendo rigorosamente un'occupazione di memoria **strettamente lineare** $\mathcal{O}(n)$, supportando al contempo ricerche veloci del pattern P. Questa sfida conduce naturalmente alle due strutture cardine della letteratura: il **Suffix Array (SA)** e il **Suffix Tree**.

16.2 La Struttura Dati Suffix Array (SA)

16.2.1 Definizione Formale e Convenzione del Carattere Sentinella

Introdotto da Manber e Myers nel 1993 [26], il **Suffix Array** (SA) costituisce una delle strutture dati più eleganti e compatte per l'indicizzazione a testo pieno.

Definizione 16.2.1 (Carattere Speciale Sentinella \$)**.** Sia data una stringa $T \in \Sigma^*$. Si definisce testo arricchito la sequenza $T\$ = T[1 \dots n]\$$, ottenuta concatenando in coda a T un carattere sentinella speciale denotato con \$, tale che:

$$\$ \notin \Sigma \quad \wedge \quad \forall c \in \Sigma,\ \$ <_{\text{lex}} c \tag{16.10}$$

Il simbolo \$ possiede convenzionalmente il rango lessicografico più basso in assoluto nell'alfabeto esteso $\Sigma_{\$} = \Sigma \cup \{\$\}$.

L'introduzione della sentinella \$ garantisce due proprietà algebriche fondamentali:

1. **Assenza di Prefissi Propri:** Poiché il simbolo \$ compare esclusivamente al termine del testo, nessun suffisso di $T\$$ può essere un prefisso proprio di un altro suffisso.
2. **Ordinamento Stretto e Univoco:** Dati due distinti suffissi S_i e S_j ($i \neq j$), essi differiranno necessariamente prima o in corrispondenza del simbolo \$, garantendo una relazione d'ordine totale stretto:

$$S_i <_{\text{lex}} S_j \quad \vee \quad S_j <_{\text{lex}} S_i \tag{16.11}$$

Definizione 16.2.2 (Suffix Array)**.** Il Suffix Array associato a un testo $T\$$ di lunghezza n è una permutazione degli indici posizionali $\{1, 2, \dots, n\}$:

$$SA : \{1, \dots, n\} \to \{1, \dots, n\} \tag{16.12}$$

tale per cui i suffissi puntati risultano ordinati in ordine lessicografico strettamente crescente:

$$S_{SA[1]} <_{\text{lex}} S_{SA[2]} <_{\text{lex}} \cdots <_{\text{lex}} S_{SA[n]} \tag{16.13}$$

In altri termini, $SA[r]$ memorizza l'indice iniziale nel testo $T\$$ del suffisso che occupa il rango lessicografico r.

16.2.2 Costruzione ed Esempio per $T =$ mississippi\$

Consideriamo il classico testo di riferimento:

$$T\$ = \text{mississippi\$} \tag{16.14}$$

avente lunghezza totale pari a $n = 12$ caratteri. La mappatura degli indici posizionali (1-based) è la seguente:

Indice i	1	2	3	4	5	6	7	8	9	10	11	12
$T[i]$	m	i	s	s	i	s	s	i	p	p	i	\$

Tabella 16.1: Mappatura posizionale dei caratteri per il testo $T\$ =$ mississippi\$ ($n = 12$).

Elenco Esaustivo dei 12 Suffissi non Ordinati. Ciascun indice $i \in \{1, \ldots, 12\}$ definisce un suffisso $S_i = T[i \ldots 12]$:

$$\begin{aligned}
S_1 &= T[1 \ldots 12] = \text{mississippi\$} & \qquad S_7 &= T[7 \ldots 12] = \text{sippi\$} \\
S_2 &= T[2 \ldots 12] = \text{ississippi\$} & \qquad S_8 &= T[8 \ldots 12] = \text{ippi\$} \\
S_3 &= T[3 \ldots 12] = \text{ssissippi\$} & \qquad S_9 &= T[9 \ldots 12] = \text{ppi\$} \\
S_4 &= T[4 \ldots 12] = \text{sissippi\$} & \qquad S_{10} &= T[10 \ldots 12] = \text{pi\$} \\
S_5 &= T[5 \ldots 12] = \text{issippi\$} & \qquad S_{11} &= T[11 \ldots 12] = \text{i\$} \\
S_6 &= T[6 \ldots 12] = \text{ssippi\$} & \qquad S_{12} &= T[12 \ldots 12] = \text{\$}
\end{aligned}$$

Derivazione del Suffix Array tramite Ordinamento Lessicografico. Ordinando i 12 suffissi in ordine lessicografico crescente (tenendo presente che \$ precede alfabeticamente qualunque lettera dell'alfabeto), si ottiene la corrispondenza biunivoca sintetizzata nella Tabella 16.2.

Rango r	$SA[r]$ **(Offset in** T**)**	**Suffisso Associato** $S_{SA[r]}$
1	**12**	\$
2	**11**	*i*\$
3	**8**	*ippi*\$
4	**5**	*issippi*\$
5	**2**	*ississippi*\$
6	**1**	*mississippi*\$
7	**10**	*pi*\$
8	**9**	*ppi*\$
9	**7**	*sippi*\$
10	**4**	*sissippi*\$
11	**6**	*ssippi*\$
12	**3**	*ssissippi*\$

Tabella 16.2: Suffix Array ordinato per il testo $T\$ = \text{mississippi}\$$.

Il Suffix Array corrisponde quindi al vettore di interi:

$$SA = [12,\ 11,\ 8,\ 5,\ 2,\ 1,\ 10,\ 9,\ 7,\ 4,\ 6,\ 3] \tag{16.15}$$

16.2.3 Occupazione Spaziale del Suffix Array

La struttura completa allocata in memoria primaria è composta dalla coppia (T, SA):

- **Il Testo Originale** T**:** Memorizza i caratteri grezzi del testo continuo. Se codificato in standard ASCII o UTF-8 a 1 byte per simbolo, occupa esattamente n byte.
- **Il Vettore** SA**:** Memorizza la permutazione degli m indici interi compresi nell'intervallo $[1, m]$. Ciascun elemento necessita di $\lceil \log_2 m \rceil$ bit. Nei sistemi reali, per testi fino a $2^{32} - 1 \approx 4$ GB si impiegano interi a 32 bit (4 byte); per collezioni più estese si adottano interi a 64 bit (8 byte).

Lo spazio complessivo occupato risulta:

$$\text{Spazio}(SA + T) = n \cdot \text{sizeof(int)} + n \cdot \text{sizeof(char)} = \Theta(n) \text{ byte} \tag{16.16}$$

Risoluzione dell'Esplosione Quadratica

Il Suffix Array abbatte la ridondanza quadratica $\Theta(n^2)$ mediante la **condivisione indiretta del testo**: i caratteri dei suffissi non vengono mai duplicati in memoria, ma referenziati al volo attraverso l'indice iniziale $SA[r]$ all'interno dell'unico array continuo T.

16.2.4 Algoritmo di Ricerca Binaria per un Pattern $P[1 \dots m]$

La proprietà algebrica fondante del Suffix Array risiede nel fatto che tutti i suffissi che condividono un determinato prefisso comune P occupano un **intervallo contiguo di indici** all'interno del vettore SA.

Lemma 16.2.1 (Contiguità dei Prefissi nel Suffix Array)**.** *Sia dato un pattern $P \in \Sigma^*$. L'insieme dei ranghi dei suffissi che ammettono P come prefisso costituisce un intervallo chiuso $[L, R] \subseteq [1, n]$ (eventualmente vuoto se P non occorre in T):*

$$\forall r \in [L, R], \quad S_{SA[r]}[1 \dots |P|] = P \tag{16.17}$$

Dimostrazione. La dimostrazione discende direttamente dall'ordinamento lessicografico totale dei suffissi. Se $S_{SA[L]}$ e $S_{SA[R]}$ iniziano entrambi con il prefisso P, allora qualsiasi suffisso $S_{SA[k]}$ intermedio con $L \leq k \leq R$ deve soddisfare:

$$S_{SA[L]} \leq_{\text{lex}} S_{SA[k]} \leq_{\text{lex}} S_{SA[R]} \tag{16.18}$$

Poiché i due estremi condividono il medesimo prefisso P, ogni stringa lessicograficamente compresa tra di essi deve obbligatoriamente iniziare con il medesimo prefisso P. □

Procedura di Duplice Ricerca Binaria. Per determinare l'intervallo $[L, R]$, si eseguono due ricerche binarie indipendenti sul vettore SA:

1. **Prima Ricerca Binaria (Estremo Sinistro L):** Localizza il primo rango $L \in \{1, \dots, n\}$ tale che $S_{SA[L]} \geq_{\text{lex}} P$. Se tale suffisso non ammette P come prefisso, il pattern non compare in T e l'algoritmo termina.

2. **Seconda Ricerca Binaria (Estremo Destro R):** Localizza l'ultimo rango $R \in \{1, \dots, n\}$ tale che P sia ancora prefisso di $S_{SA[R]}$. Formalmente, ciò equivale a cercare la posizione di inserimento di $P \circ \#$ (dove $\#$ è un carattere sentinella convenzionale superiore a ogni simbolo di

Σ, $\forall c \in \Sigma,\ c <_{\text{lex}} \#$), oppure il primo suffisso strettamente maggiore di qualunque estensione di P.

Algorithm 21: Ricerca Binaria su Suffix Array per Substring Search

Input: Testo $T[1 \dots n]$\$, Suffix Array $SA[1 \dots n]$, Pattern $P[1 \dots m]$
Output: Intervallo delle occorrenze $[L, R]$, oppure $\emptyset$

```
// Fase 1: Ricerca Binaria per l'Estremo Sinistro L
low ← 1;   high ← n;   L ← n + 1
while low ≤ high do
    mid ← ⌊(low + high)/2⌋
    if S_{SA[mid]}[1...m] ≥_lex P then
        L ← mid
        high ← mid − 1
    else
        low ← mid + 1
if L > n or S_{SA[L]}[1...m] ≠ P then
    return ∅                    // Pattern non presente nel testo
// Fase 2: Ricerca Binaria per l'Estremo Destro R
low ← L;   high ← n;   R ← L
while low ≤ high do
    mid ← ⌊(low + high)/2⌋
    if S_{SA[mid]}[1...m] == P then
        R ← mid
        low ← mid + 1
    else
        high ← mid − 1
return [L, R]
```

Esempio Guidato: Ricerca del Pattern P = "si" in mississippi\$. Ispezioniamo la Tabella 16.2:

1. La prima ricerca binaria individua il primo suffisso che inizia con ‘"si"‘:

$$r = 9 \implies SA[9] = 7 \implies S_7 = \text{"sippi\$"} \tag{16.19}$$

   Dunque $L = 9$.

2. La seconda ricerca binaria identifica l'ultimo suffisso che inizia con ‘"si"‘:

$$r = 10 \implies SA[10] = 4 \implies S_4 = \text{"sissippi\$"} \tag{16.20}$$

Dunque $R = 10$ (il suffisso successivo al rango 11 è ‘"ssippi$"‘, che diverge già al secondo carattere).

3. **Conteggio delle Occorrenze:**
$$\text{count}(P) = R - L + 1 = 10 - 9 + 1 = 2 \tag{16.21}$$

4. **Localizzazione delle Posizioni in T:** Leggendo direttamente le celle del Suffix Array tra L e R, si estraggono gli indici:
$$\{SA[9],\ SA[10]\} = \{7,\ 4\} \tag{16.22}$$
Il pattern compare esattamente agli indici 4 e 7 del testo originale.

Analisi della Complessità Computazionale.

- **Tempo di Ricerca dell'Intervallo:** Ciascuna delle due ricerche binarie esegue $\mathcal{O}(\log n)$ iterazioni. Ad ogni iterazione, il confronto tra il pattern P e il suffisso $S_{SA[mid]}$ richiede al più m ispezioni di caratteri nel testo T. Il costo complessivo per determinare $[L, R]$ è:
$$\text{Tempo di Search} = \mathcal{O}(m \cdot \log n) \tag{16.23}$$

- **Tempo di Restituzione (Reporting):** Una volta individuato l'intervallo, stampare le $t_{\text{occ}} = R - L + 1$ occorrenze richiede una semplice scansione lineare di SA in tempo $\mathcal{O}(t_{\text{occ}})$.

- **Complessità Temporale Complessiva:**
$$T(n, m) = \mathcal{O}(m \log n + t_{\text{occ}}) \tag{16.24}$$

- **Complessità di I/O:** Nell'ipotesi in cui T e SA risiedano interamente nella memoria primaria RAM, non viene eseguito alcun accesso a memoria secondaria:
$$\text{I/O} = 0 \tag{16.25}$$

16.3 La Struttura Dati Suffix Tree

16.3.1 Definizione e Proprietà Topologiche

Il **Suffix Tree**, introdotto originariamente da Weiner nel 1973 [27] e successivamente perfezionato da McCreight [28] e Ukkonen [29], rappresenta la versione indicizzata su albero digitale compresso (*compacted trie*) dell'insieme esaustivo di tutti i suffissi di un testo $T\$$.

Definizione 16.3.1 (Suffix Tree)**.** Il Suffix Tree $\mathcal{T}(T\$)$ associato al testo $T[1 \dots n]\$$ è un albero radicato che soddisfa le seguenti proprietà strutturali:

1. **Numero di Foglie:** Possiede esattamente n foglie, numerate da 1 a n. Ciascuna foglia ℓ_i è associata biunivocamente al suffisso $S_i = T[i \dots n]\$$.

2. **Branching Factor dei Nodi Interni:** Ciascun nodo interno u (esclusa eventualmente la radice se $n = 1$) possiede un grado di uscita maggiore o uguale a 2:
$$\deg(u) \geq 2 \tag{16.26}$$
Non esistono nodi non diramati (nodi unari); ogni cammino privo di biforcazioni è compresso in un singolo arco.

3. **Limite sui Nodi Interni:** In virtù della proprietà di biforcazione, un albero con n foglie possiede al più $n - 1$ nodi interni:
$$N_{\text{interni}} \leq n - 1 \tag{16.27}$$
Il numero totale di nodi (foglie + nodi interni) è superiormente limitato da $2n - 1$.

4. **Etichette degli Archi:** Ogni arco (u, v) è etichettato con una sottostringa non vuota di $T\$$. Archi uscenti dal medesimo nodo interno iniziano necessariamente con caratteri distinti.

5. **String-Depth (Profondità di Stringa):** Per ciascun nodo u, la concatenazione ordinata delle etichette lungo il cammino che scende dalla radice a u definisce una stringa $\text{path}(u)$. La lunghezza $|\text{path}(u)|$ prende il nome di **string-depth** del nodo u.

16.3.2 Compressione delle Etichette: Rappresentazione in Spazio Lineare

Se registrassimo esplicitamente le sottostringhe testuali sui singoli archi del Suffix Tree, lo spazio complessivo degenererebbe nuovamente a $\Theta(n^2)$ caratteri nel caso peggiore (ad esempio su testi periodici in cui molti archi memorizzano etichette lunghe $\Theta(n)$).

Per preservare rigorosamente la linearità dello spazio, si applica la tecnica di rappresentazione a tuple di indici già esaminata nel sottosezione 14.4.3:

- Ciascun arco (u, v) non memorizza alcun carattere, bensì unicamente una coppia di indici interi:
$$\text{Arco}(u, v) \equiv [\text{start},\ \text{end}] \iff \text{etichetta} = T[\text{start} \dots \text{end}] \tag{16.28}$$

avente lunghezza pari a $\text{end} - \text{start} + 1$.

- Poiché il numero totale di archi è limitato da $2n - 2$, l'insieme di tutti gli archi necessita unicamente di $2 \cdot (2n - 2) = \mathcal{O}(n)$ interi.
- Mantenendo in memoria l'unico testo $T\$$ (n caratteri) unitamente a $\mathcal{O}(n)$ nodi e archi, si ottiene:

$$\text{Spazio}_{\text{SuffixTree}} = \mathcal{O}(n) \text{ parole macchina} \tag{16.29}$$

16.3.3 Rappresentazione Topologica del Suffix Tree di mississippi$

La Figura 16.2 illustra la topologia completa del Suffix Tree compatto per il testo $T\$ = \text{mississippi\$}$ ($n = 12$). Nella convenzione standard dell'Algorithm Engineering (alla Weiner [27], McCreight [28] e Ukkonen [29]):

1. **Etichette compresse lungo gli archi:** L'informazione testuale risiede **esclusivamente lungo gli archi**, codificati in memoria tramite tuple $[\text{start}, \text{end}]$ sul testo $T\$$. Qualora un arco colleghi un nodo direttamente a una foglia senza ulteriori biforcazioni (ad esempio i rami della radice etichettati con \$ e mississippi\$), esso memorizza in un unico arco compresso l'intera sottostringa residua.

2. **Nodi interni:** Ciascun nodo interno u riporta la rispettiva **string-depth** $|\text{path}(u)|$ e la stringa contratta $\text{path}(u)$ formata dalla concatenazione delle etichette lungo il cammino che scende dalla radice. Ciascun nodo interno ha grado di uscita ≥ 2 e tutti i rami uscenti esibiscono iniziali mutuamente distinte. I nodi interni a string-depth 2 includono il nodo `"si"` e il nodo `"ss"`, mentre il nodo `"issi"` presenta string-depth 4.

3. **Foglie e Suffix Array:** Le foglie (in verde) non replicano il testo, bensì registrano **esclusivamente l'indice intero di partenza** $i \in \{1, \ldots, 12\}$ del corrispondente suffisso $S_i = T[i \ldots 12]$. Ordinando le diramazioni di ogni nodo in ordine alfabetico crescente da sinistra a destra, la sequenza delle 12 foglie allineate sulla riga di base genera esattamente il **Suffix Array**:

$$SA = [12,\ 11,\ 8,\ 5,\ 2,\ 1,\ 10,\ 9,\ 7,\ 4,\ 6,\ 3] \tag{16.30}$$

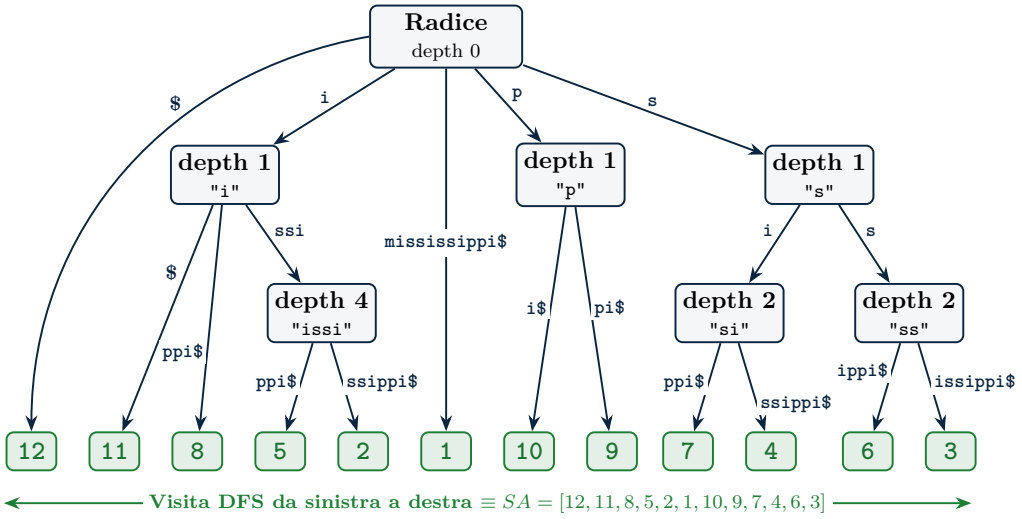


Figura 16.2: Topologia completa del Suffix Tree compatto per $T\$ =$ mississippi\$. I nodi interni riportano la string-depth e la stringa contratta dal cammino radice-nodo; le foglie (riquadri verdi) contengono esclusivamente l'indice iniziale i del suffisso S_i. L'ordine da sinistra a destra delle 12 foglie allineate riproduce fedelmente la sequenza del Suffix Array SA.

16.3.4 Ricerca del Pattern nel Suffix Tree: Ottimalità Temporale $\mathcal{O}(m)$

Nel Suffix Tree, la ricerca di un pattern $P[1 \dots m]$ si articola in una navigazione deterministica top-down a partire dalla radice:

1. **Discesa Guidata dai Caratteri:** Presso il nodo corrente u, si esamina il carattere $P[k]$ corrispondente alla profondità corrente. Si individua l'unico arco uscente la cui etichetta inizia con tale carattere.

2. **Confronto dell'Etichetta:** Si confrontano i caratteri successivi dell'etichetta dell'arco con i corrispondenti caratteri di P:

   - Se si verifica un mismatch prima di aver consumato P, il pattern P non è sottostringa di T. L'algoritmo termina restituendo zero occorrenze.
   - Se l'intero pattern P viene consumato con successo, la discesa si arresta presso un nodo interno v o lungo un arco. Tale punto viene definito il **locus** del pattern P.

3. **Enumerazione delle Occorrenze:** Tutte e sole le foglie appartenenti al sottoalbero radicato nel locus di P rappresentano suffissi che

ammettono P come prefisso. Visitando tale sottoalbero (tramite una Depth-First Search), si estraggono direttamente tutte le t_{occ} posizioni di inizio nel testo.

Analisi della Complessità.

- **Tempo di Ricerca del Locus:** Se l'alfabeto ha dimensione costante $\sigma = \mathcal{O}(1)$ (oppure impiegando una tabella ad accesso diretto sui figli dei nodi), la selezione dell'arco richiede tempo $\mathcal{O}(1)$. Il numero totale di caratteri confrontati durante la discesa è esattamente m. Il tempo per raggiungere il locus è:

$$\text{Tempo di Search} = \mathcal{O}(m) \tag{16.31}$$

  (oppure $\mathcal{O}(m \log \sigma)$ se gli archi uscenti sono mantenuti in vettori ordinati).

- **Tempo di Restituzione (Reporting):** Poiché ogni nodo interno del sottoalbero ha grado almeno 2, il numero di nodi visitati nella DFS è proporzionale al numero di foglie, ovvero $\mathcal{O}(t_{\text{occ}})$.

- **Complessità Temporale Complessiva:**

$$T(n, m) = \mathcal{O}(m + t_{\text{occ}}) \tag{16.32}$$

Ottimalità Asintotica della Ricerca

Il tempo di ricerca $\mathcal{O}(m+t_{\text{occ}})$ garantito dal Suffix Tree è **strettamente ottimo**: è lineare nella dimensione dell'input della query (m) e dell'output (t_{occ}), ed è **completamente indipendente dalla lunghezza n del testo indicizzato**.

16.4 Algoritmi di Costruzione e l'Array LCP

16.4.1 Costruzione Ingenua del Suffix Array tramite Ordinamento

Il metodo concettualmente più intuitivo per costruire il Suffix Array consiste nell'istanziare un array di puntatori a carattere che indirizzano ciascun suffisso all'interno del testo originale T, e nell'ordinarli tramite un algoritmo di ordinamento comparativo convenzionale (come il QuickSort della libreria standard C, 'qsort').

Implementazione del Codice Sorgente (dagli Appunti del Corso). Il listato seguente riporta l'implementazione descritta dal docente:

Listing 16.1: Costruzione ingenua del Suffix Array tramite puntatori e QuickSort standard.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Funzione di confronto lessicografico per qsort
int suffix_cmp(const void *p, const void *q) {
    const char *s1 = *(const char **)p;
    const char *s2 = *(const char **)q;
    return strcmp(s1, s2);
}

void build_suffix_array_naive(const char *T, int n,
                              const char **SA) {
    // 1. Inizializzazione puntatori ai suffissi di T
    for (int i = 0; i < n; i++) {
        // SA[i] punta al suffisso iniziale T[i]
        SA[i] = T + i;
    }

    // 2. Ordinamento rapido dei puntatori ai suffissi
    qsort(SA, n, sizeof(char *), suffix_cmp);
}
```

Analisi della Complessità Asintotica. L'analisi del tempo di esecuzione si compone di due fattori moltiplicativi:

1. **Numero di Confronti del QuickSort:** Su un array di n elementi, QuickSort esegue in media $\mathcal{O}(n \log n)$ comparazioni di elementi (Capitolo 5).

2. **Costo di Ciascun Confronto ('strcmp'):** La funzione 'strcmp(s1, s2)' non richiede tempo $\mathcal{O}(1)$: per due stringhe che condividono un lungo prefisso comune, 'strcmp' scansiona i caratteri uno ad uno fino al primo simbolo divergente. Poiché i suffissi hanno lunghezza media $\mathcal{O}(n)$, nel caso generale una singola invocazione di 'strcmp' costa $\mathcal{O}(n)$.

Moltiplicando i due termini, si ottiene:

$$\text{Tempo Medio}_{\text{Naive}} = \mathcal{O}(n \log n) \cdot \mathcal{O}(n) = \mathcal{O}(n^2 \log n) \tag{16.33}$$

Degenerazione nel Caso Peggiore a $\mathcal{O}(n^3)$

Su testi patologici altamente periodici (ad esempio $T = a^n\$$), tutti i suffissi condividono prefissi lunghi $\Theta(n)$. Se l'ordinatore rapido seleziona sbilanciamenti sfavorevoli sui pivot, il numero di confronti sale a $\Theta(n^2)$ e ogni confronto esamina $\Theta(n)$ caratteri, portando il tempo totale a degenerare a:

$$\text{Tempo Caso Peggiore}_{\text{Naive}} = \mathcal{O}(n^3) \tag{16.34}$$

Cenni agli Algoritmi di Costruzione Lineare $\mathcal{O}(n)$. Come evidenziato dal docente a lezione, la costruzione quadratica è puramente didattica. Nella ricerca avanzata sono stati sviluppati algoritmi ottimi in grado di costruire il Suffix Array direttamente in tempo lineare:

$$\text{Costruzione Ottima } SA = \mathcal{O}(n) \text{ tempo} \tag{16.35}$$

Tra gli algoritmi cardine figurano:

- **Algoritmo DC3 / Skew (Kärkkäinen e Sanders, 2003) [30]:** Applica una tecnica *Divide-et-Impera* basata sul campionamento delle posizioni $i \not\equiv 0 \pmod 3$, riducendo ricorsivamente il problema a una taglia pari ai due terzi e ricombinando i suffissi tramite un merge lineare.
- **Algoritmo SA-IS (Nong, Zhang e Chan, 2009):** Algoritmo basato sulla classificazione dei suffissi in tipi S e L con *induced sorting*, universalmente riconosciuto come l'algoritmo pratico più veloce a memoria contenuta.

16.4.2 L'Array LCP (Longest Common Prefix)

Per colmare il divario di prestazioni tra la ricerca binaria del Suffix Array ($\mathcal{O}(m \log n)$) e l'ottimalità del Suffix Tree ($\mathcal{O}(m)$), Manber e Myers hanno introdotto una struttura ausiliaria fondamentale: l'**Array LCP**.

Definizione 16.4.1 (Array LCP)**.** Dato il testo $T\$$ e il relativo Suffix Array SA, l'array LCP è un vettore di n interi in cui la cella $LCP[i]$ (per $i \in \{2, \ldots, n\}$) memorizza la lunghezza del massimo prefisso comune condiviso

tra il suffisso $S_{SA[i]}$ e il suo immediato predecessore lessicografico nel Suffix Array, $S_{SA[i-1]}$:

$$LCP[i] = |\text{LCP}(S_{SA[i]},\ S_{SA[i-1]})| \tag{16.36}$$

Per convenzione algebrica, l'estremo iniziale è posto a zero:

$$LCP[1] = 0 \tag{16.37}$$

Calcolo Analitico dell'Array LCP per $T =$ mississippi\$ Esaminiamo coppie consecutive di suffissi ordinati nel Suffix Array ricavato nella Tabella 16.2:

i	$SA[i]$	Suffisso Corrente $S_{SA[i]}$	Suffisso Precedente $S_{SA[i-1]}$	Prefisso Comune Condiviso	$LCP[i]$
1	12	\$	—	—	**0**
2	11	*i*\$	\$	Nessuno	**0**
3	8	*ippi*\$	*i*\$	'i'	**1**
4	5	*issippi*\$	*ippi*\$	'i'	**1**
5	2	*ississippi*\$	*issippi*\$	'issi'	**4**
6	1	*mississippi*\$	*ississippi*\$	Nessuno	**0**
7	10	*pi*\$	*mississippi*\$	Nessuno	**0**
8	9	*ppi*\$	*pi*\$	'p'	**1**
9	7	*sippi*\$	*ppi*\$	Nessuno	**0**
10	4	*sissippi*\$	*sippi*\$	'si'	**2**
11	6	*ssippi*\$	*sissippi*\$	's'	**1**
12	3	*ssissippi*\$	*ssippi*\$	'ssi'	**3**

Tabella 16.3: Calcolo passo-passo dell'array LCP per il testo $T\$ =$ mississippi\$.

Il vettore risultante è:

$$LCP = [0,\ 0,\ 1,\ 1,\ 4,\ 0,\ 0,\ 1,\ 0,\ 2,\ 1,\ 3] \tag{16.38}$$

Costruzione Lineare $\mathcal{O}(n)$ tramite Algoritmo di Kasai et al. Nel 2001, Kasai, Lee, Arimura, Arikawa e Park [31] hanno dimostrato che, una volta disponibile il Suffix Array, l'array LCP può essere calcolato in tempo **strettamente lineare** $\mathcal{O}(n)$ senza alcun confronto ridondante:

$$\text{Tempo di Costruzione } LCP = \mathcal{O}(n) \tag{16.39}$$

L'intuizione chiave dell'algoritmo di Kasai si basa sull'ordinamento posizionale dei suffissi nel testo originario: passando dal suffisso S_i al suffisso successivo S_{i+1}, la lunghezza del prefisso condiviso con il rispettivo predecessore nel Suffix Array può diminuire al più di 1 (in virtù della rimozione del primo carattere), consentendo di calcolare tutti gli LCP con un numero totale di confronti limitato da $2n$.

16.4.3 Applicazioni Avanzate: Ricerca di Sottostringhe Ripetute

L'accoppiamento tra il Suffix Tree e il binomio (SA, LCP) consente di risolvere con eleganza problemi complessi di biologia computazionale e analisi di sequenze.

Esempio 16.4.1 (Problema Formulato a Lezione)**.** Dato un testo $T[1 \dots n]$, determinare se esiste una sottostringa di lunghezza almeno L che compare con una frequenza di almeno k occorrenze in posizioni distinte di T:

$$\exists\, \text{sottostringa } \alpha \in \Sigma^* \quad \text{tale che} \quad |\alpha| \geq L \quad \wedge \quad \text{occorrenze}(\alpha, T) \geq k \tag{16.40}$$

Risoluzione con Suffix Tree. Nel Suffix Tree, ogni sottostringa che compare più volte nel testo corrisponde univocamente a un nodo interno u:

- La sottostringa $\alpha = \text{path}(u)$ compare esattamente tante volte quante sono le foglie presenti nel sottoalbero radicato in u ($\text{leaves}(u)$).
- **Criterio di Verifica:** Il problema si riduce a individuare se esiste almeno un nodo interno u tale che:

$$|\text{path}(u)| \geq L \quad \wedge \quad \text{leaves}(u) \geq k \tag{16.41}$$

- **Algoritmo:** Una singola visita in profondità (DFS) calcola in post-order per ciascun nodo u il numero di foglie del suo sottoalbero, verificando la condizione in tempo lineare:

$$\text{Tempo} = \mathcal{O}(n) \tag{16.42}$$

Risoluzione con Suffix Array ed Array LCP. Nel Suffix Array, tutti i suffissi che condividono un prefisso comune di lunghezza almeno L occupano un intervallo contiguo di indici.

Teorema 16.4.1 (Proprietà di Minimo LCP su Finestra Contigua)**.** *Se una sottostringa di lunghezza L compare in k suffissi distinti, tali suffissi formano un blocco contiguo di k elementi nel Suffix Array. Di conseguenza, nel vettore LCP deve esistere una finestra di $k-1$ elementi contigui di indici $[i, i+k-2]$ tale che:*

$$\min_{j=i}^{i+k-2} LCP[j] \geq L \tag{16.43}$$

Dimostrazione. L'LCP tra due suffissi generici non consecutivi $S_{SA[a]}$ e $S_{SA[b]}$ ($a < b$) nel Suffix Array è dato dal minimo valore di LCP lungo l'intervallo compreso tra di essi:

$$|\text{LCP}(S_{SA[a]},\ S_{SA[b]})| = \min_{j=a+1}^{b} LCP[j] \tag{16.44}$$

Se tutti i valori di $LCP[j]$ per $j \in [i, i+k-2]$ sono maggiori o uguali a L, allora il minimo prefisso condiviso tra il primo suffisso $S_{SA[i-1]}$ e l'ultimo suffisso $S_{SA[i+k-2]}$ ha lunghezza almeno L. Ne consegue che tutti i k suffissi da $i-1$ a $i+k-2$ condividono i medesimi primi L caratteri, certificando la presenza di almeno k occorrenze della sottostringa. □

Algoritmo a Finestra Scorrevole (Sliding Window) in Tempo Lineare. Per individuare l'eventuale esistenza della finestra:

1. Si scorre una finestra di ampiezza fissa pari a $k-1$ lungo il vettore LCP.

2. Per mantenere il minimo corrente della finestra in tempo ammortizzato $\mathcal{O}(1)$ a ogni scorrimento, si impiega una coda a doppia estremità (Deque) monotona decrescente.

3. L'intera scansione dell'array LCP richiede tempo lineare:

$$\text{Tempo}_{\text{Sliding Window}} = \mathcal{O}(n) \tag{16.45}$$

   e spazio addizionale limitato a $\mathcal{O}(k)$ per la deque.

16.5 Quadro Comparativo e Sintesi Conclusiva

16.5.1 Tabella Sinottica delle Metodologie

La Tabella 16.4 sintetizza il confronto asintotico e ingegneristico tra le metodologie esaminate per la ricerca di sottostringhe in un testo $T[1 \dots n]$ con pattern $P[1 \dots m]$.

Struttura / Approccio	Tempo Ricerca (Existence/Count)	Tempo Restituzione (Reporting)	Occupazione di Memoria	Tempo di Costruzione
Scansione Ingenua (Brute-Force)	$\mathcal{O}(n \cdot m)$	Incluso	$\mathcal{O}(n)$ (Solo il testo T)	$\mathcal{O}(0)$ (Nessuna pre-elaborazione)
Suffix Array ($SA + T$)	$\mathcal{O}(m \log n)$	$\mathcal{O}(t_{\text{occ}})$	$\Theta(n)$ (n interi $+n$ caratteri, $\approx 5n$ byte)	$\mathcal{O}(n^2 \log n)$ naive $\mathcal{O}(n)$ ottimo (DC3/SA-IS)
Suffix Tree ($\mathcal{T}(T\$)$)	$\mathcal{O}(\mathbf{m})$ (Indipendente da n)	$\mathcal{O}(t_{\text{occ}})$	$\Theta(n)$ ($2n$ nodi, archi a coppie $[s, e]$, $\approx 20n - 40n$ byte)	$\mathcal{O}(n)$ (Ukkonen / McCreight)

Tabella 16.4: Quadro sinottico comparativo delle metodologie per il Substring Search.

16.5.2 Trade-off Ingegneristico: Fattore Costante di Spazio vs. Efficienza Temporale

L'analisi comparativa mette in luce un classico compromesso architetturale dell'Algorithm Engineering:

- **Il Limite Pratico del Suffix Tree:** Sebbene il Suffix Tree vanti un tempo di ricerca ideale $\mathcal{O}(m)$ indipendente da n, nella pratica software ciascun nodo interno richiede puntatori al primo figlio, al fratello successivo, al padre e alla coppia di interi d'arco $[start, end]$. Su architetture a 64 bit, ciò si traduce in un'impronta di memoria reale pari a circa $20 - 45$ byte per carattere indicizzato. Per collezioni genomiche di diversi Gigabyte, il Suffix Tree satura rapidamente la memoria RAM di sistema.
- **La Superiorità Applicativa del Suffix Array:** Il Suffix Array richiede unicamente n interi da 4 byte più gli n byte del testo, occupando circa 5 byte per carattere (fino a 8 volte meno spazio rispetto al Suffix Tree). Nonostante il costo di ricerca contenga il fattore logaritmico $\mathcal{O}(m \log n)$, nella pratica ingegneristica per un testo di un miliardo di caratteri $\log_2(10^9) \approx 30$ iterazioni, un costo del tutto trascurabile rispetto ai vantaggi di compattezza della memoria e alla straordinaria località spaziale dei dati nei vettori continui.

16.5.3 Il Ruolo Unificante dell'Array LCP e Prospettive Future

L'introduzione dell'array LCP colma definitivamente il divario teorico tra Suffix Tree e Suffix Array:

1. **Accelerazione della Ricerca a $\mathcal{O}(m + \log n)$:** Arricchendo il Suffix Array con le informazioni di prefisso comune estratte da LCP e alberi di tipo RMQ (*Range Minimum Query*), è possibile evitare confronti ridondanti durante la ricerca binaria, abbattendo il tempo a $\mathcal{O}(m + \log n)$ [26].

2. **Emulazione Topologica del Suffix Tree:** Manzini e Ferragina hanno dimostrato che la terna formata da Testo, Suffix Array e Array LCP contiene esattamente la medesima informazione topologica del Suffix Tree, consentendo di simulare qualunque algoritmo su albero (visite DFS, Lowest Common Ancestor, motivi ripetuti) con codice sequenziale iterativo operante su array compatti.

3. **Verso gli Indici Compressi di Seconda Generazione:** La combinazione del Suffix Array con la Trasformata di Burrows-Wheeler (BWT) e le strutture di rank/select condurrà alla nascita del celebre **FM-Index** (*Ferragina-Manzini Index*), capace di comprimere il testo e l'indice in uno spazio inferiore a quello del testo grezzo originario ($\leq n$ byte complessivi), rappresentando l'attuale frontiera mondiale della bioinformatica e dei motori di ricerca a testo pieno.

Bibliografia

[1] Donald E. Knuth. *The Art of Computer Programming*, volume 1. Addison-Wesley, Reading, MA, 3 edition, 1997.

[2] Alok Aggarwal and Jeffrey S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.

[3] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.

[4] Robert Sedgewick. Implementing quicksort programs. *Communications of the ACM*, 21(10):847–857, 1978.

[5] Vladimir Yaroslavskiy. Dual-pivot quicksort algorithm. *Research Disclosure*, 2009. Available at http://iaroslavski.narod.ru/quicksort/DualPivotQuicksort.pdf.

[6] Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.

[7] C. T. Fan, Mervin E. Muller, and Ivan Rezucha. Development of sampling plans by using sequential (item by item) selection techniques and digital computers. *Journal of the American Statistical Association*, 57(298):387–402, 1962.

[8] Jeffrey S. Vitter. Random sampling with a reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57, 1985.

[9] William Pugh. Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6):668–676, 1990.

[10] Raimund Seidel and Cecilia R. Aragon. Randomized search trees. *Algorithmica*, 16(4-5):464–497, 1996.

[11] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[12] Jean Vuillemin. A unifying look at data structures. *Communications of the ACM*, 23(4):229–239, 1980.

[13] Edward M. McCreight. Priority search trees. *SIAM Journal on Computing*, 14(2):257–276, 1985.

[14] Michael A. Bender and Martin Farach-Colton. The LCA problem revisited. In *Proceedings of the 4th Latin American Symposium on Theoretical Informatics (LATIN)*, pages 88–94. Springer, 2000.

[15] Ricardo Baeza-Yates. A fast set intersection algorithm for sorted sequences. In *Combinatorial Pattern Matching (CPM)*, volume 3109 of *Lecture Notes in Computer Science*, pages 400–408. Springer, 2004.

[16] Jon Louis Bentley and Andrew Chi-Chih Yao. An almost optimal algorithm for unbounded searching. *Information Processing Letters*, 5(3):82–87, 1976.

[17] Alistair Moffat and Justin Zobel. Self-indexing inverted files for fast text retrieval. *ACM Transactions on Information Systems*, 14(4):349–379, 1996.

[18] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK, 2008.

[19] Yehoshua Perl, Alon Itai, and Haim Avni. Interpolation search—a $\log \log n$ search. *Communications of the ACM*, 21(7):550–553, 1978.

[20] Jon L. Bentley and Robert Sedgewick. Fast algorithms for sorting and searching strings. In *Proceedings of the 8th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 360–369, 1997.

[21] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2 edition, 1998.

[22] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley Professional, 4 edition, 2011.

[23] Edward Fredkin. Trie memory. *Communications of the ACM*, 3(9):490–499, 1960.

[24] Donald R. Morrison. PATRICIA—practical algorithm to retrieve information coded in alphanumeric. *Journal of the ACM*, 15(4):514–534, 1968.

[25] Paolo Ferragina and Roberto Grossi. The string b-tree: A new data structure for string search in external memory and its applications. *Journal of the ACM*, 46(2):236–280, 1999.

[26] Udi Manber and Gene W. Myers. Suffix arrays: A new method for on-line string searches. *SIAM Journal on Computing*, 22(5):935–948, 1993.

[27] Peter Weiner. Linear pattern matching algorithms. In *14th Annual Symposium on Switching and Automata Theory (SWAT)*, pages 1–11, 1973.

[28] Edward M. McCreight. A space-economical suffix tree construction algorithm. *Journal of the ACM*, 23(2):262–272, 1976.

[29] Esko Ukkonen. On-line construction of suffix trees. *Algorithmica*, 14(3):249–260, 1995.

[30] Juha Kärkkäinen and Peter Sanders. Simple linear work suffix array construction. In *Automata, Languages and Programming (ICALP)*, volume 2719 of *Lecture Notes in Computer Science*, pages 943–955. Springer, 2003.

[31] Toru Kasai, Gunho Lee, Hiroki Arimura, Setsuo Arikawa, and Kunsoo Park. Linear-time longest-common-prefix computation in suffix arrays and its applications. In *Combinatorial Pattern Matching (CPM)*, volume 2089 of *Lecture Notes in Computer Science*, pages 181–192. Springer, 2001.